On the Provenance of Software Systems:

Automating Software Traceability with Knowledge Graph and Large Language Model Synergy

By

Tyler Thomas Procko

Under

Dr. Omar Ochoa

A Dissertation

Embry-Riddle Aeronautical University

Daytona Beach, Florida

Spring 2023 – Spring 2025

THE PRESENT DISSERTATION WAS WRITTEN ENTIRELY BY THE HUMAN AUTHOR. DESPITE THE CENTRAL THEME BEING THE CONSIDERATE, STRATEGIC USE OF GENERATIVE ARTIFICIAL INTELLIGENCE, CONSTRUCTS AS SUCH PLAYED NO PART IN THE FINAL FORMATION OF THE PRESENT ARTIFACT. THE AUTHOR HOPES THAT HIS FELLOWS MAY CONTINUE TO THINK AND CREATE AS INSPIRED BEINGS, PROPERLY MAINTAINING THE DELINEATION BETWEEN MAN AND MACHINE AS WITH HAND AND HAMMER, WHILE REALIZING THE POSSIBILITY OF MAN-COMPUTER SYMBIOSIS.

ACKNOWLEDGEMENTS

The dissertation artifact is the culmination of an endeavour of many years of learning and research. In such time, and prior to its formation, acquaintances are made who influence the trajectory of said artifact, in various ways and capacities. To these individuals, I must give thanks. To my advisor and closest friend, Omar, I am indebted for his taking me under the wing as a fledgling software engineer and instilling in me a deep interest in research. I thank Nick for an early cultivation of interest in that domain so important to my early development as a researcher, the Semantic Web. To Tim, my best friend, colleague, and personal comedian, I am most appreciative. To my father and his father before him, I probably owe my curiosity and academic penchant. To my mother and the members of my mother's family, my aunts, uncles and cousins, I must thank for instilling in me a penchant for doing things with my own two hands; and for making dying with one's boots on the highest aspiration for a man. I am grateful to my milkmaid, for giving me perspective. I must thank my Creator, through whom all things are made possible. I am humbly thankful for the words of those authors and thinkers, operating in various arenas of life, who have shaped me as an integrated man, forming my philosophy of life, my Weltanschauung, and who have brought me struggling out of the mire of apathy into the splendor of positive masculinity: Alighieri, Aristotle, Bacon, Carnegie, Deida, De Tocqueville, Dumas, Glubb, Greene, Hume, Hyde, Jacques, Jay, Lee, Licklider, Ma, Machiavelli, McCarthy, Mentzer, Mishima, Musashi, Rothfuss, Roosevelt, Sowell, Strauss, Tomassi, Tolkien, and uncountable others, each affecting me in some way. Admittedly I possess no hope of a satiation of the desire for knowledge; accordingly, the list of names shall grow. In truth, it seems impracticable for me to close my dissertation's work into a single artifact, as there is always tangential knowledge to identify and integrate. Notwithstanding, for the purpose of providing to the greater scientific community a well-scoped, focused artifact from which to potentiate new work, I provide, here, some two-hundred seventy pages of my accumulated understanding of proper, human-centric knowledge representation in the context of software engineering in an age of automated systems.

A SOFT, EASY LIFE IS NOT WORTH LIVING, IF IT IMPAIRS THE FIBRE OF BRAIN AND HEART AND MUSCLE. WE MUST DARE TO BE GREAT; AND WE MUST REALIZE THAT GREATNESS IS THE FRUIT OF TOIL AND SACRIFICE AND HIGH COURAGE... FOR US IS THE LIFE OF ACTION, OF STRENUOUS PERFORMANCE OF DUTY; LET US LIVE IN THE HARNESS, STRIVING MIGHTILY; LET US RATHER RUN THE RISK OF WEARING OUT THAN RUSTING OUT. – *Theodore Roosevelt*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

The present dissertation delineates a system that enables those engaged in software development to automatically generate and maintain project life cycle provenance. All projects are implemented and made manifest with the development of artifacts, e.g., papers, code files, etc. Tools exist to accelerate artifact creation, but little focus is paid to the processes that produce them. In terms of Ontology, or, from Ancient Greek, the *study of being*, the two most basic entities in reality are Continuant and Occurrent, or, roughly, "Artifact" and "Process". This dissertation posits that for any created artifact, its process of creation, i.e., its life cycle *provenance*, must also be captured and maintained. Artifacts are often delivered without an explicit trace of their evolution. This is particularly unacceptable for critical systems, where requirements documents, codebases, and other meta-artifacts are revisited without a corresponding history of how or why they came to be, leading to confusion and rework.

While the software development life cycle (SDLC) incorporates meta-artifacts like traceability matrices to improve artifact provenance, these are typically informal, heavy with natural language, and lack structured explainability. This work proposes that each artifact should be attended by a machine-readable, human-interpretable, extensible provenance record, implemented in the form of a knowledge graph, backed by well-established ontologies. The developed system, ProvTracer, leverages structured knowledge via PROV-O and the Basic Formal Ontology alongside generative natural language capabilities via the Generative Pretrained Transformer (GPT) series of Multimodal Large Language Models (MLLMs), to create real-time, traceable, and explainable links between development activities and their resulting artifacts. By

capturing these provenance trace links automatically through multimodal signals, e.g., screenshots, peripheral device input, etc., ProvTracer aims to bridge the gap between implicit processes and explicit traces, enabling developers to understand, query, maintain, integrate, and trust the evolution of their projects and systems.

The synergy between knowledge graphs and MLLMs enables a novel form of interactive, explainable software development. Natural language queries of provenance trace link knowledge graphs can reduce information overload, extract developer rationale and decision histories, support task assignment, and a range of project management activities. This aligns with a burgeoning trend in research demonstrating that structured knowledge improves machine learning trust, transparency and reproducibility. The present dissertation addresses the challenges of traceability and explainability in the SDLC by presenting a system that automatically captures artifact provenance and operationalizes it for practical use in real-world software development.

Chapter 1: INTRODUCTION

In the year 1958, the American mathematician, John Tukey, used the term "software" in a printed article [1]; this marked the beginning of a storied area of research and development. The Software Development Life Cycle (SDLC) is a generalized abstraction of the processes, artifacts and actors involved in the development of some software artifact, from the initial steps, e.g., requirements elicitation, to the end, e.g., with maintenance and even phaseout, or disposal. Various models have been popularized in attempting to streamline software development, e.g., the early Waterfall or Cascade models [2, 3], and the Agile methods of the 1980s-1990s [4, 5]. While these methods provide developers with frameworks under which to operate over the course of development, they do not consider the importance of provenance to software artifacts, i.e., *how* and *why* software artifacts were created in the first place. In other words: software artifact provenance may be considered as a similarly important sibling concern to the developed artifacts themselves. The intent of the present paper is to demonstrate the importance of software development provenance, and to propose a method for automatically capturing it, directly and seamlessly embeddable into the typical SDLC.

1.1. PROVENANCE

Prior to 1979, scientific publications including the term "provenance" in their titles had no association with information systems. Interestingly, most of these early papers on provenance were written in the fields of geology, archaeology, and anthropology [6]. Etymologically, the word "provenance" was (and is) often used synonymously with the terms "lineage" or "pedigree", both of which have a connotation with genetics [7]. The earliest known publication on provenance as it

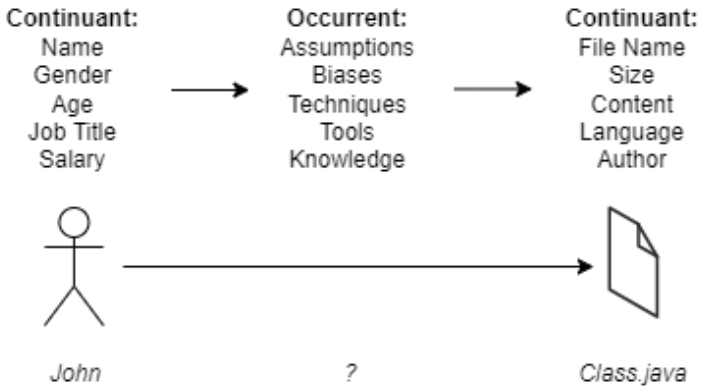

Fig. 1 The difficulty in identifying artifact provenance.

pertains to an information system is Richard H. Lytle's 1979 dissertation, *"Subject Retrieval in Archives: A Comparison of the Provenance and Content Indexing Methods"* [8]. This was followed by a 1985 article, *"The Power of the Principle of Provenance"*, in which Bearman and Lytle delineate a data retrieval system for data archivists that uses provenance (in their terms, the activities that an organization may have performed in generating some requested artifact) to resolve queries, as opposed to indexing subjects [9].

Provenance is the documentation of the process of a created artifact's life cycle. The World Wide Web Consortium (W3C) defines provenance as the "… information about entities, activities, and people involved in producing a piece of data or thing, which can be used to form assessments about its quality, reliability or trustworthiness" [10]. Many would argue that the provenance of an artifact is as important as the artifact itself. For example, a car buyer does not purchase a vehicle without investigating the previous owners, maintenance records, etc. A contemporary software system does not exist without a source control system, e.g., Git, backing it. The Open Provenance Model states that the desire for better provenance in the online science community is increasing, since provenance is seen as a "crucial component of workflow systems that can help scientists ensure reproducibility of their scientific analyses and processes" [11]. A literature review that

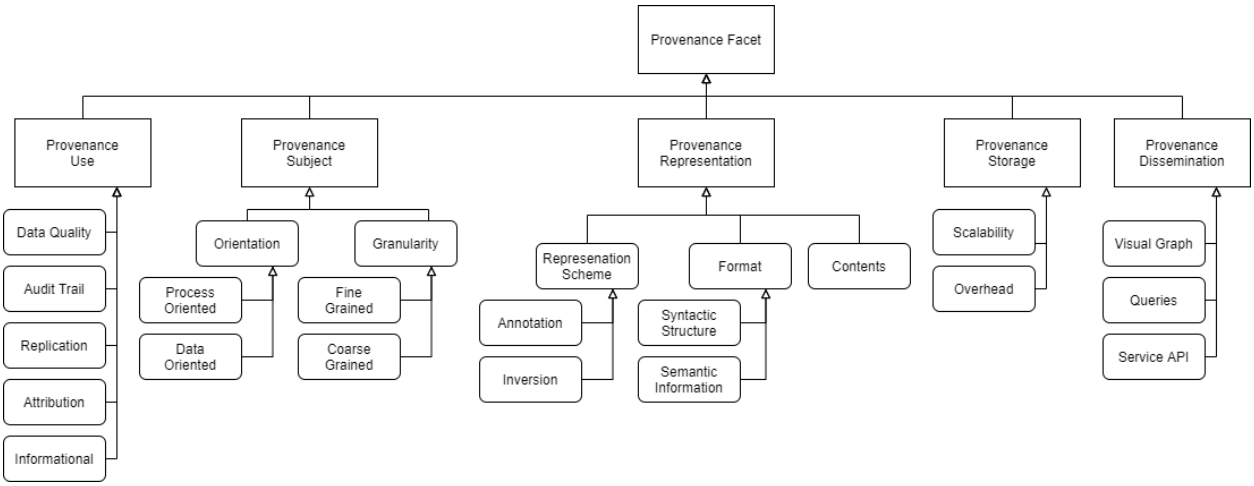

Fig. 2 The provenance taxonomy (re-created from Simmhan et al. [19]).

produced a taxonomy of IT project failure root causes states that project management issues cause most of the symptoms of IT project failure [12]. Project management issues often arise from miscommunications that may arise from poor provenance. Some real-world uses of provenance in information systems include supply chain management, scientific experiments, and complex data processing [13].

There is much consideration on the creation of artifacts for projects, but little attention is given to the processes that these artifacts are engendered from. See Fig. 1. Copious mainstream tools exist that streamline the development of artifacts, e.g., word processors, integrated development environments, web browsers, etc., but the common use of provenance capturing applications is novel.

A simple provenance model for Web-based research is browser history, but this scheme only records the chronological access of Web resources, and does not link them to parts of artifacts, e.g., as one Webpage may incite the formation of an entire section in a given artifact. Another example of provenance capture is found in software development work, which typically employs the use of a version control system, like Git. However, Git only records file modifications, not the

TABLE I Differences between metadata and provenance.

Aspect	**Metadata**	**Provenance**
Definition	Descriptive information about data	History, origin and changes of data
Focus	Characteristics like format, structure, size	Processes, actions, transformations
Purpose	Organization, discovery, management	Traceability, accountability, trust
Examples	Author, creation date, file size, format	Who modified data and how
Use Cases	Organizing, searching, interpreting data	Ensuring authenticity, tracing errors, workflows

reasoning behind such; and, although developers can provide messages accompanying their changes, these messages are volitional, which results in poor descriptions of provenance, becoming worse as projects lengthen, and become more complex, over time.

In a word, provenance allows the ascertainment of data quality, inasmuch as it provides data lineage. Simmhan et al. present the data provenance taxonomy, which covers the lifecycle of provenance. See Fig. 2. The system developed herein addresses all facets of provenance, including use, representation, storage, and dissemination.

1.2. METADATA AND PROVENANCE

To avoid confusion, it essential to delineate metadata and provenance, as the two terms are often conflated as representing the same category of thing, when, in fact, they are conceptually distinct. Per Riley, in association with the National Information Standards Organization (NISO), metadata is loosely defined as structured information that describes, explains, locates, or otherwise makes it easier to retrieve, use, or manage an information resource [14]. From the W3C, provenance is defined as "… information about entities, activities, and people involved in producing a piece of data or thing, which can be used to form assessments about its quality, reliability or trustworthiness" [10]. Summarizing: provenance is process-oriented metadata focusing on *how*

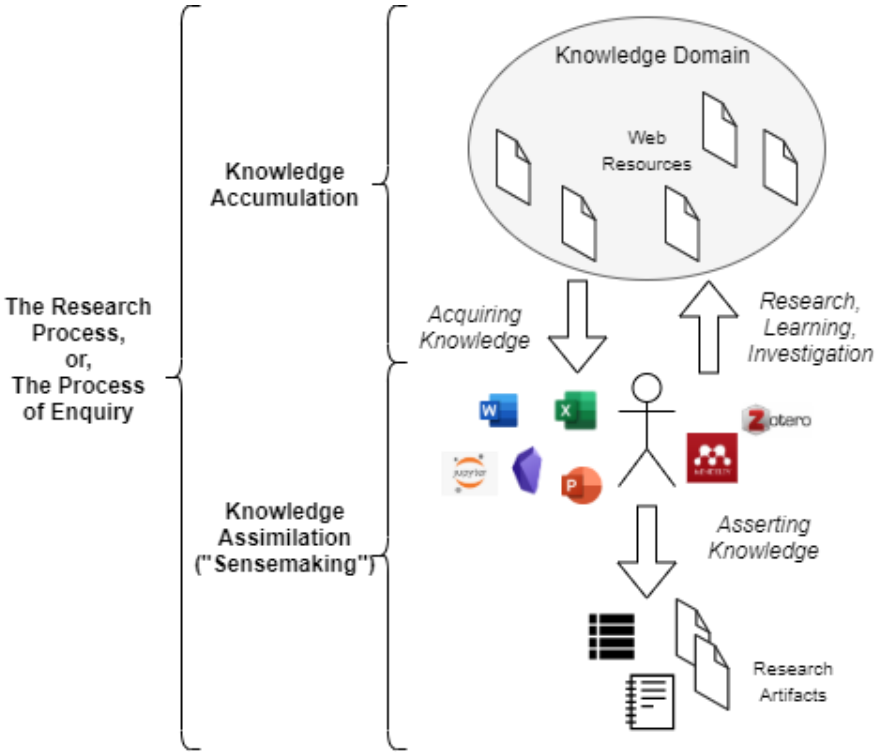

Fig. 3 The complex process of enquiry in the digital age.

data was created and changed, while metadata itself is descriptive information that provides context about data attributes (see TABLE I).

With respect to the present dissertation, an important factor is the ease of automation for the capture of these two description types. While metadata capture is easily automated and managed, e.g., by file systems, provenance is not. Provenance pertains to information about *how*, not just the *what*.

1.3. THE PROBLEM OF PROVENANCE

Attending the creation of any artifact is its provenance; this is an absolute inevitability of any intentional process. There is a duration of time that some agent, or knowledge worker, endures, during which he volitionally takes knowledge garnered from various influencing sources, and makes it explicit in the desired artifact. This process is laden with insightful provenance that can

benefit downstream analysis, querying, and understanding. The difficulty lies in capturing artifact provenance, as it is a multi-faceted problem of interest, intent, intuition, assertion, and modification. See Fig. 3. For instance, a software engineer creating a class diagram, when parsing through a customer requirements backlog, defines a new class, “ServerSideParallelRenderer”. This is a complex modification to the artifact, requiring considerable thought on the part of the knowledge worker, e.g., the translation of natural language requirement(s) to a brief, descriptive class name; and, unless the provenance of this action is captured, it is lost, and the insight with it. Software engineers (ideally) use additional artifacts like traceability matrices to explicitly link specifications to requirements, but this is another artifact laden with semantic provenance that is hard to validate, especially with larger sets of requirements. Moreover, this technique of provenance capture is strictly manual, requiring human effort to produce. The system developed in the course of the present thesis intends to automate this process.

Manual data entry is prone to issues of human fallibility, i.e., incorrectness, inconsistency, slowness, etc., which may affect pressing concerns such as compliance, e.g., with standards or governing bodies. The use of automation tools for mundane tasks has been exhibited as addressing such human issues in myriad fields [15, 16]. Regarding provenance capture, insofar as some provenance capture system is automated, its scalability to larger and more longitudinal projects is improved; and, any necessitation of manual provenance entry is detrimental to provenance maintenance.

Fig. 4 Relevant and irrelevant knowledge events for some artifact, e.g., a Microsoft Word file, over some time, t; the artifact of interest is denoted by a version, e.g., V_x (recreated from [18]).

Speaking in terms of the Basic Formal Ontology (BFO), the two basic classifications of entities in reality are *Continuant* and *Occurrent*, or, put informally, "*Artifact"*[1] and "*Process"* [17]. The properties associated with artifacts are easy to identify, as they exist in time with the artifact itself, e.g., this document is written in the English language using an American dialect, the color of the text is black, etc. However, the properties of the process that spawned this document are more difficult to capture, e.g., this document was informed by several other scientific papers the author had written previously, citations were taken from other papers and from Google Scholar, etc. This is provenance.

The process of artifact creation is laden with semantic meaning, implicit thinking, assumptions, and outside influences. These are all forms of provenance that affect the output artifact, and as such, they should be captured so that the artifact and its process form a self-

[1] In BFO's mid-level extension, the CCO, an Artifact is something intentionally created by some Agent. The term Artifact is used here as a more digestible synonym of Continuant, and ignores Continuants *not* created by some Agent, like a leaf, which grows "naturally", i.e., through a series of physical processes and without the explicit direction of some Agent. The present dissertation only considers Continuants that are Artifacts, i.e., that are created directly and volitionally by some Agent.

describing knowledge unit. This is important in larger projects where several artifacts contribute to a larger goal, such as that in the SDLC.

1.4. CHRONOLOGICAL AND CONTEXTUAL PROVENANCE

Thaul posits that, within the context of personal knowledge management, knowledge units are composed of three distinct parts: content, descriptive metadata, and relationships with other knowledge [18]. Further, knowledge events, or sub-processes occurring between new versions of artifacts, are classified as either relevant (contributing to a new version of said artifact) or irrelevant (noise). See Fig. 4. Several event types are defined, including phone call, email, website, etc. These event types are limited to a handful within Thaul's proposal, and much of the conceptual architecture indicates that his system is inflexible and limited to a very specific subset of digital work. Many event types cannot be reliably addressed, or are otherwise blacklisted, as they are noise with respect to some particular project of interest. With a more general architecture leveraging a LLM, as the present dissertation proposes, these event types can be more dynamically addressed. In the context of the SDLC, some relevant provenance event types may include the SRS, a tracked issue or bug report, a Scrum / Kanban board, a developer comment, etc.

There are two distinct provenance types established as the underpinning of the present dissertation: chronological and contextual. Although this bifurcation may be considered as fiat (or arbitrary, w.r.t. BFO [17]), the delineation of the two is important.

- Chronological provenance: the "what" and "when" of artifact modifications over time, i.e., the timeline of artifact modifications
- Contextual provenance: the "why" behind artifact modifications, i.e., motivations, influences

One may consider these two provenance types as synonymous with data-oriented and process-oriented provenance, respectively, e.g., as Simmhan et al. posit in their taxonomy of provenance [19]. The two may also be thought of as descriptions and explanations, or simply metadata and provenance (see section 1.2), respectively. The former, roughly synonymous with metadata, is not the interest of the present dissertation, although it is a facet to be captured by traditional means, e.g., file versioning, Git, etc. The latter, contextual provenance, is the primary interest of this work, encompassing the various semantic sources from which ideas, inspirations, motivations, influences, and assumptions contribute to the evolution of digital artifacts. Such sources of contextual provenance may occur in visited Webpages, in consulted online publications, within a digital team chat, in email chains, local files, etc.; or, more informally, in conversations with co-workers, and even one's own knowledge and skill. These sources of contextual provenance, and their precise contribution to developed artifacts, are the focus of the automated provenance capture system proposed in the present dissertation, alongside traditional chronological provenance.

1.5. PROVENANCE AND TRACE LINKS

It may be said that, for provenance capture, there are two perspectives, or approaches, to the task: post-activity capture and real-time capture, or retrospective and prospective capture, respectively [20]. The former is typically a manual process, involving human remembrance and annotation, and is therefore not amenable to reproducibility, due to the diversity of minds and stochasticity of thought. The latter is the perspective of the present dissertation, that provenance capture should occur in real-time, with little to no delay between events, with as little human involvement in the capture and serialization as possible.

This dichotomy is mirrored in the software engineering literature regarding software traceability. A software trace is a semantic link between two or more software artifacts. Software practitioners have established two main ways of achieving traceability: trace recovery and trace capture [21].

- Trace recovery: post-activity traceability; an approach to create trace links after the artifacts that they associate have been generated or manipulated [21, 22]
- Trace capture: real-time traceability; the creation of trace links concurrently with the creation of the artifacts that they associate [21, 22]

As yet, there is no established conceptual bridge between the notion of provenance, which is deeply entrenched in the Semantic Web movement of the World Wide Web Consortium, and software traceability, which is a concern of software practitioners engaged in client work. There is very little research in this conceptually overlapping area. Despite the dated nature of software traceability, being defined as early as 1990 in IEEE standard 610.12-1990 [23], there is no consensus on the set of trace link terms, or relations, to be used in traceability efforts. Mustafa and Labiche present a preliminary set of requirements for a trace links taxonomy based on the Resource Description Framework [24]. Abdeena et al. present a method for forming trace links based on a taxonomy, as opposed to simple, direct trace links [25]. An early, very general definition of traceability is more applicable to the present dissertation; it is given as follows: "… the ability of tracing from one entity to another based on given semantic relations" [26]. While the primary use case herein is the SDLC, the real-time generation of trace links between source and target artifacts engendered through any digital research process is the goal of future work.

The Center of Excellence for Software and Systems Traceability (CoEST) states that trace links "… can be created manually as part of the software development process. Much [state-of-the-art] work focuses on automating the trace link creation process using information retrieval, machine learning, AI approaches, or even eye-tracking devices; or leveraging the software developing environment to prospectively capture trace links" [27].

"Real-time", as a prepended modifier to other terms, such as "system", and not in the domain of theoretical computer science [28], is typically associated with systems incorporating hardware and software, in addition to direct human input or concerns, e.g., an electric vehicle's autopilot system. Such an example is a hard real-time system, i.e., results generated after some deadline may result in a catastrophic outcome, e.g., a loss of human life [29]. The concept of a real-time AI system, i.e., one in which reasoning is triggered by events signaled by knowledge sources, was proposed in a paper describing the general architecture, as early as 1989 [30]. The system proposed in the present dissertation is a soft real-time system, with very loose properties regarding such a deadline, as the only negative outcome would be a loss of information. The primary concern is the explainability of the output provenance traces (operationalized traceability). In a later chapter detailing future work, the importance of real-time speed and responsiveness will be discussed, as there is a possibility of integration with hardware devices, such as cameras for eye tracking.

1.6. MOTIVATION AND GOAL

A major facet affecting project transparency is the representation of the steps undertaken for a given research process. The discovery and integration of new knowledge with developed artifacts is only one part of the process of research, where another, perhaps more important, part is, as

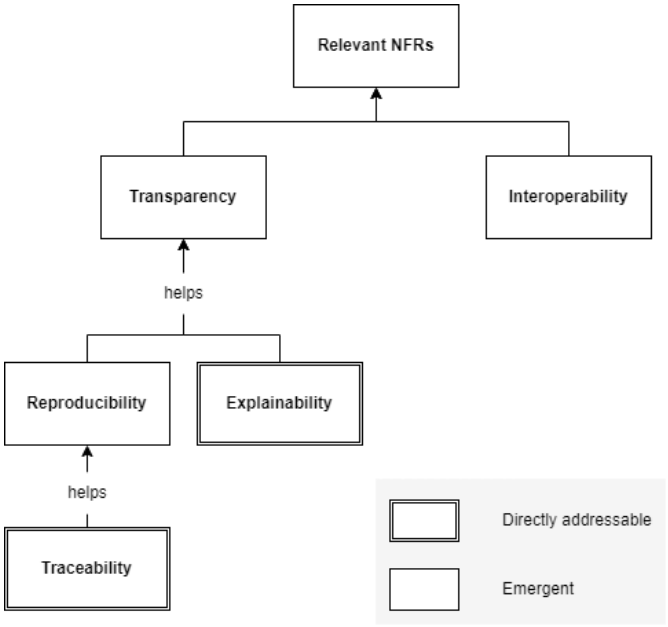

Fig. 5 The NFRs of interest to the present dissertation. Traceability and explainability are directly addressed, engendering reproducibility and transparency, which are emergent. Interoperability is a sibling NFR engendered with the use of a common ontology.

posited in the present dissertation, overlooked, even though contemporary computing can allow for real-time capture of such knowledge. "*Basic scientific research is concerned with the discovery of new phenomena and their integration into coherent conceptual models of major physical or biological systems… [engineering research is] concerned with the discovery and systematic conceptual structuring of knowledge*" [31].

Although the principal process focused on in the present dissertation is the SDLC, much consideration is also given toward the general process of scientific research, as activities in the SDLC, though they are obfuscated through decades of software engineering professional practices, are scientific activities, requiring logical investigation, hypotheses, testing, evaluation, and reporting. Shavelson and Towne define six guiding principles for scientific inquiry [32]:

1. Pose Significant Questions That Can Be Investigated Empirically
2. Link Research to Relevant Theory

3. Use Methods That Permit Direct Investigation of the Question
4. Provide a Coherent and Explicit Chain of Reasoning
5. Replicate and Generalize Across Studies
6. Disclose Research to Encourage Professional Scrutiny and Critique

Scientific principles 2, 4, 5, and 6 would directly benefit from the use of a real-time, graph-based research process provenance capture system, because (2) research artifacts would be directly linked to their corresponding knowledge graph, where the graph contains the provenance of the artifact, (4) the chain of reasoning would explicitly be preserved in the knowledge graph, (5) replication and generalization across studies would be improved because of the generalized ontology backing the knowledge graph, and (6) research output would not simply be locked in artifacts. e.g., documents, but the process of research itself, intimately linked with the artifact, would be publishable alongside it as a knowledge graph.

Thaul discusses the difficulty in tracing personal knowledge formation for many reasons, with two outstanding: information overload and knowledge fragmentation [18]. The former is a pressing concern for any knowledge management system, as its purpose is presumably to enable users to store and use their knowledge, i.e., to help overcome information overload. Knowledge fragmentation is the complication of any research or development process, in that knowledge is scattered, or fragmented, over several physical locations: the brain, on paper, on hard drives, etc.; i.e., it is difficult to accurately aggregate and synthesize all sources of tacit and explicit process knowledge, i.e., provenance. The generation of provenance graphs for SDLC artifacts, which are queried in natural language with a LLM, addresses the issue of information overload because

LLMs are good summarizers. Knowledge fragmentation is addressed insofar as multiple provenance graphs can be integrated with a LLM for querying.

The goal of the present dissertation is provided by utilizing the goal definition template of Wohlin et al. for experimentation in software engineering [33], which builds on the earliest codified framework for applying the experimental method to software [34]. The goal of the present dissertation is as follows:

> Goal definition**:** To **analyze** the digital research process, in particular, the SDLC, **for the purpose of** evaluating and capturing relevant provenance sources **with respect to their** ability to engender transparency through explainable provenance traces, while avoiding information overload **from the point of view of the** software developers **in the context of** personal computer work.

The present dissertation intends to directly address two non-functional requirements (NFRs): traceability and explainability. See Fig. 5. The former directly contributes to reproducibility, insofar as the act of reproduction is a tracing and recreation of taken steps; the two together contribute to transparency. Interoperability is an emergent provision born of the chosen graph serialization.

Artifacts, in whatever form they are created, are insulated forms of explicit knowledge; but the processes that must occur to create these artifacts are inevitable and universal. These processes of research are common across all disciplines; they transcend the boundaries of professional societies and tenuously connect the disparate knowledge of man. Artifacts do not exist by their very nature; no, they are created through some process or set of processes, i.e., the ultimate goal and output of a research process is the production of some knowledge artifact(s). These processes

can be abstracted to a generalized process model and represented formally; and then real-world research processes (e.g., the software development life cycle) can have their provenance captured explicitly using it as a backbone structure. The present dissertation investigates the automatic capture and representation of artifact creation (or, research) processes, using the agnostic graph language of the World Wide Web Consortium as a provenance representation engine, and contemporary NLP techniques, in combination with low-level programmatic tools, to populate provenance models, such that every research artifact engendered is attended by its corresponding provenance graph. The principle use case is the software development life cycle, but any digital research process may have its provenance captured, e.g., the writing of a scientific paper. These future considerations will arise in the later sections of the present dissertation.

1.7. RESEARCH QUESTIONS

Following are the research questions that guide this work and inform the development of the proposed system.

1.7.1. R.Q. 1

How can relevant provenance sources be distinguished from noise during digital software development work?

In the process of digital work, various sources of provenance influence output artifacts. However, it is inevitable that some encountered entities constitute noise, rather than relevant provenance; such noise has no appreciable influence on artifacts labored upon. Noise may be personal in nature, e.g., emails, social media, incidental content, etc. For both the present research and future work it may inform, it is essential to establish a principled delineation between relevant provenance and noise. Such a boundary enables more precise provenance capture, clearer trace interpretation, and

more trustworthy provenance trace link knowledge graphs. Because noisy provenance capture is assumed to be inimical to proper provenance dissemination and use, this RQ is introduced first insofar as, in seeking its answer, the entirety of the work considers it.

1.7.2. R.Q. 2

How can the integration of a knowledge graph-based provenance system enhance the traceability and explainability of artifact development and maintenance within the SDLC?

It is important to evaluate what improvements can be had within the SDLC when a knowledge graph-based provenance system is used, e.g., as the biomedical industry has seen knowledge graphs with provenance information perform better in drug-disease classification than knowledge graphs without such [35]. In the 2020s landscape of large-scale generative AI, data authenticity and consent are broken; a proper, standardized data provenance has been proposed as the solution [36]. The automatic generation of provenance necessitates the querying of the generated provenance graphs and reporting of its usefulness for the purpose of explaining or tracing software progress.

1.7.3. R.Q. 3

What are the technical challenges and limitations in developing a system for the automatic, real-time serialization of SDLC provenance, and how can these challenges be mitigated?

This RQ embodies the ultimate intent of this dissertation artifact because, as is discussed throughout, such an automatic system does not exist. Much of the work in answering this question, and implementing a solution, pertains to the appropriate elucidation of the necessary components,

principally the real-time software observer of user actions based on a LLM. The system proposed here is preliminary and evolving; there will be facets not addressed due to technical constraints of the LLM, privacy and security concerns, and current hardware limitations beyond the scope of the present dissertation. This RQ establishes the forward-looking feasibility of continuing development of the proposed system for the real-world and application to any digital research project.

1.8. Addressing Generative AI

Given the current zeitgeist of large-scale generative artificial intelligence (AI) (mid-2020s) being used in ever-increasing capacities in every conceivable field, it is pertinent to address the issue of such constructs being leveraged for academic artifacts, e.g., this very dissertation. Generative AI has had no contributory action to the present dissertation artifact; it is written entirely by the human author. While generative AI, e.g., LLMs, are a core facet of the system developed under this dissertation, generative AI has had no direct input to the present dissertation artifact. As witness, the author's prior history of publication in refereed outlets, many of which are IEEE affiliated, lends credence to this assertion.

It may be possible to augment the present dissertation with a PDF-embedded signature, which relies on the seminal public/private key cryptography method [37]; but this is a method for verifying sender identity and document non-modification, not of ensuring a lack of AI use. The use of a distributed ledger, such as a blockchain is much the same [38]. The use of watermarking or steganography, the practice of embedding hidden information in the document [39], could be utilized to verify document integrity, but, again, such an approach does not ensure a reader that AI was not used. For instance, a simple watermark may be to replace a certain set of words with a

certain set of synonyms, without altering the semantics of the text; but this alters the overall word distribution of the corpus, allowing adversarial detection; moreover, a re-training (fine-tuning) of a LLM can account for such a modification.

Instead, what is proposed is an open invitation for any interested reader to take the content of the present dissertation and parse it through any desired system which purports to detect AI-generated text. There are various methods of detecting AI-generated text which are either white-box or black-box [40]. It should be noted that such methods experience difficulty in real-world scenarios, as they are easily fooled by techniques such as spoofing [41]. As witness, one may consider OpenAI's AI Classifier, purporting to detect AI-generated text, that was subsequently shut down due to its low accuracy in a matter of months[2].

As stated earlier, the interested reader may procure the prior publications of the author, which predate the current explosion of large-scale generative AI, to verify his personal ownership of the present dissertation and its entire process of creation.

1.8.1. JUSTIFICATION

There is arising in software-related discourse a tendency for researchers, developers, and project managers to appeal to the general knowledge and accuracy of LLMs as the "catchall" software component, insofar as the need for intelligent software development and coherent knowledge representation is no longer seen as the solution to digital problems. For large enterprise systems comprising hundreds or thousands of gigabytes of data and information, where such artifacts are scattered across disparate filesystems, databases, document stores, repositories, etc., it is more

[2] The webpage for this is no longer available, and even Internet Archive copies of it are missing: https://platform.openai.com/ai-text-classifier

common, now, to witness a stakeholder say something to the effect of, “Just vectorize it all into a LLM”; where, before this current age, an effort of software development would take place, wherein a system purpose-built would be rigorously evinced, designed, implemented, tested, and maintained.

For the present dissertation, and its idea of automatic provenance capture for SDLC work, there comes the question: “Why not just ask a LLM to do it all?”. Such a question presupposes the correctness of some LLM, or the possibility of using it correctly, in the context of the idea of the present dissertation. This dependence on an outside entity for the providence of a dissertation artifact and its accompanying system is the antithesis of the dissertation’s purpose. The dissertation is a personal effort in the systematic structuring of knowledge for the purpose of advancing research in one’s field of expertise. Considering this, it would be entirely trivial, and certainly duplicitous, to labor upon a dissertation artifact, writing a body of text, replete with citations, that effectively demonstrates nothing new, inspired or human.

Furthermore, in the grand context of software development history, LLMs are relatively novel constructs; and as such, they cannot be trusted implicitly or without caution. Authors applying LLMs to SDLC task automation warned in 2023 that such constructs should be approached cautiously, as the depth of their capabilities are not fully understood, and, therefore, they still require human intervention where automation is the goal [42]. There is voluminous literature on LLM hallucination, or the phenomenon that sees some LLM generate text that looks plausible, but is entirely misleading and/or false [43]. Facebook shut down their “scientific” LLM, Galactica, within a week of release because of its utterly dismal outputs, some of which encouraged users to self-harm, all while maintaining an academic prose [44]. Some have referred to LLMs as

"stochastic parrots", indicating their uncertain guesses at best output [45]. Certain fields, such as scientific research, are unlikely to ever fully integrate with LLMs, as indicated by past studies of trust in science, particularly with respect to the process of peer-review [46].

LLMs have obtained their capabilities because of massive quantities of human-generated artifacts and human feedback, so it is imperative for humans to keep generating semantically meaningful content, otherwise future LLMs may have their training tainted by masses of AI-generated content, a phenomenon referred to as data poisoning, data contamination [47] or model collapse. Some assertions about LLMs influencing the present dissertation are given below:

1. A LLM is not a silver bullet: LLMs are generally accurate on most subjects, but they lack updated understanding, as their embeddings are typically outdated by weeks, months or years, unless re-trained, so, when prompted on topics outside of their embedded knowledge, LLMs suffer in reliability [48]; therefore, the integration of an LLM with structured knowledge, e.g., knowledge graphs, augments LLM understanding positively
2. Knowledge graphs are cogent: there is a World Wide Web full of unstructured knowledge, or any number of unstructured documents, that an LLM can be augmented with to make its output more reliable; knowledge graphs provide a common, cogent, semi-formal means of encoding pertinent information for a LLM to utilize more effectively and efficiently
3. LLMs need knowledge injection: LLMs operate by being prompted with information relevant to a desired output; additionally, LLMs are shown to provide better output when augmented with external knowledge [48]

The ultimate provision of the present dissertation, its system, is effectively a reconciliation of large-scale generative AI with structured knowledge, placing it within a vein of research known

as hybrid, or augmented, intelligence [49]. The system proffered is an engineered, NFR-driven software application with general purpose use, grounded in the use case of SDLC traceability. In sociological terms, the proliferation of LLMs has allowed for "combinatorial breakthrough" [50], resulting in the system of the present dissertation. The next chapter provides background information necessary for a complete understanding of the topics discussed in the later chapters.

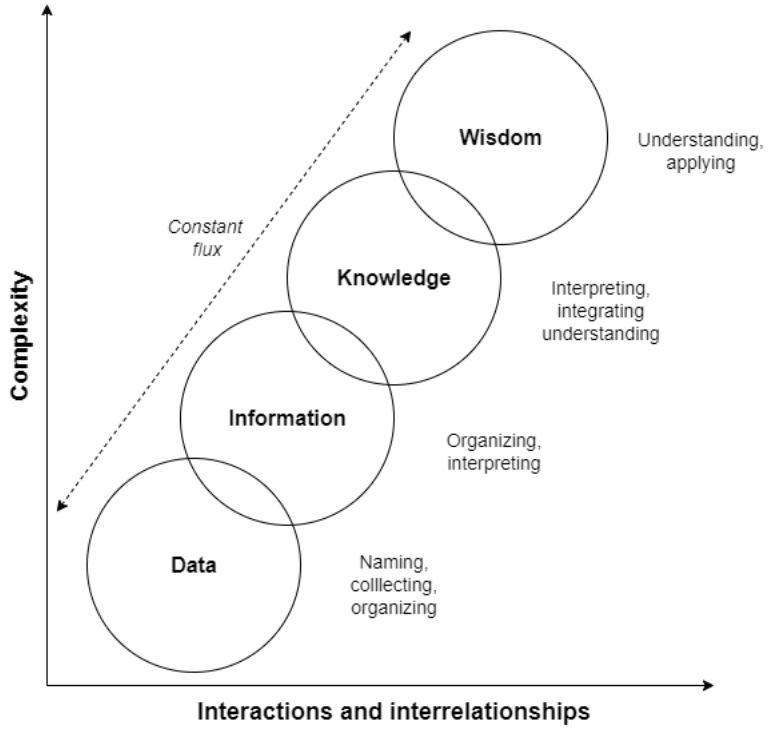

Fig. 6 The Data-Information-Knowledge-Wisdom continuum (approximated recreation of Nelson and Joos' graphic, version 2).

Chapter 2: BACKGROUND

This section delineates necessary background information about the software development life cycle, knowledge representation, the Semantic Web, provenance models and large language models. In general, the present dissertation subscribes to what may be considered a primary part of the Socratic Method, that is, for all discourse to begin first with the providence of definitions of the things being discussed [51]; therefore, the sections herein typically begin with definitions.

2.1. DATA, INFORMATION, KNOWLEDGE CONTINUUM

To establish a proper basis of diction for the present dissertation, the terms data, information, and knowledge must be distinguished and defined. Data, information, and knowledge were delineated

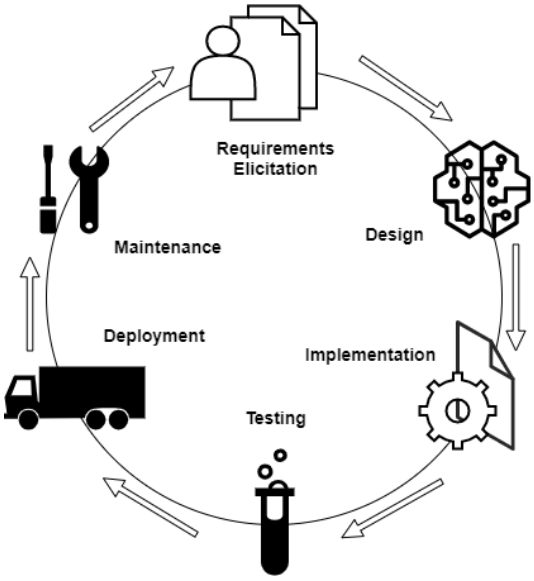

Fig. 7 The general software development life cycle.

from one another in the 1980s with a conceptual model by Grave and Corcoran for the field of nursing informatics [52]. The terms are defined in this context as:

- Data: discrete entities described objectively without interpretation
- Information: data that are interpreted
- Knowledge: information synthesized to identify and formalize interrelationships

Data may also be defined as raw facts, noise or stimuli; information as processed (meaningful) facts; and knowledge as groups of related information assimilated by an agent. Surveyed from academics operating in various fields of interest, 130 definitions of data, information, and knowledge have been enumerated and dissected [53]. Nelson and Joos augmented the aforementioned model by adding wisdom after knowledge, resulting in the Data, Information, Knowledge, Wisdom (DIKW) continuum model. Wisdom is the ability to apply knowledge to human problems [54]. There are three versions of the DIKW model embodied in images; a simple recreation of its general theme is given in Fig. 6. Flux was introduced in version 2 to represent the

notion of continual interaction, overlap, and transition to and from each of the four conceptual states [52].

2.2. THE SOFTWARE DEVELOPMENT LIFE CYCLE

The SDLC has been a subject of discussion since at least the early 1950s [2, 3]. As an abstract concept, it intends to subsume all the activities software practitioners, clients, and stakeholders partake in, in whatever measure, from product inception to product disposal. See Fig. 7. Since the inception of the software development profession, various models of the SDLC have been proposed that purport to provide some repeatable technique to the SDLC. The most common historical models are Waterfall, Spiral, and V-Model. Boehm wrote about the Waterfall model as early as 1983 in his paper, "*Seven Basic Principles of Software Engineering*" [55].

These earlier SDLC models had a heavy emphasis on up-front deliberation and consistent documentation, which resulted in software projects often taking longer than estimated because of an inability to react dynamically to changing customer demands. So, several lightweight methods emerged in the late 1980s-1990s, e.g., eXtreme programming (XP) and SCRUM, which sought to elevate customer needs above rigid documentation practices [56, 57]. These practices were unified under the Agile Manifesto in 2001 [58].

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Although Agile methods have dominated software development work since this time, notably due to their provision of early prototypes to customers, there is contention among software

practitioners that transitioning to Agile development, especially on large, complex engineering projects, can be a difficult task [59]. It is possible that this reservation may be due to the lack of recorded documentation, or provenance, in large Agile development projects, which directly hinders developer understanding. The present proposal intends to address this problem of provenance, allowing the typical Agile or Waterfall-esque SDLC to maintain seamless project provenance.

2.2.1. SOFTWARE REQUIREMENTS

The first phase of the SDLC pertains to requirements, particularly the elicitation, elucidation, and recording of them. Requirements are garnered through customer interviews and myriad activities, e.g., use case scenarios. Software Requirements Specification (SRS) documents aggregate these requirements for developers to then implement [60].Barry Boehm provided an overview of the software engineering discipline in the late 1970s; in his paper, he provides insight into the state-of-the-art tools and techniques used in the software engineering life cycle [61]. He asserts software requirement specification artifacts as belonging to one of three types: *informal*, *formatted* or *formal*. Formatted specifications are defined as: "… specifications expressed in a standardized syntax, which provides a framework for organizing the specifications and performing basic consistency and completeness checks. They generally require a moderate level of training to read and write well... Their formatted nature precludes certain sources of ambiguity, but their imprecise semantics implies that other sources of error are still present…" [61]. The SRS documents created commonly for software products are formatted specification artifacts. SRSs are, then, a serialized form of SDLC provenance, in that they directly influence the development of other software

artifacts, e.g., source code. SRSs are themselves informed by other sources, e.g., discussions with customers, which are tacit sources of provenance.

For open-source systems, which are traditionally maintained in an entirely decentralized manner through the Web, it is often the case that software requirements are not rigorously specified, e.g., within an SRS; they are, rather, elicited and recorded through a variety of identified constructs, referred to as software informalisms, or descriptive schemes based on semi-structured narratives [62]. There are eight identified software informalisms used by open-source software practitioners in managing requirements:

1. Community communications: e.g., discussion forums, email lists, network news groups and instant messaging
2. Scenarios of usage as linked Web pages: narrative descriptions of how a system operates
3. "How To" guides: descriptions of how to perform some function of a system
4. External publications: technical articles and books
5. Open software Web sites and source Webs: a community website including pages and directories, acting as a form of organizational memory
6. Software bug reports and issue tracking: dynamic information stores centering around problems with some system
7. Traditional software system documentation: documentation intended for offline or printed use
8. Software extension mechanisms and architectures: e.g., APIs and embedded scripting languages

The SRS, and the aforementioned software informalisms, are mechanisms through which software requirements, the underpinning of the SDLC, are expressed insofar as developers can implement specifications in code. These are prime, early sources of software provenance.

2.2.2. SOFTWARE DESIGN

Following the requirements phase is that of design, where the software "blueprint" is conceived. Software design is generally accepted to have two distinct sub-types: high-level and low-level. The former, also referred to as architectural design, includes an outlining of overall system structure, its modules, functions, and how they interact with one another; the latter specifies in greater detail the logical structure of the modules and functions defined in the architecture, i.e., low-level design is about algorithms, data structures, etc.

Software design artifacts are typically visualizations. Visual artifacts, as opposed to lexical artifacts, are unique, in that implicit provenance is not as obvious to infer, e.g., as some source code may carry a comment from the developer, a design artifact may just be an image.

2.2.3. SOFTWARE IMPLEMENTATION

The phase of the SDLC where most of the appreciable work occurs is that of implementation, wherein developers write code and produce releasable software. There is a litany of provenance sources during this phase.

An obvious source of recorded provenance in the software implementation phase can be witnessed within version control systems (VCSs), e.g., Git. Developers can track issues, tag other developers, push code with commit messages, and so on. One less obvious source of provenance, often informal in practice, is found in code comments left by developers: notes for use, information for future developers, rationale in implementation, etc. Then, there are extraneous sources of

provenance that may indirectly influence source code development, e.g., problem solutions taken from online forums, etc. These sources of provenance are of interest to the present proposal, as well as others that will invariably be discovered during the research process.

2.2.4. OTHER PHASES

The remaining phases of the SDLC, to wit, testing, deployment, and maintenance, are of lesser interest for the scope of the present proposal, inasmuch as these phases do not witness the abundance of created artifacts as are present in the other three phases (requirements, design, and implementation). Notwithstanding, there are some sub-processes involved in these phases that are provenance oriented in nature and laden with artifacts. For instance, the decision to deploy an iteration of a software system is very dependent on stakeholder requirements and proper testing, if the release is to be positively accepted. Much of software maintenance is dependent on provenance, e.g., in a scenario where the software tester finds a bug, which the developer must fix, the provenance trail includes the means by which the bug was found, how it was reported, who it was assigned to, etc.

2.3. NON-FUNCTIONAL REQUIREMENTS

In software development, a non-functional requirement (NFR) is a specification that defines a criterion that can be used to judge the operation of a system, rather than its function. NFRs typically pertain to system qualities, e.g., usability, reliability, performance, etc. However, quantifying software quality is difficult, as embodied in the following statement from Cavano and McCall in 1978: “Software has always been viewed as an abstraction. Unlike hardware, it has no physical presence. This concept has contributed to the difficulty in assessing the quality of software” [63]. Granting this concern, there is not extant a formal definition or complete list of NFRs [64]; but,

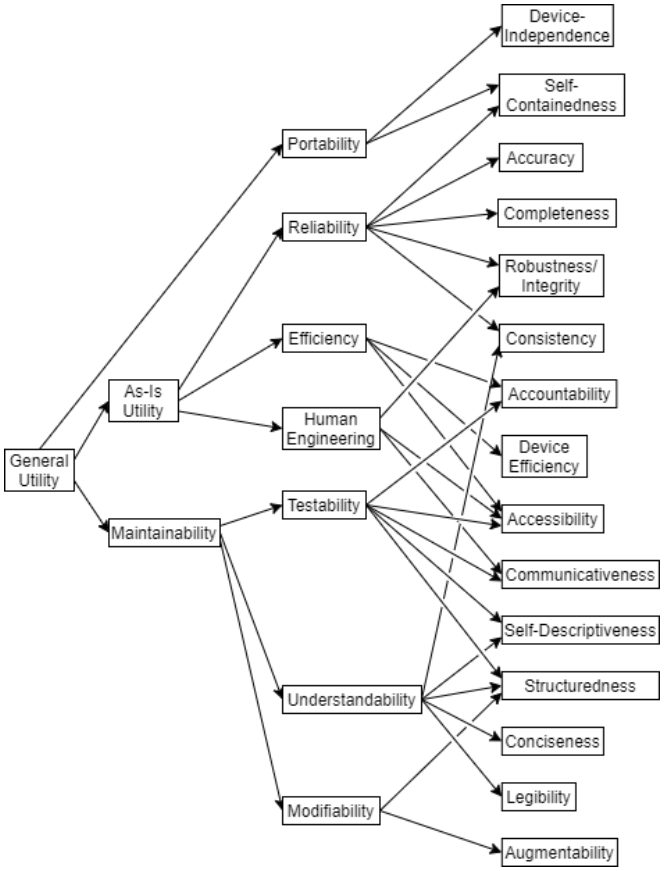

Fig. 8 The earliest software quality characteristics taxonomy, by Boehm et al. in 1976 [61].

there have been proposed various software quality models that aim to capture NFRs of importance within certain contexts.

In 1976, Boehm et al. presented the first "tree of software quality characteristics" [65]. See Fig. 8. Cavano and McCall extended Boehm's work with a software quality assurance (SQA) framework [66]. The International Organization for Standardization and International Electrotechnical Commission (ISO/IEC) holds the most broadly recognized standard for NFRs. In 1991, ISO/IEC 9126: Software Engineering – Product Quality was released. It has since been replaced in 2011 by ISO/IEC 25010:2011: Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) [67]. See Fig. 9. This model was last updated (and replaced) in 2023 [68].

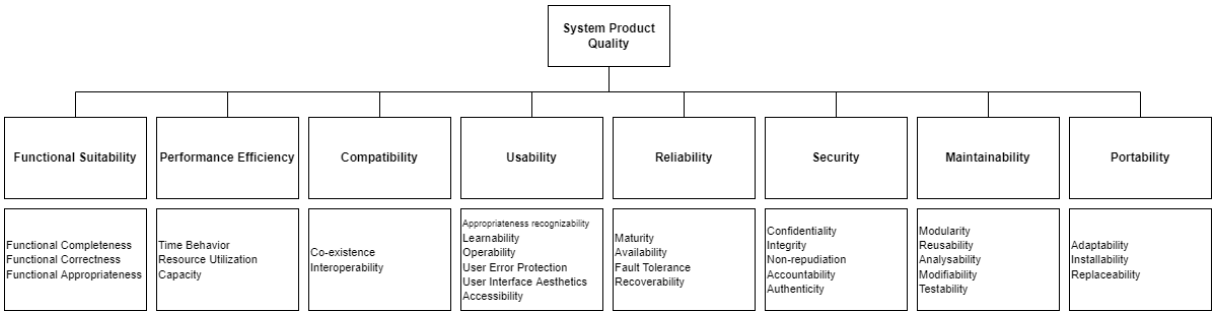

Fig. 9 The ISO/IEC 25010 NFR model [68].

As stated earlier, the NFRs of interest to the present dissertation are traceability and explainability, with reproducibility, transparency, and interoperability being emergent concerns.

2.3.1. REPRODUCIBILITY

Within the context of ESE, Madeyski and Kitchenham define reproducible research as "the extent to which the report of a specific scientific study can be reproduced (in effect, compiled) from the reported text, data and analysis procedures, and thus validated by other researchers" [69]. Proponents of reproducible research in the digital domain, beginning in the 1990s with software artifacts attendant with makefiles (programs containing batches of tasks), posit that the ideal research artifact is some paper and its computational environment, which includes data and algorithms used [69]. I.e., some software research project is only reproducible if it is explicitly attended by its provenance. A lack of included source artifacts, e.g., data or code, in addition to poor documentation, have led to the scientific reproducibility crisis, affecting various domains of interest, including artificial intelligence [70]. Although the extent of the crisis is debated, a 2016 survey of 1,576 researchers found that 52% of researchers agree a significant crisis of reproducibility does loom over the scientific research landscape [71].

In a purpose-oriented sense, reproducibility is the ease with which one may trace artifact development steps to associated early influences, and the copying of them to reproduce some

process and its output; i.e., if traceability is achieved, then reproducibility is more likely attendant. So, in the present dissertation, traceability is the NFR that is directly addressed, where reproducibility is emergent.

2.3.2. TRACEABILITY

Per the Center of Excellence for Software and Systems Traceability (CoEST), software traceability is defined as "the ability to interrelate any uniquely identifiable software engineering artifact to any other, maintain required links over time, and use the resulting network to answer questions of both the software product and its development process" [72]. ISO/IEC/IEEE standard 24765-2017 defines traceability as the "degree to which a relationship can be established between two or more products of the development process, especially products having a predecessor-successor or master-subordinate relationship to one another" and a traceable system as "having components whose origin can be determined" [73]. The Software Engineering Body of Knowledge (SWEBoK) says that developing software typically involves the "use, creation, and modification of many work products such as planning documents, process specifications, software requirements, diagrams, designs and pseudo-code, handwritten and tool-generated code, manual and automated test cases and reports, and files and data. These work products may be related through various dependency relationships (for example, uses, implements, and tests)" [74]. Spanoudakis and Zisman define software traceability as "the ability to relate artefacts created during the development of a software system to describe the system from different perspectives and levels of abstraction with each other, the stakeholders that have contributed to the creation of the artefacts, and the rationale that explains the form of the artefacts." [75].

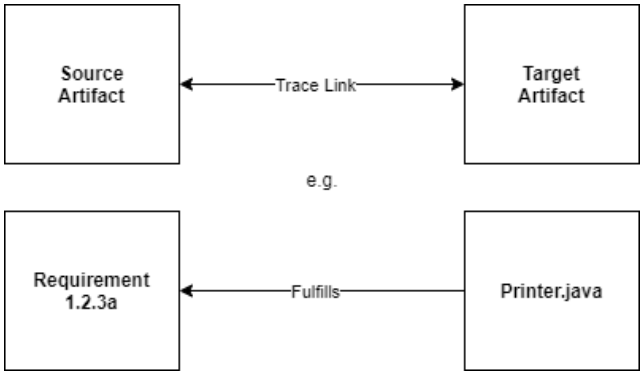

Fig. 10 A simplified example of source and target artifact trace linking.

While traceability is typically connoted with safety-critical systems, e.g., in aviation, traceability as an NFR is a desirable quality in any software system, insofar as improved traceability allows stakeholders to ensure that requirements have been satisfied and generally maintain their software more easily. Traceability also enables software practitioners to assess change impact once some software system is released, since relationships to other software work products can be more easily traversed [74].

Research into requirements traceability in the 1990s revealed much confusion among software practitioners. Along with conflicting definitions and a lack of coherence with regard to the actual purpose of traceability, Gotel and Finkelstein noted two separate problem areas in requirements traceability implementation [76]:

- Pre-requirements specification traceability: tracing from specified requirements to their elicitation sources, e.g., customer interviews; very difficult and not well understood, due to the informal and dynamic nature of the processes and environments in which elicitation activities take place

- Post-requirements specification traceability: tracing from developed software artifacts/fragments to their specified requirements; most traceability literature and techniques are particular to this task

While both traceability phases of the SDLC engender much provenance and should therefore be considered for an automated provenance capture system, post-requirements specification traceability is the principal focus of the present dissertation, as, prior to requirements being specified, much of the provenance is tacit or non-digital, e.g., as requirements engineers may interview software customers in-person, so there may be entire bodies of dialogue preceding individual requirements that are then extracted through more conversation, and only later refined into specified requirements. These provenance sources are beyond the scope of the present work, but are discussed in a later chapter on future work.

A large 2020 mapping study found that the most frequently traced software artifact was requirements, followed by source code; and that the most frequently traced pairing of software artifacts was requirements and source code [21]. Schwarz et al. posit that there are six distinct traceability-related activities [77]:

1. Definition: the determination of entities to be traced and the relations to do so
2. Recording: the storage of traces, e.g., in spreadsheets
3. Identification: the discovery of new traces using the instances from the definition activity
4. Maintenance: updating extant traces
5. Retrieval: finding relevant traces that have been identified and recorded
6. Utilization: uses previously retrieved traces in an application scenario

Most traceability processes use auxiliary artifacts, e.g., spreadsheets, to record trace links [25]. The types, or terms, of trace links are not universally agreed upon, nor is there any apparent reason for choosing a given trace term for a given application [78]. Trace links are established between traceable entities, e.g., stakeholders, requirements, code elements etc., with the general model resulting in links between source and target artifacts [77]. See Fig. 10. Graphs are, therefore, amenable to traceability work. Creating and maintaining software trace links is difficult in a social sense, as communication between stakeholders is often poor, as well as a technical sense, since physically operating on hundreds or thousands of interrelated trace links is not trivial [27].

In 2014, Cleland-Huang et al. cogently describe the difficulty in automating software trace capture. "[Prospective trace capture] sets out to capture and infer trace links from the project environment and from the actions of the project stakeholders… trace links can be inferred as a natural byproduct of software engineering activities… the challenge of instrumenting an environment by building and evolving plugins for many different versions of different tools, distributed across potentially different networks, is quite challenging… [researchers must] address systems and software engineering issues that go far beyond algorithmic solutions and reach into the technical infrastructure of the project, particularly into the user interaction layer where plugins reside, the repository layer where data are stored, and the trace engine layer which needs to implement algorithms that make sense of the captured data" [72]. The present dissertation posits that this identified difficulty is directly addressable by the generalizable capabilities of modern LLMs, which gained prevalence in the early 2020s.

2.3.3. EXPLAINABILITY

In Latin, an explanandum is an information bearing entity, such as a sentence, describing a phenomenon requiring explanation (not the phenomenon itself); an explanans is offered as explanation of such a phenomenon. Explanations are, in effect, the means by which "why" questions can be answered, as opposed to simpler "what" questions. The deductive-nomological model of Hempel and Oppenheim, striving to explain explanations, as it were, established the importance of a proper explanans for some explanandum. "It may be said… that an explanation is not fully adequate unless its explanans, if taken account of in time, could have served as a basis for predicting the phenomenon under consideration… It is this potential predictive force which gives scientific explanation its importance: only to the extent that we are able to explain empirical facts can we attain the major objective of scientific research, namely not merely to record the phenomena of our experience, but to learn from them, by basing upon them theoretical generalizations which enable us to anticipate new occurrences and to control, at least to some extent, the changes in our environment" [79]. In the philosophical view of causal realism, explanations are "descriptions of causality, expressed as chains of causes and effects" [80, 81].

Explainability, the measured quality of some set of explanations, is a subjective concern, as it varies per individual; e.g., what is easily explainable by one (or *to* one) may be confusing for another. As a term describing this positive quality for some artifact, explainability may be asserted to be directly proportional to the number of explanations purporting to accurately "answer" any concerns about the artifact [80]. Explainability in the context of software development evolved to be an important concern as AI systems became increasingly complex, with researchers in this area promulgating literature regarding explainable AI (XAI) [82]. Although the dominant context for

explainability is AI work, this NFR is a concern for many aspects of many types of software systems, as indicated in a large survey by Droste et al. [83]. Chazette and Schneider provide a definition for explainability by first considering explanations: "[E]xplanations (objective factor) are operationalizations of explainability. They can be a way of improving the understanding of a system by conveying information, thus influencing interpretability or understandability (subjective factor)" [84]. More formally, Chazette et al. define explainability in the context of software and requirements engineering thusly:

> *A system S is explainable with respect to an aspect X of S relative to an addressee A in context C if and only if there is an entity E (the explainer) who, by giving a corpus of information I (the explanation of X), enables A to understand X of S in C* [85].

The enabling of an addressee to understand an aspect of some system is not a guarantee, due to the subjective nature of explanation and individual differences in understanding. Interpretability and understandability, as qualities of a software system, are born out of explanations that some agent has individually processed, insofar as the agent recognizes the system to be explained well enough that it is, ideally, not regarded as confusing. Based on the Merriam-Webster dictionary definition, and specific to the context of machine learning, Doshi-Velez and Kim define interpretability as the "ability to explain or present in understandable terms to a human" [86]. The traceability of *what* information in *what* form of presentation depends on *who* wants it and *why* they want it [76]. As discussed in a prior section on traceability, provenance traces between digital artifacts provide more information to an interested individual than "what" the artifact is; they provide the opportunity to answer "why" such artifacts were made, and "how" such occurred. This is the essence of an explanation, which is the objective means of achieving

explainability. Explanations are, therefore, a means of achieving transparency; so, explainability is a key NFR underlying transparency.

2.3.4. TRANSPARENCY

In April 2016, the European Parliament set forth a number of regulations regarding the collection, storage and use of personal information in the General Data Protection Regulation (GDPR) [87]. Of particular interest is the "right to explanation" [88]. This body of legislation intends to protect the privacy of everyday consumers in a Web-enabled world, particularly from data mining and an ever-expanding retinue of online AI systems. The underpinning of the GDPR is that organizations must obtain explicit consent from individuals before collecting or processing their data; and this must be made transparent to individual users.

Lipton posits that transparency is, informally, the opposite of opacity, or that quality of systems which renders them conceptually as black-boxes [89]. Leite and Cappelli define transparency by decomposing the concept into 33 softgoals (NFRs) within a softgoal interdependence graph (SIG) [90]. Their SIG is a three-level hierarchy, with transparency itself taking the terminal parent position. The second level is composed of five NFRs: accessibility, usability, informativeness, understandability and auditability, with each of these five mid-level NFRs being related to three or more sub-NFRs. Each lower level NFR in this transparency SIG directly influences the fulfillment of its parent to some degree.

It is posited that, in generally addressing traceability and explainability, the concerns of reproducibility and transparency are fulfilled by degrees. In other words: transparency is not directly addressable, but is addressed implicitly in the fulfillment of other NFRs – in the present

dissertation, these are traceability (and, by extension, reproducibility) and explainability. By implementing explainable traces, transparency is emergent.

This brief discourse on software transparency is concluded by remarking that the transparency SIG identified by Leite and Cappelli is conceptually similar to the Findable, Accessible, Interoperable, Reusable (FAIR) principles for data stewardship [91], insofar as findable is roughly synonymous with a third-level SIG term, availability, accessible is equivalent to the second-level SIG term accessibility, and reusability is roughly synonymous with the SIG term reusability and its sub-NFRs. Interoperability, the "I" in FAIR, is a concern requiring separate treatment.

2.3.5. INTEROPERABILITY

In contexts outside of software development, e.g., the defense sector, interoperability is defined with respect to the ability of physical systems and units to coordinate with one another [92]. For the domain of software development, interoperability is defined as "the ability of two or more software components to cooperate despite differences in language, interface, and execution platform" [93]. Interoperability assessments tend to be qualitative, specifically, ordinal, in nature, with very few purely quantitative measurement frameworks extant [92, 94].

In the present dissertation, interoperability is a NFR not directly addressed, but engendered minimally through the use of the standardized Resource Description Framework (RDF) language and two standardized, ubiquitous ontologies, the Provenance Ontology (PROV-O) of the World Wide Web Consortium and the Basic Formal Ontology (BFO) of the Open Biomedical Ontologies Foundry. The utilization of these constructs enables basic interoperability that is not quantified through any metric, but may ease integration with other systems that are based on the same

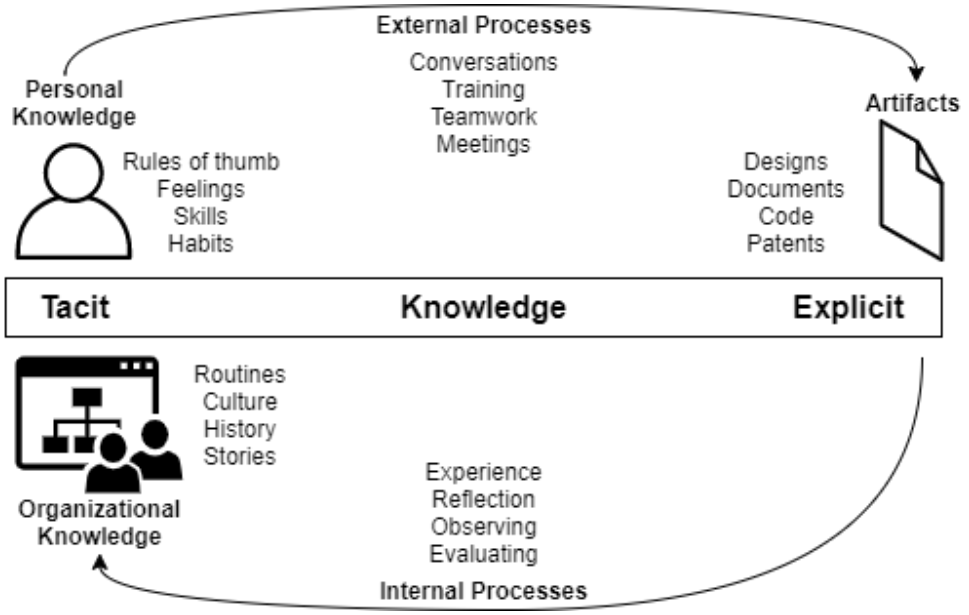

Fig. 11 The dynamic tacit-explicit knowledge continuum [100].

constructs. This is an aspect of data, information or object model interoperability, which is addressed by many of the seminal interoperability measures [92].

2.4. INFORMATION OVERLOAD

A situation in which so much relevant, and *potentially* relevant, information is available to an agent, insofar as it is a hindrance rather than a help, is an instance of information overload [95]. Information overload is an issue in business organizations, where it has been cited as negatively affecting worker job performance and interpersonal relationship quality [96]. “Information has always been a source of power, but it is now increasingly a source of confusion. In every sphere of modern life, the chronic condition is a surfeit of information, poorly integrated or lost somewhere in the system” [97].

Filtering is perhaps the most common technique employed in addressing information overload. Filtering within some information system operates by defining some criteria for inclusion, and therefore exclusion, then applying the criteria algorithmically to incoming data

sources. Proper information architecture, e.g., with taxonomies and appropriate user experience (UX) interfaces which aim to minimize the choices and information a user is exposed to, are means of addressing information overload [95].

Irrelevant information within provenance systems, i.e., noise, is a noted issue [18]. Within the present dissertation, information overload is a central concern, given the real-time generation of provenance traces while software developers work on their computers. This is intrinsically addressed, in part, due to the digestive, summarizing nature of LLM output.

2.5. KNOWLEDGE REPRESENTATION AND ONTOLOGY

Knowledge representation (KR) is best defined by breaking apart its latter word: *re-presentation*. KR is the re-presentation of facts, in whatever form such a representation may take, whether it be an image, a book, a speech, a code repository or a first-order logic system. Bergman (founder of the KBpedia project) defines KR as "a field of artificial intelligence dedicated to representing information about the world in a form that a computer system can utilize to solve complex tasks" [98]. In contemporary research, KR is the longarm of the philosophical field of Epistemology, which concerns itself with knowledge, and how men acquire it [99].

There exist (barring philosophical debates and nominalism) two types of knowledge: implicit (or tacit) and explicit. The former is stored in human minds, being experience, or know-how; the latter is serialized, or *represented*, through some medium for re-use [100]. See Fig. 11. Both types of knowledge can inform the other through the various processes an individual is involved in with the development of a system, such as a software system produced during the SDLC.

The matter of *how* knowledge is represented is the purview of the philosophical field of Ontology; from Greek: Οντολογία, or, *the study of being* [101]. As a discipline, Ontology centers around the concepts of existence, the structure of reality, and the classification and categorization of all entities [102, 101]. Exercise in Ontology results in the creation of *an* ontology [102]. An ontology can be defined in a number of ways (see Gruber [103] or Borst [104]), but the definition of Arp, Smith, and Spear is most appropriate: "a representational artifact, comprising a taxonomy as proper part, whose representations are intended to designate some combination of universals, defined classes, and certain relations between them" [17].

A key part of this definition is the constraint that an ontology contains a taxonomy, or hierarchy, as proper part. Taxonomy is the "orthodoxy of ontology", in that most ontological undertakings are predicated on the taxonomy, e.g., as XML, HTML, and JSON, are all hierarchical by design, and make up a great portion of the World Wide Web [105]. Furthermore, the taxonomy, or the entity subsumption hierarchy, is the central construct for graph-based representations of data.

2.5.1. ONTOLOGY STRATA

Guarino defines three types of ontologies: top-level ontologies (TLOs), domain and task ontologies, and application ontologies [106]. See Fig. 12. These are defined below:

1. Top-level ontologies: contain general concepts like space, time, matter, object, event, action, etc., which are not particular to any domain
2. Domain and task ontologies: describe the vocabulary of some domain, e.g., software engineering, or a generic task/activity, e.g., requirements elicitation, by specializing terms from top-level ontologies

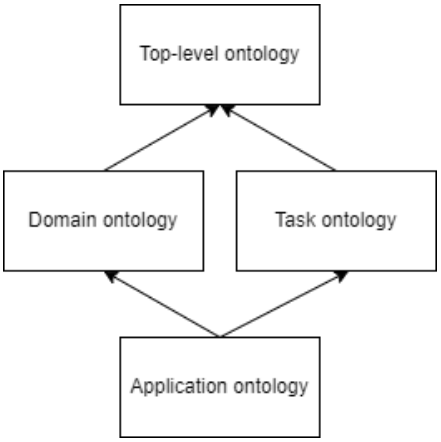

Fig. 12 The three ontology types, defined by Guarino [106].

3. Application ontologies: describe concepts specializing both a domain and task, e.g., use case diagrams

The research landscape for TLOs is broad and diverse: the many philosophical orientations of various researchers have resulted in a wide range of implemented TLOs. A 2020 survey presents consideration on over two dozen TLOs, including BFO, COSMO, DOLCE, FrameNet, GFO, KKO, SUMO, UFO, and many others [107]. In practice, the use of all three types of ontologies is common for the representation of some domain of interest. In the context of the present dissertation, the top-level ontology of interest is the Basic Formal Ontology.

2.5.2. BASIC FORMAL ONTOLOGY (BFO)

As discussed previously, there are any number of ways to represent some knowledge of interest, but the taxonomy is commonly employed. The taxonomy took its first recognized form with Aristotle. The Basic Formal Ontology (BFO), the core ontology of the Open Biomedical Ontologies Foundry (OBO), adheres to the Aristotelian notions that (1) all entities in reality are classified within a dichotomic tree, starting with continuant and occurrent, and (2) all entities in reality are children (or sub-classes) of exactly one other entity, i.e., BFO models knowledge in a

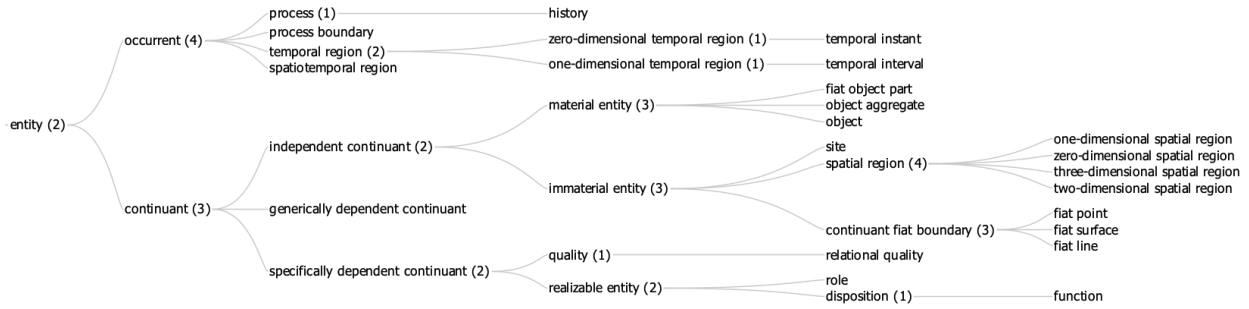

Fig. 13 The BFO mono-hierarchy.

tree of single inheritance, or a monohierarchy, in which every class can be traced up to entity by one path.

BFO immediately bifurcates reality into two distinct types of entities, *Continuant* and *Occurrent*. See Fig. 13. It is this distinction that, in the first place, planted the seed of the idea that the present dissertation is written around. Moreover, BFO is publicly available, open for discussion, consistently used in various domains (most notably, the biomedical domain), and well-documented, having extremely precise natural language definitions for every facet of the ontology. These definitions are of the Aristotelean form:

$$S = a\ G\ that\ Ds$$

Where S is the species being defined, G is the genus (or parent of S) and D is the differentia, or what distinguishes S from G (while still remaining a G). Definitions as such are *unpackable*, or reducible, up to the most general thing (in BFO, this is the term “Entity”), which can only be defined by way of use cases and examples.

The Common Core Ontologies (CCO)[3], an extension of BFO, arose out of a collaborative effort between the National Institute of Standards and Technology (NIST) and CUBRC, a research and development firm. While BFO is purely a top-level ontology, the CCO exist as domain and task ontologies, extending BFO at the mid-level with a wide array of entities and relations applicable across all domains of interest [108].

There are various philosophical interpretations of reality, resulting in many top-level ontologies (TLOs), e.g., the Unified Foundational Ontology (UFO) [109], the KBPedia Knowledge Ontology (KKO) [107] and BFO itself. Notably, BFO possesses the distinct qualification as the first and only ISO/IEC standardized top-level ontology, as of 2021 [110]. BFO is therefore considered in the present proposal as the most interoperable, forward-looking means of representing SDLC provenance.

2.6. EARLY PROVENANCE IDEAS AND THE INTERNET

In 1989, Tim Berners-Lee (now Sir Timothy Berners-Lee), member of the European Organization for Nuclear Research (CERN), proposed a system in his paper, "*Information Management: A Proposal*" [111]. In an interesting historical fragment, Tim's advisor wrote the comment "*Vague but exciting…"* above the proposal's abstract. This paper is often (erroneously) credited as the start of the modern Internet as it is known today. As Tim has stated himself, there is an important distinction between the *Internet* and the *Web*: *"I was lucky enough to invent the Web at the time when the Internet already existed - and had for a decade and a half. If you are looking for fathers*

[3] See the CCO official GitHub: *github.com/CommonCoreOntology/CommonCoreOntologies*

of the Internet, try Vint Cerf and Bob Kahn who defined the "Internet Protocol" (IP) by which packets are sent on from one computer to another until they reach their destination" [112].

The reference to Vinton Cerf and Robert Kahn acknowledges their role in the development of the Department of Defense's (DOD) Advanced Research Project Agency Network (ARPANET). In 1974, Vinton and Robert outlined a new protocol for network communication using packets, called the Transfer Control Protocol (TCP), which is one of the main protocols of the modern internet suite [113]. This public protocol came out of the government funded ARPANET project in the late 1960s. In 1966[4], Robert William Taylor, an untrained computer networking pioneer employed as director of ARPA's Information Processing Techniques Office (IPTO), established the ARPANET project [114]. This was the formal beginning of the project that spawned the Internet.

Yet, the history of the Internet can be traced back farther still. In April of 1963, Joseph Carl Robnett Licklider (a 1937 triple major in physics, mathematics and psychology) sent several memorandums to his colleagues while employed as ARPA's IPTO director, designating the memorandum for the "Members and Affiliates of the Intergalactic Computer Network". This "Intergalactic Computer Network", as Licklider delineated in his series of nascent notes, quite literally defined the Internet as it is known today, including complex concepts like cloud computing [115]. Robert Taylor said the following about Licklider: "… most of the significant

[4] The starting year for ARPANET is variously contested. See the RFC Series (ISSN 2070-1721), started in 1969, for an in-depth history of the Internet: *rfc-editor.org*

advances in computer technology… were simply extrapolations of Lick's vision. They were not really new visions of their own. So he was really the father of it all" [116].

Even considering J. C. R. Licklider, the history of the Internet can be found to go back even earlier. In the 1960s, SAFARI, an online text-processing system, came about to help information analysts construct personalized files for research. SAFARI was intended to assist researchers with the problem of information overload. "*SAFARI is… designed to be used by a group of information analysts for building their own personal files. The system allows an analyst to scan through documents… to select relevant facts which are then processed linguistically and stored in the computer in the form of logical content representations. To locate information, the analyst can initiate a search through the stored representation... When an item of interest is found, the original text from which it was derived can be recovered…*" [117]. In 1945, Vannevar Bush proposed the Memory Expansion (memex) system, a hypothetical electromechanical device for interfacing with microform (shrunken reproductions of text files, images, etc.) documents, as an aid to the individual and as a means of storing and consulting a record of the human species [118]. Bush also discussed the formation of *associative trails*, or chains of linked microfilm frames, including personal comments, both of which are key facets of contemporary provenance systems.

It is even possible to consider the earliest history of provenance systems in encyclopedias, where one of the first real printed attempts came about in 1751, when Denis Diderot and Jean le Rond d'Almbert published the *Encyclopedia, or a Systematic Dictionary of the Sciences, Arts and Crafts*, better known as *Encyclopédie*. Before this, there were several written encyclopedias, from the 11th century in China, 7th century in India, the 5th century in Christian countries and the 1st century in Rome.

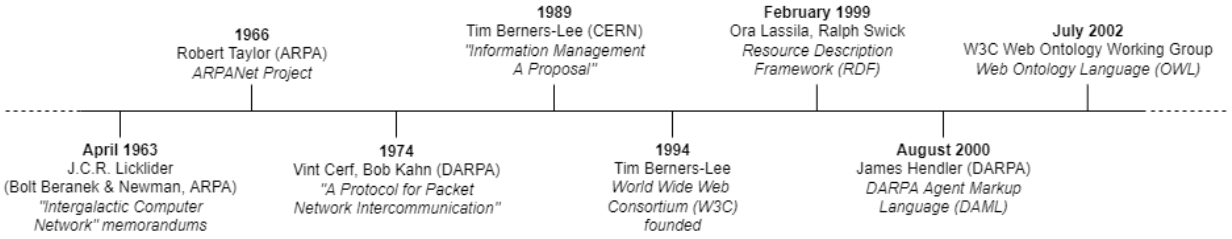

Fig. 14 A brief timeline of the Internet, Web and Semantic Web.

The principal development that allows for massive expansion of distributed knowledge is the hypertext system, where parts of artifacts have internal structure and are linkable to one another. The Internet, as a term, encompasses the technical aspects of distributed network communication. The Web is data connected via hypertext documents and URLs. The Semantic Web, "*... in naming every concept simply by a URI, lets anyone express new concepts that they invent with minimal effort. Its unifying logical language will enable these concepts to be progressively linked into a universal Web. This structure will open up the knowledge and workings of humankind to meaningful analysis by software agents, providing a new class of tools by which we can live, work and learn together*" [119].

Similarly, Taylor and Licklider have this to say in a 1968 paper: "*... we are entering a technological age in which we will be able to interact with the richness of living information – not merely in the passive way that we have become accustomed to using books and libraries, but as active participants in an ongoing process, bringing something to it through our interaction with it, and not simply receiving something from it by our connection to it*" [120].

The management of knowledge through provenance capture has been a subject of research and effort since man has recorded knowledge in written books. With the Internet, World Wide Web and, as will be discussed later, large-scale generative AI, it is possible to effectually automate

the capture of digital provenance, providing digital workers with real-time knowledge management and a means of addressing information overload.

2.7. LINKED DATA AND THE SEMANTIC WEB

In 1999, James Hendler of the United States Defense Advanced Research Projects Agency (DARPA) began the DARPA Agent Markup Language (DAML) program, which intended to provide machine-readable data for the Web, in addition to facilitating the development of Tim Berners-Lee's vision of the Semantic Web [121]. DAML was first introduced by Hendler and McGuinness in 2000 [122]. In 2002, the Ontology Inference Layer (OIL) was presented, and the DAML + OIL ontology language for the Semantic Web was born [123]. See Fig. 14.

The Semantic Web is commonly portrayed as a stack, or "layer cake", of technologies. This stems from a diagram created by Tim Berners-Lee in 2006 [124, 125]. See Fig. 15. The Resource Description Framework (RDF) was first introduced by Lassila in 1998 [126]. It became a W3C recommendation in 1999 [127]. RDF allows the representation of knowledge as a graph, through its node-edge paradigm of triples, of the form subject-predicate-object. Building upon RDF, RDF Schema (RDFS) allows for the creation of class hierarchies [128].

Tim-Berners Lee now heads a startup company, Inrupt, backed by the Solid project, that intends to enable personal data governance. The Solid project states that the Semantic Web and Linked Data never took off, but the project uses some of their constructs because they enable interoperability and reuse [129]. In any case, is common in provenance work, because graph data is intuitive and extensible, making it ideal for capturing provenance, which is an evolving process that does not conform well in a traditional data structure, e.g., the relational database, or the table.

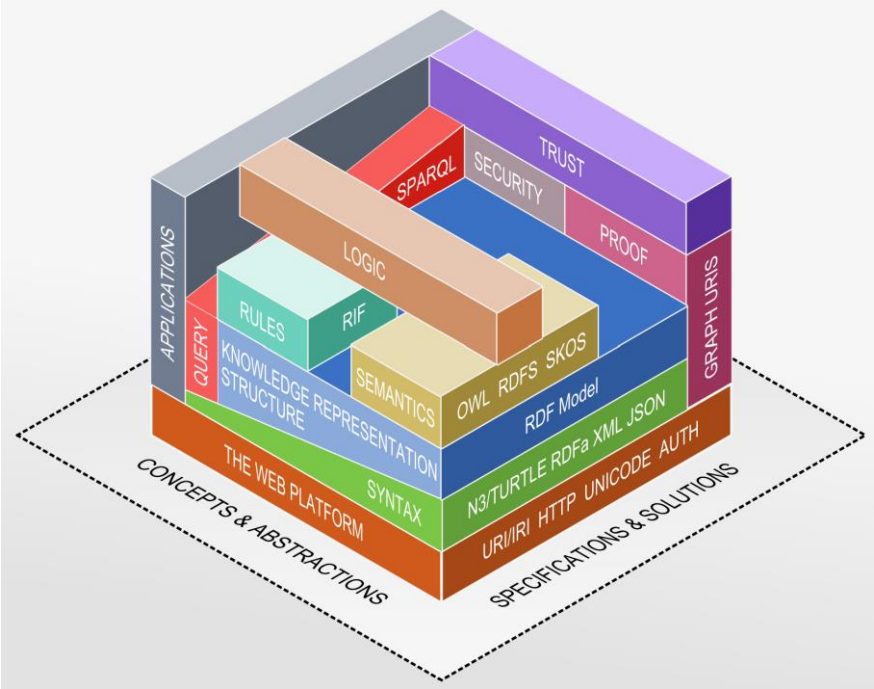

Fig. 15 The Semantic Web technology stack, by Gängler [125].

Linked Data is a specific practice intended to manifest the Semantic Web vision; it is a set of best practices for publishing data on the Web, facilitating discoverability, reuse, etc. Linked Data is defined in four principles, all of which are addressed with proper knowledge graph formation [130]:

1. Use URIs as names for things
2. Use HTTP URIs so that people can look up those names
3. When someone looks up a URI, provide useful information, using the standards (RDF*, SPARQL)
4. Include links to other URIs so that they can discover more things

The only surviving fragment of the Semantic Web movement still employed at any appreciable scale is Resource Description Framework in attributes (RDFa). RDFa is a W3C specification for embedding structured data within XML-based markup languages, e.g., HTML,

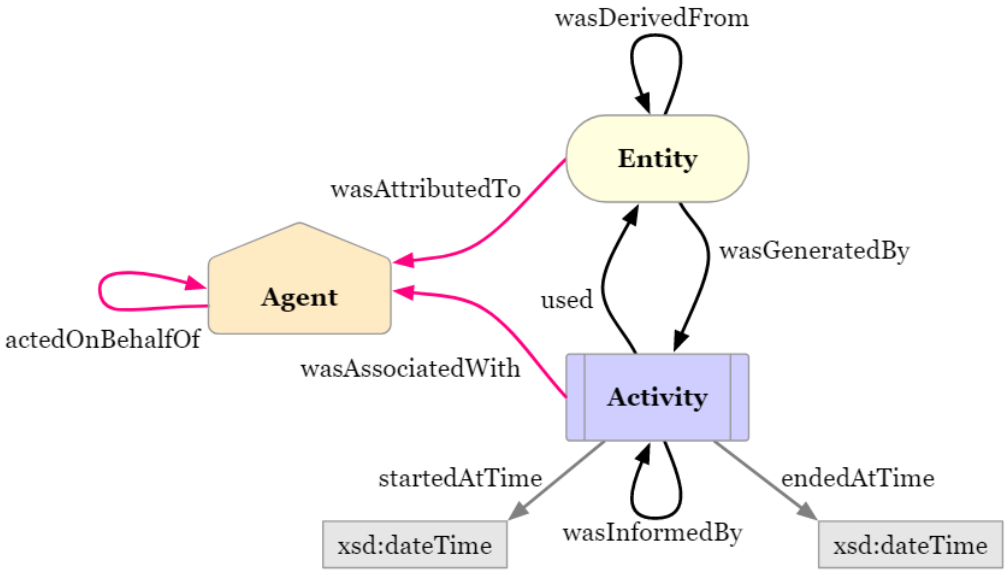

Fig. 16 The three Starting Point classes of PROV-O and their properties [136].

essentially allowing for the proliferation of distributed, tacit knowledge graphs without the rigor required by RDF/RDFS [131].

2.7.1. KNOWLEDGE GRAPHS

Instance knowledge adhering to an ontological basis can form a Knowledge Graph (KG) [132]. KGs are composed of nodes, or entities and concepts, and the links between them, which are their relations. Knowledge graphs are intuitive to humans, borrowing from the longstanding tradition of natural language [98]. Natural language expresses facts in the form of *subject-predicate-object* statements. This well-established syntactical linguistic component was adopted by the W3C, in their provision of RDF, which allows the creation of triples [127].

There are numerous means of storing represented knowledge, e.g., tables and databases, but none are so flexible as knowledge graphs. KGs allow for seamless expansion, deletion, modification, and transmutation in the face of new and emerging data. For complex systems undergoing rapid change, e.g., as the typical software development project is, knowledge graphs

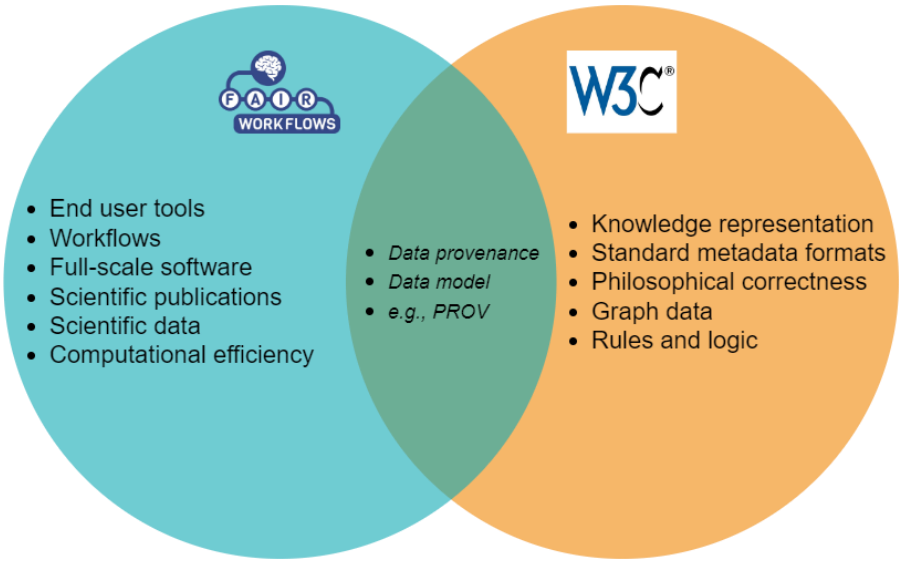

Fig. 17 Elements of FAIR and the Semantic Web [142].

are an ideal representational form for provenance information. The W3C standard suite for knowledge graph representation is in RDF/RDFS and OWL, but other forms exist, e.g., the labelled property graph (LPG), used by graph platforms like Neo4j [133]. The benefit of LPGs over RDF-based graphs is that LPGs allow predicates to have properties attached to them, where this is not possible in RDF; also, LPGs are optimized for transactional performance, so they can be appreciably faster than RDF-based graphs when queried. However, the LPG lacks the standardization and longstanding use that RDF/RDFS and OWL have, so this construct is not ideal as a futureproof form of KR. In addition, as RDFa is prevalent in much of the Web as a W3C standard for rich HTML annotation, LLMs trained with online information have ingested massive quantities of RDF triples. Granting this, the RDF suite for KG formation is the basis of KR and LLM interaction in the present dissertation.

2.8. PROVENANCE MODELS

There are extant a number of codified models of provenance or metadata which have been used across various domains. One such model originates from a W3C working group on provenance.

2.8.1. PROV-O

The Open Provenance Model (OPM) was proposed as early as 2007 [134]. Its 2011 specification defines the OPM as an annotated causality graph, with its core nodes and edges, several examples, constraint rules, considerations on the temporal nature of provenance and its implementations (e.g., in XML and RDF/OWL) [11]. The OPM is now a W3C recommendation and has its own working group, under the shortened name PROV, for provenance [135]. An OWL serialization of PROV exists, named PROV-O [136]. It is backed by the PROV data model, PROV-DM [137]. PROV-O finds use in many areas as a metadata recording ontology, as its terms are general, understandable and amenable to LD workers for linking resources. PROV rests upon the use of three core entities: Agent, Entity, and Activity (as opposed to OPM's Agent, Artifact, and Process). See Fig. 16. The PROV data model also provides several properties that can relate things, in addition to an expanded list of classes, e.g., Collection, Organization, Quotation, etc.

2.8.2. FAIR DATA

In 2016, the seminal FAIR (Findable, Accessible, Interoperable, Reusable) principles for scientific data management were published [91]. The FAIR principles are not a technical implementation or a system: they are a collection of non-functional recommendations for scientific data and research that intend to engender reusability, transparency, trust, and quality of scientific workflows [138]. The FAIR principles experienced a widespread resurgence in 2019, with the advent of the COVID-

19 incident, in which policy decisions made without transparent scientific evidence resulted in a loss of public trust [139].

- Findable: data use unique and persistent identifiers, are described with rich metadata and are registered in a searchable resource
- Accessible: data are retrievable by their identifiers using a standard protocol, the protocol is freely open and metadata are available, even when data are not
- Interoperable: data use a formal, shared language, and use vocabularies that follow FAIR principles
- Reusable: data are described with accurate and relevant attributes, are released with clear usage licenses, are associated with detailed provenance and meet domain standards

FAIR data must meet particular criteria [91]:

Others have presented work attempting to map or evaluate Linked Data principles and projects, e.g., PROV-O, to FAIR principles [140, 141, 142]. See Fig. 17. However, experts in the FAIR community have asserted that FAIR and the Semantic Web are not equal. "[FAIR] emphasizes the machine-actionability of data and metadata. This implies… that resources that wish to maximally fulfill the FAIR guidelines must utilize a widely-accepted machine-readable framework for data and knowledge representation and exchange. While there are only a handful of standards and frameworks that could, today, fulfil this requirement, other potentially more powerful approaches may appear in the future. As such, the FAIR Principles explicitly do not prescribe the use of RDF or any other Semantic Web framework or technology" [143].

FAIR is of interest to the present proposal insofar as the ultimate providence of this work is to be an open-source, findable, accessible, interoperable and re-usable software artifact, which itself generates FAIR software artifacts.

2.9. LANGUAGE MODELS

Natural Language Processing (NLP), as a distinct subset of Artificial Intelligence (AI), and, specifically, Machine Learning (ML), has been extant since the late 1940s. Alan Turing cemented the place of AI in history in 1950, by asking the question "Can machines think?" [144]. In 1994, Jones defines the four phases of NLP work: machine translation (late 1940s – late 1960s), artificial intelligence (late 1960s – late 1970s), grammatico-logical (late 1970s- late 1980s), and statistical language data processing (1980s onward) [145]. The 2000s-2010s saw a resurgence in Machine Learning (ML), principally Deep Learning (DL) models, which allow for more complex model architectures and result in greater capabilities [146].

Language modeling is a practice incorporating methods from linguistics, psychology, mathematics, and computer science. This practice may be said to find its roots in 1949, with Waren Weaver's memorandum *Translation*, in which the use of probabilistic methods for translating languages are discussed [147], building off of the earlier 1948 work by Claude Shannon, of Bell Labs, who established the theory underlying information theory, most notably, information entropy, spawning Weaver's idea of using probabilistic models for text prediction [148]. A language model (LM) is intended to predict the likelihood of a sequence of words. Given previous words within some context window, e.g., a sentence, a language model can estimate the likelihood of the next word, insofar as entire bodies of text can be predicted and, for example, output for use.

Early applications of language models commonly included speech recognition and machine translation between different human languages [149, 150].

In 2007, Google released a paper containing a technique for training LMs on up to 2 trillion tokens (specific sequences of characters, e.g., words), resulting in LMs capable of remembering 300-billion word combinations [151]. With the advancement of particular ML constructs, e.g., the Rectified Linear Unit (ReLU) activation function, in addition to more powerful graphics processing units (GPUs) being offered, neural networks could leverage more layers, becoming deeper and more complex [152, 153, 154]. In 2017, a seminal neural network architecture, the Transformer, which utilizes a novel bidirectional self-attention algorithm, was described in a published paper [155]. Bidirectionality, e.g., as the Bidirectional Encoder Representations from Transformers (BERT) model exemplifies [156], allows the transformer to consider context before *and* after every token, where previous unidirectional models only considered context *after* every token, and therefore suffered in performance. The Transformer architecture is the basis of contemporary LMs, allowing for very large context windows and a large prediction space.

The early 2020s experienced a frenetic proliferation of generative AI, principally due to the advanced, generalizable capabilities of Large Language Models (LLMs), which have been trained on inordinate quantities of Internet data [144]. The ultimate appeal of LLMs is their ease of use: they are simply prompted in natural language and return the same. Multi-modal LLMs (MLLMs) can interact and respond with various modalities of expression, e.g., music, images, videos, etc. ML is defined as a "field of study that gives computers the ability to learn without being explicitly programmed" [157]. LLMs have brought this capability to the general public, where they are used in many domains and in various capacities, e.g., by content creators for

suggestions and editing, by students for tutoring and studying, software developers in code checking, etc. [158]. A LLM has even been leveraged to assist ML non-experts to more effectively prototype ML systems [159].

In the context of software development, when given appropriate instruction, and relevant context, e.g., source files, contemporary LLMs can perform a variety of tasks for the incentivized developer, who may even build a codebase with stored prompts around such an LLM. In other words: for contemporary (2024 and beyond) software developers, LLMs remain a central component in both prototyping and actual implementation work, often entirely replacing purpose-written code. Some research indicates that LLMs compete with, and may someday replace, certain software development demographics, e.g., online software help resources like Stack Overflow, which were traditionally community-driven [160].

2.9.1. Large Language Model Hallucination

A prime concern of LLM output, aside from simply ensuring correctness given some need, is that of hallucination. Derived from the same term of psychology, LLM hallucination is the tendency of an LLM to output content that is fluent and correct in appearance, but is semantically wrong, nonsensical or fabricated [43]. As LLMs are, in effect, "best guessers", some have questioned their usefulness, referring to them as stochastic parrots [45]. Hallucination is not a theoretical concern but one with real-world consequences. On February 6th, 2023, a Google LLM, Bard, hallucinated a correct-appearing response in a live demonstration; in the following days, the stock value for Alphabet, the parent company of Google, lost $100 billion [161].

Mitigating LLM hallucination is therefore a task of utmost importance. The field of prompt engineering provides some leverage in this regard.

TABLE II Prompt engineering techniques.

Prompt Engineering Technique	**Definition**	**Example**
Direct Task Specification	A 0-shot prompt telling a model to do a task without any additional context; a model is given *what* without *how*	*"Translate this into that"*
Task Specification by Demonstration	A 1- or few-shot prompt telling a model to do a task with a demonstration; a model is given *what* and *how*	*"Reformat this text X to adhere to this new format Y"*
Task Specification by Memetic Proxy	A prompt using an analogy or proxy for open-ended tasks	*"You are my mentor: explain Hegel's dialectic to me as a student"*
Constraining Behavior	An intentionally complete prompt intended to restrict possible model interpretations	*"Translate the following phrase from Latin to English: 'Cogito, ergo sum' "* is a more predictable prompt than *"Translate this: 'Cogito, ergo sum' "*
Serializing Reasoning for Closed-Ended Questions	A prompt, or series of sequential prompts, that ask(s) a model to extend its reasoning and consider intermediate verdicts	*"Let's consider each possible answer by breaking them apart into their respective steps"*
Metaprompt Programming	A prompt that begins with a specific task question followed by a general, agnostic request	*"f(x) = x*x + x. What is f(f(5))? Let's solve this by breaking it down into steps."*

2.9.2. PROMPT ENGINEERING

Reducing the process of content generation with LLMs to the simplest terms is as follows: prompt and receive response. This abstraction of the underlying NLP work resulted from an evolution, over the years, from early fully-supervised learning with feature engineering and neural network architecture research, which advanced into the pre-train and fine-tune paradigm in the late 2010s, and which has proceeded into the current paradigm of "pre-train, prompt, predict" [162]. However trivial prompting seems, there has arisen a dedicated discipline out of innumerable empirical trials referred to as prompt engineering (prompt programming), which intends to make LLM output more consistent and reliable, through various techniques [163, 164]. This quality of LLMs to be more amenable to the intent of the prompt engineer is referred to as steerability.

Prompt engineering has been applied in the domain of software development, where common SDLC activities, e.g., requirements elicitation, code refactoring, etc., have been addressed by effective LLM prompting with prompt patterns, deriving from traditional software patterns, which are reusable solutions to common problems in software development [165, 42]. There are various prompting techniques reported to provide better or more consistent output from language models [162]. As natural language can be ambiguous, one method to mitigate this is 1- or few-shot prompting, which provides 1 or more examples of desired output in the prompt itself, effectively giving an LLM what is expected, so it may more readily conform to such. This technique works by reinforcing a model with correct behavior, steering it toward a desired output, as long as the examples are clearly distinguished from the prompt itself, and do not contaminate its semantics [163].

In discussing the few-shot prompting technique, the authors present six prompting techniques that they posit, just as in human discourse, should be woven together to redundantly reinforce a given model [163]. These frameworks are summarized in TABLE II. In general, the heuristic for closed-ended prompts is to provide as much correct contextual information alongside the prompt, so long as the semantics of the task request are not contaminated, and it is clearly distinguished from supporting information.

Outside of the perspective of the SDLC given above, empirically validated prompting techniques established by prompt engineering researchers are a means of steering LLM output toward desired states. Prompt engineering is utilized in computer vision to produce more consistent image and video outputs [166], in academic writing to mitigate ambiguity [167], and so on. Prompt

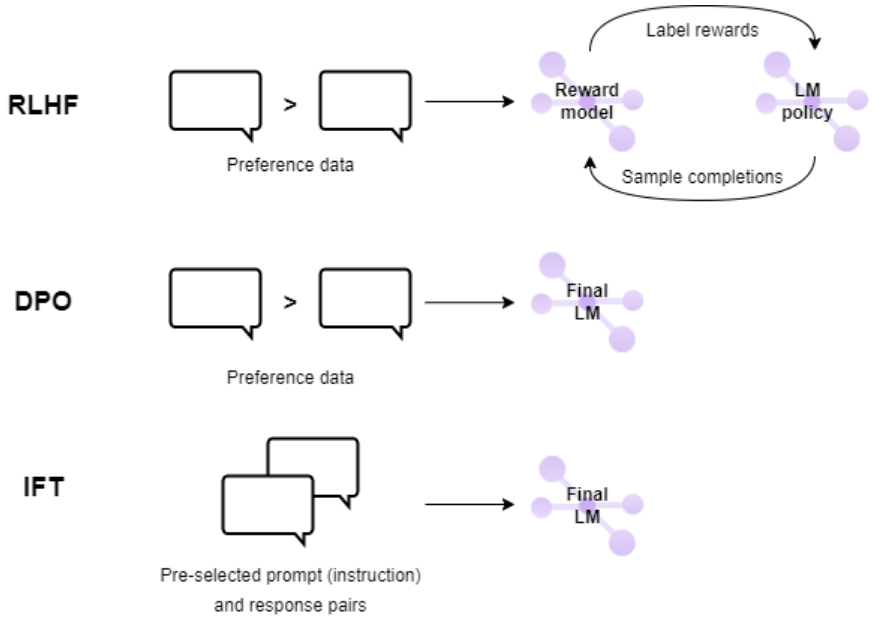

Fig. 18 The three primary LLM fine-tuning techniques [168].

engineering is an "after the fact" means of steering LLM behavior. Prior to LLM use, LLMs are fine-tuned to improve their usefulness to prompters; this is discussed in the following section.

2.9.3. Large Language Model Fine-Tuning

In the domain of applied ML, there are two distinguishable approaches to model deployment, which are adhered to differently, depending principally on the goals and requirements of some project:

1. Specific models for specific use cases: highly efficient, easily deployable models with high accuracy on specific tasks; the tradeoffs are such models cannot handle tasks outside of their scope, and developing new models for new tasks can be costly
2. General models transferred to specific use cases: highly versatile models with good accuracy on most tasks; the tradeoffs are that fine-tuning is a complex task, and most pre-trained models are computationally expensive due to their size

The ML field has generally transitioned to transferring general models to specific use cases (the "pre-train and fine-tune" paradigm) since the late 2010s and the advent of public LLMs [162]. Given a model pre-trained for some task, transfer learning applies the model to a different, but related task, by extracting new features given a new dataset, and training new layers for the new task. Fine-tuning is a specific form of transfer learning where a pre-trained model is further trained, i.e., fine-tuned, on a task-specific dataset, without training new layers, but adjusting the extant pre-trained layers.

Because the pre-training of LLMs is an unsupervised process, where large quantities of data generated from real humans are vectorized, resulting in broadly knowledgeable models, these models can come to embody some of the undesirable qualities of human speech, and writing, such as typing errors, incorrect assertions, biases, etc. [168]. Some LLMs have hundreds of billions of parameters, but being larger does not translate to greater practicality for the human user [169]. It is necessary, then, to involve some measure of supervised learning on pre-trained LLMs, to correct undesirable behaviors and to make LLMs as useful as possible for end users, while adhering to pre-defined human values, which may be hidden for corporate LLMs [170]. These supervised learning techniques are forms of post-training fine-tuning, not in the traditional sense of fine-tuning the model for atomic tasks, but to mitigate hallucination and other issues embedded from the model's training data. There are three identifiable means of LLM fine-tuning, all involving human input [171]. See Fig. 18.

1. Instruction Fine-Tuning (IFT): a model is fine-tuned on a curated dataset of pre-approved instruction-response pairs; datasets can be automatically generated from LLMs [172], curated by a large human effort [173] or made with a middle-ground approach, where a

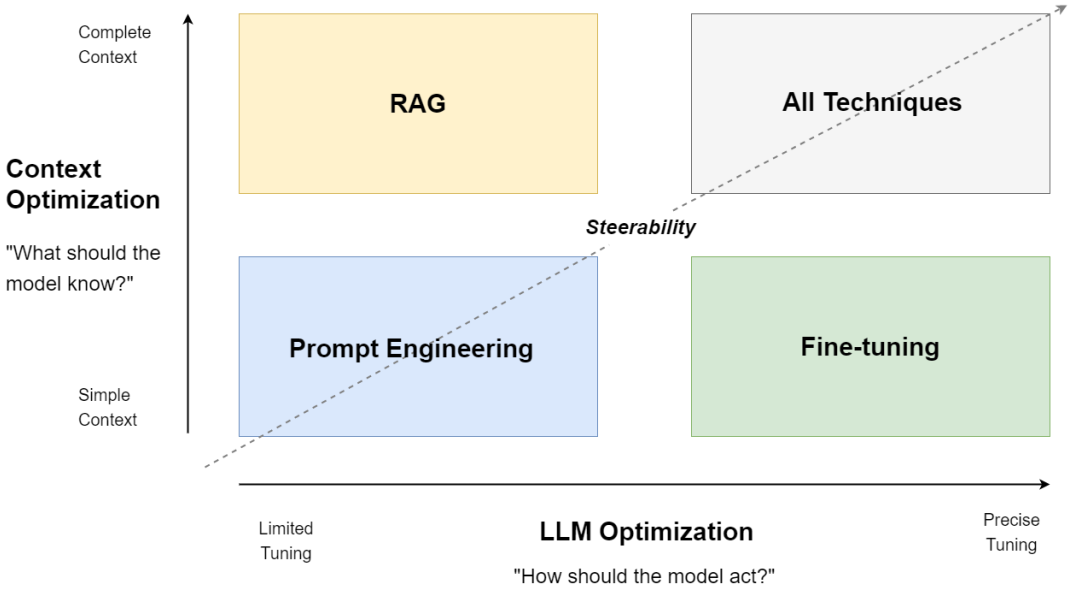

Fig. 19 LLM optimization techniques and their relationships to model steerability (adapted from [171]).

small set of seed tasks is used to generate an entire instruction dataset from the model itself, e.g., as Self-Instruct demonstrates [174]; the primary tradeoff is dataset curation can be time-consuming

   a. A parent technique to IFT may be considered in Supervised Fine-Tuning (SFT), e.g., as DeepMind's Sparrow leverages to mitigate unhelpful or "unsafe" outputs; some SFT rules may include no threats, no insults, no emotions, no legal advice, etc. [175]

2. Direct Preference Optimization (DPO) [176]: for a series of prompts, a model generates sets of response options, from which a human trainer selects their preferred responses, and which are then used to fine-tune the model; DPO is computationally simple but time-consuming
3. Reinforcement Learning from Human Feedback (RLHF): a model is fine-tuned through prompting and continual feedback from human trainers, often using a hybrid, dynamic

reward model; an example is InstructGPT [169]; RLHF is a computationally complex form of fine-tuning

2.9.4. RETRIEVAL AUGMENTED GENERATION

Post-training fine-tuning alters a model iteratively with internal, predefined criteria so it is better suited for its intended task, which can be quite broad, as with LLMs. LLMs operate on two types of memory [48]:

1. Parametric memory: a pre-trained language model, i.e., a model's learned associations of texts
2. Non-parametric memory: external sources of information that a model can query in real-time for information (prompts containing one- or few-shot examples may be considered non-parametric model memory)

Leveraging non-parametric memory, e.g., with a retrieval mechanism to fetch relevant contextual information from an external corpus, is effectively a means of continuous, real-time model fine-tuning. This method, very flexible in practice, is codified as Retrieval-Augmented Generation (RAG) by Lewis et al., of Facebook (Meta) AI Research [48]. Both fine-tuning and RAG approaches are human-centric, insofar as they aim to align model output with human expectations and needs; the former through direct feedback, and the latter by accessing curated information sources. See Fig. 19.

As RAG leverages external documents of any desired type, these corpora can be rather noisy, i.e., filled with additional information not directly relevant to a given prompt; evaluations have demonstrated that, with increased document noise, LLM accuracy suffers when employing RAG [177]. So, a less noisy RAG corpus is desirable [178]. RAG is, at a practical level, a

prompting technique, as traditional RAG sees extracted snippets appended to prompt bodies as additional context.

As discussed in previous sections, knowledge graphs proliferated from the Semantic Web movement of the late 1990s and early 2000s, but have since been relegated to research projects, enterprise knowledge bases and distributed web semantics in the form of RDFa. However, due to the adaptability and structured nature of knowledge graphs, they are ideal augmenters of LLM embeddings, i.e., they are useful for improving LLM output. The recent adoption of LLMs within the RAG architecture for software systems in various domains has brought knowledge graphs back to prominence as a means of reliably improving LLM efficacy [178].

A difficulty in large software systems that interact with a variety of file types is extracting information in a structured manner. Traditionally, such systems would require bolstering through purpose-written code to adequately account for varying file types, encoding formats, etc.; moreover, different media modes, e.g., text and images, require different approaches with varying degrees of complexity to extract structured information, that may then be synthesized and fused for use. This problem of multimodal information extraction is addressable using the RAG method and a multimodal, vision LLM, e.g., the Generative Pre-Trained Transformer (GPT).

2.9.5. GENERATIVE PRE-TRAINED TRANSFORMER (GPT)

The Generative Pre-trained Transformer (GPT) from OpenAI is at the center of the current (early- to mid-2020s) zeitgeist of large-scale, generative AI being rapidly adopted in every conceivable domain, personal, professional or otherwise [144]. GPT is accessible freely, or with paid premium plans, over the Internet, meaning the powerful LLM computes responses on more capable hardware than a typical local user could provide. There are various other powerful LLMs, e.g.,

Google's Fine-tuned LAnguage Net (FLAN), and Facebook's Bidirectional and Auto-Regressive Transformers (BART) [179], but none have achieved the universal adoption that GPT has. In the Holistic Evaluation of Language Models (HELM) project, over two dozen LLMs were evaluated on several metrics, with GPT-4 achieving a mean win rate of nearly 97% over the other models [180, 181]. For instance, with GPT-4, users can upload files and the LLM can ingest and interact with these files [170]. Images, videos, and audio can be uploaded alongside textual prompts, allowing for multi-modal prompt engineering. In terms of efficacy, OpenAI has stated that the capabilities of GPT-4 are, in exam evaluations, nearly equivalent for both the base model and RLHF model, so RLHF does not significantly alter its efficacy [170].

On May 13, 2024, OpenAI announced GPT-4o, a multi-modal model capable of reasoning across audio, vision and text [182]. On July 18, 2024, the GPT-4o mini model was released, a small model which is nearly as effective as GPT-4o, albeit with a much smaller cost of use [183]. On September 12, 2024, OpenAI released their o1 model, which is trained with reinforcement learning to reason more effectively; it surpassed every LLM thus far in its evaluations, rivalling the reasoning performance of human experts [184]. On October 3, 2024, OpenAI released Canvas, a more collaborative means of interacting with GPT during iterative development of text or code [185]. On February 27, 2025, OpenAI released GPT-4.5, a scaled unsupervised model with a great breadth of world knowledge [186]. The development of GPT models is clearly fast paced and continuous through 2023, 2024, into 2025, and beyond. The usefulness of GPT is apparent; it is discussed in the context of the current proposal because its capabilities are easily integrated with programmed software systems, inasmuch as the full GPT models can be accessed through

OpenAI's application programming interface (API). This fact is essential to the development of the proposed system.

2.9.6. Other Large Language Models

Although the GPT series of LLMs dominates in mainstream use, these models from OpenAI are not amenable to proper open-source development, as nothing is known of their architecture, training data, training methods, etc. The technical report for GPT-4 is very intentional in obscuring this information, providing mostly performance evaluations. "Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar" [170]. The GPT-4 technical report is useful for an understanding of the public policies enacted the ensure model fairness, security, etc. [187], but the exact architecture and training of the model remains unknown, so it can only be considered for prototyping purposes. Other LLMs exist and have been shown to perform very well in comparison to the GPT series, e.g., Mistral 7B [188], LLaMA 1 (7B, 13B, 33B, 65B) [189], and LLaMA 2 (7B, 13B, 70B) [190] from Meta (formerly Facebook) AI, FLAN [191], and so on [192, 193]. Many flagship LLMs are not open-source, and the classification of each model is generally on its underlying architecture: encoder only, decoder only, or encoder-decoder. See Fig. 20.

The GPT suite of models is used in the present dissertation inasmuch as it is a "common ground" for LLM development in the time of this dissertation's progression. The GPT series is effectively a baseline for the future work of the present dissertation, where, ideally, any LLM accessible through an API (or executable locally) capable of (1) multimodal reasoning and (2) returning plaintext responses, could be used instead.

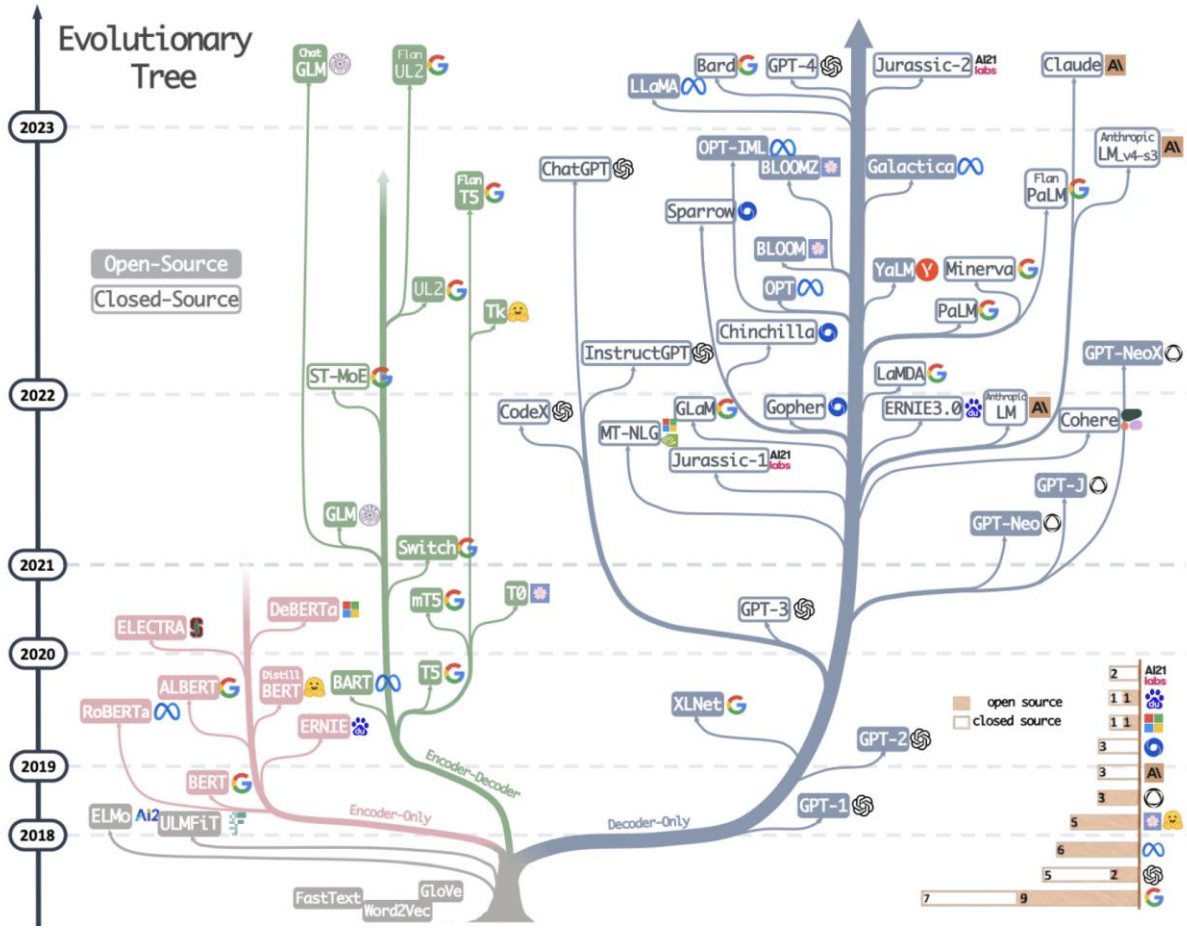

Fig. 20 LLM family tree (taken from [193]).

2.9.7. VISION-LANGUAGE MODELS

The research area of visual recognition has witnessed the same shift to the "pre-train, fine-tune, predict" paradigm as has language modeling; this is true of all deep learning applications beginning in the late 2010s. Training a deep neural network for a planned visual recognition task is time-consuming and costly. Similarly to LLMs, Vision-Language Models (VLMs) or Multimodal language Models (MLLM) have embedded associations between image-text pairs at the scale of the Web, insofar as they are applicable to any number of domain tasks with high accuracy [194]. A VLM is a multimodal construct capable of reasoning across visual and textual input, and

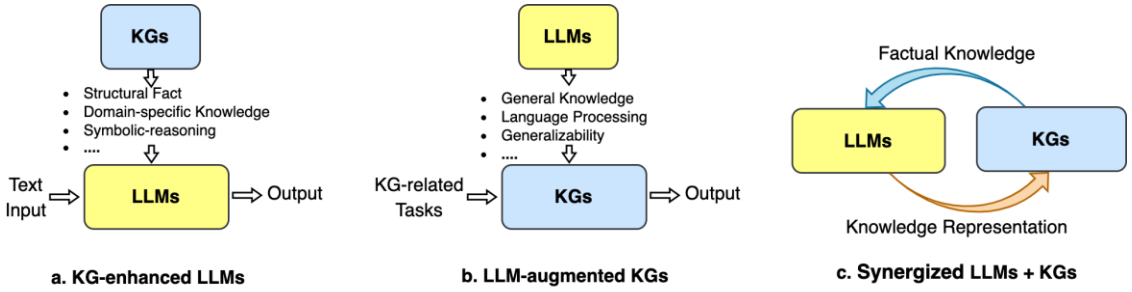

Fig. 21 Three paradigms of LLM and KG combination (taken from [196]).

outputting either modality. GPT-4-Turbo, GPT-4o, and GPT-4o-mini are capable MLLMs[5] underlying the system developed in the present dissertation.

2.10. Knowledge Graph and Large Language Model Synergy

As discussed in previous sections, knowledge graphs proliferated from the Semantic Web movement of the late 1990s and early 2000s, but have since been relegated to research projects, enterprise knowledge bases and distributed web semantics in the form of RDFa. However, due to the adaptability and structured nature of knowledge graphs, they are ideal augmenters of LLM embeddings, i.e., they are useful for improving LLM output. The recent adoption of LLMs within the RAG architecture for software systems in various domains has brought knowledge graphs back to prominence as a means of reliably improving LLM efficacy, primarily as a means of reducing noise, and therefore hallucination [178].

Elvira et al. present a large survey on the intersection of knowledge graphs and language models [195]. They identify five paradigms within which works combining knowledge graphs and language models exist:

[5] GPT-4o is, colloquially, considered the state-of-the-art (SOTA) MLLM as of late 2024.

- Knowledge Graph Generation: generating a KG from natural language text
- Knowledge Graph Completion: predicting missing relations and entities in a KG
- Knowledge Graph Enrichment: augmenting LLM input with KGs; equivalent to RAG
- Ontology Alignment: mapping serialized ontologies to create semantic equivalence
- Language Model Probing: extracting KGs assessing LLM knowledge retention

These five paradigms see a leveraging of knowledge graphs and language models, both in some capacity, towards some explicit goal, as the two are considered to be complementary. Similarly, Pan et al. present three paradigms of KG + LLM use: KG-enhanced LLMs, LLM-augmented KGs and synergized LLMs + KGs [196]. See Fig. 21. As LLMs possess massive quantities of parametric memory, i.e., embeddings, knowledge graphs contain non-parametric, structured knowledge – in similar fashion to the human brain, which Atkinson and Shiffrin (1968) proposed to be bifurcated into working (or short-term) memory and long-term memory [197]. See TABLE *III* for a comparison of KG and LLM aspects.

Natural language is inherently tree-like, or roughly hierarchical (but not necessarily subsumptive), as bodies of text contain paragraphs that are broken down into sentences, then words, then characters; broadening the perspective, series contain books, which contain chapters, and so on. Procko et al. posit that LLMs learn the hierarchical structure of natural language, and demonstrate preliminarily the preference of GPT-4 to represent knowledge domains as taxonomies [198]. Earlier evaluations of the BERT LM revealed that BERT captures tree-like representations of natural language [199]. This phenomenon is important to consider, as taxonomies (subsumption hierarchies) typically form the basis of KGs.

TABLE III Knowledge graph and LLM comparison.

Aspect	**Knowledge Graphs**	**Large Language Models**
They model…	Facts	Sequences of characters
Using…	Language-independent concepts	Strings specific to languages
Having the strengths…	Factual accuracy and coherent reasoning	Linguistic diversity and man-machine communication
Focusing on…	Semantics	Syntax
Storing meaning…	In computers	In human users
Created by…	Humans	Algorithms
Based on…	Written materials, human expertise	Written materials
Through…	Expert review, tools	String frequency
With reliability that is…	Very high	Variable
With cost that is…	High	High
With speed that is…	Slow	Fast

The synergy between LLMs and KGs arises in various ways [200]. See TABLE III. KGs can be incorporated in the training stage of LLMs, to help them learn structured knowledge; KGs can also be used to help interpret the reasoning of LLMs [195, 196]. KGs can be incorporated during active use, i.e., by implementing the RAG architecture [48]. The synergistic effect of these two constructs used in combination has been established in a variety of fields. GPT has been shown to provide more accurate learning recommendations and explanations for students when augmented with a structured KG [201]. LLMs augmented with KGs demonstrate reduced hallucination and increased explainability within a biomedical context [202]. In short: LLMs, despite their generalizability, are prone to issues of hallucination and accuracy because of the stochastic nature of ML [203], and KGs are loosely formal knowledge representation artifacts lacking usability; so the two in combination are more amenable for real-world task deployment than if deployed independently. LLMs are limited by input token length, either by local

computation or (typically paid-for) cloud computation, so the incorporation of cogent, non-noisy RAG data is preferable to unstructured, noisy data [178]. The combination of LLMs and KGs in the context of SDLC provenance capture is a primary aspect of the present dissertation.

Chapter 3: Overview of the Proposed System

As established, the lack of explainability and traceability in digital artifact work, e.g., the SDLC, is ubiquitous; this, in turn, lends to a dearth of reproducibility, transparency, and interoperability. Therefore, there is a need for an automated provenance capture system for digital artifact modification, which also mitigates the issue of information overload. LLMs provide the most agnostic means of interpreting user action in the digital space, and knowledge graphs provide an intuitive means of storing provenance traces; the two in combination are synergistic.

While the KMS of Thaul focuses on artifact-specific provenance, i.e., knowledge traces specific to individual files [18], the system proposed herein is less restrictive, and therefore more broadly applicable, as it is intended to be applied across multiple artifacts within working projects. In digital work, artifact modification is not an atomic process, i.e., some knowledge worker is unlikely to only operate on one singular artifact in a given work session. Realistically, a knowledge worker involves several artifacts during their work sessions, with influences, and modifications occurring both from and to them. In the present dissertation, the system scope is at the project level, not at the individual artifact level. Moreover, Thaul's proposed system is effectively an embedded operating system application with dependence on manual provenance annotation, which introduces complexity and unnecessary overhead if an LLM is to be the underpinning of the provenance automation. The system proposed and implemented here is lightweight, applicable to *any* digital work (not selected file types, e.g., Word files), and executable on any operating system.

The developed system is, at the highest level, composed of three primary entities (see Fig. 22):

1. The knowledge worker or user, i.e., the software engineer

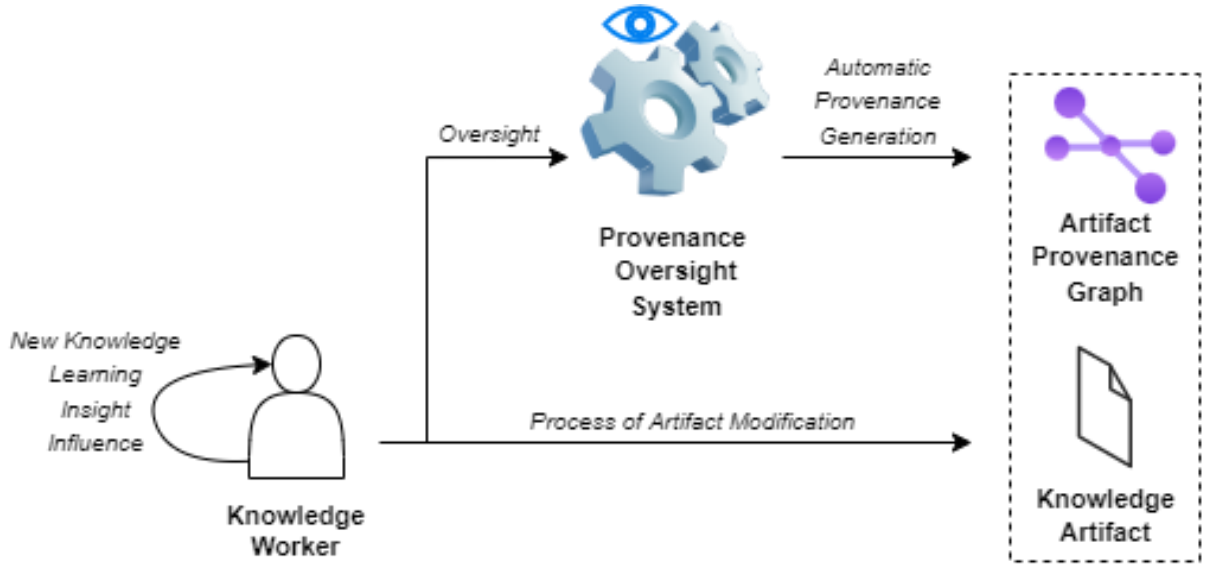

Fig. 22 High-level representation of the implemented provenance capture system.

2. The provenance oversight system, i.e., the means of automatically capturing digital provenance
3. The knowledge artifacts and their associated provenance graphs

The proposed system is an automatic, or observed, provenance system [204], spanning all PKM tasks [18]. A principal reason for asserting the feasibility of developing the proposed system is the recent shift in thinking of large software systems as Extract, Transform, Load (ETL) constructs, into Extract, Contextualize, Load (ECL) constructs, ultimately manifesting in the RAG architecture around powerful LLMs. This is due primarily to the generalizable capabilities of LLMs, which can effectively replace the transformation pipelines traditionally needed for data ingestion, by providing contextualized prompts alongside un-transformed data; i.e., LLMs are able to "make sense" of large and varied data without codebases dedicated to data transformation. This capability of LLMs is crucial to the proposed system, insofar as it may ingest provenance data from various sources, in various forms, and in various quantities, so a programmed "catchall" ETL

TABLE IV Key terms of the developed system.

Term	**Definition**
ProvTracer	The system/application developed
Provenance capture / recording / collection	The primary goal of ProvTracer
Session / trace session / work session	The occurrent that a knowledge worker operates within to produce provenance while laboring upon some digital artifact(s)
Work action	Any user action taken that may result in a provenance trace
Trace / trace link	A captured provenance fragment in the form of a knowledge graph node and link
Timeslice	The set interval of time each session is divided into to capture manageable chunks of work
Context item / provenance signal	An individual context component collected within the bounds of a timeslice
Timeslice packet	The aggregated work context components collected during a timeslice, in addition to the LLM prompt body
Screenshot loop / provenance loop	The heart of the system, generating provenance on a set interval
Provenance trace link knowledge graph / provenance trace KG	The ultimate output of a work session

system is infeasible, whereas a dynamic, contextualized ECL system built on a LLM is entirely feasible, particularly for the evolving nature of a system developed in the course of a dissertation.

3.1. KEY TERMS

There are a number of terms used when describing the system and its functionality. These are selected to be as agnostic as possible while retaining their semantics. See TABLE *IV*. To describe the system lexically with these terms would be as follows: the user begins taking work actions within a trace session, which operates on a timesliced loop, involving various work context components aggregated into timeslice packets, sent along to a MLLM to produce trace links integrated into a provenance trace knowledge graph.

3.2. ASSUMPTIONS

Some assumptions dictate the development of the system; these are discussed in turn.

The view of Asuncion and Taylor, that trace links are first class objects, i.e., they are represented as "separate and independent entities from the artifacts they connect", underpins the implemented system [205]. It is infeasible to associate a unique provenance KG with every individual artifact, as doing so would require integration with operating system and file versioning libraries. A key consideration of the system is to remain lightweight, so trace link KGs are considered project-wide, or session-wide, for a given worker. It is asserted that every individual be considered as a knowledge worker or subject matter expert (SME), whose implicit knowledge is made explicit in project artifacts.

In keeping with the principles of Agile software development [58], it is assumed that software evolution is dynamic, so it cannot be expected of a ProvTracer user to enter an explicit plan for a work session. It is posited that plans for some work session will almost invariably deviate as the work session proceeds [206], so ProvTracer is intended to be as lightweight and dynamic as possible, by only requiring a modicum of initialization information. Regardless, a later section on the potential application of ProvTracer for task-oriented software development is given (see section 8.2.2).

A key decision made prior to the development of the system was the technical reason for automatic capture, i.e., the trigger causing provenance capture. There were identified two distinct means of doing so:

1. On file events: capturing provenance deltas, or chunks, based on file events, e.g., modifications or file saves
2. On a set interval: capturing provenance on the threshold of a predefined, set time interval, or timeslice

The latter, to use a set time interval, or timeslice, to record provenance chunks, was selected as the most lightweight technique. The primary issue with the first technique is the unbounded nature of file modification events and file deltas. A user may make rapid modifications in a set period, or larger modifications over a longer period; the problem is therefore unscalable without a clear definition of what exactly constitutes a proper "file delta", rather than a small change. Use of a set timeslice was chosen as the technique for automatic provenance capture insofar as it leaves the problem bounded.

In automatic provenance capture, there is an inherent mismatch between observed and recorded provenance; this manifests in three core challenges: granularity, versioning and cycles [204]. Granularity is addressable by filtering particular events, e.g., irrelevant events, and only including those deemed relevant to the artifacts in question, e.g., as Thaul does [18]. For the system developed herein, granularity is addressed by only recording a small set of provenance context sources. Versioning is a more complicated matter, as some automatic provenance capture system must assert what actually constitutes a modification, e.g., as the Provenance-Aware Storage System (PASS) uses a copy-on-change method. The last challenge is the provenance cycle, which can occur when so many file versions are created; the PASS authors freely admit that cycle avoidance was unsolvable [207]. These two challenges are avoided with the chosen set time interval technique.

The most important information captured by the system are the MLLM-generated descriptions of screen activity during each timeslice interval; these provenance fragments, taken together, make up the provenance of a given work session. As discussed in section 1.4, this contextual provenance is the primary interest of the present dissertation, but the chronological

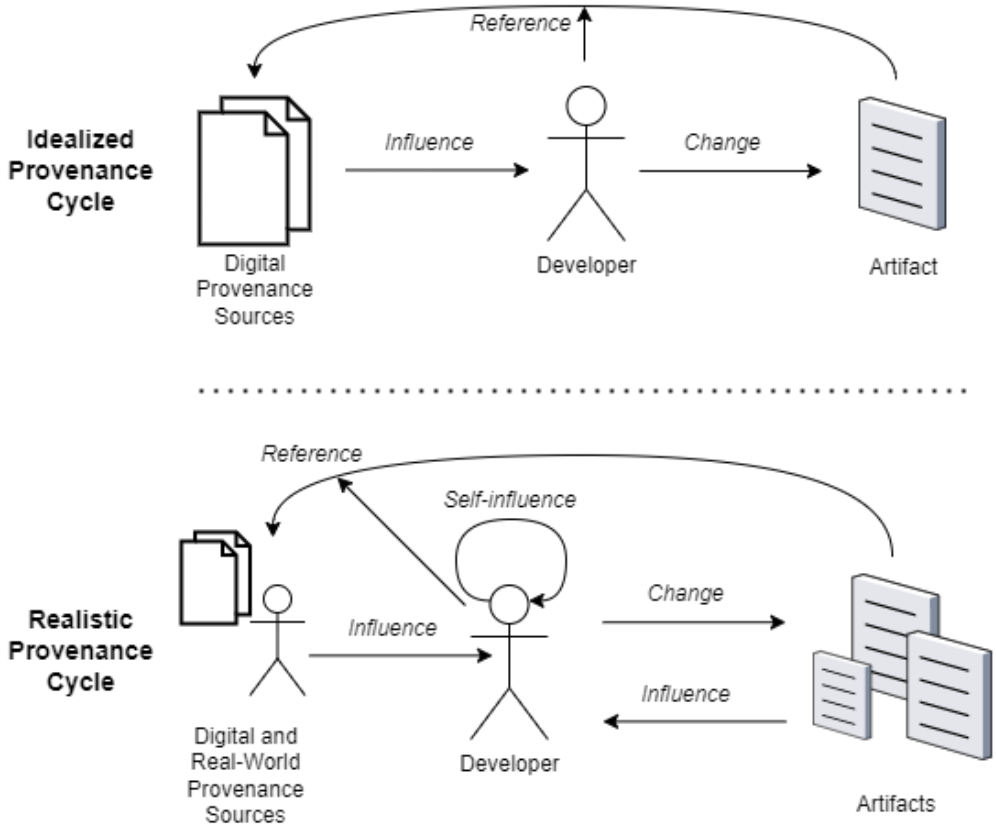

Fig. 23 Idealized vs. realistic provenance. In reality, there are various influencing sources of provenance, digital and real-world, including one's own experience and the artifacts being labored upon themselves.

provenance, or metadata, e.g., timestamps and user information, are easily recorded as well with no MLLM input, so such items are incorporated with the output KGs to retain as complete a picture of a work session as possible.

Another assumption is the manner of system usage. Herein, it is assumed that some system user will analyze their output provenance graphs after their work sessions are complete. Granting this, some slight delay in the MLLM response garnering is acceptable. This is discussed in a later section detailing MLLM latency (see section 4.1.5).

The system is intentionally limited to one screen, even in multi-screen environments. This is to lessen the associated latency and cost of the employed MLLM. Additionally, in digital work,

it sometimes occurs that a user will employ a secondary monitor (or set of monitors) for content not directly related to their work, e.g., music, videos, personal interests, etc. The one screen screenshotted is, by default, the user's primary monitor, which is a stored value in the OS. In the future, it may prove beneficial, for greater adoption of the system, to allow for multi-screen screen captures.

3.3. LIMITATION OF SCOPE

There are certain external influences that cannot adequately be accounted for in digital artifact provenance. See Fig. 23. For instance, if a software developer's superior informs him in-person of some code element that should be addressed, and the developer later fixes it, it is not possible to record this provenance, as it exists outside of the developer's computer. Barring later manual annotation (which is likely not amenable to developer workflows or would be seen as a chore), or a physical oversight system, e.g., an audio recording device or a brain implant [208], provenance as such is of interest for future work, but not part of the current focus. A future concern also out of the present scope is that of eye tracking, as developers may view particular elements in influencing artifacts, e.g., SRS documents, without explicitly clicking them or otherwise interacting with them, resulting in an implicit source of influencing provenance effectively "lost in translation" between the human eye and the computer screen without eye tracking hardware. These out-of-scope items are treated in greater depth in a later section on future work.

For some artifact, there is an identifiable piece of "root" provenance that is the beginning impetus of influence for the development of said artifact. For instance, the "root" provenance for some group of software developers on some team creating some contracted software system may be the requirements document provided to them from their senior engineers. For the developers,

this requirements document is, effectually, their root, or ground truth, provenance; nothing precedes it. But, within the context of the senior engineers, who likely engaged in interviews with the customer, their root provenance is found within the auditory or textual interviews held with the customers; and, further, the root provenance of the customers may be their tacit identification of a need for some software system given personal or business-oriented circumstances. This chain of provenance roots within the differing contexts can be traced backward ad infinitum, and serves no practical purpose. The boundary of what is feasible is therefore relegated to some digital artifact (or aggregation of them) and the relations to preceding provenance sources that can be recorded, i.e., those that exist on the computer of the knowledge worker and are interacted with.

For the demonstration and evaluation of the system, the SDLC artifacts of primary interest are implemented code files, i.e., the implementation stage is the most feasibly addressable SDLC stage by the system. This is because deriving provenance for, e.g., the design phase, where developers are working in diagramming software, is much more semantically complex than when writing code derived from textual requirements. This assertion will be evinced in the description of the system development in later sections. As another example, consider the SDLC phase of requirements elicitation, which may not be conducted entirely within computer systems, e.g., as many software development firms meet with clients in-person and use pen-and-paper materials to brainstorm requirements – none of this provenance is recordable without either manual annotation or a physical oversight system. In any case, all SDLC stages are considered; but the implementation stage is the operating basis.

3.3.1. AGILE PROCESSES

For the idea of automated provenance capture in the SDLC, and digital research in general, a boundary of feasibility must be established so that the developed system is not too far-reaching in scope. The type of digital research best suited for the system is Agile in nature, i.e., any digital work that is not accompanied by up-front, extensive documentation, e.g., requirements documents. This is because such documentation, for the developer engaged in active development, constitutes much of their development provenance, although other sources may influence their development as well.

Although explanations *can* be useful, they can often confuse users more than a lack of explanations; this is the "double-edged sword effect" of explainability. A lightweight, Agile approach to explanation generation is therefore preferable, focusing on usability for end users [84]. The digestive nature of LLM output, and the steerability of LLMs through additional context and adjusted prompts, is in line with the aspiration of end usability. Regardless, all forms of SDLC work, Agile or traditional, have provenance that can be captured and recorded automatically by the proposed system. Agile work simply possesses less formally codified provenance, so it remains the ideal use case for the system.

3.3.2. TRACE HALLUCINATION

A primary limitation, or operationalized concern, as it were, of the implemented system, is LLM hallucination; this cannot be completely avoided, although proper prompt engineering techniques can mitigate the issue [162]. A LLM is, in the most mundane sense, a "best guesser", and will always return *something* when prompted, even if the response is not entirely truthful or accurate. This is brought about as a limitation of scope because, despite best efforts in prompt engineering,

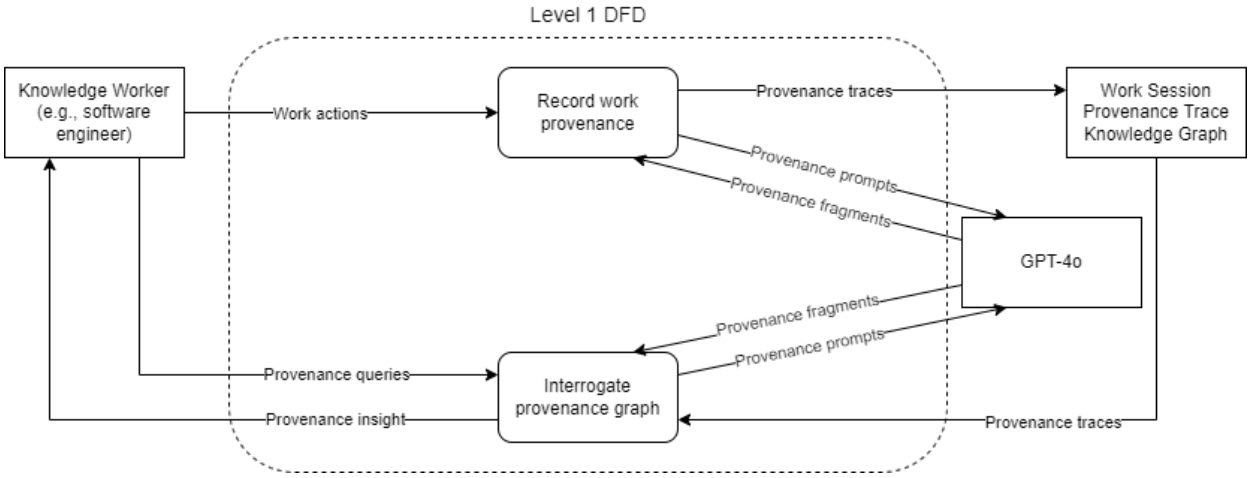

Fig. 24 High-level, level 1 data flow diagram (DFD) of the system.

as the implementation description evinces, the chosen MLLMs occasionally return improper responses[6]. The scope of system ability is limited by hallucination and general model stochasticity; these are evinced and will be discussed in the system implementation discourse.

3.3.3. Choice of LLM

The MLLMs chosen as part of the implementation of the system is the GPT-4 series from OpenAI. GPT-4-Turbo, GPT-4o, and GPT-4o-mini demonstrate exceptional ability in visual tasks, which is integral to this work, as it operates on screenshot data. This MLLM suite from OpenAI is not an ideal choice for software systems intended as free and open-source software (FOSS) projects, insofar as its architecture, training, and maintenance are closed-source. This is why the author is very careful to declare the company name, OpenAI, as one single word with the first letter capitalized, to avoid confusion with the concept of open AI, which concerns transparency and

[6] "Occasionally" is an ambiguous term, and determining what constitutes an improper response is a somewhat subjective exercise. This is discussed in the next chapter on system implementation.

accessibility in AI work [209]. Notwithstanding, access to a MLLM such as GPT-4o via an API is effectively a black-box function call, where both input and output are strings, or plaintext responses. For most LLM users, this black-box perspective holds true, as these constructs are complex stochastic predictors too vast to explain and understand; research has even been conducted on the personalities, or "AInalities", of LLMs, using such human psychometric measures as the Myers-Briggs Type Indicator (MBTI) to better understand them [210]. Herein, the MLLM is therefore considered a black-box ingesting and producing a usable output – in this case, plaintext strings. With this perspective taken, any MLLM can be substituted in place for those selected; the members of the GPT-4 series are chosen here for their visual capability and widespread use. The system is designed and implemented agnostically, such that the use of another MLLM in the future can be done with minimal changes to the code. See section 4.2.3 for more information on the MLLM wrapper module.

3.4. General System Design

Use of the system is reducible to two primary activities, or functions, applicable to the knowledge worker, e.g., the software engineer: (1) to record digital work provenance during a work session and (2) to interrogate the resultant provenance trace knowledge graph for insight. See Fig. 24.

The timeslice technique of provenance capture manifests insofar as the system is constructed around a screenshot loop which repeats at said interval. In addition to the visual screenshot of the user's screen, other context sources, e.g., keyboard strokes and file window events, are recorded and packaged into timeslice packets to be sent alongside the LLM prompt. See Fig. 25.

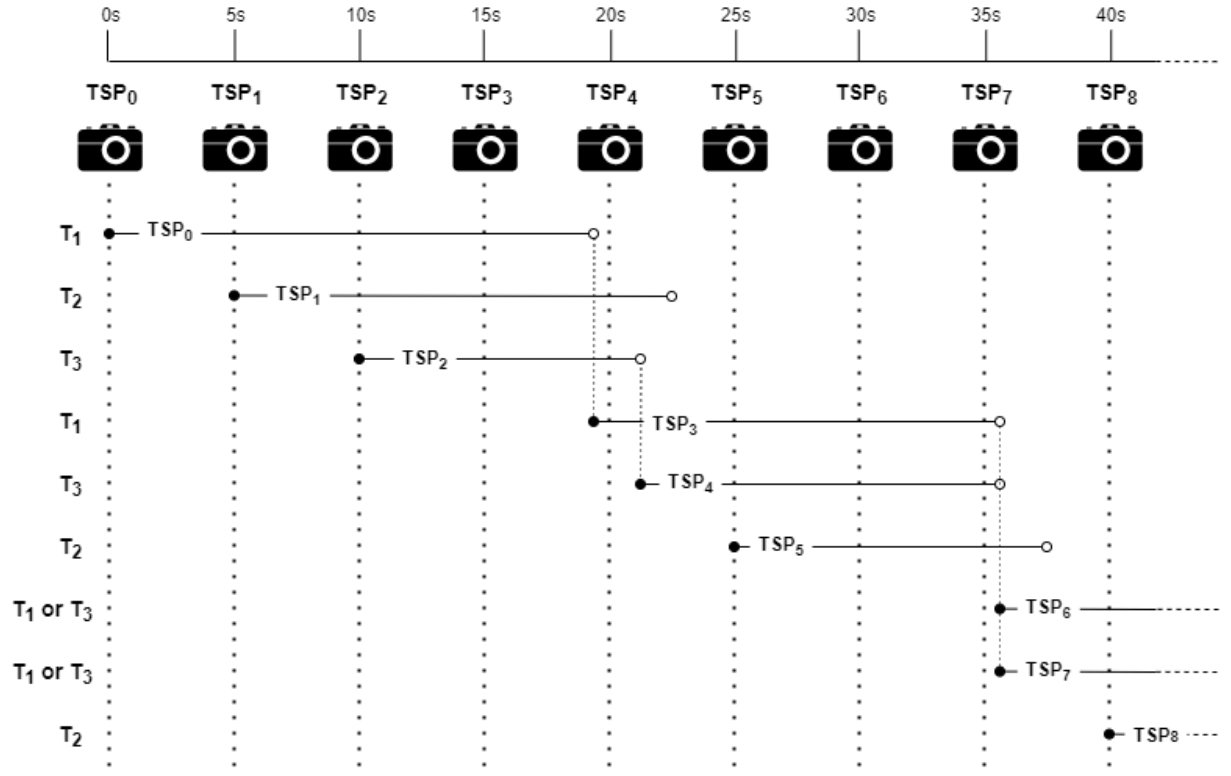

Fig. 25 Timeslice diagram of ProvTracer's threadpool of size, e.g., three, denoted T_n, with nine timeslice packets, denoted TSP_n, which are captured exactly every five seconds. The horizontal lines denote LLM response time, which varies.

Prior to the proliferation of MLLMs capable of visual input, such a task would likely involve some combination of an optical character recognition (OCR) model to extract text, and a vision model trained or fine-tuned on digital computer screens to extract broader semantics. OCR is a long-studied application area of mathematics and engineering, with its beginnings in the early 1950s [211]. Later discourse on OCR noted the importance of more context for greater OCR applicability, which is the underpinning of LLMs [212].

A general object classification model, e.g., You Only Look Once (YOLO) [213], could be utilized, but such models are trained to detect and classify real-world objects, such as people and vehicles. A YOLO model could be fine-tuned on a new corpus of annotated computer screenshots, but this is difficult to scale to different operating systems, screen sizes, application varieties, etc.,

so it is beyond the scope of the present dissertation. Research in association with the Apple company has presented the training of two ML models on iPhone screenshots to recognize screens and screen changes between apps [214], and an approach to screen parsing, or predicting user interface elements and the relationships between them based on screenshots [215]. Apple has also addressed the issue of poorly annotated UI datasets with an application crawler to continually train ML models in an unsupervised fashion [216].

The system developed here utilizes a simple screenshot loop. Smartphone users were found to take screenshots for several reasons, with the three most common verbs in their explanations of doing so being to "remember", "share", and "save" [217]. These are fitting motivators in the context of the present dissertation, which aims to record digital provenance traces for later analysis and use for individuals and among teams.

GPT-4-Turbo, GPT-4o, and GPT-4o-mini show impressive capabilities in question answering, reasoning and most visual tasks, but demonstrates some difficulty with niche vision tasks, e.g., captioning complicated images [218]. Regardless, these models are available through an API in Python and have extensive documentation and support, so they were chosen as the MLLMs underlying the system developed here.

3.4.1. PROV-BFO Representation

Provenance trace knowledge graphs output from the system are serialized as RDF graphs, following a simple schema derived from PROV-O and BFO. Refer to Fig. 26. Procko and Ochoa present a seminal mapping of PROV to the BFO/CCO taxonomy, although it is preliminary and quite speculative [219]. The mapping of the PROV-O to BFO/CCO is not trivial and not universally agreed upon, so, for the purpose of the present dissertation, the PROV ontology is the

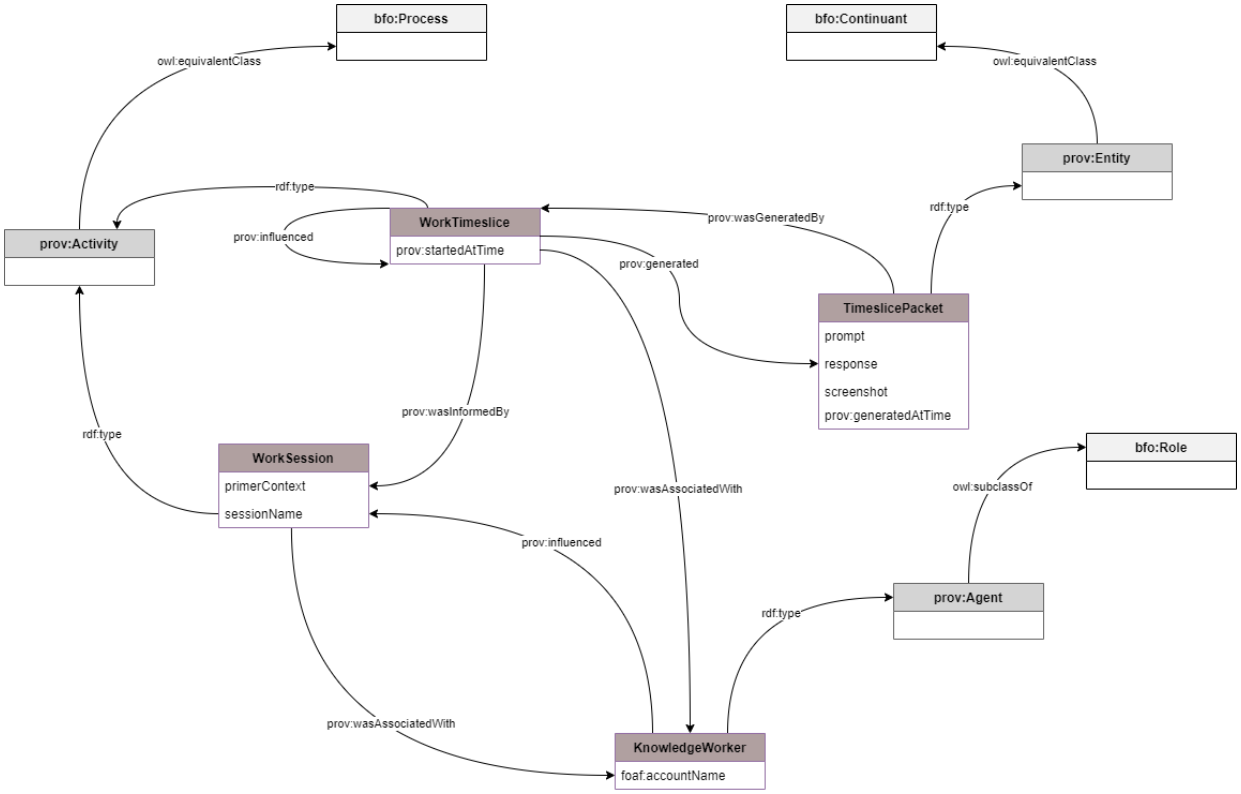

Fig. 26 The developed ontology underlying the system's graph output.

main vocabulary utilized to represent the captured provenance traces. Terms from BFO are linked to PROV terms via mapping relations, maintaining a mapping for every generated provenance graph. This engenders lightweight interoperability with the longstanding PROV standard and the more recent and burgeoning BFO standard. In the ontology presented here, Knowledge Worker is synonymous with User; this more descriptive term is used to maintain the association of ProvTracer with its knowledge representation underpinnings.

The PROV-O term, Role, is not to be confused with that in BFO. BFO is Aristotelian, following the notion of single inheritance, where PROV-O is more philosophically loose, allowing multiple inheritance. With BFO, the question of multiple inheritance is addressed with the BFO term, Role, which allows some Entity, that (ideally) has a defined superclass, to possess multiple Roles, e.g., as some human, who may be asserted as a subclass of a Material Entity, can have the

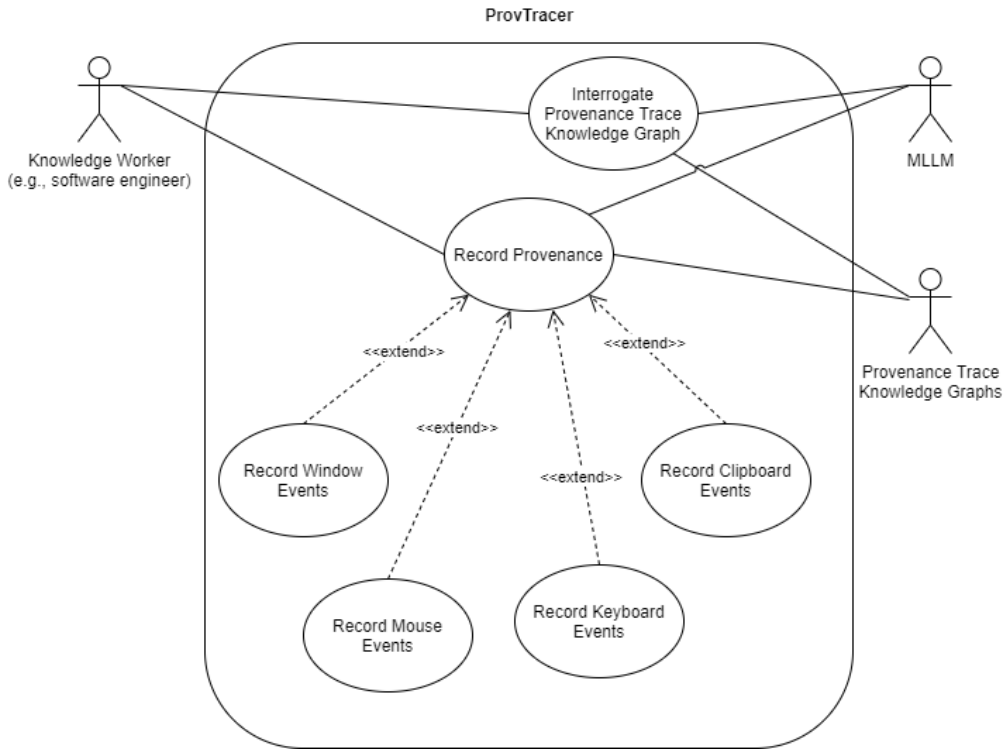

Fig. 27 System use case diagram.

defined Roles of Software Engineer, Student, etc. Insofar as some entity may or may not perform the functions of an agent at any given point in time, PROV Agent is asserted as a subclass of BFO Role; this is in keeping with the preliminary PROV-O to BFO mapping of Procko and Ochoa [219].

3.5. SOFTWARE SCENARIOS

The system is described in terms of functionality, using the use case diagram of the software engineering discipline attendant with use case scenarios. See Fig. 27. Use cases capture contracts between system stakeholders and system behavior, by describing such behavior under various conditions in response to stakeholder requests [220]. Use case diagrams are a simple visual design for demonstrating system functionality and use case scenarios act to check this design through a flow of events.

3.5.1. RECORD PROVENANCE FOR WORK SESSION

Scenario 1: Record Provenance for Work Session
Description: A knowledge worker wishes to record their work provenance during a computer work session.
Actors: Knowledge Worker, MLLM, Provenance Trace Knowledge Graphs
Precondition: The knowledge worker has an Internet connection and API credits for the MLLM.
Trigger Condition: The knowledge worker has launched the application.
Steps:

1. The system displays the primer window.
2. The knowledge worker selects the context options they desire.
3. The knowledge worker selects the Save Context button (ALT 1).
4. The system closes the primer window and displays the disclaimer window.
5. The knowledge worker accepts the disclaimer and agrees to proceed (ALT 5).
6. The system displays a persistent disclaimer window.
7. The knowledge worker operates their computer while the system records provenance (ALT 3).
8. The knowledge worker presses the Wait and Terminate button.
9. The system pauses new provenance recording and generates the trace link knowledge graph.
10. The system halts all execution and closes.
11. End of use case.

ALT 1 : Step 3: The knowledge worker presses the Skip Context button.
Step 3.1: The application displays the disclaimer window.
Step 3.2: The knowledge worker accepts the disclaimer and agrees to proceed (ALT 2).
Step 3.3: Go to step 4.

ALT 2: Step 5: The knowledge worker does not accept the disclaimer.
Step 5.1: Return to step 1.

ALT 3: Step 7: The knowledge worker selects the Pause button.
Step 7.1: The system pauses.
Step 7.2: The knowledge worker selects the Resume button.
Step 7.3: Return to step 7.

3.5.2. INTERROGATE PROVENANCE TRACE LINK KNOWLEDGE GRAPH

Scenario 2: Interrogate Provenance Trace Link Knowledge Graph
Description: A knowledge worker wishes to analyze and derive insight from a provenance trace link knowledge graph.
Actors: Knowledge Worker, MLLM, Provenance Trace Knowledge Graphs
Precondition: The knowledge worker has previously generated a trace link knowledge graph with the application.
Steps:

1. The knowledge worker locates an output provenance trace link knowledge graph from a previous work session.
2. The knowledge worker uploads the provenance trace link knowledge graph to a MLLM used by the application, e.g., gpt-4o-mini (ALT 1).
3. The knowledge worker asks questions of the MLLM with respect to the uploaded provenance trace link knowledge graph.
4. End of use case.

ALT 1 : Step 2: The knowledge worker uploads the provenance trace link knowledge graph to a graph database, e.g., GraphDB.
Step 2.1: The knowledge worker queries the uploaded provenance trace link knowledge graph with SPARQL.
Step 2.2: Go to step 4.

3.5.3. CONTEXT PRIMING

As established in background sections 2.9.2 and 2.9.4, describing prompt engineering and RAG, respectively, it is apparent that LLMs operate most reliably when bounded through appropriate input information, i.e., contextual information. For the implemented system, RAG is too heavy-handed, as the focus is lightweight, Agile development work. As such, the implemented system utilizes context priming, which is an upfront means of allowing users to input a basic description of their work; this is integrated into prompts to the MLLM. The system begins a work session by asking the user literally what their purpose in a given planned work session is. This context primer is included as a preface in prompts for provenance traces. Additionally, it is known that manual entry, e.g., of provenance, is prone to human errors. With the exception of the initial context primer (which is implemented as an optional feature, although it is recommended), the recording facet of the implemented system is entirely automated, operating without any direct user input. Other, specific sources of provenance context, all customizable by system users, are discussed in turn within the implementation description of the system.

Chapter 4: IMPLEMENTATION

This section describes the design and implementation of the automatic provenance capture system, hereafter referred to as ProvTracer. The name of the system is an incorporation of the two disciplines underlying, to wit, the Semantic Web, which places great emphasis on provenance, e.g., with the PROV-O standard, and software engineering, which emphasizes traceability from requirements elicitation forward. ProvTracer is an approach at unifying these two concerns through the use of a capable MLLM and knowledge graphs.

ProvTracer is developed in Python, as it is a language with very little overhead in development. Python is also the "default" programming language GPT responds in and is accessed with through its API. As ProvTracer is an extremely novel system, i.e., it is prototypical in nature, development in Python is beneficial inasmuch as it reduces boilerplate and the need for overly complicated system constructs. ProvTracer can be found in its GitHub repository: https://github.com/PR0CK0/ProvTracer.

4.1. MLLM EXPERIMENTATION

Historians of science note the common mismatch between what scientists do in practice and how their work is reported in publications [221, 222]; while their actual work may involve a hypothetico-deductive approach, i.e., a cyclical pattern of hypothesizing and testing, which invariably involves failures of hypotheses, scientific authors often obfuscate or entirely remove these trials in their reporting, misrepresenting their work as perfectly inductive, i.e., generalizing from phenomena to form hypotheses. Many modern academic publications in the domain of software engineering read more like product pitches than scientific reports.

The intent of this section within the present dissertation is to report on the empirical assessment of the MLLM within a series of prototypes which preceded the full system implementation. The included subsections describing the experimentation of the prototypes read, necessarily, as scientific reports; i.e., they are not abridged or edited for a positive reflection of the system. The use of a stochastic MLLM with inadequately understood visual capabilities is not an exercise in perfection, but one of finding fitness for purpose [223]. The system addresses the heretofore unresolved problem of agnostic digital provenance automation with a novel software construct, the MLLM. The words of Basili et al. (1985) are applicable here: "As any area matures, there is the need to understand its components and their relationships… [software engineering] is certainly a candidate for the experimental method of analysis… [it] involves an iteration of a hypothesize and test process" [34]. As ProvTracer is a novel, automatic provenance trace capture system incorporating a stochastic MLLM with structured knowledge graphs, some experimentation is needed.

It is necessary to ascertain the capability of a MLLM for extracting semantics from a digital research process via screenshots, to determine the feasibility of the planned system. To establish a nominal baseline, the author served as a single participant during a session of real dissertation research and development. The session was recorded using a prototype of the designed system described previously. This is similar to the "fly on the wall" technique of indirect data collection, whereby a researcher is an observer of some activity without being present [224]. Where traditional fly on the wall data collection involves the use of a video- and/or audiotape system, this particular collection process leveraged computer screenshots collected on a set interval. The downside to this technique is the later debt of manually analyzing the recordings; but as a preliminary data

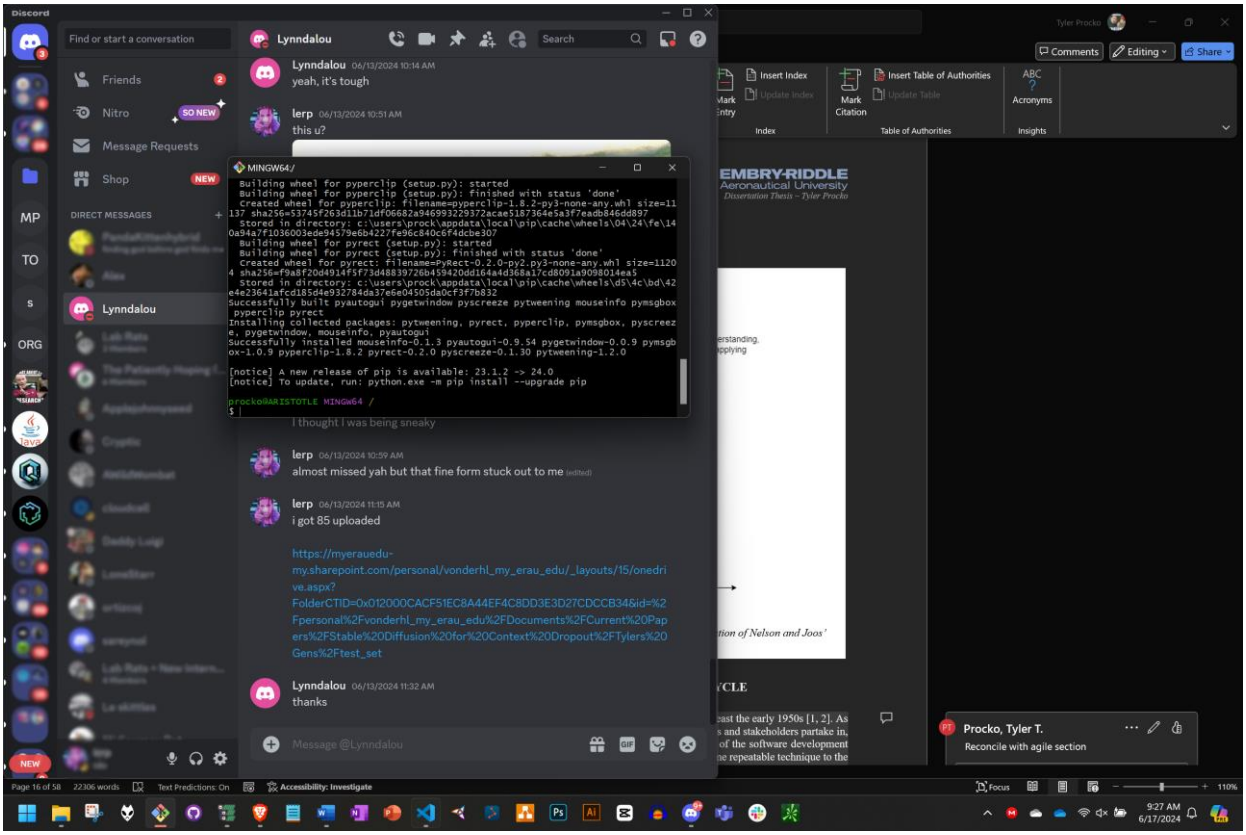

Fig. 28 A screenshot containing multiple application windows open simultaneously; used as a preliminary evaluation of GPT-4o in extracting meaning from a small fragment of a larger digital research process.

collection technique, it suffices for further experimentation. The recording took place at a physical work desk with no particularly notable equipment, save for the computer (a portable laptop) and a wireless mouse.

While the MLLMs chosen for this work are a subset of those from OpenAI, what is ascertained of them with respect to ProvTracer in the ensuing subsections can be extrapolated to other MLLMs. This is discussed in a later section on future work.

4.1.1. Ascertainment: Screenshots

To evaluate the prototype in the simplest possible scenario, its output images were uploaded individually to the ChatGPT interface, specifically using the GPT-4o model because of its flagship status as the fastest MLLM. An example screenshot demonstrating multiple application windows

open simultaneously is given in Fig. 28. The following prompt was used to elucidate its capabilities at extracting process-oriented knowledge from a screenshot given a modicum of context:

- "This is a screenshot of a person's screen as they are working. Can you tell me what this person is doing?"

Prompts varied as the experiments progressed; this simple one was used to establish a minimally replicable baseline. In one instance, GPT-4o responded with the following (copied verbatim):

From the provided screenshot, here's an analysis of what the person is doing and the applications they have open:

Applications Open:

1. Discord:
    - The person is using Discord, which is a communication app primarily used for chatting, voice, and video communication. They are in a chat with someone named "Lynndalou" and are discussing a variety of topics including uploading files and some technical issues.
2. Terminal (MINGW64):
    - A terminal window (MINGW64) is open, indicating the use of Git for Windows. The terminal shows the installation of Python packages using pip. Packages like pyuvm, pyvctrl, pyperclip, pymsgbox, pyscreeze, and others are being installed. This suggests they are setting up or working on a Python environment, possibly for a project involving these libraries.
3. Microsoft Word:
    - A document titled "Dissertation Thesis - Tyler Procko" from Embry-Riddle Aeronautical University is open. The person is likely working on their dissertation, editing, and formatting the document.
4. Web Browser:
    - There is a browser window in the background, open to a OneDrive link which seems to be related to academic papers or documents. This indicates they are possibly uploading or accessing documents related to their dissertation.
5. Other Apps and Tools:
    - The taskbar at the bottom shows icons for various other applications, including Microsoft Outlook, Adobe Photoshop, Microsoft Visual Studio, and some system utilities like a calculator and file explorer. These applications suggest the person might also be involved in tasks related to software development, graphic design, or managing emails and files.

Summary of Activities:

- Python Environment Setup: Installing various Python packages, likely for development or a project.

- Academic Work: Editing a dissertation document in Microsoft Word.
- Communication: Chatting and sharing links/files on Discord.
- Document Management: Accessing or managing files via OneDrive.

In general, the person seems to be multitasking, focusing on both development tasks and academic writing, while also staying connected through Discord.

The response of GPT-4o is a good digest of the actual situation as it occurred; further, GPT-4o extracted fine portions of text from the screenshot, including application titles and content, e.g., the Discord chat and the Microsoft Word document containing the present dissertation. The GPT-4o response is composed, it seems, of four sections of text:

- Introduction matter: an address of the prompt request and transition to the response
- Response matter: the relevant response content, organized as an indented list
- Summary matter: a brief summarization of the response itself
- Closing matter: a briefer summary acting as a conversational close

In further evaluations, this response structure is adhered to with little deviation, unless explicitly prompted for a particular output format. The introduction and closing matter are not immediately desirable as stored provenance, being mostly "contextual" information, so this extra output was noted, and theorized as containable by stricter prompting. The response matter is the main object of interest, containing the lexical representation of the work occurring in the screenshot. The summary matter is of interest insofar as it is a more cogent form of the response matter, which was posited to be useful as a means of reducing information overload in the later retrieval and application of provenance traces.

However appropriate this response seems, GPT-4o made an incorrect assumption about partially visible content. The communication application, Discord, was open between two colleagues, with a OneDrive (a cloud-enabled filesharing application) link to some project work

visible in their chat. Additionally, the Web browser, Brave, was open, but minimized, with its corresponding icon highlighted in the taskbar. GPT-4o assumed that the Web browser, which was not fully visible in the screenshot, was opened to the OneDrive link. In reality, it was not, so this assertion was hallucinated. Techniques to address this particular hallucination are given below:

- Remove the taskbar in screenshots: may, in fact, remove some semantics from which the MLLM may rely; not easily scalable to different operating systems or platforms with differently sized and/or placed taskbars
- Restrict the MLLM with a more directed prompt: the prompt can be modified to include more restrictive instructions to the MLLM; this is a basic practice in prompt engineering [165]

Because of the historical inaccuracy of VLMs employed in UI tasks, earlier work on this subject included the generation of a large (n = 335k) UI dataset for fine-tuning a VLM specific to UI tasks [225]. Notwithstanding these reservations, the GPT-4o MLLM is well capable of extracting provenance, at least in an isolated perspective, from desktop screenshots during digital research processes.

4.1.2. Ascertainment: Mouse Inputs

It was posited that additional semantics could improve the output of GPT-4o. As screenshots are static captures of a user's screen at some instant in time, the capture of mouse events in the timeslice interval leading up to the screenshot could contribute additional context useful for the MLLM.

So, mouse events were incorporated in a second iteration of the prototype. The mouse inputs included were mouse movements, scrolls, and clicks. The prototype was extended to draw

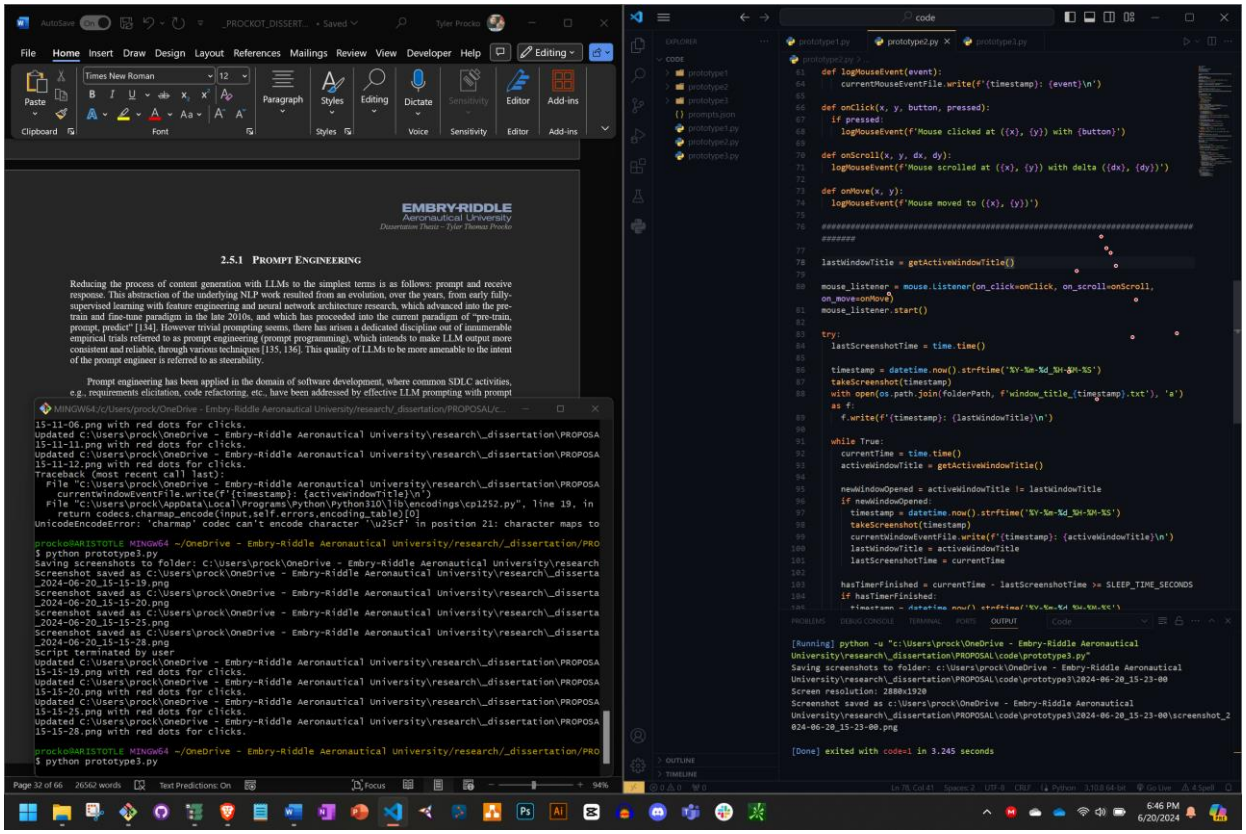

Fig. 29 A screenshot containing multiple application windows and the user's mouse clicks, indicated by small red circles with white borders.

small red circles with white borders where user clicks occurred during the duration of each screenshot, on top of every screenshot. The following prompt, modified slightly from that in the first iteration, was asked of GPT-4o in the ChatGPT interface:

- "This is a screenshot of a person's screen as they are working. The red and white dots represent their mouse clicks in the duration the screenshot took place. Can you tell me what this person is doing? Include information about the mouse clicks."

An example screenshot that was attended with this prompt can be found in Fig. 29. GPT-4o responded with the following:

```
From the screenshot provided, here is a detailed analysis of what the person is doing:

Activities Observed:

        1.  Editing and Testing Python Code:
```

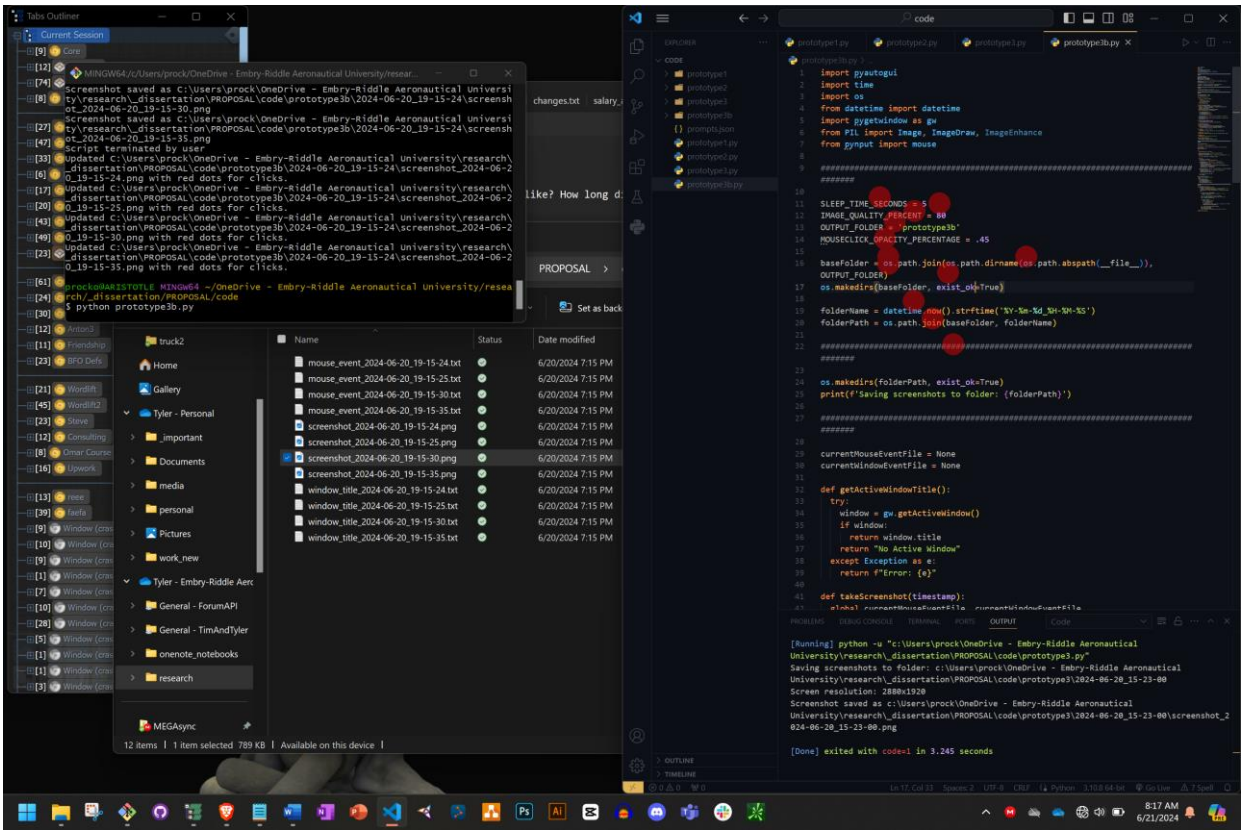

Fig. 30 A screenshot containing multiple application windows and the user's mouse clicks, indicated by large, semi-transparent red circles.

- The person is using Visual Studio Code (VSCode) to write and edit Python scripts. The script named prototype3.py is open in the editor.
- The terminal at the bottom shows the user running the prototype3.py script multiple times, as indicated by the logs.

2. Document Editing:
   - A Microsoft Word document is open, titled PROCKOT_DISSERTATION, indicating that the user is working on or reviewing their dissertation. The specific section visible is about "Prompt Engineering" related to large language models (LLMs).
3. Code Execution and Debugging:
   - The terminal logs show various operations, including saving screenshots with red dots for clicks and updating the console with the results of running the Python script.
   - The logs also show an error related to the 'charmap' codec, indicating the user is debugging encoding issues in the script.

Mouse Clicks Analysis:

- Code Editor (VSCode):

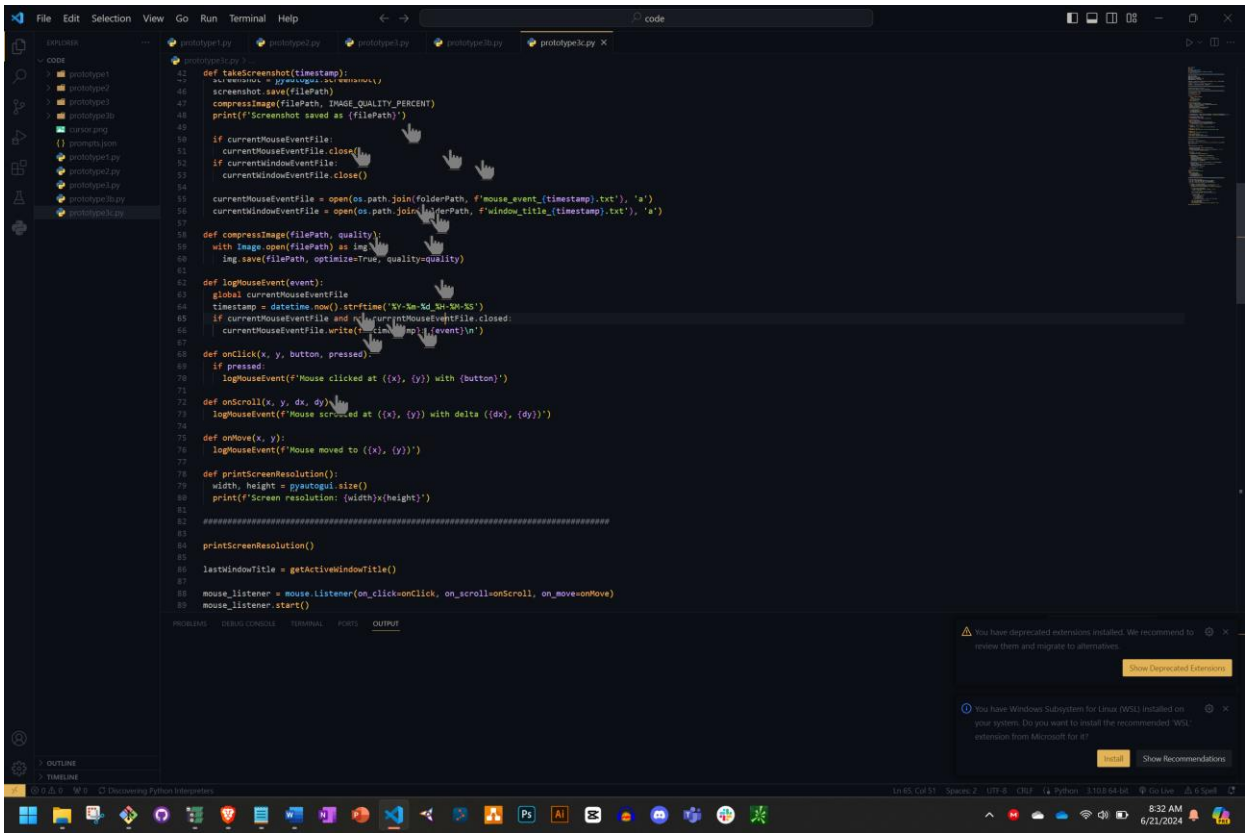

Fig. 31 A screenshot containing multiple application windows and the user's mouse clicks, now indicated by mouse click hand icons.

- Many red and white dots are concentrated around specific lines of code, indicating active editing and navigation within the prototype3.py script. The user is likely focusing on the functions for logging mouse events and taking screenshots.
- Frequent clicks near the line numbers and the text indicate navigation through the code, likely for reviewing and editing purposes.

- Terminal:
  - There are clicks in the terminal area, showing the user is running the script and reviewing the output frequently. This interaction suggests a typical development workflow of making changes to the code, executing it, and checking the results.
- Microsoft Word:
  - The Word document also has some mouse clicks, indicating the user is scrolling through or editing the dissertation.

Summary:

- The person is involved in a dual-task workflow: editing and debugging a Python script related to capturing screenshots and logging mouse events, and simultaneously working on their dissertation in Microsoft Word.

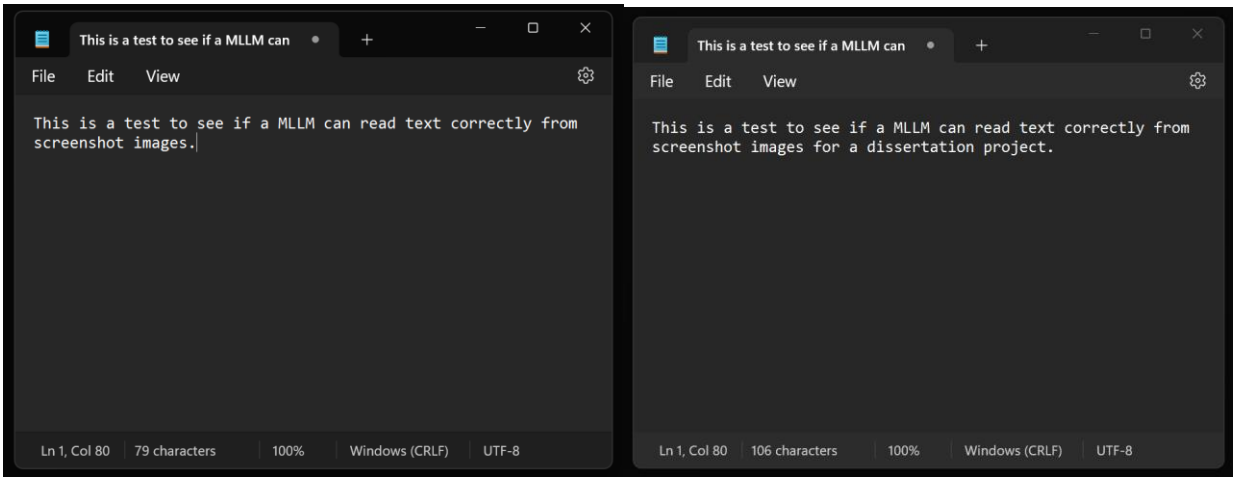

Fig. 32 Example screenshots (left and right) given to GPT-4o to test its visual text discrimination ability.

- `The mouse clicks in VSCode suggest active coding and debugging, while the clicks in the Word document suggest reviewing or editing the dissertation.`
- `Terminal interactions indicate frequent testing of the Python script to ensure it runs correctly and debugging issues as they arise.`

GPT-4o hallucinated, as there were no mouse clicks present in the terminal or Microsoft Word. It was posited that the circles were not particularly distinguishable by GPT-4o, so they were made larger and transparent, to allow other elements, e.g., text, to remain at least partially visible even if occluded by them. The prototype was updated and a slightly modified prompt, now denoting the larger red dots, used in conjunction with Fig. 30. The response of GPT-4o was similar to the previous response, with hallucinated locations of mouse clicks. It was posited that the use of circles, which are arbitrarily chosen indicators of mouse clicks known to the MLLM only through the prompt text, may be insufficient, as this is not likely present in its training set. So, mouse click (hand with one finger raised) icons were trialed next (see Fig. 31). GPT-4o continued

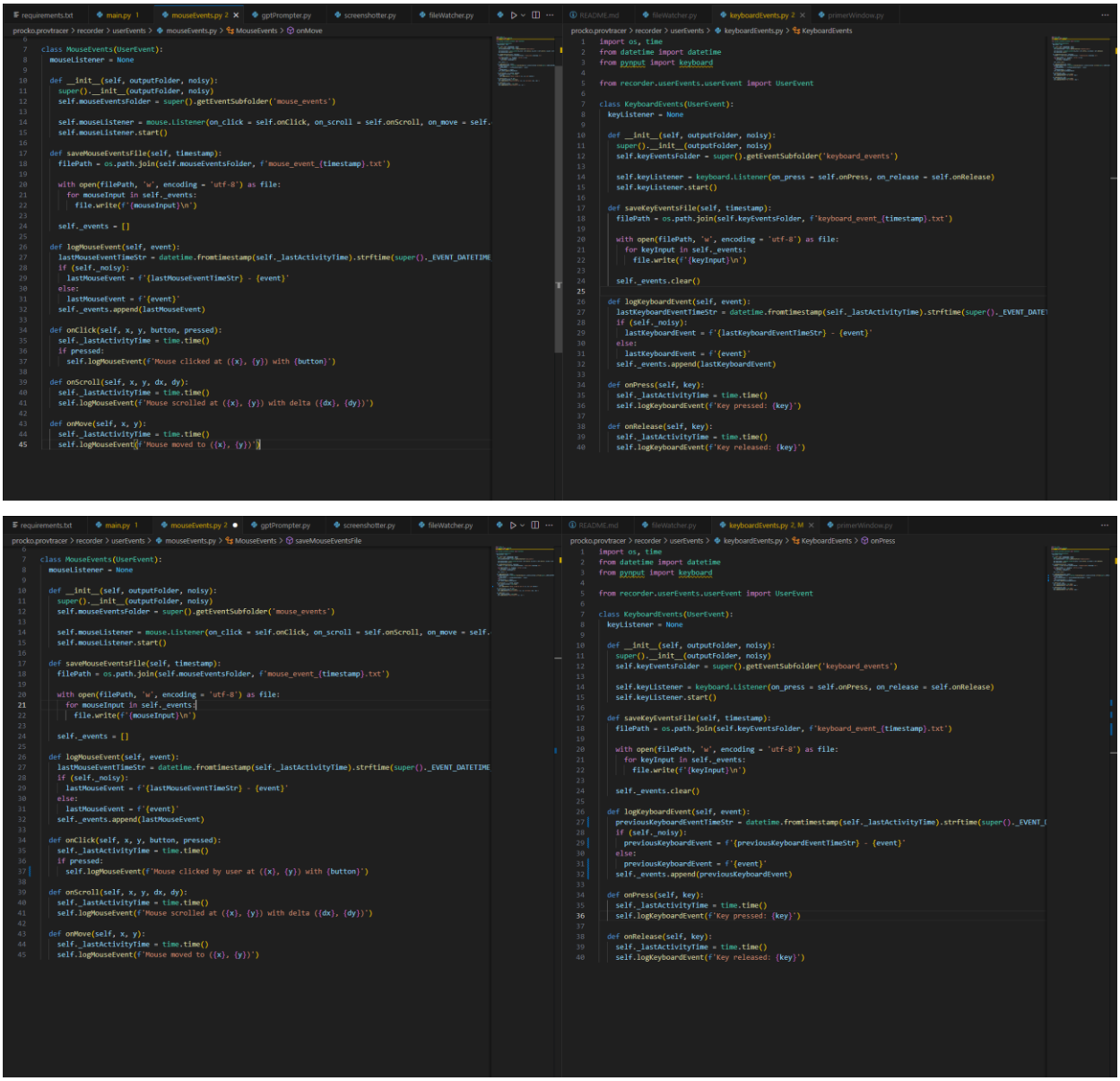

Fig. 33 Example screenshots (top and bottom) given to GPT-4o to test its visual text discrimination ability on complex code.

to hallucinate its observation of mouse click locations, so it was posited that the MLLM is not sufficiently trained on UI elements to accurately use such a provision[7].

4.1.3. ASCERTAINMENT: SCREENSHOT DELTAS

To ascertain the ability of the MLLM for textual discrimination within images, it is necessary to establish its efficacy in finding textual deltas across screenshots. To begin, GPT-4o was given two

TABLE V GPT-4o response for textual discrimination across two screenshots.

File	Section	Line(s)	Change
`mouseEvents.py`	`onClick`	37	Added “by user” to log message
`keyboardEvents.py`	`logKeyboardEvent`	29, 31, 32	Renamed `lastKeyboardEvent` → `previousKeyboardEvent`
`keyboardEvents.py`	`logKeyboardEvent`	27	Renamed `lastKeyboardEventTimeStr` → `previousKeyboardEventTimeStr`

very simple screenshots, each containing a small amount of text. The second image was identical to the first, with the exception that the text was added to, with the simple phrase “for a dissertation project”. See Fig. 32. The first image, on the left, is the baseline; the second image, on the right, is augmented from this baseline. GPT-4o was asked, “What is the difference in text between these two images?”. The MLLM detected the exact change without error. Its output is given below.

```
The difference between the text in the two images is that the second image has an extended sentence
with additional context.
---
**Text in the first image:**
> This is a test to see if a MLLM can read text correctly from screenshot images.
**Text in the second image:**
> This is a test to see if a MLLM can read text correctly from screenshot images for a dissertation project.
---
**Difference:**
The second image adds:
> **for a dissertation project.**
```

Next, the semantic complexity of the screenshots was increased by employing two screenshots of an IDE with two Python classes open, left and right of each other. See Fig. 33. The first image, on the top, is the baseline; the second image, on the bottom, is augmented from this baseline. GPT-4o detected the change without error. Its response was rather long, but included a summary section at its end. See TABLE V.

While promising, the MLLM suffered immediate semantic confusion and complete incorrectness when, in the augmented image of Fig. 33, the right-most class file was simply

scrolled down, such that the first line visible in the new augmented screenshot was line 7, instead of line 1. When this was conducted and the same experiment applied as in Fig. 33, GPT-4o continually provided incorrect responses for the given prompt. GPT-4o continued to note that the class which was scrolled down on appeared "partially collapsed", and that the "final lines differed", but it was unable to detect the same difference in code as in the previous experiment. This is a very important discovery which restricts the efficacy of the implemented system.

4.1.4. ASCERTAINMENT: KNOWLEDGE GRAPH OUTPUT

The end goal of the ProvTracer system is to output provenance knowledge graphs adhering to PROV-O and BFO. As such, it was deemed that the MLLM could be prompted for one of two output types: (1) plaintext or (2) a knowledge graph serialization, e.g., Terse RDF Triple Language (TTL, or Turtle).

In several tests with the latter, it was deemed infeasible for the MLLM to produce syntactically correct RDF output, due to a number of reasons:

1. Unbound prefix namespaces: the MLLM used namespace prefixes in the generated Turtle without first defining their URIs, resulting in invalid RDF
2. Inconsistency with requested namespaces: even when prompted specifically to use only, e.g., PROV, the MLLM would use terms from other vocabularies
3. Incomplete triples: the MLLM would often truncate its output Turtle, violating RDF syntax; this may be due to token overrun with the API

Given these issues, it was deemed that the knowledge graph output of ProvTracer is best generated programmatically, with the MLLM responses being put into appropriate triples as plaintext strings. This also improves MLLM response time and limits any possible hallucination

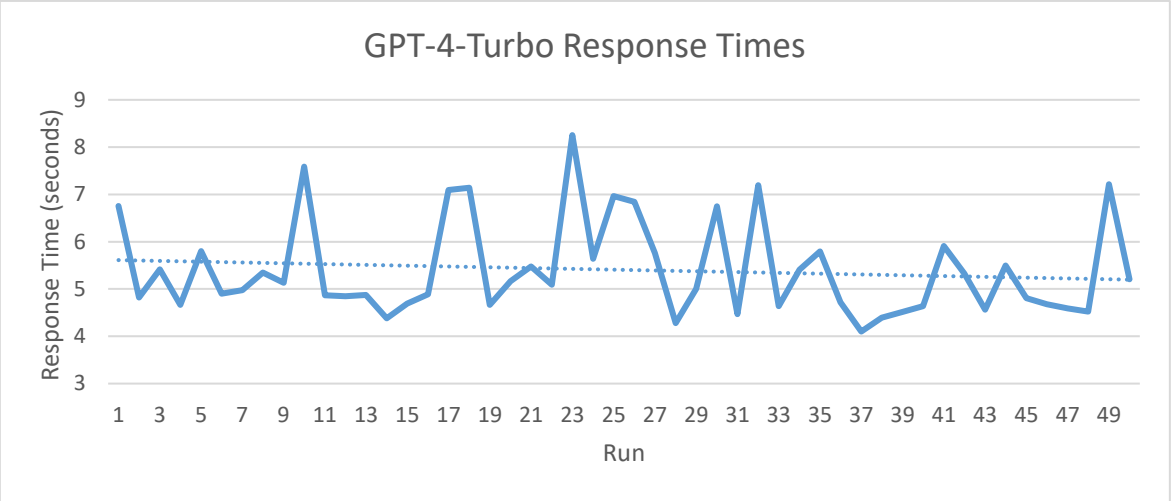

Fig. 34 GPT-4-Turbo response times over 50 runs.

to plaintext output. There is voluminous literature on the generation of KGs with LLMs [226, 227, 228], but for ProvTracer, which intends to be lightweight for Agile development, such a dependence would involve KG validation and consistency checking, which presents additional computational load that may be problematic at scale. With the purely programmatic technique, trace link KGs output from ProvTracer are valid upon creation.

4.1.5. ASCERTAINMENT: MLLM LATENCY

The MLLMs chosen for use in the developed system were evaluated in terms of their temporal efficacy. To do this, a pipeline was constructed in Python to prompt each of the three models fifty (50) times with a screenshot image of the user's screen. The averages of each model's response times are given below:

- GPT-4-Turbo: 5.40 seconds
- GPT-4o: 4.46 seconds
- GPT-4o-mini: 3.74 seconds

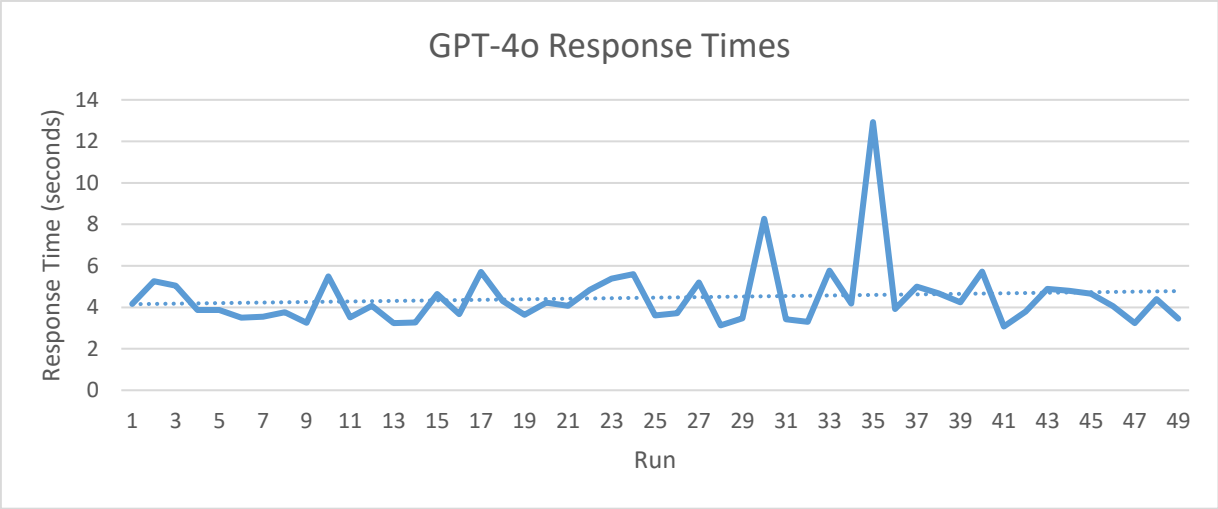

Fig. 35 GPT-4o response times over 50 runs.

These averages should be regarded with some uncertainty, due to their reliance on an Internet API, which may introduce additional latency. The prompt used was kept very simple: "Describe the content of this image in detail." GPT-4o had one response time over 600 seconds, which is likely due to throttling or Internet latency with the API of OpenAI; this time was removed from the dataset. Refer to Fig. 34, Fig. 35, and Fig. 36 for visualizations of response times. GPT-4o-mini had the fastest response times and is the cheapest model of the three, so it is set as the default model in ProvTracer.

A very notable observation that arose as more test runs were conducted was that, as the input prompt became longer, with more restriction on the LLM, its response time acted inversely, by accelerating. I.e., when the prompt was shorter and more unrestrained, response times of the LLMs varied and, in general, took significantly longer. Response times of the LLM are never particularly consistent, as expressed in the aforementioned figures, but this technique of improving response time while simultaneously constraining responses was noted for the system implementation.

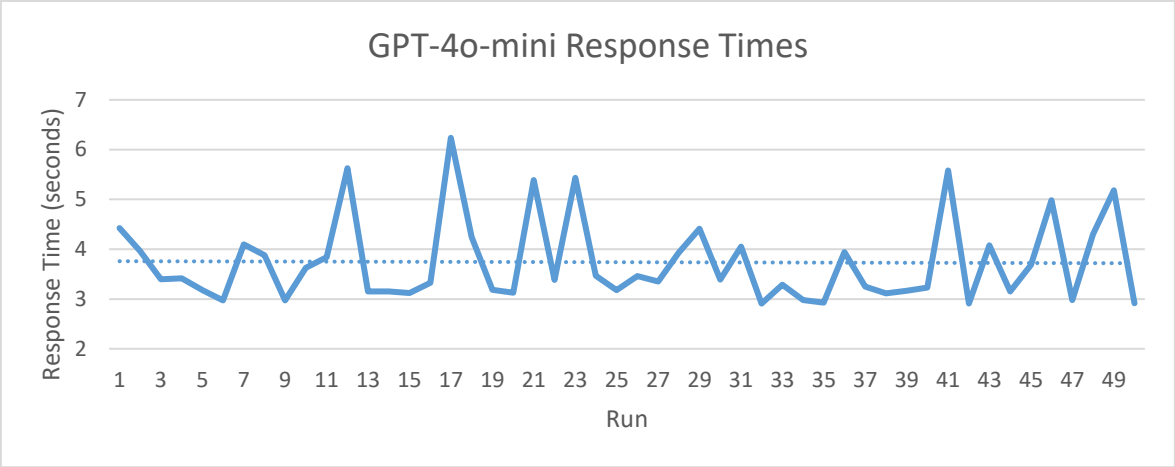

Fig. 36 GPT-4o-mini response times over 50 runs.

4.2. PROVTRACER

In this section is described the design and construction of the ProvTracer system. This includes more technical diagrams, and a description of the code and system functionalities.

4.2.1. CODED SYSTEM DESCRIPTION

ProvTracer is a designed and engineered system. It is implemented in Python using open libraries and object-oriented techniques. ProvTracer allows a user, e.g., a software engineer, to select from a handful of relevant context sources upon startup, whereafter it runs automatically, prompting a capable visual MLLM with screenshots and other timeslice information for provenance trace recording on a set timeslice interval. These timeslice packets are curated and sent to a MLLM in real-time for provenance trace responses. Upon work session termination, ProvTracer automatically converts all recorded provenance information into a knowledge graph adhering to PROV-O and BFO. ProvTracer trace link knowledge graphs can be queried with SPARQL, or by uploading them via RAG to a MLLM for interrogation in natural language. ProvTracer is organized into four primary packages, with subpackages for more organization within: grapher, helperz, recorder, and tracer.

Aside from standard Python library imports, the notable external libraries used are given below:

- pyautogui: For screenshots
- pyperclip: For clipboard events
- pynput: For mouse and keyboard events
- pygetwindow: For window events

With the exception of dependence on the chosen MLLMs from OpenAI, ProvTracer remains a FOSS project. All python libraries used are public and the system is programmed as agnostically as possible, such that integration with any other MLLM is possible with minor modification to system code. See section 4.2.3; this is also discussed in greater depth in a later section on future work.

4.2.2. State Chart

See Fig. 37 for a state transition diagram of the ProvTracer system. The main thread constitutes the Python default thread and the UI thread of the PyQt6 library. The watcher thread observes for new screenshots in the specified output directory. The worker threadpool is composed of n threads, where n is an integer in the range of 1 and the number of detected CPU cores on the host machine; these threads atomically and asynchronously prompt and wait for MLLM responses.

Fig. 37 contains an abstraction within the main thread. The context listeners may be considered separate threads, e.g., the pynput library implements listeners on separate threads. The state labelled “Timeslice Looping” can be decomposed into several sub-states. See Fig. 38 for a visualization of this composite state.

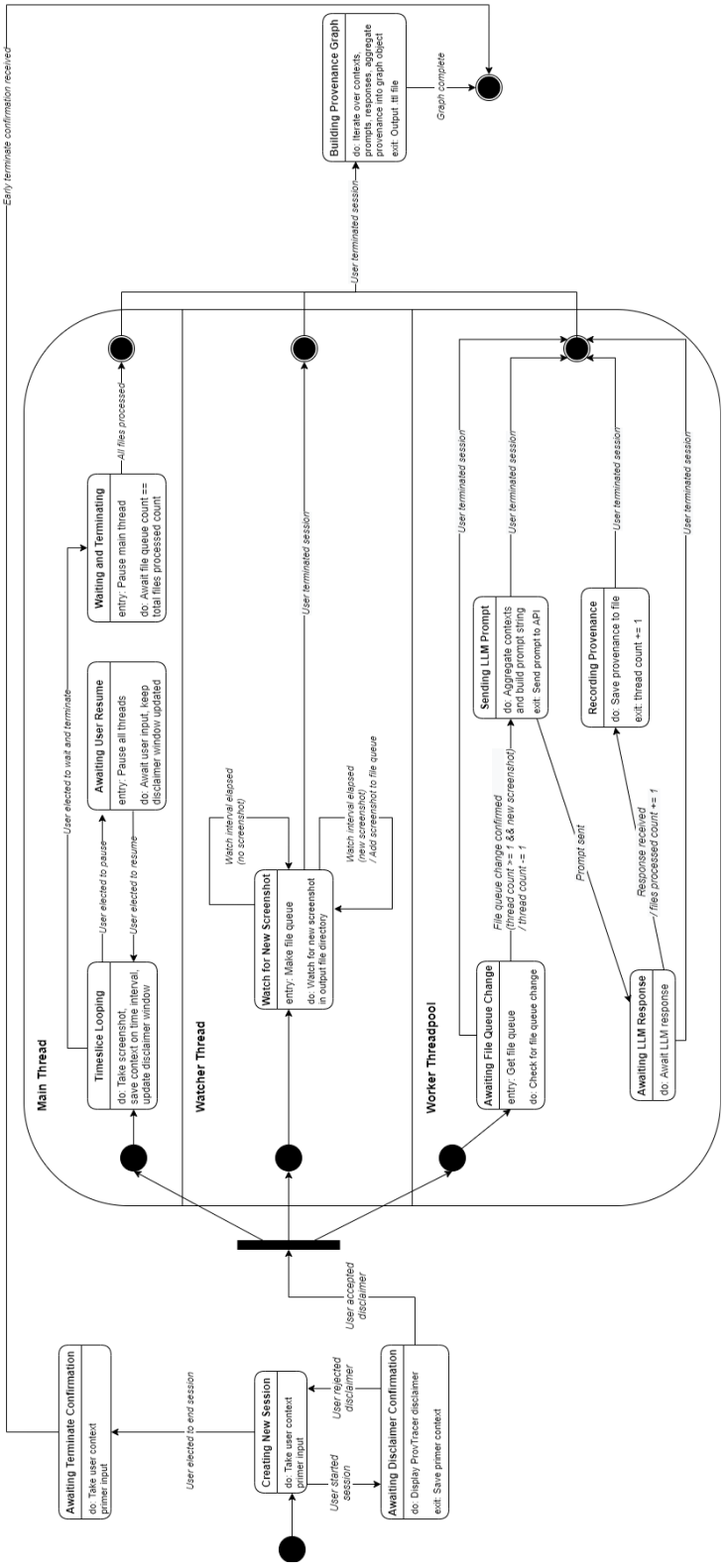

Fig. 37 Level 1 state chart of the ProvTracer system.

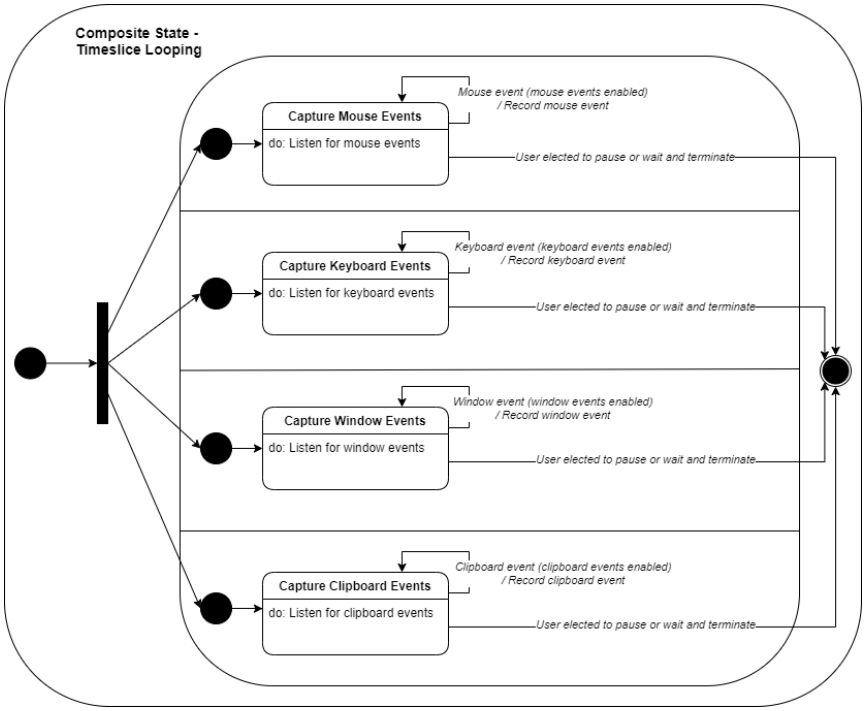

Fig. 38 Level 2 state chart of the ProvTracer system representing the timeslice looping composite state, which contains several event listeners.

4.2.3. UML CLASS DIAGRAM

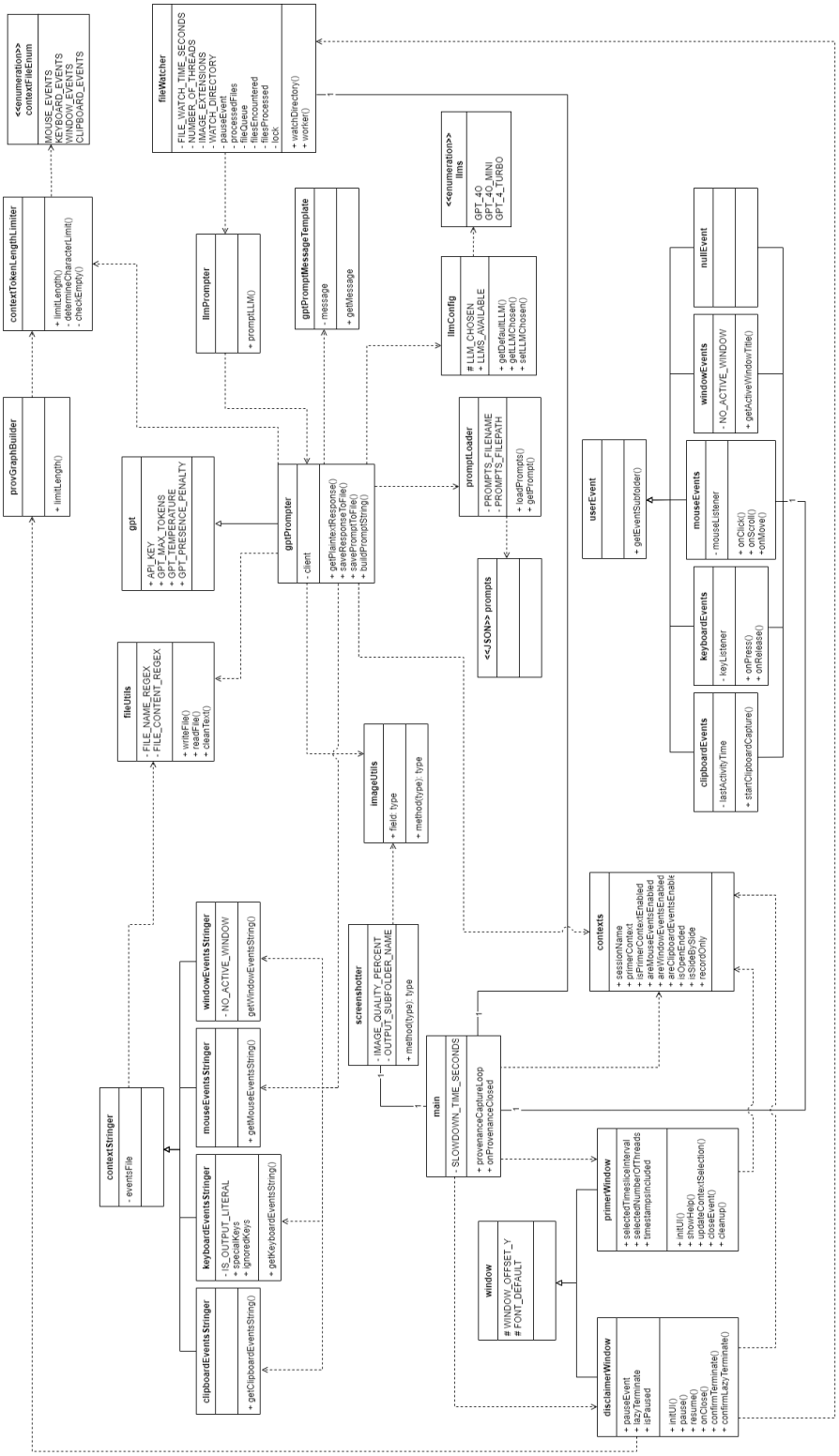

Fig. 39 ProvTracer class diagram.

TABLE VI ProvTracer context sources.

Context Source	**Definition**	**Notes**
Primer context	Initial chance for the user to inform the MLLM what their planned work is about	Limited to 350 characters
Mouse events	Mouse clicks since last screenshot	-
Keyboard events	Keyboard strokes since last event	Aggregated in pieces then actual string recreated for upload to MLLM
Window events	Window title changes since last screenshot	-
Clipboard events	Clipboard changes since last screenshot	Truncated to 500 characters to avoid MLLM token overrun

Fig. 39 presents a Unified Modeling Language (UML) class diagram for ProvTracer. For the sake of properly rendering the diagram within the present thesis artifact, some attributes and methods of the classes are ignored. The core functionalities and interrelationships of the classes are retained. The class titled *LLMPrompter* manifests the design pattern of a façade or wrapper, which encapsulate more complex function(s) under a simplified class and method(s) [229]. This is a means of future-proofing the MLLM agnosticism of ProvTracer, which currently employs GPT models from OpenAI.

4.2.4. CONTEXT SOURCES

There are five context sources available to prime and positively augment prompts to the MLLM with additional context. These are given in TABLE VI. Some of these affect MLLM response efficacy more than others; this is discussed in a later ablation study (see section 5.7).

4.2.5. WORK MODES

The ideal scenario for traceability is when some knowledge worker has a requirements source artifact, e.g., an SRS, open on one half of a screen, with development applications, e.g., an IDE, on the other. This would allow a view of both source and target and the best possible scenario for recording influencing provenance in any output work artifacts. To this end, the ProvTracer system allows users to select from an arbitrary bifurcation of work session modes. These are given below:

Fig. 40 Example of the two ProvTracer work modes: open-ended (top monitor) and side-by-side (bottom monitor).

- Open-ended: the work session will involve many applications, windows, tabs, etc., without any planned layout or order of use; i.e., the user will interact with their computer in an organic manner
- Side-by-side: the work session will involve only a source artifact and a working artifact, with each artifact taking up one half of the screen

See Fig. 40. By default, ProvTracer is set to open-ended mode, and all work can be performed in this mode without any special consideration. Side-by-side mode is introduced to determine if enforced screen semantics result in improved provenance trace output. The prompt component addressing side-by-side mode does not explicitly assert that one side of the screen or

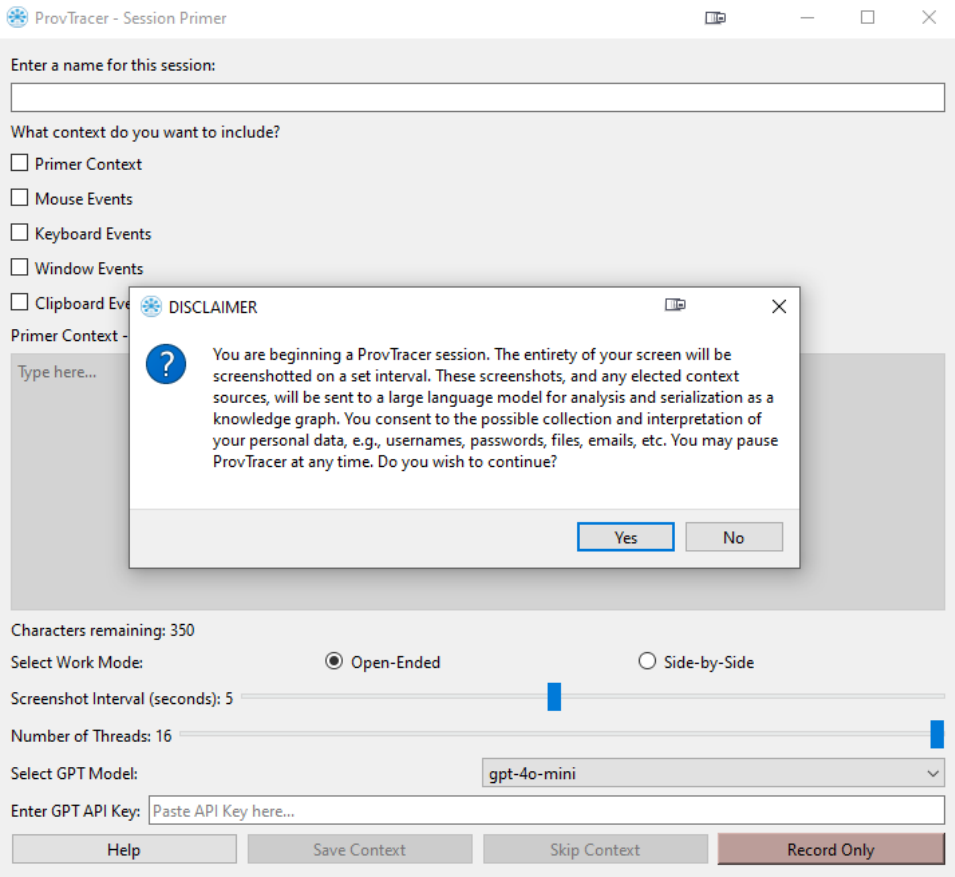

Fig. 41 Initial disclaimer popup window of ProvTracer.

the other, i.e., left and right, is the source or the working portion of the session. This is to retain work agnosticism, as different users exhibit different working patterns.

There is, obviously, no means of explicitly enforcing a ProvTracer user operating in side-by-side mode to maintain the two-part separation of source and work, so it is entirely volitional. Side-by-side mode is an "ideal" form for a provenance-considerate work session.

4.2.6. User Interface

The user interface (UI) of ProvTracer is constructed with the tkinter and PyQt6 libraries in Python. Aside from adjusting the box model and text labels to allow the UI layout to be usable, there is precisely zero code pertaining to "gold plating", i.e., unnecessary aesthetics or functionality

providing no real value to project goals [230]. In software engineering, gold plating is undesirable because (1) it wastes time and resources, e.g., financially and even computationally, during run-time, (2) it can introduce new bugs or risks because of complexity, (3) it may violate project scope, and (4) it may cause misalignment with end-user usability. ProvTracer is coded to be as lightweight and agnostic as possible, while meeting its objective of automatic provenance capture. A key component of the ProvTracer UI is discussed in the next section. There are three options to proceed to the looping component of ProvTracer:

1. Save Context: The ideal procession; saves any entered context and selected context sources to be sent alongside the screenshots to the MLLM
2. Skip Context: Skips all context and sends only screenshots to the MLLM
3. Record Only: Saves selected contexts (if any) and only records provenance and screenshots without prompting the MLLM

4.2.7. DISCLAIMER WINDOW

A key component of ProvTracer is the disclaimer window. Once primer information is entered by the user and they elect to continue, a general disclaimer popup window is displayed, informing the user of the level of access of ProvTracer. This establishes a baseline of informed consent for the user. See Fig. 41. For greater detail on the consideration of informed consent and its implications to the adoption of ProvTracer, see section 7.3.

Once the user elects to continue from this popup window, the ProvTracer system displays a persistent disclaimer window that maintains transparency between itself and the user, by literally declaring “You are being recorded”. See Fig. 42. This window is drawn on top of all other apps and remains slightly opaque to maintain the readability of other apps, becoming fully opaque when

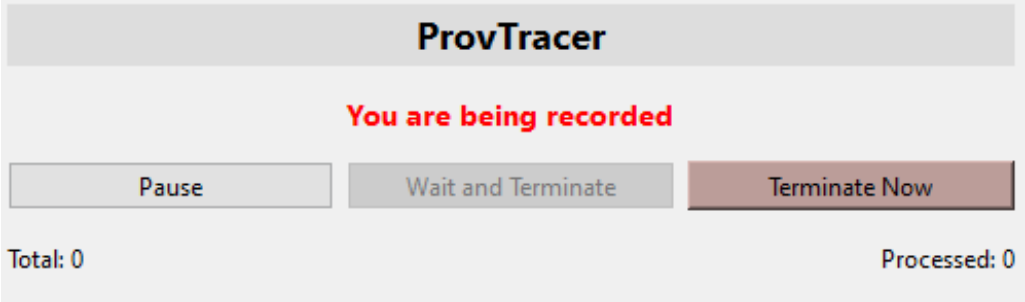

Fig. 42 The persistent disclaimer window visible while ProvTracer executes the provenance loop.

the user's mouse enters its boundaries. The bottom left of the disclaimer window persistently displays the total number of timeslices completed; the bottom right displays the number sent to the MLLM which have received a response and are accordingly stored as files in the output directory structure. These update in real-time unless the user elects to Pause, which will halt all execution. ProvTracer also includes the keyboard shortcut Ctrl + Shift + P, in place of the Pause button. The Pause button is included to mitigate concerns of privacy and security, which Thaul noted [18]. See section 7.3 for more details. If the user elects to Wait and Terminate, no new timeslices will be recorded, but all unprocessed ones will be finished as normal; upon completion of the remaining timeslices, the system terminates all execution.

4.2.8. Asynchronous Thread Pool

Every command executed in software is an occupation of resources. Employing the fewest resources possible to complete some desired task is the ideal for all software systems, i.e., reducing overhead is beneficial to the efficient execution of software systems. In ProvTracer, timeslice packet captures occur in adherence to a set interval while requests to the LLM take a variable amount of time. While the system could simply aggregate these timeslice packets and send them serially or as a batch, either during execution or after, the better use of computation and user time

is to process timeslice packets in real-time as a stream. Concurrency is therefore necessary to accommodate the asynchronous nature of the system.

The software design pattern of the thread pool was chosen as the model of concurrency for ProvTracer. This is because (1) timeslice packet captures occur on a set interval while (2) responses from the LLM vary due to the context included, prompt token length, hardware configuration, Internet connection, etc., and (3) thread pools are an efficient and compartmentalized means of completing multithreaded tasks. The task queue containing timeslice packets grows consistently over time, but the processing of timeslice packets takes some ambiguous amount of time because of MLLM latency. The thread pool allows for increased performance and low latency because threads are not being created and destroyed for relatively short-lived tasks [231]. With a thread pool, threads are created once up-front and simply re-used when needed and unencumbered; without a thread pool, thread genesis involves repeated run-time memory allocation and deallocation, which introduces overhead. The important technical factor to consider is optimal thread pool size.

In a concurrent setting, there are two means of setting the size of a thread pool: manually (or heuristically) and dynamically [232]. Manual thread pool size assignment is simpler, often done once pre-emptively but able to be changed manually later. Manual thread pool size assignment may result in suboptimal performance as there may be unallocated tasks that could otherwise be allocated if a greater number of threads were extant in the pool, or if there are too many threads available in the pool, and not enough tasks to occupy them. Dynamic thread pool size assignment typically involves adjustment based on the number of waiting tasks – more waiting tasks results in more threads in some pool, and fewer tasks results in fewer threads. Research has been conducted

on dynamic thread pool size assignment based on total average processing time (TAPT) and particle swarm optimization (PSO) [233]. For ProvTracer, it was decided that a manually determined thread pool size is the most lightweight approach, to account for diversity in personal computer central processing unit (CPU) architectures, and because the default screenshot interval is five seconds, which, in terms of computational speed, is very slow. Research on thread pool optimization is often specific to run-time systems and enterprise software ecosystems considering tens or hundreds of threads within pools [232, 233], so for the lightweight ProvTracer system executing on the individual knowledge worker's computer, a middle-ground approach to thread pool size assignment is used.

By default, ProvTracer sets the thread pool size to be equal to the number of CPU cores on the host machine. This is so that the program can make the most of parallelism while avoiding the overhead of context switching that may occur when too many threads compete for the same core. The session setup window of ProvTracer includes a slider for setting the thread pool size manually, so the individual knowledge worker has greater control over what is deemed necessary.

4.2.9. PROMPT STRUCTURE

To interface with the MLLM, ProvTracer stores prompts as a JavaScript Object Notation (JSON) array of prompt sections. These sections are accessed programmatically as needed based on user selection in the primer window. TABLE VII contains a breakdown of ProvTracer's MLLM prompt structure. With the exception of the preface, which is always sent along with a MLLM request,

TABLE VII Prompt structure breakdown for ProvTracer's MLLM integration.

Prompt Section	Used?	Content
Preface	Always	The provided image is a screenshot of a user's computer as they are working. Describe what they are doing in detail. Do not use newlines. Avoid describing personal information, such as passwords. Answer in 3-5 sentences.
Open-Ended Transition	If selected	The user has stated that their screen use is open-ended, i.e., they may use many windows in various areas of the screen.
Side-by-Side Transition	If selected	The user has stated that one side of the screen is the 'source' artifact or artifacts, and the other side is the working area. In general, the 'source' artifact(s) influence(s) the development of the working side, which may include several artifacts.
Context Transition	If 1+ context source selected	Below is some additional context provided by the user.
Primer Context Modifier	If primer context selected	Here is what the user stated they were working on when this screenshot was taken:
Window Events Modifier	If window events selected	Here is a list of application window titles that the user opened since the last screenshot:
Mouse Events Modifier	If mouse events selected	Here is a list of user mouse events since the last screenshot:
Keyboard Events Modifier	If keyboard events selected	Here is a list of user keyboard events since the last screenshot. Compare what they typed with what is present in the screenshot:
Clipboard Events Modifier	If clipboard events selected	Here is what the user copied/pasted into their clipboard since the last screenshot. It is trimmed to 500 characters:

each prompt section is aggregated together in order, if selected by the user, forming a large, contextual prompt.

4.2.10. Default Values

The previous sections described, in parts, the default, or assumed, values utilized in ProvTracer. TABLE VIII defines the default values used in ProvTracer.

4.2.11. Output Directory Structure

ProvTracer records several context sources from the knowledge worker's computer, generates LLM prompts by aggregating them, and receives LLM responses. All of these dynamically generated components are not lost during runtime, as they are preserved within an output directory structure. ProvTracer knowledge graphs employ consistent use of the XSD (XML Schema Definition) namespace, specifically, the *xsd:dateTime* term, in Turtle, to preserve timestamps. In

TABLE VIII Default values used in ProvTracer and the reasoning behind them.

Attribute	**Value**	**Reasoning**
Timeslice	5 seconds	Appropriately sized chunk for human interaction with a computer interface
Thread pool size	Number of CPU cores	Gains benefit of multithreading without overhead of context switching
MLLM	GPT-4o-mini	Fastest and cheapest model of the three chosen
Work mode	Open-Ended	Most work is too organic and user variance is too widespread to enforce the strict separation of screen content

combination with the output file preservation, this timestamping is done so that later analysis may incorporate these source artifacts, as desired. The output directory structure is given below:

- contexts/
  - clipboard_events/
  - keyboard_events/
  - mouse_events/
  - window_events/
  - *primer_context.txt*
- prompts/
- responses/
- screenshots/

Each of the subdirectories contains a number of context, prompt, response, or screenshot files, respective to the aforementioned directory structure, of count n, depending on how long the ProvTracer system is active for during a given work session. To retain some basic metadata, each of the filenames includes a datetime of capture, e.g., *screenshot_year_month_day_hour_minute_second.png*. If some context source is not selected by the user, e.g., primer context, then *primer_context.txt* will not be extant; the same applies to the four event type sub-directories. If the user elects to use ProvTracer to only record their work session, then the prompts/ and responses/ subdirectories will not be created, nor will any MLLM use occur. The contents of this directory structure are, upon work session completion, transformed

into a provenance knowledge graph adhering to PROV-O and BFO; see section 3.4.1 for a definition of the ontology used. This knowledge graph is output alongside the stated sub-directories, e.g., as *year_month_day_hour_minute_second_name_provenance_traces.ttl*, where the *name* sub-string is set by the user in the primer window, otherwise it defaults to “unnamed”.

Chapter 5: ASSESSMENT

Due to the limited timeframe and lack of external participants, the evaluation of ProvTracer is framed within the context of exploratory systems research; therefore, evaluation focuses on output generated by the system, with qualitative inspection. With a combination of (1) grounded theory analysis of trace descriptions, (2) an ablation study to determine which context sources most affect output quality, and (3) visualizations of provenance trace links, ProvTracer is interrogated qualitatively in parts. Two case studies are employed to generate ample ProvTracer output for qualitative analysis. This approach aligns with established methods in HCI tool design, where system expressiveness and design rationale may take precedence in evaluation over user studies [234].

5.1. EMPIRICAL SOFTWARE ENGINEERING AND GROUNDED THEORY

The empirical evaluation and reporting of software processes, methods, and tools is an effort of empirical software engineering (ESE). ESE has been extant since at least 1985, with Basili, Selby, and Hutchens' technical report, *Experimentation in Software Engineering*, which presents a framework for applying the experimental method in software engineering [34]. Basili noted the difficulty of the human element in practically forming an experimental method in the software engineering domain, as opposed to the longstanding experimental methods present in natural and social sciences, e.g., physics and psychology [235]. Wohlin et al. present the landmark book, *Experimentation in Software Engineering*, that provides software practitioners engaged in experimental work with an exhaustive resource on the application of basic scientific analysis, e.g., empirical studies, to the domain of software engineering, which, until its publication, was

fragmented [33]. An early paper discussing quality software engineering research centers posits general software research question archetypes, research result themes, and validation techniques, many of which are qualitative or descriptive [236].

Early evaluation of developed software systems is typically a practice in data collection and qualitative analysis, e.g., through interviews or surveys of usability, satisfaction, etc. [237]. Qualitative analysis, as opposed to quantitative analysis, is inductive, not deductive; dialectic, not linear; it is cyclical and evolving, a search for coherence within ambiguity [238]. As the output of ProvTracer is textual descriptions of provenance, there is no readily available quantitative evaluation technique, so qualitative methods must be employed. It is posited that grounded theory [239] is a suitable framework for evaluating ProvTracer, because this method allows structure and insight to emerge from the data without presupposing a formal interpretation framework. Given the absolute novelty of automatically generated provenance trace link descriptions, and the absence of prior taxonomies for such output, grounded theory provides a principled way to iteratively identify meaningful patterns, insights, and themes from ProvTracer trace links. Grounded theory mirrors the exploratory nature of ProvTracer: both are grounded in observation and generative in intent.

In the present dissertation, grounded theory was operationalized through the three-stage coding approach articulated by Charmaz [239] to derive theoretical propositions about ProvTracer's MLLM dependence:

1. Open coding: recurring terms in provenance descriptions are grouped into conceptual codes

2. Axial coding: open codes and concepts are refined by clustering them into related concepts as broader thematic categories
3. Selective coding: these themes are distilled into cohesive theoretical propositions about ProvTracer's MLLM generations

This approach is used to evaluate the output of two real-world case studies employing ProvTracer as an observer. Herein, grounded theory enables a structured yet flexible analysis of a novel provenance representation form, aligning with the principles of exploratory, empirical software engineering.

5.2. PROVTRACER OUTPUT

A more open concern of ProvTracer comes after the system has performed its primary task, which is providing trace link KGs; i.e., what is one to *do* with the automatically recorded artifact provenance? Effectively disseminating, or *using*, created provenance is always a practice in establishing use cases particular to some business or personal need; there is no silver bullet for provenance use. As Simmhan et al. note, there are three primary means of using provenance: (1) visual graph, (2) queries, and (3) service API [19]. More recently, Pimentel et al. mirror this in their large survey of script provenance, providing taxonomies for collecting, managing, and analyzing, or *using*, provenance, noting three means: (1) query, (2) visualization, and (3) comparison [240].

As discussed previously, artifact provenance provides traceability and explainability, engendering transparency and interoperability, thereby establishing trust, auditability, data preservation, etc. When software artifacts are accompanied by their attendant provenance knowledge graphs, there are a number of activities that motivated parties can partake in:

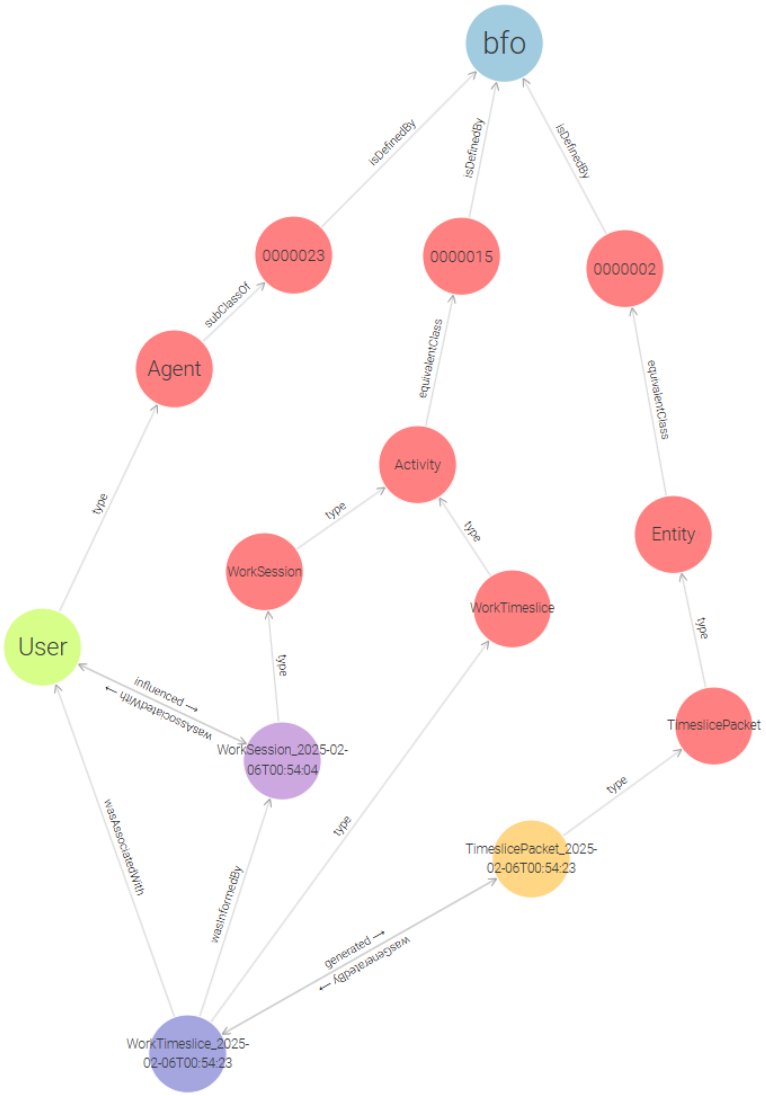

Fig. 43 Example of a single Work Timeslice output from ProvTracer.

- Data lineage visualization: knowledge graphs are easily visualized, providing users with artifact history and evolution
- Automatic compliance checks: artifacts attended with provenance graphs can more easily be audited for compliance against regulations or standards
- Worker knowledge representation: as software developers contribute to projects and associate themselves with more provenance graphs, their skills, interests, and capabilities

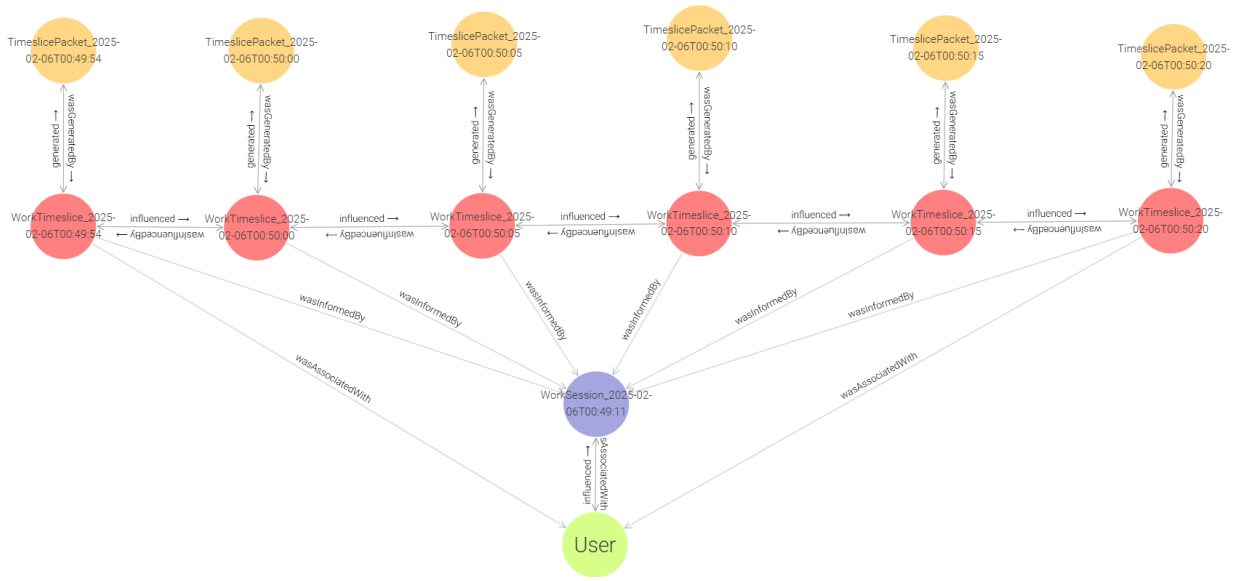

Fig. 44 A ProvTracer timeslice chain.

are serialized, and project managers can query these to assign developers to appropriate roles

The second half, as it were, of the developed system, involves means of searching and interrogating the generated provenance knowledge graphs for insight. Knowledge graphs can be queried, either with a query language, e.g., SPARQL Protocol and RDF Query Language (SPARQL), or by incorporating them, or fragments of them, in prompts to a LLM; this can reveal patterns and dependencies useful for individual developers, project managers, Agile teams, etc.

ProvTracer provenance trace link knowledge graphs are visualized using GraphDB. Fig. 43 contains a single Work Timeslice output, including its connection to the PROV-BFO hierarchy described in section 3.4.1.

A collection of ProvTracer Work Timeslices form a chain of Work Timeslices, connected by the PROV-O relations, *prov:influenced* and *prov:wasInfluencedBy*. Fig. 44 contains an example

of a ProvTracer provenance trace link chain. A real, example Timeslice Packet, generated by ProvTracer, is given below in Turtle format:

```
< https://github.com/PR0CK0/ProvTracer/tree/main/procko.provtracer/grapher/provtracer-o.ttl#
TimeslicePacket_2025-03-26T00:03:57> a pto:TimeslicePacket ;
    pto:prompt "The provided image is a screenshot of a user's computer as they are working. Describe what they are
doing in detail. Do not use newlines. Answer in 3-5 sentences. The user has stated that their screen use is open-
ended, i.e., they may use many windows in various areas of the screen." ;
    pto:response "The user appears to be coding in Python, working on a project related to data provenance tracking
as indicated by the visible files and code snippets. They are using an integrated development environment (IDE)
that displays various programming files, with a prominent focus on a script named `main.py`. Within this script,
the user is likely debugging or running a program that interacts with keyboard and clipboard events, as shown by
the related imports and functions. Additionally, there's an open README file that details project requirements and
usage instructions, suggesting that the user is in the process of setting up or troubleshooting aspects of the
project. The terminal section at the bottom shows output logs related to the execution of the code, indicating they
are actively monitoring the program's performance and any errors that may occur." ;
    pto:screenshot "screenshot_2025-03-26_00-03-57.png" ;
    prov:generatedAtTime "2025-03-26T00:03:57"^^xsd:dateTime ;
    prov:wasGeneratedBy <http://example.org/terms#WorkTimeslice_2025-03-26T00:03:57> .
```

The associated Work Timeslice of the previous Timeslice Packet is given below, in Turtle format:

```
<https://github.com/PR0CK0/ProvTracer/tree/main/procko.provtracer/grapher/provtracer-o.ttl#
WorkTimeslice_2025-03-26T00:03:57> a pto:WorkTimeslice ;
    prov:generated <http://example.org/terms#TimeslicePacket_2025-03-26T00:03:57> ;
    prov:influenced <http://example.org/terms#WorkTimeslice_2025-03-26T00:04:02> ;
    prov:startedAtTime "2025-03-26T00:03:57"^^xsd:dateTime ;
    prov:wasAssociatedWith pto:KnowledgeWorker ;
    prov:wasInformedBy <http://example.org/terms#WorkSession_2025-03-26T00:03:41> .
```

5.3. QUERYING OUTPUT GRAPHS

As established, there are two means of querying provenance trace link knowledge graphs output from ProvTracer, (1) using SPARQL and (2) in natural language, with a LLM, utilizing the RAG

technique. With SPARQL, several possible quantitative analyses can be performed with ProvTracer graphs, because of the proper use of timestamping with *xsd:dateTime*:

1. Time-bounded filtering: Provenance events can be queried within specific time windows, during specific sessions, etc.
2. Temporal distance between events: Durations between events can be calculated, e.g., times between file edits, times between tasks, etc.
3. Rate of activity over time: Queries can be written to count how many events occurred per hour, day, etc., detect idle versus productive periods, etc.
4. Temporal provenance windows: Windows around key events can be queried, e.g., "what happened in the 10 minutes prior to this commit?"

For instance, the following query finds all timeslice packets between noon and 1 PM on a particular day (the prefixes are omitted for brevity; *pto* is the namespace for the ProvTracer ontology):

```
SELECT ?packet ?timestamp
WHERE {
  ?packet a pto:TimeslicePacket ;
          prov:generatedAtTime ?timestamp .
  FILTER (
    ?timestamp >= "2025-03-22T12:00:00"^^xsd:dateTime &&
    ?timestamp <= "2025-03-22T13:00:00"^^xsd:dateTime
  )
}
ORDER BY ?timestamp
```

Querying output provenance trace link knowledge graphs with a LLM in natural language is more intuitive and better suited to more qualitative questions; however, as established throughout the present dissertation, LLM prompting suffers from hallucination, even with a structured document being utilized as a RAG corpus, so this method of querying must be performed cautiously and, ideally, in tandem with manual lexical searching of output knowledge

graphs, or with SPARQL, to verify the veracity of output information. In order to assess ProvTracer provenance output quality using grounded theory, LLM prompting is used in the next two sections containing case studies of actual ProvTracer use in real-world software development.

5.4. CASE STUDY: PYTHON PROGRAMMING

As a case study of the ProvTracer system, ProvTracer was run for a period of approximately 30 minutes while the user worked within the Visual Studio Code IDE, working on a series of Python classes for a project centering around string manipulation. The entire case study can be found in the GitHub repository of ProvTracer: https://github.com/PR0CK0/ProvTracer/case_studies/1_python_programming. Fig. 45 shows the primer window selections for this case study. After running ProvTracer for the duration of 30 minutes, it is possible to quantify some aspects of the output graph with SPARQL:

- Total timeslice packets: 298
- Total elapsed time: 25 minutes 2 seconds
- Total keyboard events: 224
- Total mouse events: 28
- Total window events: 13
- Total clipboard events: 2

There is very little mouse activity inasmuch as programming within an IDE can be performed with the keyboard, and navigation specifically with the arrow keys; window events were minimal as the IDE was the only application employed, and clipboard events were the least utilized context source, which is expected, due to the low frequency of use in general and when compared to, e.g., keyboard strokes.

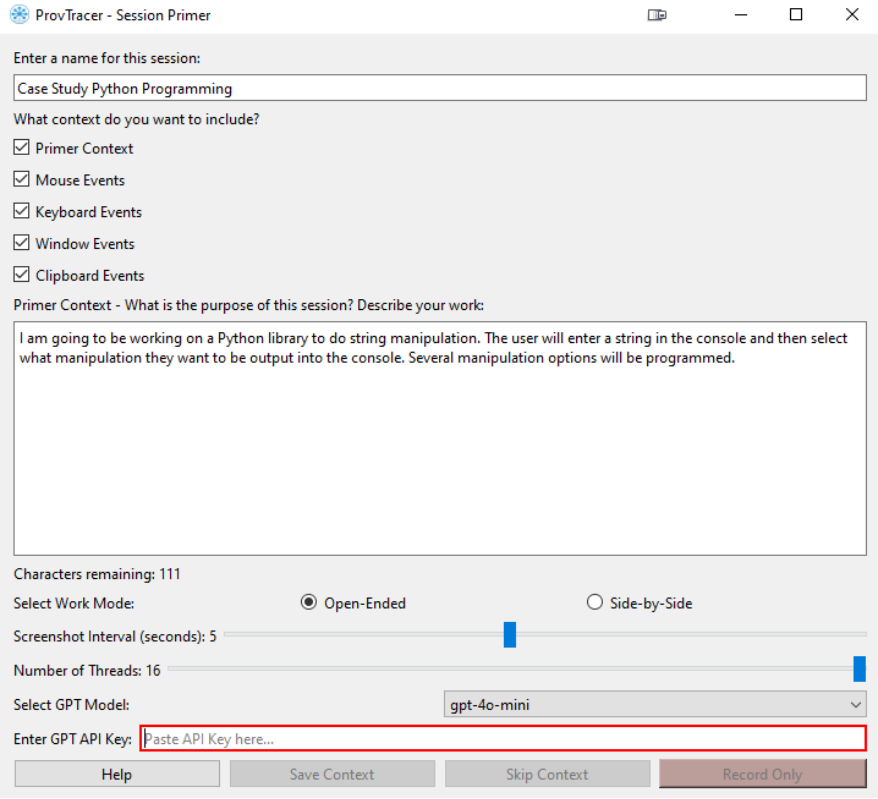

Fig. 45 The primer window selections for the first ProvTracer case study involving Python programming.

From the wordcloud of Fig. 46 and manual lexical analysis of the output provenance descriptions, it is possible to begin a grounded theory analysis of the MLLM generations underlying ProvTracer. First, open coding is employed by grouping terms with concepts, then axial coding is used to group these codes into broader categories:

1. Axial code: User Agency and Intent Perception
    a. Example terms: user, actively, appears, engaged, currently, enter, interacting
    b. Concept: Descriptions frequently frame the user as a central, active agent engaged in purposeful behavior. This suggests that the MLLM chosen tends to adopt the role of an observer.
2. Axial code: Development Environment Awareness
    a. Example terms: python, code, script, function, console, editor, visible

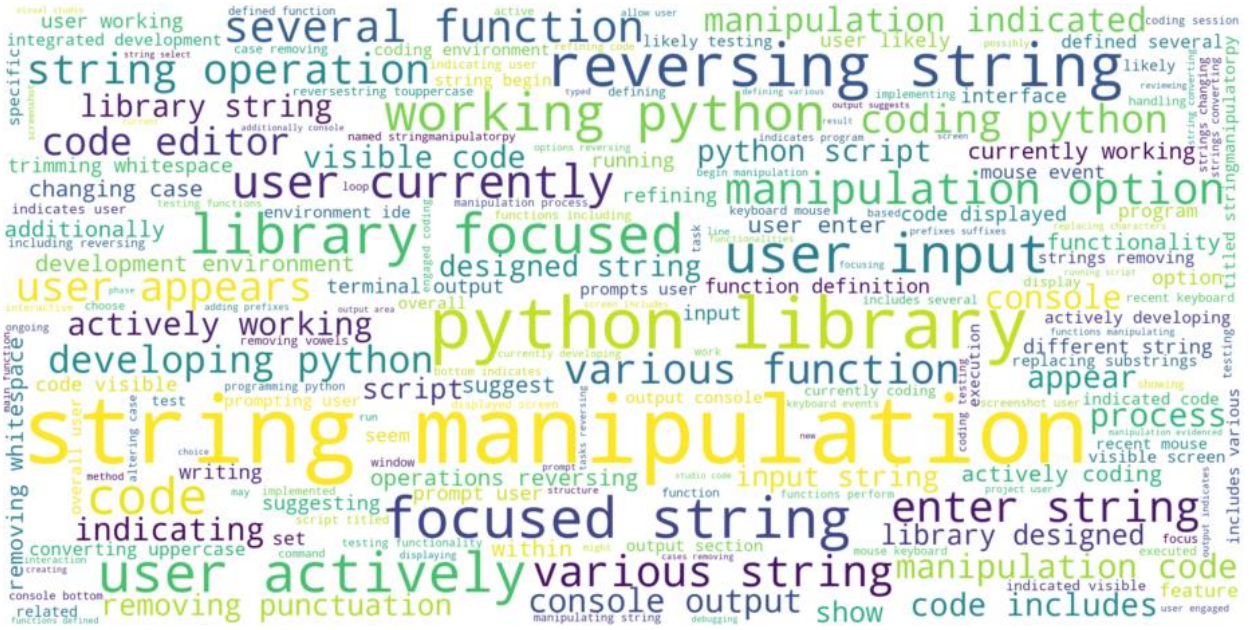

Fig. 46 Wordcloud of MLLM-generated provenance descriptions for the first case study involving a user programming a string manipulation program in Python.

   b. Concept: Strong indicators of the development context; the chosen MLLM routinely identifies high-level cues, e.g., language, library usage, etc.

3. Axial code: Task Semantics Recognition
   a. Example terms: string, manipulation, substring, reversing, changing case, removing vowels, inserting dashes
   b. Concept: A session's task (in this case, string manipulation, stated explicitly in the primer context) seems to be surfaced by the chosen MLLM.
4. Axial code: Feature Utilization Framing
   a. Example terms: library, designed, visible, output, includes, functionality
   b. Concept: The chosen MLLM reflects on the capabilities of the code and libraries employed; this may support understanding how the user is working.
5. Axial Code: Interaction Loop Detection

a. Example terms: console, terminal, prompting, output, interface
b. Concept: The chosen MLLM is good at summarizing interaction patterns, especially with visible system behavior or response.

From these axial codes derived from an application of grounded theory, it is possible to posit a selectively coded theory of ProvTracer's ability in the context of Python programming:

In the context of programming, MLLM-generated provenance descriptions in ProvTracer abstract low-level interface events into high-level user intentions by emphasizing semantic operations and inferred actions, constructing narratives that anthropomorphize the user and prioritize task-oriented reasoning over raw interaction data, while being limited by a lack of spatial awareness.

This grounded theory perspective may indicate that ProvTracer, because of its dependence on a MLLM, does more than simply capture activity – it may be said to actually interpret it. The system consistently elevates raw interface interactions into meaningful task descriptions, often highlighting *why* a user is doing something, not simply *what*. This narrative construction positions the chosen MLLMs as intelligent (within the bounds of computation) observers, capable of inferring intent, summarizing tool and application usage, and identifying semantic patterns in the context of programming tasks. However, the analysis reveals current limitations: spatial context is rarely referenced, and ambiguous input can lead to interpretive overreach, i.e., hallucination. These insights provide a foundation for improving MLLM integration in provenance modeling.

5.5. Case Study: Code Documentation

As a second case study of the ProvTracer system, ProvTracer was run for approximately 20 minutes while the user worked in several applications, focusing on the creation of a README

document for the previously programmed Python library for string manipulation. As program documentation is a common practice within the SDLC, this was chosen as a representative use case for the furthered qualitative evaluation of ProvTracer. The entire case study can be found in the GitHub repository of ProvTracer: https://github.com/PR0CK0/ProvTracer/case_studies/2_readme_creation. Fig. 47 shows the primer window selections for this case study. After running ProvTracer, it is possible to quantify some aspects of the output graph with SPARQL:

- Total timeslice packets: 376
- Total elapsed time: 19 minutes 8 seconds
- Total keyboard events: 336
- Total mouse events: 136
- Total window events: 27

Total clipboard events: 10

There is more mouse, window, and keyboard activity than with the prior case study in Python programming, because the documentation work employed the use of external resources about Markdown, and much boilerplate Markdown content was copied-and-pasted to save time.

From the wordcloud of Fig. 48 and manual lexical analysis of the output provenance descriptions, a grounded theory analysis of the MLLM generations underlying ProvTracer is performed:

1. Axial code: Documentation Structure Awareness
    a. Example terms: readme, markdown, formatting, content, file, heading, table, layout

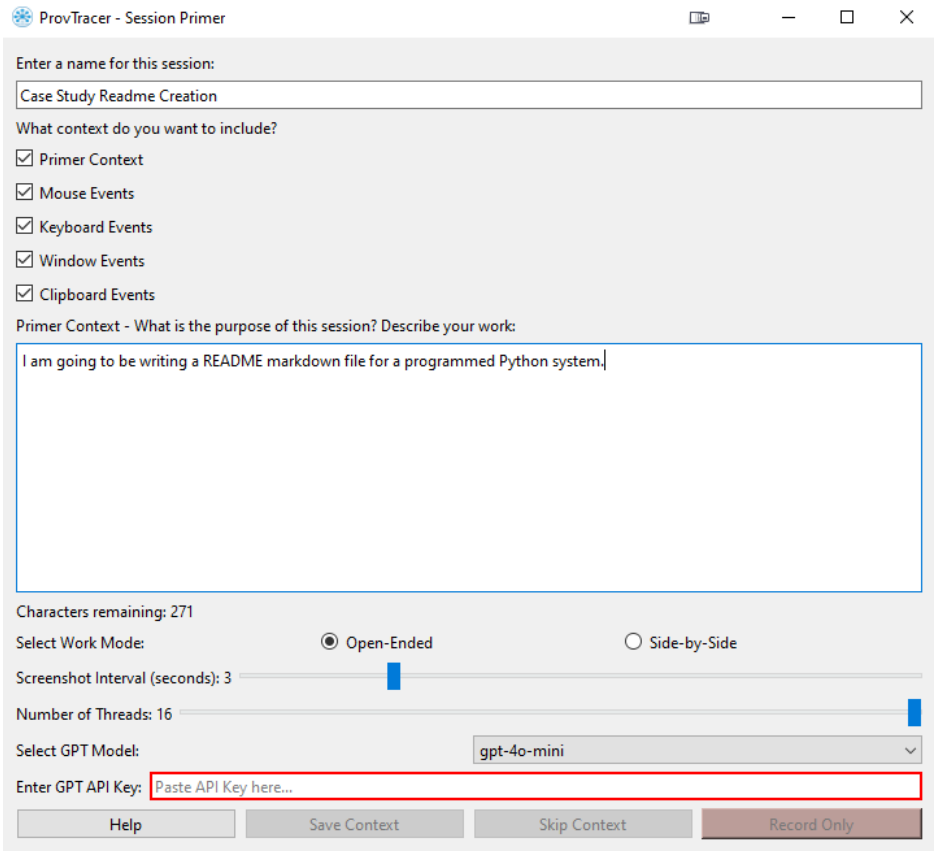

Fig. 47 The primer window selections for the second ProvTracer case study involving README creation for a programmed Python system.

b. Concept: The chosen MLLM understands that the user is constructing structured documentation; it accurately picks up on formatting constructs, e.g., section headers and documentation segmentation.

2. Axial code: Feature Description Framing
    a. Example terms: features, functionality, usage, description, instructions
    b. Concept: The chosen MLLM recognizes that the user is listing and elaborating on software features; this suggests some understanding of the rhetorical nature of documentation, i.e., explaining what some code does and how it functions.
3. Axial code: Task Intent Recognition

Fig. 48 Wordcloud of MLLM-generated provenance descriptions for the second case study involving a user creating a Markdown README file for a coded Python system.

   a. Example terms: working, creating, writing, focused, currently working, appears

   b. Concept: The chosen MLLM continues to anthropomorphize and assign intent to user actions; it may be said to "see" the user as purposefully constructing a README, not just typing randomly.

4. Axial code: Tool and Environment Contextualization

   a. Example terms: python, project, string, program, tool, system, screen

   b. Concept: The chosen MLLM seems to realize context clues to then infer that the README relates to a specific Python project, often referring back to the codebase.

5. Axial code: Instructional Tone Detection

   a. Example terms: guide, installation, usage, command line, indicate, ensure, include

b. Concept: Many of the provenance descriptions seem to possess an understanding that the user is writing instructional content, reflecting that the chosen MLLM may have awareness of common documentation tropes.

From these axial codes derived from an application of grounded theory, it is possible to posit a selectively coded theory of ProvTracer's ability in the context of Python programming:

In the context of code documentation, the provenance descriptions generated by ProvTracer's chosen MLLM infer higher-order tasks of instructional writing, extrapolating semantic structure, e.g., features, usage, installation, and rhetorical intent, e.g., guiding, informing, and explaining, from low-level context sources. The framing of the MLLM seems to depend on structural cues, e.g., markdown syntax and keywords, to generate summaries that mirror conventional documentation practices.

This case study demonstrates that the generated provenance descriptions of ProvTracer are not only reactive to user interface events, but are also reflective of genre-specific writing patterns, e.g., instructional writing. The grounded theory analysis suggests that the chosen MLLM can function for various task types, e.g., both algorithm implementation and instructional documentation, and that the model is capable of adapting its narrative frame accordingly. Using cues like headings, the provenance descriptions reconstruct user activity as a structured, goal-oriented process. This lends credence to the capability of the model to infer semantic rationale and intent, even when direct code execution is absent; in turn, this reaffirms the suitability of ProvTracer for capturing the provenance of diverse software development activities.

5.6. COMBINED THEORETICAL PROPOSITION ON PROVTRACER'S EFFICACY

As a general observation, the MLLMs evaluated tend to use "framing" verbs, e.g., *appears, indicating, suggesting, focused,* etc., which seem to be words employed to hedge or otherwise hypothesize about user behavior that is not concretely provable given a screenshot and some associated context. Granting this a general theoretical proposition on ProvTracer's efficacy as a provenance capture engine can be posited:

Across various SDLC tasks, the MLLM-generated provenance descriptions of ProvTracer abstract low-level user behavior into meaningful narratives by leveraging contextual cues to infer task type, user intent, and domain structure, using hedging language to frame these inferences with interpretive caution.

5.7. ABLATION STUDY: CONTEXT SOURCES

As detailed in an earlier section on the implementation of the system, there are a number of input context sources that can be used to augment the underlying screenshot provenance. The present section contains the description of a small ablation study [241] wherein the individual context sources are apprised as to their contribution to the quality of provenance outputs by systematically removing them; refer to section 4.2.4 for more details on the context sources implemented in ProvTracer. The study is broken apart into three steps:

1. Baseline: no context, simply screenshots included in timeslice packets
2. Systematic addition: adding each context type individually (while not addressing every combination of context types) to timeslice packets
3. All context: all context types included in timeslice packets

TABLE IX Context sources and their importance to ProvTracer output.

Context	**Importance**	**Reasoning**
Primer context	High importance for any task	Provides the MLLM with a general understanding of the intent of a work session
Mouse events	Low applicability	The chosen MLLMs cannot properly make use of spatial context (see sections 4.1.2 and 4.1.3)
Keyboard events	High importance for any task	Provides a simplistic "delta" for text from the previous timesliced screenshot
Window events	High importance for any task, particularly those employing multiple application windows	Provides general semantic cues for the work being performed
Clipboard events	High importance for particular tasks, e.g., those with large amounts of boilerplate or repeated text that is copied-and-pasted to save time	Provides a direct working cue for the work being performed

For each configuration, output provenance traces are qualitatively analyzed with respect to their usefulness in a manual fashion, by inspecting the output MLLM responses. TABLE IX summarizes the results of the ablation study. Ideally, every combinatoric possibility of the five context sources would be tested, resulting in 32 independent context source sets ($2^5 = 32$), but such exhaustive evaluation is beyond the scope of the present dissertation given practical constraints on time and resources. This short ablation study establishes key insight into the usefulness of each context source, with primer context demonstrating the most value to the MLLM.

An additional ablation study that may provide insight into MLLM efficacy would seek to establish the effect of the screenshot interval as it is changed. This is not performed in the present work, but a discussion on the temporal limitations of ProvTracer is given in section 7.2.3.

5.8. EVALUATING PROVENANCE NFRS

NFRs describe how, rather than what, a system accomplishes its objectives. For provenance capture, downstream NFRs for SDLC tasks include explainability, traceability, and interoperability. These are critical to address, particularly with automated provenance capture. The following sections discuss how ProvTracer addresses these key NFRs.

5.8.1. ADDRESSING TRACEABILITY

Traceability is a foundational NFR in provenance systems and refers to the ability to trace the path of actions and artifacts through a system. ProvTracer explicitly supports traceability at multiple levels. At the structural level, the use of the PROV-O ensure that each *prov:Entity* is linked to a *prov:Activity*, and that activities are temporally linked through the *prov:wasInformedBy* and *prov:influenced* relationships. These temporal chains allow an interested user to trace the evolution of a workflow. At the semantic level, traceability is enhanced by embedding high-level provenance summaries in the form of natural language descriptions into each timeslice. When paired with the structure of the underlying knowledge graph, this enables hybrid traceability: an interested user can trace both *what* happened, via entities and relations, and *why* it likely happened, via MLLM responses. Furthermore, timestamped identifies and *prov:generatedAtTime* annotations allow the querying of trace data using SPARQL, as demonstrated in sections 5.3, 5.4, and 5.5.

In terms of practical traceability engineering, requirements satisficing may be done as simply as employing a spreadsheet, where each requirement is given a row, followed by a checkbox representing incompletion/completion, a cell for its source, i.e., where the requirement was stated, and a cell for its implementation location, e.g., a particular section of code in some source file. Such a method can be linked with ProvTracer's own output, by connecting specific

provenance assertions to known requirements, e.g., via the *prov:qualifiedAssociation* relationship or custom terms, thus facilitating semi-automated tracing. Altogether, ProvTracer enables both human-readable and machine-queryable traceability in digital workflows.

5.8.2. Addressing Explainability

Explainability refers to the degree to which a system's behavior and outputs can be understood by a human operator. This NFR is particularly relevant in provenance systems, where the ability to interpret how artifacts were produced is central to a great number of downstream tasks. ProvTracer addresses explainability by integrating a natural language interface layer, to wit, a MLLM, to summarize system activity timeslices based on screenshots and chosen context sources. These descriptions are not opaque logs, but human-readable summaries.

While this introduces challenges related to consistency and factual grounding (see section 2.9.1 on hallucination), the ability to query output provenance trace link knowledge graphs in natural language allows domain experts, stakeholders, or end users to interpret work sessions at a glance, without needing to reconstruct event histories manually. Additionally, because the prompts that elicited output descriptions are included as part of traces, an interested user can inspect the reasoning context supplied to the MLLM during generation, effectively exposing the *why* behind each summary.

5.8.3. Addressing Interoperability

Few quantitative interoperability measures exist; most are quantitative and stem from the defense sector, with particular emphasis on physical, deployed systems [242]. The ProvTracer system developed retains interoperability at the data, or terminological, level with other provenance systems. Quantifying the interoperability of two systems with some measure is not trivial [92, 94],

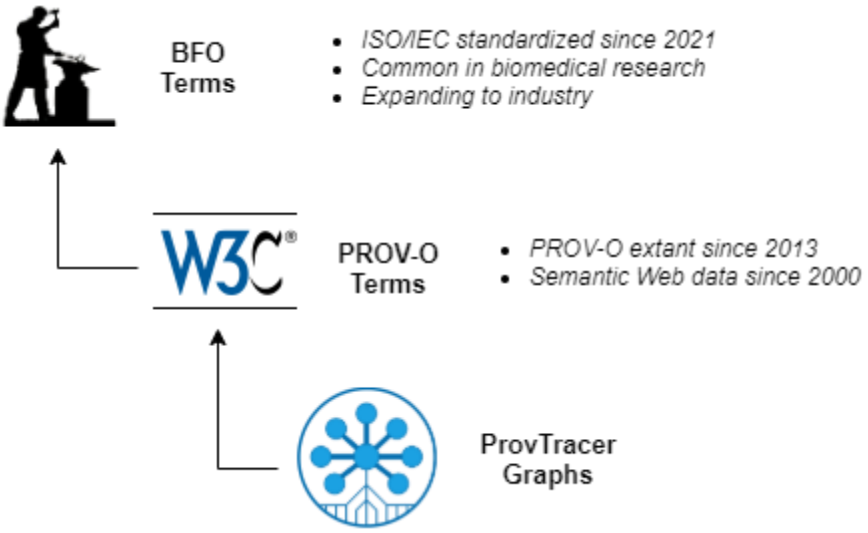

Fig. 49 Interoperability hierarchy of ProvTracer.

so what follows is a short discourse on the evolving nature of ProvTracer's interoperability with emerging provenance systems in the face of large-scale generative AI, as a result of its use of PROV-O and BFO.

Johns et al. provide a systematic survey of provenance capture technologies in the biomedical research domain [243]. 41% of abstract data models surveyed were graph, 18% of implemented data models were in RDF, with 23% adhering to the PROV data model and 7% adhering to the OPM. As the OPM is effectively a precursor to PROV, it can be extrapolated from the survey that, approximately 30% of provenance representation efforts in the domain of biomedical purport to be compliant with the W3C's PROV standard. Pimentel et al. present a survey on provenance capture and analysis techniques for scientific experiment scripts; they reveal that a small amount of the surveyed techniques employ the PROV-O, the OPM, RDF/RDFa, or graph constructs [240].

The ProvTracer system developed herein uses PROV-O as its data model. Given this, it may be speculatively asserted that the ProvTracer system is interoperable at a terminological level

with approximately one-third of provenance systems in the biomedical domain, and a burgeoning percentage of scientific workflow provenance systems. This is likely to improve over time, as there have been recent calls for the use of standard provenance models, e.g., PROV-O, because of the rapid expansion of massive AI models, datasets, and ML workflows [244]. As example, in the domain of scientific ML, the Provenance-based Holistic Data View of the Lifecycle of Scientific ML (MLHolView) knowledge graph, compliant with the PROV data model, has been shown to be scalable in an oil and gas use case within a HPC environment [245]. Data provenance is important; of deeper importance is the use of a proper standard for representing such provenance to engender widespread adoption. As Longpre et al state: "[There is an] urgent need for a standardized data provenance framework to address the complex challenges of AI development. The proliferation of AI models, their diverse training data sources and associated ethical, legal, and transparency concerns have culminated in a critical need for a comprehensive approach to data documentation." [36]. The present paper utilizes the widely-known W3C-standardized PROV-O for its core terms in representing digital workflow provenance.

Additionally, the used PROV-O terms are mapped to corresponding terms in the more general BFO, using the preliminary mapping by Procko and Ochoa [219]. BFO is ISO/IEC standardized as of 2021 [110]; also, BFO and CCO were adopted by the DoD in January of 2024 as baseline standards for development work [246], so adhering system terms to BFO is promising for interoperability [247].

5.8.4. Mitigating Information Overload

As Simmhan et al. note, provenance granularity is either fine or coarse; additionally, the scalability of some provenance system, and its incurred overhead, are issues of concern dictated by a number

of factors including the number of datasets (in the case of ProvTracer, its work sessions), their granularity, provenance depth, and so on [19]. These are summarized by the concern of information overload, which can be user-oriented, e.g., when users are faced with too much raw or unstructured information, or system-oriented, e.g., when large amounts of data must be processed, stored, or queried. Information overload is exhibited by many symptoms in human users, e.g., loss of work efficiency, decision paralysis, stress and cognitive strain, etc. [248]. ProvTracer attempts to mitigate information overload in a number of ways.

Information overload is not simply relegated to the volume of information, but its relevance; MLLMs can help prioritize the most meaningful provenance information. Prompt responses from a MLLM about an image can be considered as statistical aggregates, i.e., they are terse summaries of complex visual artifacts. Knowledge graphs, particularly those serialized in Turtle format, are very cogent knowledge artifacts. Those generated by ProvTracer contain a modicum of boilerplate information, where the majority of generated provenance content pertains to actual instance information. These trace link KGs are able to be uploaded to a MLLM and queried in natural language, with natural language responses returned. This is a direct means of mitigating information overload by leveraging the general knowledge of MLLMs with RAG [95]. ProvTracer allows users to electively include context sources, i.e., users are able to choose how much context they provide to the MLLM, thus filtering the noise. ProvTracer users are also able to adjust the timeslice interval, which allows them to take in more or less information with each timeslice.

ProvTracer's approach to automated provenance capture aligns with human-centered provenance design, where the goal is not to capture *all* information, but to capture and expose

relevant information in such a way as to be functionally useful for downstream purposes. Earlier studies on information overload emphasize that both the volume and presentation of information affect the performance and understanding of users [248]. ProvTracer mitigates cognitive burden through its configurability, e.g., selectable context sources, timeslice adjustability, and MLLM summarization. This idea is consistent with principles from cognitive load theory (CLT), which suggests that reducing unnecessary information during problem solving can support comprehension and reduce cognitive load [249]. Finally, by encoding provenance in structured knowledge graphs backed by a developed ontology, and querying them via natural language, ProvTracer lends to intelligent information filtering, delivering a sufficient amount of traceability without overwhelming a user.

5.8.5. TRANSPARENCY AND REPRODUCIBILITY

Because ProvTracer automatically captures multimodal provenance signals, e.g., screenshots, keyboard strokes, etc., on a fixed interval and codifies them into queryable KGs, it supports traceability and explainability by design. As a consequence of this design, ProvTracer inherently provides transparency and reproducibility as additional NFRs. The presence of a persistent disclaimer window ensures that users remain aware of ProvTracer's status at all times, satisfying the notion of informed consent and reinforcing transparency as a user-facing value. Additionally, because ProvTracer stores provenance data such as keyboard strokes and window events alongside MLLM responses about user activity, the system supports reproducibility in digital workflows. In short: as traceability tells an interested party *what* happened and explainability tells them *why*, transparency demonstrates *that* it happened, and reproducibility allows it to more easily *happen again*. These emergent NFRs are not directly implemented in ProvTracer's system code; rather,

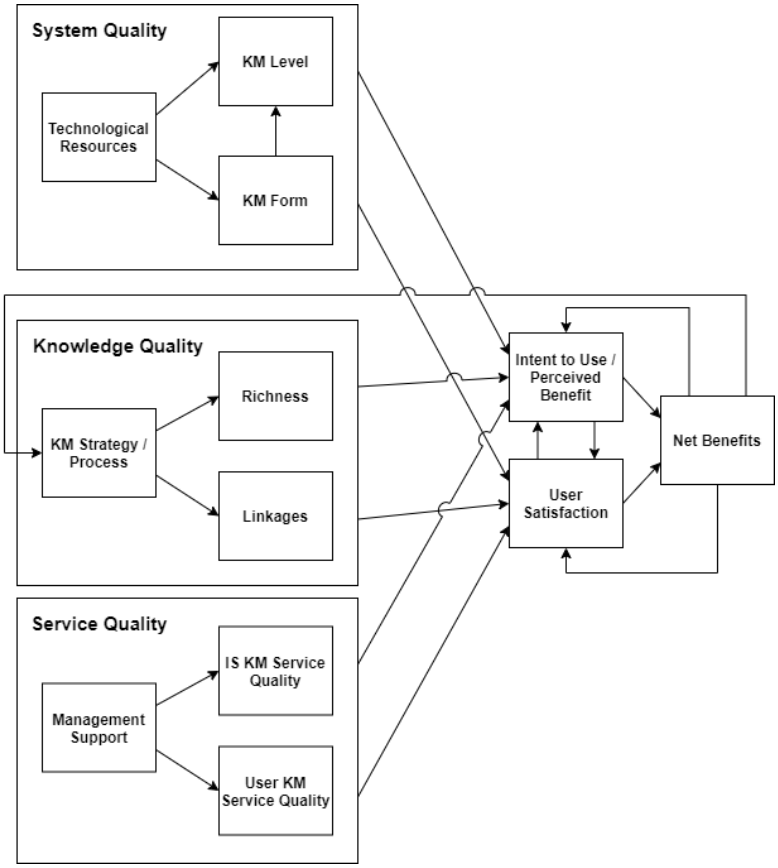

Fig. 50 KMS success model of Jennex and Olfman (recreated from [250]).

they are intrinsic to the design of ProvTracer as a lightweight, human-centric provenance capture system, manifesting in the downstream use of its output knowledge artifacts.

5.9. ESTABLISHING KMS SUCCESS

Mirroring the qualitative evaluation of Thaul's theoretical KMS [18], this section considers ProvTracer with respect to the KMS success model of Jennex and Olfman, which was adapted from information systems to knowledge management [250]. This is performed after the grounded theory analysis inasmuch as no formal user study was possible for ProvTracer. Their KMS success model addresses six dimensions of quality (see Fig. 50):

1. System quality: the technical performance of the KMS

   a. Technological resources: the capability of an organization to make use of KM
   b. KM form: the extent to which the knowledge and knowledge processes are serialized and made useful
   c. KM level: the ability to use knowledge on current activities
2. Knowledge quality: the richness, accuracy, and relevance of the knowledge captured by the KMS
   a. KM strategy / process: the processes for identifying knowledge users and knowledge for capture
   b. Richness: the accuracy and timeliness of stored knowledge
   c. Linkages: the knowledge maps available to identify resources to users
3. Service quality: the degree to which the system supports user needs
   a. Management support: the support an organization must provide to properly use KM
   b. User KM service quality: the support provided by organizations to help personnel use KM
   c. IS KM service quality: the support provided by the IS organization to users
4. User satisfaction: satisfaction of KM use, as reported by users
5. Intent to Use / Perceived Benefit: perceptions of the benefits of KM by users
6. Net Benefits: the overall impact on a user's performance as a result of KM

While the proper assessment of some KMS against the KMS success model is intended to be performed after a usability study given some population of users, it is possible, here, to provide discourse on the KMS success of ProvTracer in a thematic narrative style, given the grounded

theory analysis presented earlier. Not every dimension of the KMS success model is addressable, given the lack of a formal user study; the ones that are addressable are expounded on below.

5.9.1. System Quality

ProvTracer addresses the system quality dimension of KMS success in that it addresses "knowledge creation, storage/retrieval, transfer, and application" [250] because of generalized MLLMs. The system quality sub-dimensions of KM form and KM level are posited to be of high quality for ProvTracer. KM form is addressed inasmuch as ProvTracer automatically and unobtrusively observes, serializes, and makes queryable, provenance as knowledge graphs. KM level is also addressed because output provenance trace link knowledge graphs can be queried in SPARQL or in natural language with prompts to a LLM.

5.9.2. Knowledge Quality

The knowledge quality sub-dimension of richness is addressed directly by ProvTracer, as the system produces provenance trace link knowledge graphs containing machine-readable timestamps, serialized context inputs, and MLLM-generated descriptive summaries. These descriptions often reflect specific user actions, e.g., code edits, window activity, etc. Grounded theory analysis identified user purpose and temporal sequencing as frequent trace features, indicating strong support for the richness dimension. Additionally, the sub-dimension of linkages is addressed insofar as ProvTracer uses knowledge graphs to serialize its observations, including user information and provenance descriptions.

5.9.3. Net Benefits

Because ProvTracer operates independently of user action, it is beneficial insofar as it is unobtrusive to digital workflows, only acting as a positive augmenter of such workflows after the

fact, when business questions need to be answered and concerns relating to projects or tasks must be addressed. ProvTracer is entirely voluntary and scalable to future needs, as it captures low-level metadata programmatically, only relying on a MLLM for provenance descriptions.

Chapter 6: RELATED WORK

This section presents related work; the literature review performed for its construction was not systematic, inasmuch as a "systematic" literature review is not precisely feasible [251]. In practice, the snowballing technique of Wohlin [252] was used, which sees the tracing of relevant literature from the citations of already-found relevant literature, such that citations are "snowballed" from one another. This is a serendipitous method of citation garnering that lends to a very organic discussion, e.g., of related work, which is not the primary contribution of the work, but rather, a supporting facet. Snowballing is also an effective means of locating relevant literature that other authors have identified, that one's own searches may have missed. The information sources used to locate relevant literature are given below:

- Google Scholar
- ScienceDirect
- ACM Digital Library
- Wiley Online Library
- IEEExplore

The search phrases used varied over the course of searches, but were generally informed from the defined research questions and the trail of research that unfurled over the course of the dissertation's development. Some of the notable search phrases used are given below:

- (provenance OR metadata OR lineage OR pedigree OR history OR source OR background OR ancestry OR traceability OR reproducibility) AND (model OR scheme OR tool OR system OR technique OR method OR construct OR recording OR capture)

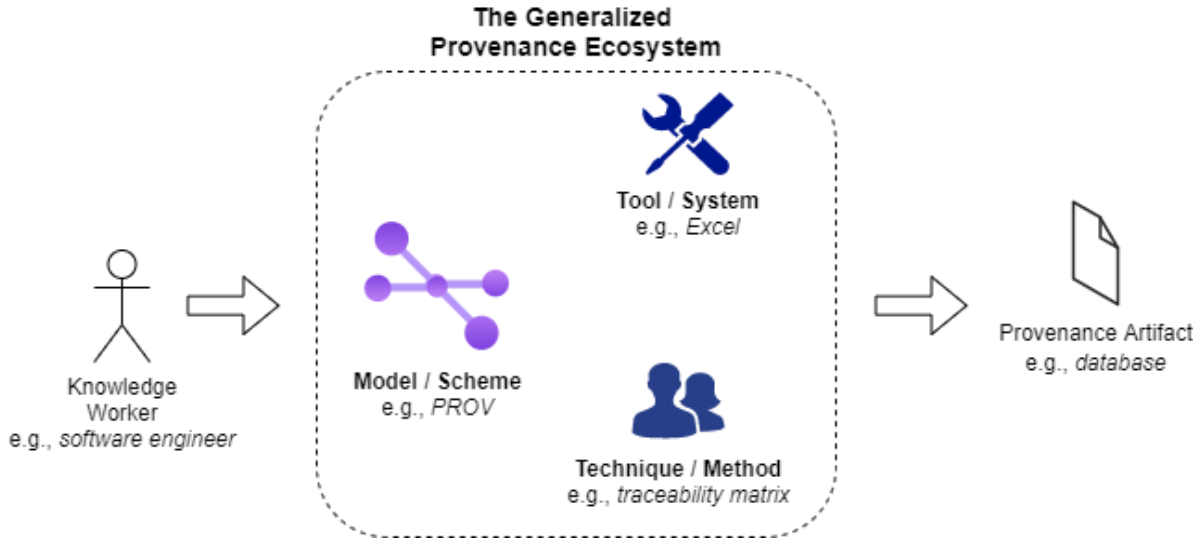

Fig. 51 The three core pieces of any typical provenance ecosystem: models, tools, and techniques. A knowledge worker capture provenance may use any combination of these.

The search string above contains two groups of OR-separated phrases linked by Boolean AND. Note that the term metadata is included, although section 1.2 clearly delineates the difference between metadata and provenance. This is because, in the research landscape and in enterprise work, the two terms are often conflated as representing the same concept; so, to capture as much relevant literature as possible, the term is included. From the literature search, it is possible to assert that there are three common constructs in traditional provenance work. See Fig. 51. Each construct has its own strengths and drawbacks depending on implementation, and all three are generally incorporated to form a working provenance ecosystem for some end application purpose. These constructs are: models or schemes (form of capture), techniques or methods (way of capture) and tools or systems, which are the coded, developed artifacts typically incorporating the two prior constructs into a functional whole. These three core components of the traditional provenance ecosystem form the "toolbox" of the knowledge worker engaged in provenance capture.

Related work on artifact provenance is plentiful and relevant to myriad fields, including general scientific research, bioinformatics, business workflows, etc.

From the snowballing survey, it is possible to assert that an automated provenance capture system utilizing MLLMs and KGs is not extant. ProvTracer purports to be the first engineered system with this distinction.

6.1. SIMILAR TERMS

Finding research related to that discussed in the present dissertation is not trivial, given the wide variance in terms used by other researchers, and the often novel terms employed. *Provenance capture* is the term of principal interest, but varied searches led to the discovery of several related terms. The literature survey is not systematic or exhaustive, nor does it purport to be, as claiming such may be unsubstantiated [251]. Most of the literature on provenance pertains to early 2000s Semantic Web and related concerns, and most of the literature on software traceability pertains to professional applications in software development. As argued in the Introduction section of the present dissertation, there is very little research overlap between the dated notion of provenance and the professional concern of software traceability. TABLE X contains a list of the various terms encountered that are relevant or tangentially relevant, e.g., metascience, aware applications, Informatology [253], etc.

A very relevant paper about personal knowledge management (PKM) was submitted in 2014 as a dissertation: *"Supporting Learning by Tracing Personal Knowledge Formation"* by Thaul. The paper presents an advanced concept to support contemporary learning, principally for students. The proposed system is described as an "intelligent, intermediate layer, which traces the different steps our knowledge entities are going through" [18]. The system is an XML-based GUI that allows the user to record the evolution of their knowledge artifacts, similar to that proposed here, except that it is strictly manual. Although its implementation seems to be purely theoretical,

TABLE X Similar terms and their domains.

Term	**Acronym**	**Applicable Domain(s)**	**Definition/Clarification**
Aware Applications	-	Computer Science	Software applications that can react to their environment/context meaningfully
Business Intelligence	BI	Business, Data Analysis	Technologies, applications, strategies and practices used to collect, integrate, analyze and present business information
Corporate Memory	-	Business, Information Science	The accumulated body of data and knowledge in an organization's existence
Findable, Accessible, Interoperable, Reusable	FAIR	Research, Science	Used primarily in tandem with the term 'data': FAIR data; also, FAIR workflows
Information Fusion	-	Information Science	Integrating information from multiple sources to produce more useful information
Informatology	-	Information Science	Similar to information science or informatics; derived from metascience
Knowledge Management	KM	Business, Information Science	Creating, sharing and using knowledge for businesses, firms or organizations
Life Cycle Metadata	-	Information Science	Metadata about an artifact, including its creation, modification and usage history
Metascience	-	Science, Philosophy	The study of the scientific process, i.e., science about science
Personal Knowledge Management	PKM	Information Science	For personal notetaking, studying, research, etc. in daily life
Process Documentation	-	Business, Quality Assurance	The act of detailing the steps/activities involved in particular processes
Research Process Optimization	-	Research, Science	Techniques applied to improve the efficiency/effectiveness of research processes
Research Profile	-	Research, Science	Description of some research process, i.e., its "research profile"
Scientific Workflow Management Systems	SWfMS	Computer Science, Bioinformatics	Software systems designed to compose, execute, and monitor scientific workflows
Workflows	-	Business, Information Science	Sequences of industrial, administrative or other processes through which artifacts pass until completion
Zettelkasten	-	Information Science	German for "slip box" or "card file"; related to PKM

the author provides great insight into the classification of knowledge worker actions and the automated population of relevant actions into a provenance database. The distinction between relevant and irrelevant knowledge worker actions is peculiar to the individual knowledge worker, as, for instance, the action of checking emails during artifact development may be irrelevant to a music artist making digital music, but for a software engineer forming a requirements document, it may be directly informing the artifact. This notion of irrelevant action, i.e., noise, was integral to the conceptual design of the present dissertation.

A 2012 paper delineates an automatic provenance discovery system that detects entity derivations using clustering and semantic similarity with linked data [254]. A 2006 paper provides insight into the difficulties in automatic provenance collection [204]. Notably, the authors distinguish between observed and disclosed provenance systems, stating that most provenance systems use disclosed, i.e., manually annotated provenance; where observed provenance systems were, at the time, difficult to implement due to concerns of provenance granularity and versioning.

The system developed in the course of the present dissertation, ProvTracer, is an automatic, lightweight, unobtrusive means of capturing workflow provenance that can later be analyzed. There was identified no unified body of work that purports to do the same, although there are a number of relevant provenance systems. These are noted in the imminent subsections.

6.2. PROVENANCE SYSTEMS

Simmhan et al. (2005) present a survey of data provenance technologies, including the Lineage Information Program (LIP), the Virtual Data Grid (VDG), myGrid, Collaboratory for Multi-Scale Chemical Science (CMCS), Provenance Aware Service-oriented Architecture (PASOA), Earth System Science Workbench (ESSW), Tioga, and others [19]. Many of these provenance

technologies are intended for use in scientific or otherwise experimental work, where many trials and runs abound and provenance is generated en masse. The myGrid project represents e-Science provenance using RDF [255].

In delineating the Holistic Knowledge Formation System, Thaul (2014) asserts that there are seven PKM tool types: indexing and searching, associative linking and searching, online meta-searching, web capturing, organizing and mapping, collaborating, and organizing/sharing tools [18]. The current dissertation presents a system addressing the functionalities of all these tool types. As discussed previously, this is possible due to the generalization capabilities of LLMs, which were not extant beyond conceptualization in 2014, with the seminal transformer architecture arriving in 2017 [155]. Thaul's system is a table-based observer that records transformative knowledge events for artifacts such as word processor files, effectively capturing provenance for PKM.

The Semantic Desktop is a construct of relevance in the early-mid 2000s, principally in the context of PKM [256]. Many of the semantic desktop applications described in the early literature, e.g., the Networked Environment for Personal, Ontology-based Management of Unified Knowledge (NEPOMUK), used RDF, and other related Linked Data models, to store and retrieve knowledge. The primary use case for semantic desktops is in reducing information overload for knowledge workers at their own computers. Such systems were very low-level and often relied on manual user annotation, but never reached widespread use because of their tendency to become overloaded and slow as more data were collected [257].

Aside from semantic desktops, some other provenance systems are extant. Although not necessarily intended to be used as such, the Obsidian project, a private note-taking app allowing

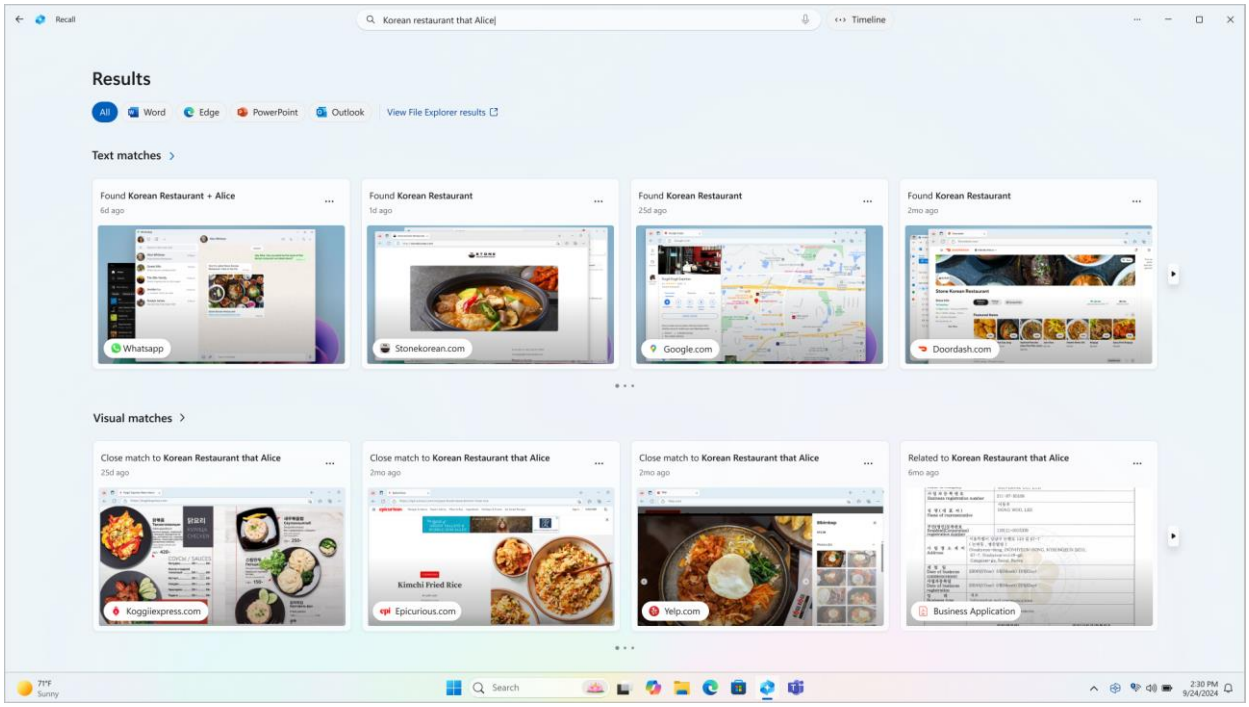

Fig. 52 A screenshot of the Microsoft Recall application.

graph-based visualizations of personal knowledge, can be used as a manual provenance system by the motivated user [258]. Except for the paper of Thaul [18], no other work describing an automated, real-time, end-to-end provenance capture and analysis system was located in the literature survey of the present dissertation. It is posited that this is due to the relatively recent novelty of LLMs, which demonstrate great ability in performing general tasks, e.g., desktop computer work.

6.3. MICROSOFT RECALL

Beginning in early 2024, Microsoft offered for sale Copilot+ personal computers (PCs), carrying a new hardware component, the neural processing unit (NPU), which is dedicated to performing AI-intensive tasks. One of the applications leveraging the NPU is Microsoft Recall, which allows users to query the past of their PC use as a timeline. Microsoft Recall is useful for returning to

some text, media, application, web page, etc., by simply asking about such in natural language; both text and images can be searched over for desired content. From Microsoft's website, Recall operates by taking screenshots (snapshots) "… every five seconds while content on the screen is different from the previous snapshot" [259][8]. Results are displayed as either text matches or visual matches; and, further, close matches or related matches. See Fig. 52. Close matches include at least one of the search terms or images representative of the query; related matches are somewhat related to the search terms. Microsoft Recall timeline activity is broken up into segments, defined as the "… blocks of time that Recall was taking snapshots while… using [the] PC" [259].

Microsoft Recall is, in effect, a real-time, continually updating RAG system, where most of the RAG knowledge base is embedded in the semantics of images, although some lexical fragments persist, e.g., hyperlinks. Presumably, Microsoft Recall has an embedded model capable of extracting text from captured images for direct application to a LLM as lexical prompts, e.g., OCR, but this is unclear. Microsoft Recall is closed source and not extensible; further, it does not explicitly capture provenance, but captures a user's screen on a set interval, essentially offloading any provenance work onto the user, who must consider the proper queries. Microsoft Recall therefore diminishes provenance in two clear ways:

1. By embedding it as RAG inputs: provenance, for Microsoft Recall, is a fuzzy cloud to be searched with "close enough" natural language prompts; no serialized provenance artifacts exist, save for what can be searched for and returned transiently

[8] It must be noted that Microsoft Recall's use of a 5-second screenshot interval is assumed to be arbitrarily set. ProvTracer defaults to the same value, but allows uses to adjust the interval as desired within a set range.

2. By not associating it with artifacts: there is no explicit correlation between provenance events and artifact modification, only tacit links that must be discovered manually through prompting

Another concern with Microsoft Recall is information overload. With larger sets of computer use, Microsoft Recall may return many relevant results given a prompt for insight; this may overload the user with too many elements to consider. ProvTracer addresses this by being voluntarily started, named, described, etc., before beginning a work session, so all provenance is broken down into manageable chunks; additionally, screenshots are reduced in size, provenance descriptions are terse MLLM-generated summaries, and KGs serialized in Turtle are compact.

6.4. JIRA

Jira is a proprietary bug and issue tracking application for Agile project management. As software issues are not always atomic, i.e., they may have some arbitrary relations to other issues, Jira allows users to link issues. By default, a Jira installation is equipped with four issue link types [260]:

- relates to / relates to
- duplicates / is duplicated by
- blocks / is blocked by
- clones / is cloned by

Jira lets users add new link types as necessary, a consideration typically raised at the team or corporate level. These issue links are a manual form of software provenance curation. While an interesting consideration for the underlying provenance model, with respect to the present dissertation, Jira issue linking suffers from the same issues of manual provenance annotation noted

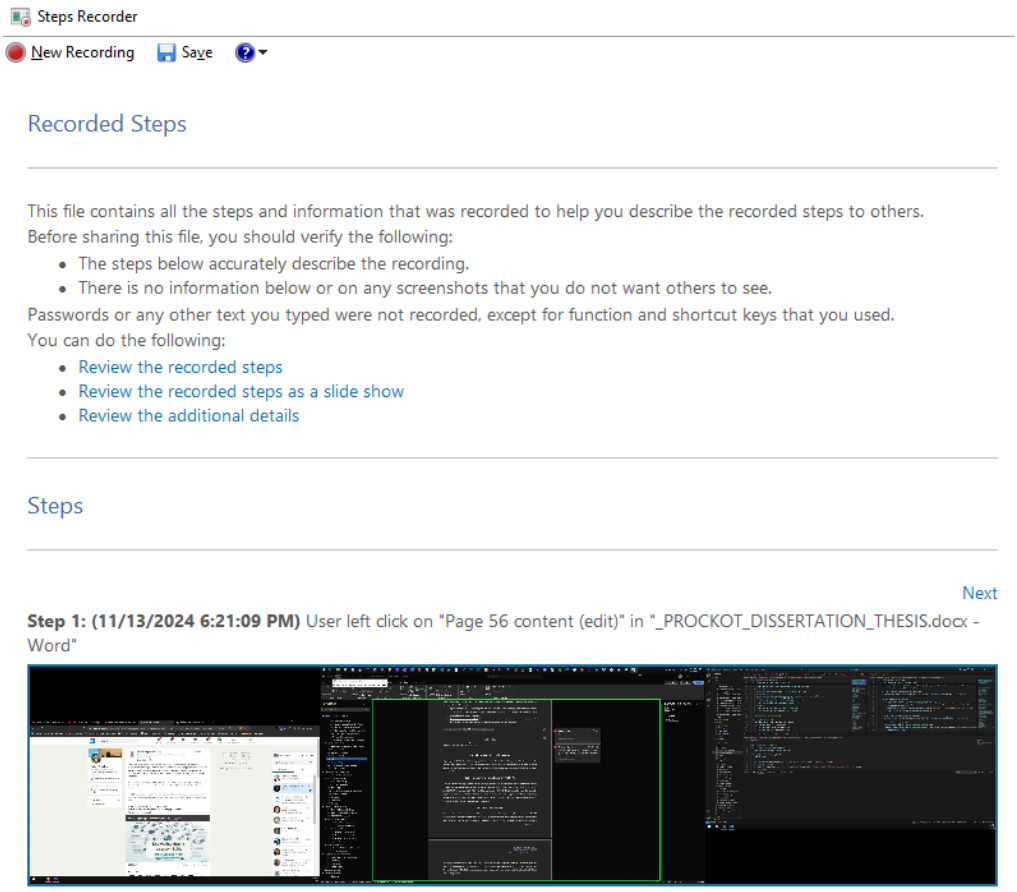

Fig. 53 A snippet of the Microsoft Steps Recorder application.

in earlier sections. ProvTracer uses PROV-O and BFO, which correlates with greater interoperability as its trace link terms are well-established.

6.5. MICROSOFT STEPS RECORDER

Beginning with the Windows 7 operating system from Microsoft, the Microsoft Steps Recorder, also known as the Problems Steps Recorder (PSR), was introduced as a built-in tool that allows a user to record screenshots of their desktop alongside annotated user actions, or steps, for the purpose of "playing back" problems encountered for sharing with others for assistance. See Fig. 53. This application also allows users to annotate steps as desired, i.e., it is intended to be augmented manually with user intent, thought processes, etc. Microsoft Steps Recorder has special

functionality with particular desktop applications, e.g., Microsoft Word, as it will automatically annotate steps with additional information, e.g., “left click on page 56”; but which applications it has such functionality with, and to what degrees, are unclear, as, for general application use, its step annotations remain very agnostic, e.g., with simple click and keystroke event notes. With respect to the system developed in the present dissertation, Microsoft Steps Recorder covers a very simple subset of its functionality, while also suffering from the typical need for manual user annotation; it also has no means of analysis aside from what can be read or seen.

6.6. REDSEEDS GRAPH TRACEABILITY

Schwarz, Ebert, and Winter demonstrate a graph-based approach to all traceability activities: definition, recording, identification, maintenance, retrieval, and utilization; they apply their approach to the Requirements Driven Software Development System (ReDSeeDS) [77]. They identify the challenge of comprehensively addressing all activities of traceability with a single approach. Their approach utilizes directed TGraphs, which are typed, attributed, and ordered. Their underlying ontology, the Traceability Reference Schema, defines the TraceableEntity and the TraceabilityRelationship, which forms the core of their TGraph composition. TraceableEntitiy has defined sub-entities, e.g., Stakeholder, TestCase, Requirement, etc.; TraceabilityRelationship has defined sub-relations, e.g., IsResponsibleFor, ConflictsWith, DependsOn, Tests, etc. TGraphs are retrieved and utilized with the Graph Repository Query Language (GREQL).

Their approach employs graph-based modeling to represent and analyze traceability activities using a defined vocabulary. There is no reference to an outside lexicon or a well-established upper ontology, and the modeling and retrieval is manual. While promising, this

approach suffers from the same issues of manual provenance trace annotation as noted throughout the present dissertation.

6.7. DATED PROJECT MODELS

The Description of a Project (DOAP) model is an early-2000s RDF/RDFS vocabulary used to describe and annotate software project metadata, principally for open-source software [261]. DOAP extends heavily from the Friend of a Friend (FOAF) vocabulary and Dublin Core Metadata Initiative (DCMI). DOAP prescribes terms for annotating projects, specifications, branches, developers, testers, releases, repositories, operating systems, programming languages, and so on; but its terms are not organized with respect to any TLO, and it has been without maintenance for several years [262], so it remains a historical sidenote.

There are a few serialized models of the SDLC, which are not specific to provenance, but pertain to overall SDLC structure. Procko and Ochoa present a survey of SDLC ontologies, notably, the Software Evolution Ontologies (SE-ON) and the Software Engineering Ontology Network (SEON), both of which are serialized in OWL [263]. Both of these ontologies possess great detail of the SDLC, albeit from different perspectives, with SE-ON viewing the SDLC through its processes, and SEON viewing the SDLC mostly through its continuant objects. SEON is based on the UFO TLO by Guizzardi [264]. Both of these ontologies have limited real-world use and are included here briefly for the sake of completeness. Similar to DOAP, these are data models, not executable applications, so they are only useful insofar as some system may utilize them to frame information against.

Chapter 7: DISCUSSION

This chapter presents a discussion of the findings and research contributions of the present dissertation, with respect to both the research questions established in section 1.7 and the implemented ProvTracer system; it is structured in three primary parts. First, each of the three guiding research questions is addressed, placing the dissertation's conceptual and technical contributions into the context of the greater scientific landscape. Second, the success of the implemented ProvTracer system is evaluated through the lens of KMS theory, connecting its provenance specific features to broader aspirations of organizational knowledge. Finally, a critical reflection on limitations and concerns is presented, with attention to both theoretical challenges and practical constraints of the implementation. This discussion aims to contextualize the contributions of the present dissertation within the wider provenance and traceability research landscape.

7.1. ANSWERING RESEARCH QUESTIONS

The research questions established in the introduction section of the present dissertation guided its development. These are addressed in the ensuing subsections.

7.1.1. ANSWERING RESEARCH QUESTION 1

The first research question concerns the core challenge of separating meaningful provenance signals from incidental noise in digital work environments; this establishes what is useful for the implemented system, ProvTracer, from what may cause information overload.

How can relevant provenance sources be distinguished from noise during digital software development work?

There exists little synthesized research on such irrelevant sources of digital activity, e.g., a taxonomy, but a great amount of research centering on digital distractions is present in the scientific landscape. A great deal of knowledge work is performed in digital environments; digital distraction occurs commonly in digital environments as a result of accessing or being made aware of, often unintentionally, entertaining content that derails mindful work [265]. Most literature on digital distraction focuses on university students, social media and smartphone distraction [266, 267], with a great deal being published as a result of the COVID-19 incident [268], yet some posit that digital distraction occurs as a result of one's own mind and psychological factors, particularly in remote work scenarios [269]. Orhan et al. discuss technology overload, technology distraction and digital interruption in the workplace [270]. They note and define several interruption types, e.g., intrusions, distractions, breaks, surprises, multi-tasking, etc. In the context of provenance capture, the present dissertation posits that digital distractions and interruption are noise that should be filtered.

Fig. 54 contains a taxonomy of digital work distraction, or noise, sources. The taxonomy presented is theory-driven, using the principles of grounded theory for its induction [239]; it does not purport to be exhaustive, but rather, functionally useful, informed by observations during system development, general experience, and the aforementioned related literature on digital distractions. The taxonomy is ad-hoc and not conformant to any ontological standard, e.g., as BFO provides, because of its novelty and focus on human computer interaction; it is, at best, a simple categorization scheme, and not a rigorous baseline for a proper ontology. The taxonomy, an organized knowledge structure of digital work noise sources, effectively a blacklist, is posited to

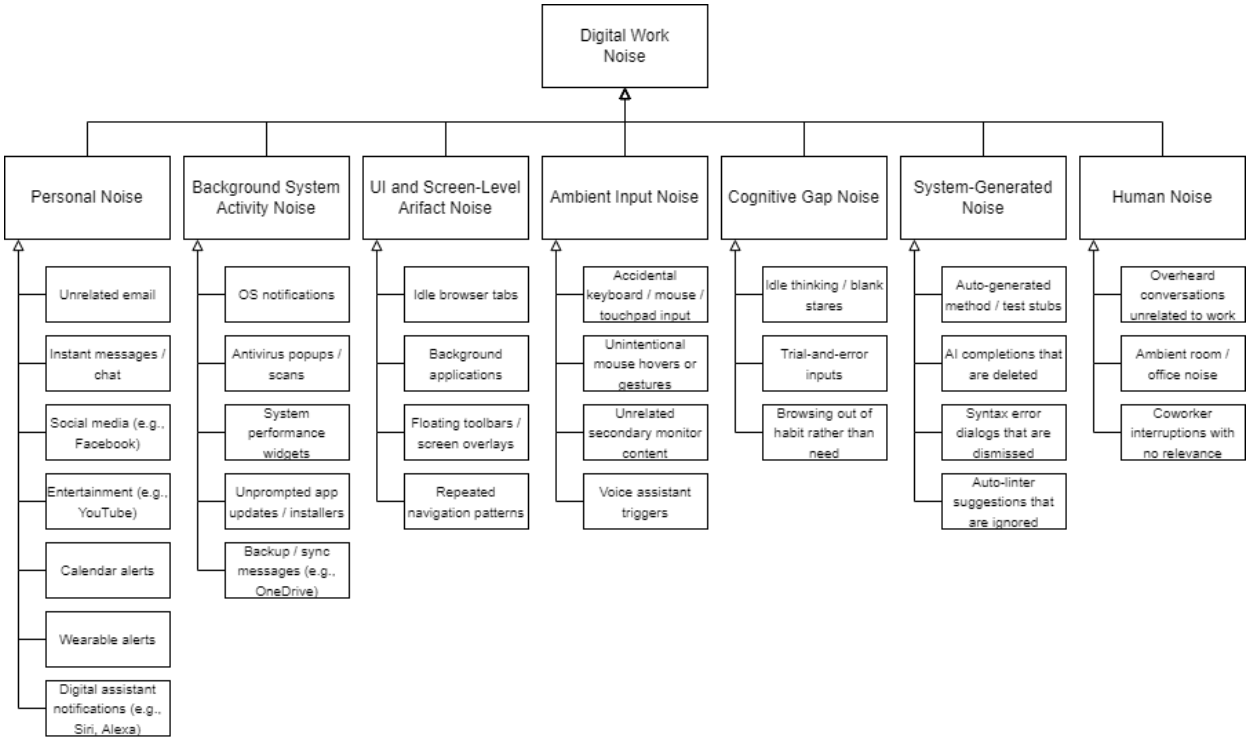

Fig. 54 Digital work noise source taxonomy for general blacklist use to inform whitelists.

be useful for the purpose of whitelisting provenance sources for representation in the output provenance trace link knowledge graphs of ProvTracer.

It is generally accepted that blacklisting is a fragile and unscalable means of input validation, because the set of undesirable inputs is combinatorially unbounded and always evolving (especially in open-ended digital work), while whitelisting allows for more precise control by only allowing what is explicitly trusted [271]. Blacklisting risks the chance of finding false negatives (missing new noise) and false positives (blocking relevant provenance), because every new application or UI element can introduce new noise requiring additional filters. In short: blacklisting is *reactive* and incomplete, while whitelisting is *proactive* and complete with respect to some designed task. In the context of automated digital provenance capture, the space of possible distractions, false cues, or incidental interactions is effectively unbounded, so the

declaration of a noise source blacklist is not feasible. In keeping with this principle, it is posited here that the digital work noise source taxonomy be used as a general guide for the formation of relevant provenance source whitelists, to be tailored to the precise needs of personal, team, or corporate work.

The taxonomy of digital work noise sources is decomposed into sub-categories, each containing specific exemplars of the category. As an example, the Personal category contains an exemplar digital noise source, “Wearable alerts”, which may include output information from devices such as smartphones, smart watches, etc. Full, Aristotelian-style definitions are given for each of the category and exemplar terms in Appendix B.

The digital work noise source blacklist taxonomy of Fig. 54 is generally applicable to any work context, not just the SDLC; its intent is to inform the creation of domain- or task-specific whitelists, mirroring the ontology strata model of Guarino [106]. In the case of ProvTracer, such a whitelist would translate to additional prompt context, explicitly informing the chosen MLLM as to what is to be accepted as relevant provenance information, as opposed to noise. As an example, using the approach of grounded theory, it is possible, given the research forming ProvTracer and its evaluation, to induce a taxonomy of proper SDLC provenance. This is given in Fig. 55. It is informed by an early paper describing software informalisms in software requirements management [62], the SEON and SE-ON ontologies (see section 6.7), and general SDLC experience. As with the more general blacklist taxonomy, the SDLC provenance source whitelist taxonomy does not purport to be exhaustive or complete, but functionally useful. The taxonomy is heuristic in nature, and not philosophically rigorous. The leaf classes are exemplars of real-world

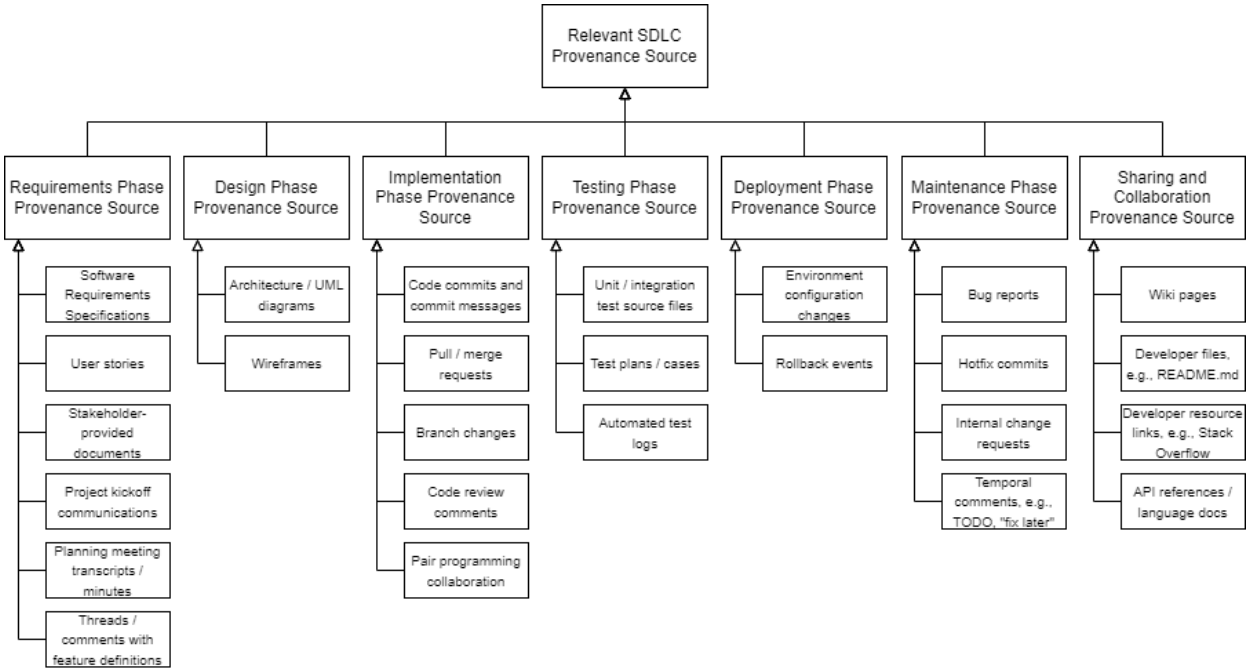

Fig. 55 SDLC provenance source taxonomy for whitelist use.

SDLC provenance sources, but the set of them is not complete. Full definitions of each term in the taxonomy are given in Appendix C.

The two taxonomies presented herein are not serialized outside of the current dissertation artifact, e.g., as an OWL file, inasmuch as they are novel and presented for qualitative purposes.

7.1.2. Answering Research Question 2

The second research question pertains to the main goal of the present dissertation.

How can the integration of a knowledge graph-based provenance system enhance the traceability and explainability of artifact development and maintenance within the SDLC?

The integration of a knowledge graph based provenance system, as demonstrated by ProvTracer, enhances traceability by structurally linking actions and artifacts over time, and improving explainability by embedding natural language provenance descriptions into provenance trace link knowledge graphs. Grounded theory analysis of real-world case studies preliminarily

confirm that such integration enables both human understanding and machine queryability within the SDLC.

7.1.3. Answering Research Question 3

The third research question focuses on the difficulties inherent in the implementation of a system intended to automatically capture digital work provenance, which are many and varied. The answer includes a synthesis of all prior discourse into an organized table.

What are the technical challenges in developing a system for the automatic, real-time serialization of SDLC provenance, and how can these challenges be mitigated?

As is inevitable in the course of a dissertation's work, several challenges were encountered. Due to the relatively novel nature, and the rapidly evolving research landscape, of the present work, the research question addressed in this section specifically pertains to such challenges. These are detailed in turn; TABLE XI summarizes them.

Comprehensively addressing provenance capture is a longstanding challenge. In the domain of software traceability, this is a pressing concern as well, as Schwarz, Ebert, and Winter note: "… a pressing challenge of traceability research is the development of a comprehensive approach, i.e., an approach supporting all traceability-related activities from defining, recording, and identifying, to maintaining, retrieving, and utilizing traceability information… the traceability approach has to be seamless… a consistent conceptual and technological foundation for definition and recording of traceability information is existing, spanning all abstraction levels of a software system. This enables the integration… of different identification and maintenance techniques. Finally, uniform retrieval and utilization of traceability information is facilitated… regardless of

TABLE XI Challenges identified in the implementation of this dissertation.

Challenge	**Details**	**Mitigation**
Information overload	Generating timeslice data on a set interval, e.g., 5 seconds, results in a significant amount of information that may be hard to digest	TTL KGs are terse, prompt responses are shortened aggregates, KG analysis by a LLM returns plaintext
Context sources	There are various possible influencing sources, where only a distinct subset of them can be captured consistently on a computer	Define and implement the selection of the distinct subset of context sources able to be feasibly captured
Privacy	Automatic provenance capture using screenshots may introduce privacy concerns, e.g., passwords that are seen and captured	See section 7.3.1 on privacy and informed consent
Automation	Automatically capturing provenance as it is witnessed is a complex semantic action	Low-level metadata is captured with basic code libraries, high-level provenance with a MLLM
Hallucination	LLMs may produce inaccurate or fabricated provenance descriptions	Add consistency checks; fine-tune prompts with domain- or task-specific constraints
Computational scalability	Frequent screenshot processing and LLM prompting may stress system performance	Optimize prompt granularity; use local caching; employ batch processing where latency allows
Fiscal scalability	Repeated API calls to LLMs are costly at scale	Reduce snapshot frequency, e.g., with ProvTracer settings; explore open-source or locally hosted LLMs
Provenance use and analysis	Once provenance is generated, its use / analysis / dissemination has no general heuristic, is primarily qualitative, and specific to domains	Use LLM to analyze provenance in a RAG workflow (see section 5.2)
Noise differentiation	Irrelevant inputs may be interpreted as provenance, e.g., Spotify, background apps	Apply whitelist-based filtering; use taxonomy of known noise categories
Temporal disconnect	Measuring work deltas between screenshot intervals is difficult	See section 7.2.3 on temporal disconnect
Trace granularity	Overly fine-grained or overly coarse traces can reduce usability	Allow user to select provenance sources to be considered, e.g., as ProvTracer allows context source selection
User trust	Users may distrust generated traces, especially if incorrect or involving personal information	Enable user correction or annotation

the types of the involved entities and relationships" [77]. They go on to assert and validate that a graph-based approach facilitates the challenge of comprehensive traceability management. The system implemented in the present dissertation, ProvTracer, addresses these activities holistically. The final activity, provenance utilization, is the most pressing challenge of the present dissertation.

It can be asserted that the final purpose of provenance, i.e., use and analysis, is the most underserved in practice and in related literature [19, 240]. There is voluminous literature on modeling, defining, recording, and maintaining provenance, but literature on retrieval and use (or analysis) is always relegated to particular implementation constraints, business concerns, or domain interests. There is no general heuristic or technique for provenance use; much provenance use is qualitative in nature. Assessment of the implemented ProvTracer system in Chapter 5 is qualitative and based in grounded theory.

The challenges identified in this section are accompanied by the next section on limitations, which are more technical and specific to the coded implementation of ProvTracer.

7.2. LIMITATIONS

This section details the limitations of the present dissertation, including its developed system and analysis. A discussion of limitations is important insofar as it places some work within context and lends to a discussion of future work [272]. This is especially important for the present work, which was developed in the context of early MLLM advancement from 2023 to 2025 – a dynamic time period for AI and software systems in general. Large-scale generative AI constructs are only evolving and curating more general applicability to wider areas of interest because of widespread use, so, the limitations of using such constructs for the developed system must be identified. These are discussed in the following subsections.

A generic limitation that was addressed in section 3.3 detailing project scope is that of provenance tangibility. Some sources of provenance are not, as yet, feasible to identify and capture, e.g., inspiration, intuition, word-of-mouth, etc. Section 8.1 and its ensuing subsections delineate potential means of capturing some of these currently intangible sources of provenance.

Additionally, upon evaluation of the MLLM GPT series, it was found that, in particular, the spatial reasoning of the MLLMs is poor. This is detailed in section 4.1.1. The MLLMs evaluated are exceptional at extracting textual content from computer screenshots, but they are impotent in terms of their ability to extract spatial semantics from screenshots, e.g., location, position, size, etc. So, a distinct limitation of ProvTracer with respect to the SDLC is the design phase, which typically includes iteration over graphical design artifacts. With the currently employed MLLMs, GPT-4-Turbo, GPT-4o, and GPT-4o-mini, the design phase of the SDLC is not able to be properly addressed.

Thaul's KMS allows users to assign weights of influence to artifacts interacted with, establishing greater semantic depth to artifact evolution serialization [18]. Because ProvTracer intends to be lightweight, with minimal user input to avoid work interference and the issues of human error, such a provision is not present in ProvTracer. However, Thaul's reasoning for the incorporation of influence weighting is appropriate when considering influence from multiple applications, so the lack of provenance trace weighting is noted as a limitation of ProvTracer.

7.2.1. User Interface Variance

The purpose of ProvTracer is to allow provenance capture in an agnostic, generalized manner, regardless of the user engaged with it. However, a primary consideration that may, upon future evaluation, prove to be a legitimate limitation, is that user interfaces vary widely across work

domains and between individual workers. For instance, in the same software engineering firm, two co-located developers may use entirely different operating systems, different IDEs, coloring schemes, fonts, and styles, etc.; furthermore, their screen layouts will likely differ: where one developer may prefer to work fullscreen and switch between tabs as needed, the other may prefer to split his screen up with the needed tabs. ProvTracer is developed in English and tested with primarily English text on-screen, so there is no proof of efficacy with respect to different written languages. The prompts underlying the LLM interface are written in as general a manner as possible to mitigate context dependence on any particular user interface elements, but the natural, inevitable variance in human user interface preference may limit the efficacy of the system output. As another example, some developer may prefer to use an external device, such as a phone or tablet, to check off their assigned issues through Git, with their desktop being dedicated to the development work itself; this theoretical separation of devices injures the incoming provenance.

7.2.2. NOISE, NOISE, NOISE

Heuristically, workers make mistakes and forget guidelines from time to time. An example of this can be witnessed in the Personal Software Process (PSP), which is used by software developers to track progress and establish more accurate time estimates, among other benefits. PSP relies on manual data entry by individual developers, but this method results in poor data quality and difficult analysis [273], because, e.g., developers do not always faithfully record their times, sometimes even forgetting to do so. Granting this, the ideal solution would be entirely automated, such as ProvTracer proposed here; but an automated solution suffers from the issue of noise [18]. Because the system runs until a user chooses to shut it down, much noisy, or irrelevant, data are likely to be harvested, which may adversely alter the provenance outputs of the system.

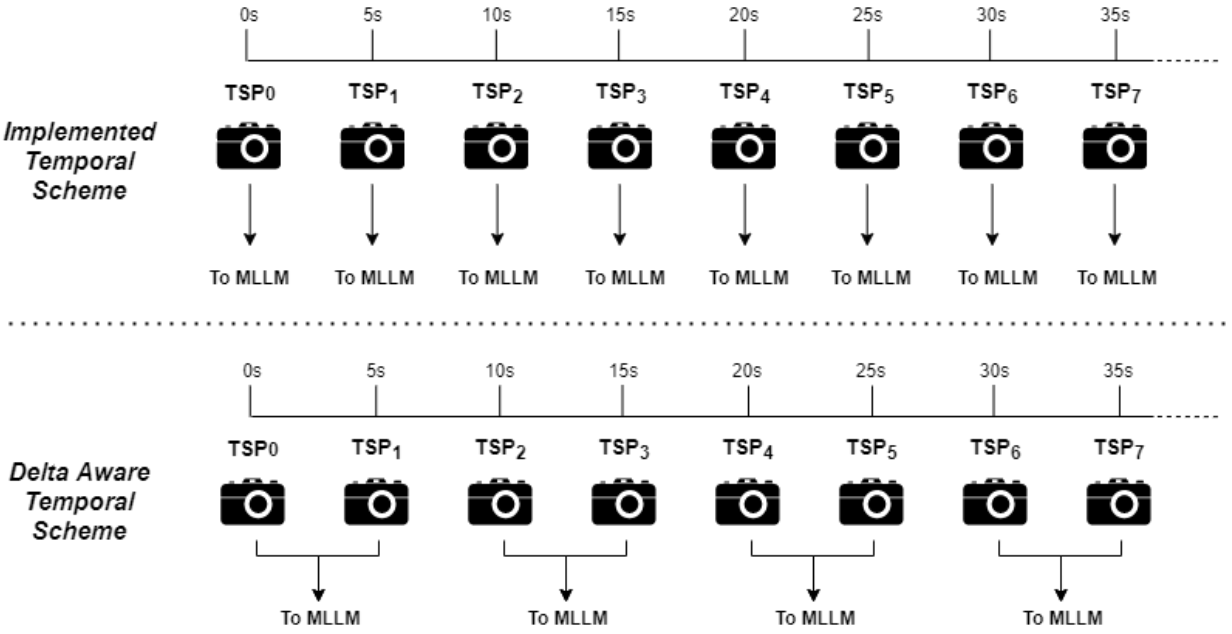

Fig. 56 The implemented temporal timeslice scheme compared to one where the delta between timeslice packets (denoted TSP_n) is made apparent to the MLLM in prompts.

Real-world users may choose to have a background video open, e.g., on YouTube, while working, or they may have a messaging application open in a side window, etc. This is an unbounded problem with a theoretically infinite number of noise sources. A technique for mitigating such a problem is found in whitelisting, which sees the delineation of a small number of desired elements, with the provision that everything else is to be blacklisted, or ignored [274]. Typically used in malware detection and prevention systems, whitelisting drastically reduces the complexity of the problem by shifting the perspective of what is desired, instead of what is not. For ProvTracer, applications could be whitelisted at the operating system level by noting their process IDs or window titles, where only actions occurring within the whitelisted PID / windows are captured. This would have to occur on a case-by-case basis, with the user manually selecting these applications before beginning a trace session, or perhaps at the corporate level, with specified lists of whitelisted work applications. It is not currently implemented because of the dependence

on manual user input. Appendix B contains definitions for terms in the digital work noise source taxonomy introduced in section 7.1.1.

7.2.3. TEMPORAL DISCONNECT

As described in the system implementation, ProvTracer relies on screenshots taken at set intervals, or timeslices. Each timeslice screenshot, along with any included contextual data, is sent to the LLM alongside a built prompt; these inputs deduct from overall token length, limited practically by spending and/or computation limits, which are either personal or corporate concerns. Ideally, each timeslice packet would include the response for the previous timeslice for greater context, effectively providing a delta between each interval. See Fig. 56. However, this would result in two issues:

1. System slowdown: the reliance on previous slice LLM response would injure overall system speed, as new slices would not queue without first receiving the previous response, effectively un-parallelizing the system
2. Token overrun: LLM responses may vary in length; additional prompt input may cause token overrun due to the relatively unbounded nature of LLM output (addressable with more restricted prompts)

An earlier thought was to send the LLM two screenshots at once, by beginning at the 1st screenshot, not the 0th, so the sequence of timeslice screenshots pairs would be 0-1, 1-2, 2-3, etc. This was impractical because prompts to the GPT Completion API are best utilized as atomic transactions, and sending two would incur additional cost. A means of "meeting in the middle" would be to combine timeslice screenshot pairs into single images with an image editing library, but this would spawn new issues.

1. System slowdown: larger images may slow down or even halt API computation; this is a potentially unbounded problem as users have varying screen sizes and monitor counts; this could be addressed by normalizing images to a set width and height or file size
2. Semantic confusion: the LLM may become confused because of the doubling of semantics over a larger image; new prompts could be written to guide it, but this increases complexity significantly

One may also consider the scenario wherein one or more working artifacts are being operated on at the same time, perhaps with the user switching between fullscreen views of said artifacts as needed with a keyboard shortcut. These artifacts are directly influencing one another, but the timesliced screenshot loop only captures, for example, one of them over a large period of time, perhaps five minutes. This introduces a limitation where the possible influence of one working artifact cannot be directly captured in the screenshot. The incorporation of window events into prompts, as discussed in the implementation description, act as a "stopgap" measure for this issue, but it remains a limitation nonetheless.

Given these practical limitations, it was decided that simply sending the LLM one timeslice image at a time, along with any user-enabled context, would suffice to demonstrate system efficacy. The LLM responses are therefore "strung together" using system timestamps, so the provenance traces are guaranteed to be linked in temporal space. This provides a basic chain of derivation for any digital work performed. However, as the evaluation demonstrates, more context generally provides better LLM output, so the inclusion of screenshot deltas would likely improve LLM output further. This is an obvious limitation of the system that induces future work.

Not evaluated is the effect of the timeslice interval on MLLM efficacy. The default value of 5 seconds was established in section 4.2.10 and reinforced by the value set by Microsoft Recall (see section 6.3). It is tentatively posited that longer timeslice intervals would result in less useful provenance descriptions, where shorter timeslice intervals would result in better provenance descriptions. This position is tampered by the practical limitation of fiscal and computational performance as a result of prompting the MLLM more frequently. Taking this position to its lower, logical extreme, where the timeslice interval approaches 0, it may be posited that the use of a live video recording system loop, as opposed to a screenshot loop as employed by ProvTracer, would be preferable. Theoretically, such would provide much finer-grained provenance capture, allow for task initiation, switching, and completion detection because of the lack of temporal gaps, etc.; video capture would effectively allow the timeslices to be chunked as necessary and as desired given, e.g., a business use case. But video capture is significantly more computationally and fiscally expensive, and current integration of MLLMs with video input is not so commonplace as with static images, so ProvTracer employs the latter.

7.2.4. VISUAL WORK

In as broad a sense as possible, for the system of the present dissertation, it is posited that there are two types of digital work: textual and visual. Textual work includes user actions and textual modifications that can be input directly to a LLM for use within a prompt. Visual work is more semantically deep; e.g., a developer working on a software diagram during the design stage makes modifications on visual artifacts that may be difficult to describe textually, where keeping images of the before and after changes may provide better insight to the incorporated LLM. Visual work is not the focus of the present dissertation, as the image interpretation capabilities of the chosen

LLMs, GPT-4-Turbo, GPT-4o, and GPT-4o-mini, are limited. However, as LLM capabilities expand, this secondary facet of digital work may positively augment the input textual work data to produce better provenance trace output.

7.2.5. PROVENANCE HALLUCINATION

In discussing LLM prompt patterns for SDLC task automation, the authors present two considerations for software developers [42]:

1. The true capabilities of LLMs are not fully known
2. Human involvement is still needed

This first point is attended with the concern of hallucination. Hallucinated LLM output looks legitimate but is false when verified [43]. Because a LLM generates an aggregated guess culminating in the most statistically plausible response, some have referred to LLMs as stochastic parrots [45]. In the case of ProvTracer, MLLM hallucination may arise from insufficient input context, e.g., screenshots lacking enough semantic cues for reliable inference, model overgeneralization, e.g., the MLLM may complete correct-appearing provenance even when no actual causal relations exist, etc. Such hallucination may arise as inferred user rationale or intent that may be plausible, but in reality, is incorrect. Hallucination as such is not simply a performance limitation, but one of traceability and reliability, inasmuch as provenance outputs from ProvTracer are intended to be used for common provenance purposes, e.g., downstream decisions, auditing, compliance, replication, etc.

Hallucination limits the usefulness of ProvTracer insofar as its provenance about timeslice packets is generated by a MLLM. ProvTracer mitigates this as much as possible by capturing low-level metadata programmatically, isolating the provenance descriptions by the MLLM to a single

triple per timeslice packet in output graphs. As LLMs have propagated beyond the flagship GPT 3.5, with the GPT-4 series, o1 and o3, GPT-4.5, etc., the research landscape has deepened and the understanding of LLM capabilities has improved. Hallucination is an issue actively addressed by LLM providers through various techniques [275], but it is still noteworthy as a limitation of ProvTracer. Hallucination may be addressed most directly by user feedback or interactive validation, where developers can refine inferred provenance, but this goes against the unobtrusive and Agile nature of ProvTracer, so it is not implemented in the current system.

7.3. CONCERNS

Separate from limitations are the concerns of the present dissertation and its developed system, which are mostly oriented towards less tangible topics, e.g., privacy.

7.3.1. PRIVACY AND INFORMED CONSENT

In the context of computer systems, the concern of security has been extant since at least 1972 [276]. In a comprehensive lexical analysis of the term, Schatz et al. define cybersecurity as "the approach and actions associated with security risk management processes followed by organizations and states to protect confidentiality, integrity and availability of data and assets used in cyber space. The concept includes guidelines, policies and collections of safeguards, technologies, tools and training to provide the best protection for the state of the cyber environment and its users" [277]. This definition highlights the commonly referenced Confidentiality, Integrity, Availability (CIA) triad of the cyber security domain.

Confidentiality, roughly synonymous with privacy, is a concern of the utmost relevance to the present dissertation, insofar as the system proposed herein is effectively a real-time personal data harvesting application integrated with an AI discriminator. Hence, the personal right to one's

own privacy may be violated, depending on the perspective or will of the person engaged with the system. Notwithstanding, the present dissertation mitigates this concern by being entirely voluntary, i.e., a user may choose to simply not use the system; moreover, when the system is active, a highly visible disclaimer persists at the top of the user's screen, informing them of the capture taking place, only disappearing when the user navigates to the disclaimer box and selects to turn the system off. In a word, the system is purely consensual and entirely transparent. This is a key parameter of the notion of *informed consent*, which is predicated upon a great body of research in diverse fields, e.g., moral philosophy, the behavioral sciences and law, and which underpins many others, e.g., biomedical practice [278]. An early (1975) definition of informed consent for the biomedical domain includes two primary facets: (1) to promote individual autonomy and (2) to protect the patient-subject's status as a human being [279]. In the digital age, the informed consent process may be augmented with digital tools. In a survey of digital tool use in gathering informed consent, authors discovered that multimedia tools were desirable and did not negatively impact any informed consent outcomes [280]. It is necessary to remark that the concern of informed consent herein is not as pressing as it is in, e.g., biomedical practice, as there is no tangible consequence for a decision made without proper informed consent, for instance, as someone with impaired cognitive ability may not fully understand the words of their doctor in consenting to some operation [281]. The authors of the Provenance-Aware Storage System (PASS) system acknowledge the Fair Information Practice (FIP) principles, e.g., individual participation, which may be scalable to those within a software development organization; but they nevertheless leave the issues of privacy and security a rather open-ended concern [204]. In any case, the early principles of retaining individual autonomy and respecting the user's humanity are met, inasmuch

as the system developed herein is purely voluntary, transparent, and interruptible at the whim of the user.

In one sense, the system developed within the present dissertation may be classified as a keylogger or spyware, although it is certainly more nuanced than that, and is fully intended to provide software developers and development firms with manageable, usable provenance for their SDLC artifacts – not, as a keylogger would, to store personal information for illicit use. If a developer refers to an email from their senior engineer when implementing requirements, their email may become part of the provenance input; so, the issue of privacy is not relegated to the single individual, but to anyone they may interact with in any capacity on their computer. To take this discourse on privacy to an extreme, consider the situation wherein a user is engaged with the system while it captures their work, and they enter a password for a personal account. The system takes a screenshot of this moment, capturing their password. Even if the user is willing to be engaged with the system during work use, it is likely both personally and corporately undesirable for passwords to be screenshotted and included in prompts to an online LLM. As an example, Microsoft Recall was controversially received, and later disabled by default on new Windows computers because of such concerns. Microsoft Recall now requires a user to log in to view their screen captures, e.g., with a personal identification number (PIN) or biometrics. It is possible to posit an intermediary software module that takes screenshots and obscures potentially sensitive information, e.g., password entry fields, but this would likely rely on some ML model to do so, as password entry elements and their accompanying text vary widely across applications. While password entry typically begins with a hidden textfield that a user optionally selects to un-hide, this particular element of privacy leakage remains an open concern.

ProvTracer addresses these privacy issues by literally telling the user what is about to occur before the system begins its session capture, allowing them to pause cancel at any time, and additionally, by drawing a constant disclaimer window on top of the screen that tells users of ProvTracer's status. ProvTracer includes a keyboard shortcut for pausing the application that may improve the user's ability to halt execution in timely fashion. See section 4.2 for the system implementation. To more concretely address the issue of privacy violation, dedicated work machines may be employed. Such an approach may be useful within a shared workplace by effectively restricting software workers to a limited set of work-related applications, but this obviously injures individual work processes that may rely on more personal applications. The side-by-side mode introduced in ProvTracer is a lightweight approach to the same issue, but it cannot be enforced on individual computers unless additional OS rules are implemented.

7.3.2. BIG BROTHER IS WATCHING YOU

The system developed in the course of the present dissertation tactfully addresses a specific subset of possible input provenance sources, while intentionally ignoring several others for the sake of completing a prototype. As such, there remains a wide variety of potential future work with respect to additional sources of provenance input. In an "ideal" scenario involving maximal provenance input, assuming the principle of information overload can be mitigated through additional filtering mechanisms, some knowledge worker would be capture with both a camera and a microphone at all phases of their digital work. However, this opens the concern of personal privacy, as addressed earlier.

To take this line of thought to an extreme, consider the theoretical scenario wherein a team of software engineers is laboring upon some software artifact. Their development provenance is

captured digitally at their desks using the system proposed in the present dissertation. But, when one engineer retreats to the restroom, his superior enters and, upon seeing him washing his hands, strikes up a casual conversation, mentioning offhandedly that he should look into some particular feature of their artifact. The engineer later addresses this feature by incorporating it into the system, but he is not capable of linking it to an issue, or a pull request, or some other provenance entity because it was spawned in the course of a physical conversation in a sensitive location, where (hopefully) recording devices were not present. The same can be said of non-working spaces within working establishments, e.g., hallways, dining halls, gymnasiums, etc.

The above scenario demonstrates the necessity of striving to keep conversations related to artifact development within the shared workspace. More ideally, meetings should be recorded or held digitally, as automated transcript software is a common feature among video conferencing applications, which gained massive prevalence as a result of the situation surrounding COVID-19 collection [282]. “There was of course no way of knowing whether you were being watched at any given moment… You had to live… in the assumption that every sound you made was overheard, and, except in darkness, every movement scrutinized.” [283]. This dated quote from Orwell’s 1984 (written in 1949) is made more awful by the fact that today, even in darkness, movement can be scrutinized, e.g., with night vision or thermal imaging. This is mentioned to highlight the concern of personal privacy conflicting with the desire for more and better provenance to feed into a digital system such as that described in the present dissertation. There must be a balance between personal privacy and optimal provenance capture.

7.3.3. Downstream Use

As established throughout the present dissertation, the use of some output provenance is typically dependent on some downstream business or personal need or use case (see section 5.2). In the context of business scenarios, a potential use case for ProvTracer output is for project managers wishing to more appropriately assign tasks to their team members, which can be made feasible by querying provenance trace link knowledge graphs in natural language with a MLLM, e.g., in asking what a particular developer worked on the most as opposed to his peers. However, it may be posited that the provenance descriptions within some output provenance trace link knowledge graph are peculiar to the individual who spawned them, making outside understanding difficult; e.g., if a developer submits his provenance trace link knowledge graphs to his senior engineer for managerial analysis, it may be asserted that this superior may not be able to glean the same meaning from the provenance descriptions as could the developer himself. This concern is mitigated by the recommendation to use a capable MLLM to interrogate the provenance trace link knowledge graphs, such as the GPT models evaluated in the present dissertation, and those used to assist in the manual lexical analysis of the output provenance descriptions in the case studies presented in sections 5.4 and 5.5. Such MLLMs are generalizable and agnostic, providing a common ground for querying.

7.4. The Chicken and the Egg

An idea that spawned early in the course of development of the present dissertation was to use the developed system (at that point, it being mostly theoretical, with minimal prototyping implemented) to capture the provenance traces of the dissertation deliverable; i.e., an initial, novel thought that gave impetus to the fulfillment of the present dissertation was to develop the

dissertation artifact alongside its engineered system, using the system to capture the artifact evolution. However, as the written dissertation serves as a description of the engineered system and the development of the system influences the dissertation, the two have, in practice, evolved simultaneously, with progress on one driving changes in the other. Idealistically, an engineered system developed in the course of dissertation work should be driven by a specific set of research questions, and/or a goal definition, e.g., as Wohlin posits [33], then fully developed and only thereafter reported on in the written dissertation itself; but, this is not realistic for a dissertation unfurling over the course of two years of research within the frenetic scene of large-scale generative AI. The tired paradoxical question, “What came first: the chicken or the egg?”, is apt here. In the case of the present written dissertation and its attendant engineered system, both spawned as a result of, and in direct accordance with, one another.

Chapter 8: FUTURE WORK

Given the novelty of the present dissertation, and the intentionally general nature of the coded system, there is a great deal of future work to consider, namely, possible sources of input provenance that may improve system efficacy in generating explainable traces. The concluding sections of the dissertation typically open more questions than are answered, by way of stimulating future researchers with broad ideas backed by the findings of the previous sections. This section addresses future work of interest to the continuation of the present dissertation, and to other provenance researchers. The provenance capture performed in the present dissertation is strictly limited to in-workstation data, i.e., that which is internal to the individual worker's computer; this future work section therefore considers external sources of provenance. Additionally, other application areas that may benefit from the use of ProvTracer are discussed.

The present dissertation accompanies what is, given a literature review, the first engineered, automated, real-time digital provenance capture system which reconciles structured knowledge with stochastic models. Notwithstanding, there is much room for future work. Warren Weaver put it aptly: "[This document] has been written with one hope only – that it might possibly serve in some small way as a stimulus to someone else, who would have the techniques, the knowledge, and the imagination to do something about it [147]".

8.1. OTHER WORK CONTEXT SOURCES

It is posited that there are innumerable potential sources of provenance context. This is mirrored in the knowledge fragmentation issue noted by Thaul, which assumes that processual knowledge is scattered over several sources, e.g., the human mind, on paper, on hard drives, and so on [18].

While acknowledging the potentially unbounded nature of the aforementioned issue, the present section introduces some potentially relevant sources of provenance context for personal computer work. There are many possible provenance sources that may positively augment system efficacy, as well as other contexts of interest aside from the SDLC which may benefit from the use of the system.

One extant context source that could be improved is mouse events; ProvTracer captures mouse clicks and scrolls. This could be extended with mouse movement stops/hovers, which may denote interest or focus, and additionally, click-and-drag events, which may denote an area of interest. More complex possible context sources are discoursed on in the ensuing subsections.

8.1.1. Pair Programming

Pair programming is a software development technique involving two developers working on some deliverable(s) together at the same workstation. While one developer writes code, the other reviews each line written; roles are often switched and in this tandem effort, less bugs are introduced into some developed system [284]. The Agile method, eXtreme Programming (XP), makes use of pair programming. When pair programming is employed, the participating developers generate verbal development provenance in their discourse between one another. Such direct development provenance is a prime source of additional context for MLLM prompting. This leads into the next potential work context source of narrator audio.

8.1.2. Narrator Audio

By default, the Windows operating system includes a screen-reading application, Windows Narrator, which makes computer use accessible to blind and visually impaired individuals. Windows was the OS used in the development of the present dissertation. To introduce more

```
1  WEBVTT
2
3
4  00:00:02.606 --> 00:00:04.606
5  Testing, testing, one, two, three.
6
7  00:00:08.544 --> 00:00:10.544
8  Testing, testing, four, five, six.
9
10 00:00:21.854 --> 00:00:23.854
11 Hello world!
12
13 00:00:34.665 --> 00:00:39.665
14 Goodbye world!
```

Fig. 57 Example of Zoom's closed captioning recording output in VTT format.

context in the LLM prompts, it was postulated that text transcriptions of Windows Narrator narrations could be used, as the Narrator application has OS-level privileges and can extract very precise actions, such as hovers over text fragments, highlights, button presses, hyperlink use, etc.

While a promising conceptual source of additional context, there was encountered a great deal of technical complexity in the attempted implementation; additionally, Windows Narrator narrations are specific to the Windows OS, so portability of the ProvTracer system would be injured. As such, the idea was ultimately abandoned.

From this idea, one may consider another, that of actual narrated audio. As some user works at their desk, they may be encouraged to audibly narrate their work as they perform it. Such a provision would enable the capture of their inner thoughts, intentions, motivations, etc. being voiced in real-time with their digital development work. This context source would be transcribed into plaintext and integrated with provenance prompts to the chosen MLLM. Perhaps even more promising than the use of Windows Narrator, the implementation of this context source possesses a number of issues:

- Temporal disconnect: if the selected timeslice interval in ProvTracer is, e.g., 5 seconds, and the average conversational speaking rate for both male and female British English speakers in 2020 is 198 words per minute [285], this means that, in the 5 second timeslice interval, roughly 16.5 words will be spoken; so, it must be considered: how many complete thoughts can be formed for effective provenance building in just under 20 words of speech? Moreover, there is the possibility of thought/sentence overlap between screenshot intervals. Temporal disconnect is discussed in greater detail in section 7.2.3.
- Microphone: dependence on a physical microphone introduces a difficult hardware problem, which is the identification and capture of the desired voices, e.g., just the developer in question, or in an XP environment, the developer and their development partner, or possibly an entire development team during a code review; in most physical settings, there is likely a great deal of literal noise in the background that may prove difficult to isolate from the desired voices; as ProvTracer employed, a small set of up-front options for delineating which voices in particular are desirable for some work session may address this issue

8.1.3. WORKPLACE RECORDING

Gotel and Finkelstein noted that, prior to requirements specifications being written, the tracing of requirements to their sources of elicitation, e.g., customer interviews, is very difficult because of the often informal nature of requirements elicitation activities, which often occur in physical spaces [76]. With the occurrence of the COVID-19 incident, which took hold of the entire world for approximately four years, the prevalence of remote work increased and became commonplace, chiefly in those domains where workers labor at personal computer workstations, e.g., software

engineering. Although there are many difficulties inherent in remote work, e.g., procrastination [286], there are a number of positives, including the possibility of perfect meeting provenance as a result of meetings being held digitally.

The digital videoconferencing application, Zoom, exploded in popular use as a result of COVID-19 for a litany of purposes, including qualitative data collection [282], interviews [287], remote learning [288], and daily work. Zoom allows for a large group of individuals (called a room) to converse, with one individual controlling what is displayed to the rest, called screen-sharing; this privilege can be passed off to others per request. Individuals can use their microphones or type in a built-in text chat to engage in conversation. Zoom allows the host of a room to record a session with closed captioning (CC) on, the content of which is transcribed into a Video Text Tracks (VTT) document. Zoom's public API has the tools to allow for real-time transcription, e.g., the real-time media protocol (RTMP), but any number of speech-to-text libraries integrated with a live audio stream may work instead. See Fig. 57.

More generally, this record-transcribe-utilize paradigm for digital meetings could potentially be applied to physical meetings through the use of recording equipment, e.g., microphones placed throughout work spaces and meeting rooms. This is simply the logical extreme of the digital workstation recording performed in the present dissertation. As discussed previously in a section regarding personal privacy, such recording should not take place without prior informed consent from all workers involved.

8.1.4. EYE TRACKING

A distinctly possible means of augmenting the system proposed herein with more accurate provenance may be found in the technique of eye tracking, which has been employed (through

analog means) in psychological studies since the 1960s, and later integrated with cameras and computer systems in the 1980s and beyond [289]. Eye tracking is defined as "an experimental method of recording eye motion and gaze location across time and task" [290]. With this definition, it is perfectly applicable here.

Eye tracking data of interest to the present dissertation may include, for instance, the length of pauses (possibly showing intent or interest) on particular on-screen elements. As the anatomy of the human eye restricts proper vision to a small portion of its overall field of view, called the fovea, the human mind necessarily points the fovea toward the stimulus currently being processed; this is the "eye-mind link" which makes eye tracking so decidedly useful [290]. The following quote mirrors the intent of the present dissertation: "Because of its high temporal sensitivity, eye tracking can provide a moment-by-moment insight into unfolding cognition rather than simply revealing the final outcome" [290]. This is the essence of provenance in artifact modification, which the CoEST notes in their outlook on trace link automation [27].

More complicated facets of eye tracking, e.g., pupillometry, which sees eye trackers measure pupil size change, could be used to note interest. The system of the present dissertation takes as input several modes of provenance input and synthesizes them using a LLM; eye tracking data would be sanitized and integrated into the input prompt structure in the same way as, e.g., keyboard strokes. Mouse clicks are currently recorded, which includes their position on-screen, using x-y coordinates; an eye tracking data sample includes a point of gaze estimate as an x-y pixel value, so the format is familiar. Eye tracking is not yet integrated with the present dissertation because it is intended to be an evolving artifact with a minimal baseline of provenance and ample consideration with respect to future work.

8.1.5. BRAIN COMPUTER INTERFACES

Brain Computer Interfaces (BCIs) are, as of 2024, a mostly novel, albeit promising, technology that are intended to directly connect living brains to external devices, e.g., computers. Early BCIs (1960s-1980s) involved sensors placed on human scalps to detect eye and brain intent to, e.g., move mouse cursors, spell words on-screen, etc. [291]. He et al. define a BCI in two parts [291]:

1. "A BCI is a system that measures CNS activity and converts it into artificial output that replaces, restores, enhances, supplements, or improves natural CNS output…"
2. "It can also be considered as a system to influence CNS activity and behavioral performance by injecting physical energy… and thereby changes the ongoing interactions between the CNS and its external or internal environment."

The second part of the definition is outside of the scope of the present dissertation, which does not intend to modify subject behavior in any way, but simply to capture such behavior. The first part of the definition is tangential to the possible future work proposed here, that a BCI could be used to derive near-instantaneous worker provenance in the form of intent, much like eye tracking provides user intent given gaze duration. Here, BCIs are treated as devices *external* to the human body, i.e., those which may be removed at the whim of the user. BCIs not considered are those internal to, i.e., embedded within, the human body, because such devices cannot be easily removed from the body, which may violate the underlying principles of informed consent. For biomedical practice, such a consideration may not be relevant; here, users must retain absolute control over their bodies as they are the generators of desired provenance. Real-world examples of internal BCIs may be considered with some constructs of the company founded in 2016 by Elon Musk and associates, Neuralink [292].

Although BCIs are not commonplace, microelectronic technologies, e.g., small antennae, earphones, etc., have been demonstrated as useful in academic cheating [208]. The cheating student is willing to use such technologies in much stricter environments than the workplace because of their belief in the potential benefit with respect to their individual academic performance. Within a workplace, a microtechnology such as a BCI may be useful not only to the individual worker, but to the larger team within which they are placed. The system developed within the course of the present dissertation demonstrated improved traceability and explainability, among other qualitative measures; BCIs properly implemented, just as with eye tracking, may further improve these measures and prove beneficial to all workers. This thinking may be used to justify the integration of some BCI with the system, although, as noted earlier, informed consent must be obtained from all individuals involved.

8.2. Application to Other Contexts of Interest

For the present dissertation, the SDLC presents the ideal use case for artifact provenance capture, because Agile development teams are small and utilize secondary artifacts, like Git issues and pull requests, to develop end software artifacts. Additionally, the author's background as a software engineer provides familiarity with the SDLC, making it a natural and practical choice for applying the developed system. Yet, there are other contexts, or domains, of interest to the present dissertation, which may benefit from the use of a digital provenance capture system, e.g., that developed herein. Because the system retains a level of generality through the use of stored prompts, augmenting it for use within other contexts is possible with new prompts. Some of these additional contexts of interest are given below.

- Open-Source and Community Projects: in much the same way as the Agile-oriented SDLC use case presented herein, open-source projects are an aggregation of community decisions and contributions;
- Scientific Research: the scientific workflow is laden with various sub-processes all including personal intuition, thought, assumption, postulation, construction, testing, analysis, writeup, etc.; these are sources of scientific provenance that may lend to greater credibility, reproducibility, etc. [20]; an end-to-end ontology for scientific workflow provenance representation has been presented by Sheeba and König-Ries [293]
- Healthcare and Clinical Trials: provenance for clinical trial data, medical research and patient care processes may support regulatory compliance, e.g., for the Food and Drug Administration (FDA) or Health Insurance Portability and Accountability Act (HIPAA) by creating audit trails
- Education and E-Learning: learning is a process of discovery and remembrance of key elements; the provenance of curriculum development and learning materials, along with documentation of student progress, may be used to generate personalized student knowledge graphs for, e.g., later class assignment
- Creative Works (Art, Music, Film, etc.): creative works are spawned as a result of skilled, iterative processes beginning with ideation, culminating in finishing or releasing them to the public; the provenance of such may support attribution and copyright enforcement for creative artifacts
- Knowledge Management (KM): knowledge management is any process involving, in general, the organization, retrieval and use of information [294]

- Personal Knowledge Management (PKM): KM at the personal level [294], not always bound to some corporate entity or with any goal other than personal interest, e.g., learning
- Continuous Integration and Continuous Delivery (CI/CD): CI/CD streamlines code testing and deliver; ProvTracer captures developer intent for these steps, linking rationale to pipeline actions for better traceability

All of these contexts see personal skill and experience utilized across digital applications resulting in an accumulation of provenance that may prove beneficial to those engaged in their work. The application of the ProvTracer system to the ML lifecycle and open-source projects is very clear. Tracing the lineage of one's own ideas is surely of interest to those engaged in KM and PKM. All other contexts discussed would benefit from automated provenance capture, but the integration with a discriminating LLM through prompts would require adjustment for each individual context.

ProvTracer was built around the idealized Agile team and lifecycle, i.e., small teams operating within short iterations, and for individual work processes. Its expansion to larger projects, e.g., those being labored upon within multi-team, distributed firms, is considered in two following subsections on ML provenance and the Personal Software Process.

8.2.1. Machine Learning Provenance

The ML lifecycle is a subset of the general SDLC, but with facets peculiar to ML modeling, e.g., dataset curation, training, testing, validation, etc. [295]. The same motivation that drives ProvTracer for the SDLC and digital work in general, i.e., automating traceability and explainability to engender reproducibility and transparency, is applicable to the ever-burgeoning field of ML, which is almost entirely digital in nature. Much provenance exists within the ML

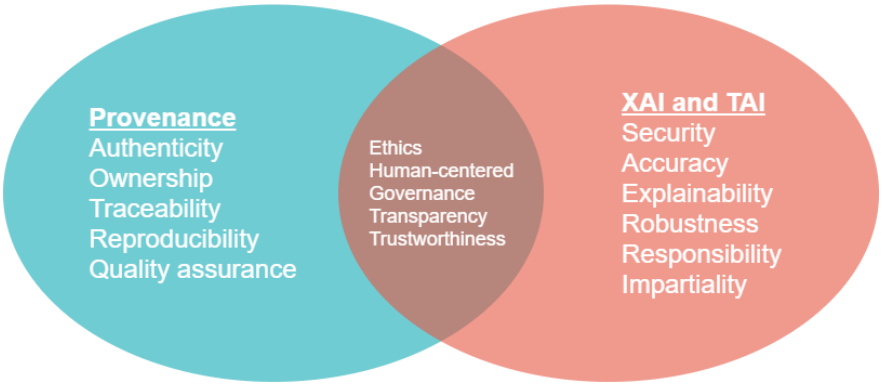

Fig. 58 Venn diagram of provenance, XAI, and TAI (adapted from [296]).

lifecycle that goes unrecorded and can benefit reproducibility, lending to FAIR ML [244]; data privacy, trustworthiness, and authenticity are also a concern, particularly for massive ML datasets, that may be addressed by the universal adoption of data provenance standards [36]. A large 2023 SLR established the importance of provenance for explainable and trustworthy AI (XAI and TAI, respectively), noting a few tools purporting to accommodate such [296]. See Fig. 58. The agnostic and automatic provenance capture provided by ProvTracer is an early means of achieving XAI and TAI for the ML lifecycle; the author hopes that other researchers may draw from the present dissertation and apply what is digested here for the purpose of bettering future ML work and research.

8.2.2. The Personal Software Process

Developed by Watts Humphrey, the "father of software quality", the Personal Software Process (PSP) is a structured framework of developing software used to understand where development time is spent [297]. The PSP is useful for optimizing developer throughput, analyzing developer failures and generating more accurate time estimates, but it relies on manual time/data entry and annotation by individual developers, so it suffers from typical issues of noise and inaccuracy [273]. There are a number of researched and developed software applications, or assistants, that purport

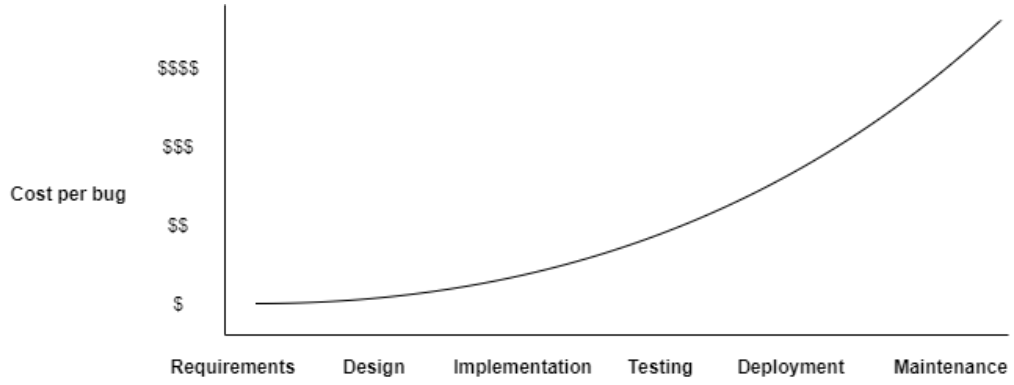

Fig. 59 Abstracted software bug detection vs. bug fix cost graph.

to streamline the PSP, e.g., Process Dashboard [298], but true automation has eluded the PSP because the wide variance in software applications used in development limits the possibility of any one PSP application capturing all necessary data. Granting this, the same idea that underlies the present dissertation may be applied to the PSP, which is that an agnostic provenance capture application underpinned by a screenshotting loop integrated with a prompted LLM can alleviate the issue of automating capture across any used applications. The ProvTracer system developed herein would be an ideal baseline for this - it would simply amount to additional stored LLM prompts tailored to PSP concerns. The same potential exists for the Team Software Process (TSP), which extends PSP concepts to team settings [299]. Such a provision would be beneficial to, e.g., project managers for task assignment.

A software development principle established early in the history of the field is that the cost, either in time, money, or both, to fix some bug increases by an order of magnitude with each stage of development; i.e., it is beneficial to identify and fix bugs as early as possible. Boehm discusses this as early as 1983 [300]. See Fig. 59. The PSP, when properly implemented and adhered to, can assist in lowering software development costs insofar as developer time is

documented and tasks can be assigned more judiciously across teams. In a more low-level context, provenance has been used to establish timing predictions for software [301].

ProvTracer, an automatic provenance capture system, can be beneficial for temporal and fiscal concerns, by documenting developer progress and work times. To accomplish this, it is necessary to theorize what additional features ProvTracer must possess; these constitute a discussion of future work and insight for other researchers. Currently, ProvTracer captures temporal information, e.g., timestamps of each timeslice packet. These may form the basis of time estimation for tasks undertaken by some ProvTracer user, but the difficulty lies in the MLLM employed being able to adequately distinguish between active tasks. For instance, some ProvTracer user may be working on some function dictated for implementation in a Git issue; the ProvTracer system may witness this work, but it is unable to "notice" when the user finishes this task and moves on to another, i.e., the capability for ProvTracer to concretely assert what task is actively being done, so it is later able to assert that said task is complete, is not currently implemented. The MLLM underlying ProvTracer is capable of generally describing what is occurring in a given screenshot, but it is unable to establish what is actively occurring via user input. This is discussed in greater detail in a section detailing temporal disconnect (see section 7.2.3) and another about delta analysis (see section 8.3.1).

8.3. TECHNICAL IMPROVEMENTS

ProvTracer was conceived, designed, architected and implemented in a frenetic period of AI expansion. Its dependence on a cloud-based MLLM was established as an early requirement to rapidly demonstrate system efficacy; notwithstanding, there are a number of technical improvements, particularly to this dependence, that may benefit the dissemination and use of

ProvTracer for SDLC work and digital work in general. These noted improvements are detailed in the following subsections.

8.3.1. Delta Analysis

One aspect of digital work not directly addressed by ProvTracer is the progression of tasks, including beginning and completion. In digital work, people undertake tasks, generating new ones as necessary, even if done so implicitly. Between the end of some timeslice and the beginning of the next, there is a delta of work performed on some task(s). If all text in a given screenshot were able to be transcribed, then it would be possible to establish deltas between timeslices. Contrastive prompts could be asked of the MLLM, e.g., "what changed since the last timeslice?". Section 4.1.3 discusses the ability of the MLLM to detect differences in text between pairs of images, but, as established there are practical limits, primarily stemming from the complexity of digital application windows. GPT-4o is not reliable in detecting textual deltas when on-screen application windows shift, e.g., when open code in IDEs are scrolled up or down between screenshots. Section 7.2.3 discusses temporal disconnect and the issues inherent in sending more than one image at a time to the MLLM in use.

Were ProvTracer capable of delta analysis, it would be feasible to address work concerns like software time estimation, as discussed in section 8.2.2 on the PSP. Nevertheless, supposing deltas can be established between a pair, or some set, of screenshots, the difficult of establishing exactly when some task is started or completed still remains. It is possible to theorize an extension to ProvTracer that, in the primer window, asks users to explicitly declare and define their set of planned tasks for a given work session, e.g., writing a function named *addNumbers()* and refactoring a class named *calculations.py*, which would then be rendered continuously on the

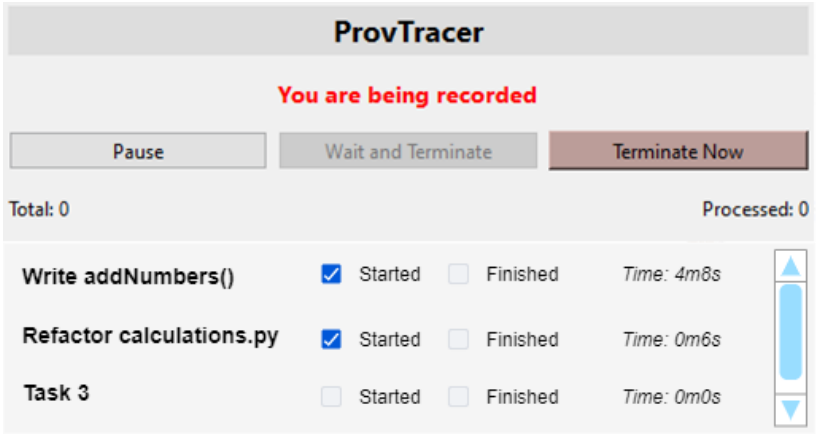

Fig. 60 Wireframe mockup of a task-oriented ProvTracer disclaimer window.

disclaimer window of ProvTracer. See Fig. 60. These tasks could be checked as being "started" by the user, and checked as "finished" by the user during live operation of ProvTracer during a work session. When such start and finish events occur, ProvTracer could take the starting screenshot and the ending screenshot, and use the MLLM to induce a delta between the two. In the current iteration of ProvTracer, this is not implemented, because the intent of the system is to remain as lightweight and automated as possible. The introduction of planned tasks violates this intent and forces a burden upon the user, which may prove to be an issue, as users can simply forget to update their tasks on time. Thaul notes this issue, stating that "the implemented algorithms need to meet the working patterns of the single user. The system will work relatively well for users which tend to digitalise their intermediate results promptly. Users who tend to collect their ideas first, and start writing them down with a certain time shift will dismiss the idea of the system. To gain the most advantages of the software, users with such behaviour would need to adopt their personal working patterns." [18]. The planned tasks feature could be implemented as an optional setting in ProvTracer, allowing for a more PSP-oriented approach to provenance capture.

A potential complication to this approach lies in the content of the starting and ending screenshots, as it is possible that the user has switched between applications, so the delta may be

so significant that it is irrelevant or otherwise useless. To mitigate this issue, it may be necessary to take a "running delta" across all screenshots in the duration of some planned task, which would involve the same temporal and fiscal issues noted earlier in section 7.2.3. As such, delta analysis remains a concern for future work.

8.3.2. SOURCE ARTIFACT RAG

A major source of influencing provenance, particularly for the side-by-side mode introduced in section 4.2.5, is source artifact RAG. This would allow a ProvTracer user to upload their source artifact(s), e.g., SRS documents, which the model could then access directly via text, rather than relying solely on visual fragments present in the screenshot-based capture loop. Source artifact RAG would effectively be the most concrete form of context priming possible for ProvTracer. Implementing source artifact RAG would require restructuring the system to support persistent document indexing and contextual retrieval during inference, likely via the GPT Assistants API or equivalent tooling. While this approach could significantly improve the contextual fidelity of provenance traces, it also introduces non-trivial tradeoffs: increased response latency, higher token consumption, and greater system complexity. For these reasons, source artifact RAG integration was deferred in the current iteration to preserve the lightweight and responsive architecture of ProvTracer.

To address the concerns of latency and cost, prompts to the OpenAI MLLMs could be cached[9], which would result in a saving of tokens and less money spent. As prompts occur on a

[9] Prompt caching is useful for repetitive content or common instructions. This technique is relegated to a few models: https://platform.openai.com/docs/guides/prompt-caching

set interval with a default timeslice of 5 seconds, and these prompts contain a good deal of repeated context prefaces, prompt caching may be beneficial for the deployment of ProvTracer at scale.

8.3.3. AGNOSTIC LLM USE

ProvTracer allows users to select from three MLLMs: GPT-4-Turbo, GPT-4o, and GPT-4o-mini. Because of the absolute opaqueness of OpenAI's MLLMs, it is posited that, to allow a more widespread adoption of ProvTracer, the system would benefit from the ability to select any MLLM desired. This would align ProvTracer with the Model Openness Framework (MOF), which intends to introduce reproducibility, transparency, explainability, accountability, fairness, collaboration, and more, to the AI landscape [302]. As of early 2025, several models by Anthropic, e.g., Claude 3.7 Sonnet, Claude 3.5 Haiku and Claude 3 Opus, and several models by Google Deepmind, e.g., Gemini 2.0 Flash and Gemini 2.0 Pro [303, 304], are the foremost competitors to the vision capable GPT models demonstrated for use in the present work.

Ideally, ProvTracer would align more closely with open-source principles, allowing even local MLLMs, i.e., those that require no dependence on remote computation, to be integrated in its provenance capture loop. The difficulty with this aspiration is primarily logistic in nature, insofar as most locally executable MLLMs require considerable computational power, and average users may not be able to accommodate such. Those locally executable MLLMs which are small for more efficient computation lack the visual capabilities of their larger counterparts. Shunting computation to a cloud service, while undesirable in the context of open-source concerns, allows ProvTracer to remain lightweight. In the future, it would be ideal for ProvTracer to allow a user to optionally select a local MLLM for use, accommodating the input as textual/visual, and receiving the same expected textual response.

8.3.4. MLLM Pipelining

An alternative approach to that taken by ProvTracer is to arrange two models into a pipeline instead of relying on one. For instance, GPT-4o may be used to generate a textual description of each screenshot, where another model, e.g., GPT-3.5 (cheaper and faster, but capable on text) may then be used to generate a RDF graph from the textual description. This thinking could be taken to an extreme. As the MLLMs evaluated herein display a distinct lack of ability in spatial reasoning (see section 4.1.1), they may be considered simply as text extraction constructs, so it could be posited that, in place of a fiscally and computationally expensive MLLM, an OCR construct could be used to generate textual transcriptions of screenshots, which could then be fed to a MLLM for purely text-based descriptions of provenance. This would remove the complexity of prompting visually and textually. However, as digital screens may contain a litany of application windows in various locations, it must be noted that an OCR construct may struggle to properly retain the semantics of the visually rendered text.

8.3.5. Session Resumption

The implementation and evaluation of ProvTracer was performed within the context of Agile work processes, i.e., those that are brief and prototypical. ProvTracer work sessions are modelled atomically, i.e., they exist as individual occurrents with associated continuant facets. When a work session is started in ProvTracer, it is intended to be completed, and its resultant provenance trace link knowledge graph output. One possible future expansion to ProvTracer would be the affordance to users to resume previous work sessions, i.e., add to the output knowledge graph with more work provenance. The most direct means of resuming previous work sessions is to begin with the ProvTracer UI explicitly asking users if they intend to begin a new session or resume a

previous one. If framed in the context of a work day, e.g., in some software development firm, then session resumption is likely irrelevant; and, users have the option to pause ProvTracer and resume an open session at any time. The ability to resume past sessions for more planned work within the same session would benefit output graph organization insofar as ProvTracer users are able to compartmentalize their work sessions in such a fashion as to re-use, and add to, extant ones. Additionally, once a work session is begun in ProvTracer past the primer window, the set of primer contexts cannot be altered. Although a user can simply terminate their ProvTracer session and begin another, and then enter primer contexts as desired anew, it may be beneficial to allow users to pause and update their initialization contexts.

8.3.6. MULTI-USER INTEGRATION

In software development work, teams commonly use Internet-enabled applications to labor upon deliverable artifacts, sometimes at the same time as one another. These are collaborative development environments (CDEs) [305]. An example is Google Docs, which allows multiple users to edit the same document together in real-time. Consider some software target artifact being labored upon by more than one developer while a ProvTracer work session is open. Edits by some other developer may occur that are captured by the timeslice loop of ProvTracer, i.e., the user with an open ProvTracer session is not making the edits. This opens an issue of capturing provenance for other users not in ownership of a given work session. An approach to this issue could be to integrate multi-user functionality within ProvTracer, perhaps in an optionally selected component, but this was not planned for in the design of the system, so it remains a possible future consideration.

8.4. LONGITUDINAL EVALUATION

The most pressing limitation of the present dissertation is the method of evaluation. While ProvTracer is fully implemented and capable of producing provenance data in realistic software engineering contexts, its evaluation is restricted to short-term, subjective feedback. The survey-based evaluation borrows from that of Thaul, who did not implement their knowledge formation capture system, but described its formation, including its components, information objects, storage, and use, then confirmed the system's theoretical usefulness with experts, thereafter substantiating this confirmation with a survey of university students [18]. No structured, quantitative benchmarking or task-based validation was feasible for ProvTracer due to the interpretive and dynamic nature of its outputs.

In the timeframe of development of the present dissertation, it was not possible to properly construct and conduct a longitudinal evaluation of the ProvTracer system over a broader population, e.g., a capstone course group or industry team. This would provide a real-world demonstration of the system, entailing a full usability study, satisfaction survey, etc. Ideally, a long-term study incorporating the feedback of several small software engineering groups would be conducted, e.g., within a student course revolving around a constructed software product. This would likely include personal work diaries and a qualitative analysis of developer comments with interviews to uncover themes of use, user expectations, positive and negative feedback, etc. Proven qualitative research approaches such as ethnography, diary studies, and semi-structured interviews [239, 306] could be applied to gather in-depth, contextual feedback about how ProvTracer integrates with workflows, influences decision-making, and either supports or impedes traceability practices. This combination of methods supports triangulation and helps bring about both expected

and emergent uses of the system. A qualitative approach is essential, given the lack of an objective ground truth for "correct" provenance, and aligns with the interpretive nature of trace construction itself. In future work, a longitudinal evaluation could offer a more grounded, human-centered understanding of the viability and value of automated provenance in live software development.

8.5. A Final Warning

As discoursed on previously, there must be a strict delineation between what is useful for the system in terms of input provenance and what is unacceptable to capture with respect to individual privacy and autonomy (see section 7.3). The encompassing section on future work discusses a number of directly useful forms of possible provenance sources, but goes no farther inasmuch as the possibility of personal privacy violation may occur. Even with informed consent obtained, some means of harvesting development provenance may be regarded as existing in a grey area with respect to moral philosophy. Such sources of provenance may include:

- Any means of capturing possible worker provenance outside of established work areas and/or hours, e.g., bathrooms
- Any means of capturing biometric telemetry, e.g., heart rate, skin conductance, movement, etc.
- Facial expression or emotion tracking via camera
- BCIs embedded in the human body, i.e., those not readily removed by the worker
- Cross-device surveillance, especially if devices are used for personal reasons

In seeking improved provenance capture, particularly sources of influencing context, the practices to do so are posited to be a slippery slope. ProvTracer demonstrates efficacy in automated provenance capture with a modicum of selected context sources which are voluntarily enabled;

any further context capture may violate personal privacy, even if voluntarily enabled, insofar as, e.g., biometric data capture is not necessarily willful or under one's direct control.

Chapter 9: CONCLUSION

The present dissertation exists as one of the first and few research artifacts bridging the knowledge chasm between research on provenance within the Semantic Web movement, e.g., the PROV ontology, and research on software traceability, a concern of software practitioners. Despite a dearth of literature associating these two, the present dissertation posits that software traceability is an operationalization of provenance, which hitherto has been relegated primarily to Semantic Web projects. In seeking a concrete realization of this position, the ProvTracer system, which automates provenance capture for the formation of provenance trace link knowledge graphs constructed in adherence to PROV-O and BFO, is implemented and demonstrated as efficacious for personal computer work. There exists a voluminous body of literature and systems focusing on the Continuant aspects of digital work, but comparatively little attention has been given to the Occurent portions – namely, the processes by which Continuants are formed by knowledge workers. This is the provenance the ProvTracer system intends to capture and make concrete. ProvTracer is evaluated in terms of its ability to engender explainability, traceability, interoperability, and its ability to mitigate information overload in the context of the SDLC. Furthermore, in establishing these non-functional requirements, ProvTracer contributes to reproducibility and transparency, which is an important emerging concern for FAIR data in the current zeitgeist of rapidly expanding large-scale AI in both research and industry work. ProvTracer is a novel system, insofar as a literature survey of related work located no automated provenance capture system for digital work incorporating structured knowledge representation with a MLLM; accordingly, the present dissertation also provides deep insight into the limitations and concerns of systems as such, as well as a detailed discussion of directions for future research.

It is the author's hope that the present dissertation artifact may provide insight into the complex problem of adequately accounting for provenance in digital work, not just in the SDLC, but within any discipline, personal or professional.

CONFLICT OF INTEREST

The author declares no conflicts of interest with respect to the research, authorship, or publication of this dissertation. Although the author is a recipient of the Department of Defense Science, Mathematics, and Research for Transformation (SMART) Scholarship for Service Program, the dissertation was conducted independently and was not influenced by any external organization, agency, or directive. No personal, financial, or professional affiliations have impacted the content, direction, or outcomes of the work presented herein.

AUTHOR STATEMENT

The present dissertation is the product of extensive research, design, development, and evaluation conducted by the author. While every effort has been made to ensure the accuracy, clarity, and academic rigor of the content, the author acknowledges that no document of this nature is ever truly final or without error. This work should be considered a living artifact: open to revision, expansion, and reinterpretation as technologies, methods, and discourse evolve. The author welcomes future critique, adaptation, and improvement of the ideas presented and its representation in this document.

The ProvTracer system can be found in its GitHub repository:

- https://github.com/PR0CK0/ProvTracer

THE MAN WHO BELIEVES THAT THE SECRETS OF THE WORLD ARE FOREVER HIDDEN LIVES IN MYSTERY AND FEAR. SUPERSTITIION WILL DRAG HIM DOWN. THE RAIN WILL ERODE THE DEEDS OF HIS LIFE. BUT THAT MAN WHO SETS HIMSELF THE TASK OF SINGLING OUT THE THREAD OF ORDER FROM THE TAPESTRY WILL BY THE DECISION ALONE HAVE TAKEN CHARGE OF THE WORLD AND IT IS ONLY BY SUCH TAKING CHARGE THAT HE WILL EFFECT A WAY TO DICTATE THE TERMS OF HIS OWN FATE. – *Cormac McCarthy*

REFERENCES

[1] J. W. Tukey, "The Teaching of Concrete Mathematics," *The American Mathematical Monthly,* vol. 65, no. 1, pp. 1-9, 1958.

[2] H. D. Benington, "Production of Large Computer Programs," *Proceedings of the ONR Symposium on Advanced Programming Methods for Digital Computers,* pp. 15-27, 1956.

[3] W. W. Royce, "MANAGING THE DEVELOPMENT OF LARGE SOFTWARE SYSTEMS," *Proceedings, IEEE WESCON,* pp. 1-9, 1970.

[4] P. Abrahamsson, O. Salo, J. Ronkainen and J. Warsta, "Agile software development methods: Review and analysis," *arXiv preprint arXiv:1709.08439,* 2017.

[5] T. Dingsøyr, S. Nerur, V. Balijepally and N. B. Moe, "A decade of agile methodologies: Towards explaining agile software development," *Journal of systems and software,* vol. 85, no. 6, pp. 1213-1221, 2012.

[6] H. Blatt, "Provenance Determinations and Recycling of Sediments," *SEPM Journal of Sedimentary Research,* vol. 37, 1967.

[7] P. Buneman, S. Khanna and W.-C. Tan, "Why and Where: A Characterization of Data Provenance," *International Conference on Database Theory,* 2001.

[8] R. H. Lytle, "Subject retrieval in archives: a comparison of the provenance and content indexing methods," *University of Maryland,* 1979.

[9] D. A. Bearman and R. A. Lytle, "The Power of the Principle of Provenance," *Archivaria*, vol. 21, 1985.

[10] L. Moreau, P. Missier, K. Belhajjame, R. B'Far, J. Cheney, S. Coppens, S. Cresswell, Y. Gil, P. Groth, G. Klyne, T. Lebo, J. McCusker, S. Miles, J. Myers, S. Sahoo and C. Tilmes, "PROV-DM: The PROV Data Model," W3C, 2013. [Online]. Available: https://www.w3.org/TR/2013/REC-prov-dm-20130430/. [Accessed 2023].

[11] L. Moreau, B. Clifford, J. Freire, J. Futrelle, Y. Gil, P. Groth, N. Kwasnikowska, S. Miles, P. Missier, J. Myers, B. Plale, Y. Simmhan, E. Stephan and J. Van den Bussche, "The Open Provenance Model Core Specification (v1.1)," *Future Generation Computer Systems*, 2010.

[12] W. Al-Ahmad, K. Al-Fagih, K. Khanfar, K. Alsamara, S. Abuleil and H. Abu-Salem, "A Taxonomy of an IT Project Failure: Root Causes," *International Managementt Review*, vol. 5, no. 1, pp. 93-106, 2009.

[13] M. Herschel, R. Diestelkämper and H. B. Lahmar, "A Survey on Provenance: What For? What Form? What From?," *The International Journal on Very Large Data Bases*, vol. 26, pp. 881-906, 2017.

[14] J. Riley, "Understanding Metadata," *National Information Standards Organization*, 2017.

[15] J. C. Bauer, E. John, C. L. Wood, D. Plass and D. Richardson, "Data entry automation improves cost, quality, performance, and job satisfaction in a hospital nursing unit," *JONA: The Journal of Nursing Administration*, vol. 50, no. 1, pp. 34-39, 2020.

[16] T. Nandwani, M. Sharma and T. Verma, "Robotic process automation–automation of data entry for student information in university portal," *Proceedings of the International Conference on Innovative Computing & Communication (ICICC)*, 2021.

[17] R. Arp, B. Smith and A. D. Spear, Building Ontologies with Basic Formal Ontology, Cambridge: The MIT Press, 2015.

[18] W. Thaul, "Supporting Learning by Tracing Personal Knowledge Formation," *University of Plymouth - PEARL*, 2014.

[19] Y. L. Simmhan, B. Plale and D. Gannon, "A survey of data provenance techniques," *Computer Science Department, Indiana University, Bloomington IN 47405* , Vols. Technical Report IUB-CS-TR618 , 2005.

[20] S. B. Davidson and J. Freire, "Provenance and scientific workflows: challenges and opportunities," *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pp. 1345-1350, 2008.

[21] S. Charalampidou, A. Ampatzoglou, E. Karountzos and P. Avgeriou, "Empirical studies on software traceability: A mapping study," *ournal of Software: Evolution and Process*, vol. 33, no. 2, 2021.

[22] J. Cleland-Huang, O. Gotel and A. Zisman, Software and systems traceability, vol. 2, London: Springer, 2012.

[23] "IEEE 610.12-1990 IEEE Standard Glossary of Software Engineering Terminology," 31 December 1990. [Online]. Available: https://standards.ieee.org/ieee/610.12/855/. [Accessed June 2024].

[24] N. Mustafa and Y. Labiche, "Employing Linked Data in Building a Trace Links Taxonomy," *ICSOFT,* pp. 186-198, 2017.

[25] W. U. M. Abdeen, A. Chirtogloub, C. P. Schimanskib, H. Golib and K. Wnuka, "Taxonomic Trace Links-Rethinking Traceability and its Benefits," 2022.

[26] C. V. Ramamoorthy, V. Garg and A. Prakash, "Programming in the Large," *IEEE Transactions on Software Engineering,* vol. 7, pp. 769-783, 1986.

[27] "Center of Excellence for Software & Systems Traceability," 2023. [Online]. Available: http://sarec.nd.edu/coest/index.html. [Accessed September 2024].

[28] S. D. Bruda and S. G. Akl, "Real-time computation: A formal definition and its applications," *International Journal of Computers and Applications,* vol. 25, no. 4, pp. 247-257, 2003.

[29] A. Kejariwal and F. Orsini, "On the definition of real-time: applications and systems," *2016 IEEE Trustcom/BigDataSE/ISPA,* pp. 2213-2220, 2016.

[30] R. T. Dodhiawala, N. S. Sridharan, P. Raulefs and C. Pickering, "Real-Time AI Systems: A Definition and An Architecture," *IJCAI,* pp. 256-264, 1989.

[31] W. R. Schowalter, D. C. Drucker, A. H. Flax, C. W. Gear, P. C. Jennings, S. Ostrach, A. R. Seebass, J. A. White Jr., B. Guile, D. Stine and J. Blake, "WHAT IS ENGINEERING RESEARCH AND HOW DO ENGINEERING AND SCIENCE INTERACT?," in *Forces Shaping the U.S. Academic Engineering Research Enterprise*, Washington, D.C., National Academy Press, 1995, pp. 3-4.

[32] R. Shavelson and L. Towne, "Scientific Research in Education," 2002.

[33] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell and A. Wesslén, "Experimentation in software engineering," *Springer Science & Business Media,* 2012.

[34] V. R. Basili, R. W. Selby and D. H. Hutchens, "Experimentation in software engineering," *IEEE Transactions on software engineering,* vol. 7, pp. 733-743, 1985.

[35] W. J. Vlietstra, R. Vos, A. M. Sijbers, E. M. van Mulligen and J. A. Kors, "Using predicate and provenance information from a knowledge graph for drug efficacy screening," *Journal of biomedical semantics,* vol. 9, no. 23, 2018.

[36] S. Longpre, R. Mahari, N. Obeng-Marnu, W. Brannon, T. South, J. Kabbara and S. Pentland, "Data Authenticity, Consent, & Provenance for AI are all broken: what will it take to fix them?," *https://arxiv.org/pdf/2404.12691,* 2024.

[37] R. L. Rivest, A. Shamir and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *ommunications of the ACM,* vol. 21, no. 2, pp. 120-126, 1978.

[38] M. Crosby, P. Pattanayak, S. Verma and V. Kalyanaraman, "Blockchain technology: Beyond bitcoin," *Applied innovation,* no. 2, 2016.

[39] I. Cox, M. Miller, J. Bloom, J. Fridrich and T. Kalker, Digital watermarking and steganography, Morgan Kaufmann, 2007.

[40] R. Tang, Y.-N. Chuang and X. Hu, "The Science of Detecting LLM-Generated Text," *Communications of the ACM 67.4,* vol. 67, no. 4, pp. 50-59, 2024.

[41] V. S. Sadasivan, A. Kumar, S. Balasubramanian, W. Wang and S. Feizi, "Can AI-generated text be reliably detected?," *arXiv preprint arXiv:2303.11156,* 2023.

[42] J. White, S. Hays, Q. Fu, J. Spencer-Smith and D. C. Schmidt, "Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," *arXiv preprint arXiv:2303.07839*, 2023.

[43] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," *Association for Computing Machinery*, 2022.

[44] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez and R. Stojnic, "Galactica: A large language model for science," *arXiv preprint arXiv:2211.09085*, 2022.

[45] E. M. Bender, T. Gebru, A. McMillan-Major and S. Shmitchell, "Bender, Emily M., Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?🦜," *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pp. 610-623, 2021.

[46] D. Nicholas, A. Watkinson, H. R. Jamali, E. Herman, C. Tenopir, R. Volentine, S. Allard and K. Levine, "Peer review: Still king in the digital age," *Learned Publishing*, vol. 28, no. 1, pp. 15-21, 2015.

[47] Y. Li, "An open source data contamination report for llama series models," *arXiv preprint arXiv:2310.17589*, 2023.

[48] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for

Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.

[49] K.-L. A. Yau, H. J. Lee, Y.-W. Chong, M. H. Ling, A. R. Syed, C. Wu and H. G. Goh, "Augmented intelligence: surveys of literature and expert opinion to understand relations between human intelligence and artificial intelligence," *IEEE Access*, vol. 9, pp. 136744-136761, 2021.

[50] Y. Yoo, R. J. Boland Jr, K. Lyytinen and A. Majchrzak, "Organizing for innovation in the digitized world," *Organization Science*, vol. 23, no. 5, pp. 1398-1408, 2012.

[51] B. H. Hugh, "Socratic Method," *The Cambridge Companion to Socrates*, pp. 179-200, 2011.

[52] C. Ronquillo, L. M. Currie and P. Rodney, "The evolution of data-information-knowledge-wisdom in nursing informatics," *Advances in Nursing Science*, vol. 39, no. 1, 2016.

[53] C. Zins, "Conceptual approaches for defining data, information, and knowledge," *Journal of the American society for information science and technology*, vol. 58, no. 4, pp. 479-493, 2007.

[54] R. Nelson and N. Staggers, "Health informatics: An interprofessional approach," *Elsevier Health Sciences*, 2013.

[55] B. W. Boehm, "Seven Basic Principles of Software Engineering," *Journal of Systems and Software*, vol. 3, no. 1, pp. 3-24, 1983.

[56] T. T. Procko, "Towards Agile Academia: An Approach to Scientific Paper Writing Inspired by Software Engineering," *Embry-Riddle Aeronautical University Scholarly Commons*, 2024.

[57] D. Cohen, M. Lindvall and P. Costa, "Agile software development," *Dacs Soar Report*, vol. 11, 2003.

[58] K. Beck, M. Beedle, A. V. Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland and D. Thomas, "Manifesto for agile software development," 2001.

[59] F. Ji and T. Sedano, "Comparing extreme programming and Waterfall project results," *2011 24th IEEE-CS Conference on Software Engineering Education and Training (CSEE&T)*, 2011.

[60] "IEEE Recommended Practice for Software Requirements Specifications," *IEEE Std 830-1998*, pp. 1-40, 1998.

[61] B. W. Boehm, "SOFTWARE ENGINEERING - AS IT IS," *IEEE Transactions on Computers*, vol. 25, no. 12, pp. 1226-1241, 1976.

[62] W. Schacchi, "Understanding the requirements for developing open source software systems," *IEE Proceedings-Software*, vol. 149, no. 1, pp. 24-39, 2002.

[63] J. P. Cavano and J. A. McCall, "A Framework for the Measurement of Software Quality," *Proceedings of the Software Quality Assurance Workshop on Functional and Performance Issues*, 1978.

[64] M. Glinz, "On non-functional requirements," *15th IEEE international requirements engineering conference (RE 2007)*, pp. 21-26, 2007.

[65] B. W. Boehm, J. R. Brown and M. Lipow, "Quantitative Evaluation of Software Quality," 1976.

[66] T. Procko and O. Ochoa, "An Ontology Based Approach to the Software Engineering Lifecycle: Application to Software Quality Assurance in the Aerospace Industry," *Available at SSRN 4703235*, 2022.

[67] "Systems and Software Engineering - Systems and Software Quality Requirements and Evaluation (SQuaRE) - System and Software Quality Models," *ISO/IEC*, 2011.

[68] ISO/IEC, "ISO/IEC 25010:2023 - Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model," November 2023. [Online]. Available: https://www.iso.org/standard/78176.html. [Accessed June 2024].

[69] L. Madeyski and B. Kitchenham, "Would wider adoption of reproducible research be beneficial for empirical software engineering research?," *Journal of Intelligent & Fuzzy Systems*, vol. 32, no. 2, pp. 1509-1521, 2017.

[70] M. Hutson, "Artificial Intelligence Faces Reproducibility Crisis," *Science*, vol. 359, pp. 725-726, 2018.

[71] M. Baker, "Reproducibility Crisis," *Nature*, vol. 533, no. 26, pp. 452-454, 2016.

[72] J. Cleland-Huang, O. C. Gotel, J. H. Hayes, P. Mäder and A. Zisman, "Software Traceability: Trends and Future Directions," *Future of software engineering proceedings*, pp. 55-69, 2014.

[73] "ISO/IEC/IEEE 24765:2017 Systems and software engineering — Vocabulary," 2017. [Online]. Available: https://www.iso.org/standard/71952.html. [Accessed June 2024].

[74] P. Bourque and R. E. Fairley, "Guide to the Software Engineering Body of Knowledge, Version 3.0," IEEE Computer Society, 2014. [Online]. Available: www.swebok.org.

[75] G. Spanoudakis and A. Zisman, "Software traceability: a roadmap," *Handbook of software engineering and knowledge engineering: vol 3: recent advances*, pp. 395-428, 2005.

[76] O. C. Z. Gotel and C. W. Finkelstein, "An analysis of the requirements traceability problem," *Proceedings of ieee international conference on requirements engineering*, pp. 94-101, 1994.

[77] H. Schwarz, J. Ebert and A. Winter, "Graph-based traceability: a comprehensive approach," *Software & Systems Modeling*, vol. 9, pp. 473-492, 2010.

[78] M. T. Tomova, M. Rath and P. Mäder, "Use of Trace Link Types in Issue Tracking Systems," *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, pp. 181-182, 2018.

[79] C. G. Hempel and P. Oppenheim, "Studies in the Logic of Explanation," *Philosophy of science*, vol. 15, no. 2, pp. 135-175, 1948.

[80] F. Sovrano and F. Vitali, "An objective metric for explainable AI: how and why to estimate the degree of explainability," *Knowledge-Based Systems*, vol. 278, 2023.

[81] W. C. Salmon, "Scientific explanation and the causal structure of the world," *Princeton University Press*, 1984.

[82] T. Clement, N. Kemmerzell, M. Abdelaal and M. Amberg, "XAIR: A systematic metareview of explainable ai (xai) aligned to the software development process," *Machine Learning and Knowledge Extraction*, vol. 5, no. 1, pp. 78-108, 2023.

[83] J. Droste, H. Deters, M. Obaidi and K. Schneider, "Explanations in Everyday Software Systems: Towards a Taxonomy for Explainability Needs," *arXiv preprint arXiv:2404.16644*, 2024.

[84] L. Chazette and K. Schneider, "Explainability as a non-functional requirement: challenges and recommendations," *Requirements Engineering*, vol. 25, no. 4, pp. 493-514, 2020.

[85] L. Chazette, W. Brunotte and T. Speith, "Exploring explainability: a definition, a model, and a knowledge catalogue," *2021 IEEE 29th international requirements engineering conference (RE)*, pp. 197-208, 2021.

[86] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.

[87] B. Goodman and S. Flaxman, "European Union regulations on algorithmic decision-making and a "right to explanation"," *AI Magazine*, vol. 38, no. 3, 2016.

[88] "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Da,"

5 2016 April. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679/oj. [Accessed June 2024].

[89] Z. C. Lipton, "The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery," *Queue*, vol. 16, no. 3, pp. 31-57, 2018.

[90] J. C. S. d. P. Leite and C. Cappelli, "Software transparency," *Business & Information Systems Engineering*, vol. 2, pp. 127-139, 2010.

[91] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes and T. Clark, "The FAIR Guiding Principles for scientific data management and stewardship," *Scientific Data*, vol. 3, no. 1, pp. 1-9, 2016.

[92] T. C. Ford, J. M. Colombi, S. R. Graham and D. R. Jacques, "A survey on interoperability measurement," *Gateways*, vol. 2, no. 3, 2007.

[93] P. Wegner, "Interoperability," *ACM Computing Surveys (CSUR)*, vol. 28, no. 1, pp. 285-287, 1996.

[94] G. d. S. S. Leal, W. Guédria and H. Panetto, "Interoperability assessment: A systematic literature review," *Computers in Industry*, vol. 106, pp. 111-132, 2019.

[95] D. Bawden and L. Robinson, "Information overload: An overview," *Oxford Encyclopedia of Political Decision Making*, 2020.

[96] A. Edmunds and A. Morris, "The problem of information overload in business organisations: a review of the literature," *International journal of information management*, vol. 20, no. 1, pp. 17-28, 2000.

[97] H. L. Wilensky, Organizational intelligence: Knowledge and policy in government and industry, Quid Pro Books, 2015.

[98] M. K. Bergman, A Knowledge Representation Practionary: Guidelines, Springer International Publishing, 2018.

[99] L. Zagzebski, "What is knowledge?," in *The Blackwell Guide to Epistemology*, 2017, pp. 92-116.

[100] C. McInerney, "Knowledge Management and the Dynamic Nature of Knowledge," *Journal of the American Society for Information Science and Technology*, vol. 53, no. 12, pp. 1009-1018, 2002.

[101] N. Guarino and P. Giaretta, "Ontologies and knowledge bases: towards a terminological clarification," 1995.

[102] N. Guarino, D. Oberle and S. Staab, "What is an Ontology?," in *Handbook on Ontologies*, 2009, pp. 1-17.

[103] T. R. Gruber, "A Translation Approach to Portable Ontology Specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199-220, 1993.

[104] W. Borst, "Construction of Engineering Ontologies," *Institute for Telematica and Information Technology*, 1997.

[105] T. T. Procko, T. Elvira and O. Ochoa, "GPT-4: A Stochastic Parrot or Ontological Craftsman? Discovering Implicit Knowledge Structures in Large Language Models," *IEEE Transdisciplinary AI (TransAI)*, 2023.

[106] N. Guarino, "Formal Ontology in Information Systems," *Proceedings of the First International Conference (FOIS '98),* June 1998.

[107] C. Partridge, A. Mitchell, A. Cook, D. Leal, J. Sullivan and M. West, "A Survey of Top-Level Ontologies - to inform the ontological choices for a Foundation Data Model," 2020.

[108] CUBRC, "An Overview of the Common Core Ontologies," 2019.

[109] G. Guizzardi, A. Botti Benevides, C. M. Fonseca, D. Porello, J. Paulo A. Almeida and T. Prince Sales, "UFO: Unified foundational ontology," *Applied Ontology,* vol. 17, no. 1, pp. 167-210, 2022.

[110] "ISO/IEC 21838-2:2021 Information technology — Top-level ontologies (TLO) — Part 2: Basic Formal Ontology (BFO)," International Organization for Standardization, November 2021. [Online]. Available: https://www.iso.org/standard/74572.html. [Accessed 2022].

[111] T. Berners-Lee, "Information Management: A Proposal," *CERN*, 1989.

[112] T. Berners-Lee, "Frequently Asked Questions," W3C, 1998. [Online]. Available: https://www.w3.org/People/Berners-Lee/FAQ.html. [Accessed 2023].

[113] V. G. Cerf and R. E. Kahn, "A Protocol For Packet Network Intercommunication," *IEEE Transactions on Communications,* vol. 22, no. 5, pp. 637-648, 1974.

[114] J. Markoff, "An Internet Pioneer Ponders the Next Revolution," *Outlook 2000 - The New York Times on the Web,* 20 December 1999.

[115] "Memorandum For Members and Affiliates of the Intergalactic Computer Network," The Kurzweil Library, 2001. [Online]. Available: https://www.kurzweilai.net/memorandum-for-members-and-affiliates-of-the-intergalactic-computer-network. [Accessed 2023].

[116] M. M. Waldrop, "The Dream Machine: J. C. R. Licklider and the Revolution That Made Computing Personal," New York, Viking Penguin, 2001, p. 470.

[117] D. E. Walker, SAFARI: an on-line text-processing system, Bedford: The Mitre Corporation, 1967.

[118] V. Bush, "As We May Think," *The Atlantic Monthly*, vol. 176, no. 1, pp. 101-108, 1945.

[119] T. Berners-Lee, J. Hendler and O. Lassila, "The Semantic Web," Scientific American, May 2001. [Online]. Available: https://web.archive.org/web/20020406010218/http://www.scientificamerican.com:80/2001/0501issue/0501berners-lee.html. [Accessed 2023].

[120] J. C. R. Licklider and R. W. Taylor, "The Computer as a Communication Device," *Science and Technology*, vol. 76, no. 2, pp. 1-3, 1968.

[121] "The DARPA Agent Markup Language," DARPA, 2006. [Online]. Available: http://www.daml.org/. [Accessed 2023].

[122] J. Hendler and D. L. McGuinness, "The DARPA Agent Markup Language," *IEEE Intelligent Systems*, vol. 15, no. 6, pp. 67-73, 2000.

[123] D. L. McGuinness, R. Fikes, J. Hendler and L. A. Stein, "DAML + OIL: An Ontology Language for the Semantic Web," *IEEE Intelligent Systems*, vol. 17, no. 5, pp. 72-80, 2002.

[124] C. Bizer, T. Heath and T. Berners-Lee, "Linked data: the story so far," *International Journal on Semantic Web and Information Systems,* vol. 5, pp. 1-22, 2009.

[125] D. M.-I. T. Gängler, "Semantic Federation of Musical and Music-Related Information for Establishing a Personal Music Knowledge Base," 2011.

[126] O. Lassila, "Web Metadata: A Matter of Semantics," *IEEE Internet Computing,* vol. 2, no. 4, pp. 30-37, 1998.

[127] O. Lassila and R. Swick, "Resource Description Framework (RDF) Model and Syntax Specification, W3C Recommendation," World Wide Web Consortium, 1999. [Online]. Available: https://www.w3.org/TR/REC-rdf-syntax/. [Accessed 2023].

[128] D. Brickley, R. V. Guha and A. Layman, "Resource description framework (RDF) schema specification," *W3C,* 1999.

[129] "Frequently Asked Questions," Solid, [Online]. Available: www.solidproject.org/faqs. [Accessed 2022].

[130] T. Berners-Lee, "Design Issues: Linked Data," W3C, 27 July 2006. [Online]. Available: https://www.w3.org/DesignIssues/LinkedData.html. [Accessed Februrary 2025].

[131] I. Herman, B. Adida, M. Sporny and M. Birbeck, "RDFa 1.1 Primer - Third Edition," W3C, 17 March 2015. [Online]. Available: https://www.w3.org/TR/xhtml-rdfa-primer/. [Accessed November 2023].

[132] T. T. Procko, A. Kiselev and O. Ochoa, "A Semi-Automated Process for Extracting a Taxonomized Knowledge Graph from the Software Engineering Body of Knowledge with a Sequence-to-Sequence Transformer," *Available at SSRN 4493644.*

[133] J. J. Miller, "Graph database applications and concepts with Neo4j," *Proceedings of the southern association for information systems conference,* vol. 2324, no. 36, 2013.

[134] L. Moreau, J. Freire, J. Futrelle, R. E. McGrath, J. Myers and P. Paulson, "The Open Provenance Model," 2007.

[135] "Provenance Working Group," W3C, 2013. [Online]. Available: https://www.w3.org/2011/prov/wiki/Main_Page. [Accessed 2023].

[136] T. Lebo, S. Sahoo, D. McGuinness, K. Belhajjame, J. Cheney, D. Corsar, D. Garijo, S. Soiland-Reyes, S. Zednik and J. Zhao, "PROV-O: The PROV Ontology," W3C, 30 April 2013. [Online]. Available: https://www.w3.org/TR/prov-o/. [Accessed 2023].

[137] "PROV-DM: The PROV Data Model," W3C, 2013. [Online]. Available: https://dvcs.w3.org/hg/prov/raw-file/default/model/working-copy/prov-dm-issue-450.html. [Accessed 2023].

[138] R. F. da Silva, H. Casanova, K. Chard, D. Laney, D. Ahn, S. Jha, C. Goble, L. Ramakrishnan, L. Peterson and B. Enders, "Workflows Community Summit: Bringing the Scientific Workflows Community Together," *arXiv preprint arXiv:2103.09181,* 2021.

[139] S. N. Mitchell, A. Lahiff, N. Cummings, J. Hollocombe, B. Boskamp, R. Field, D. Reddyhoff, K. Zarebski, A. Wilson, B. Viola, M. Burke, B. Archibald, P. Bessell and R. Blackwell, "FAIR data pipeline: provenance-driven data management for traceable scientific workflows," *Philosophical Transactions of the Royal Society A,* vol. 380, no. 2233, 2022.

[140] A. Gaignard, H. Skaf-Molli and A. Bihouée, "From Scientific Workflow Patterns to 5-star Linked Open Data," *TaPP*, 2016.

[141] A. Hasnain and D. Rebholz-Schuhmann, "Assessing FAIR data principles against the 5-star open data principles.," in *The Semantic Web: ESWC 2018 Satellite Events: ESWC 2018 Satellite Events, Heraklion, Crete, Greece, June 3-7, 2018, Revised Selected Papers 15*, Springer International Publishing, 2018, pp. 469-477.

[142] T. T. Procko and O. Ochoa, "Semantic Science: Publication Beyond the PDF," *IEEE SoutheastCon 2024*, pp. 207-215, 2024.

[143] B. Mons, C. Neylon, J. Velterop, M. Dumontier, L. O. B. da Silva Santos and M. D. Wilkinson, "Cloudy, increasingly FAIR; revisiting the FAIR Data guiding principles for the European Open Science Cloud," *Information services & use*, vol. 37, no. 1, pp. 49-56, 2017.

[144] T. T. Procko, T. Elvira and O. Ochoa, "Dawn of the Dialogue: AI's Leap from Lab to Living Room," *Frontiers in Artificial Intelligence*, vol. 7, 2024.

[145] K. S. Jones, "Natural Language Processing: A Historical Review," *Current Issues in Computational Linguistics: In Honour of Don Walker*, pp. 3-16, 1994.

[146] Y. Lu, "Artificial intelligence: a survey on evolution, models, applications and future trends," *Journal of Management Analytics*, vol. 6, no. 1, pp. 1-29, 2019.

[147] W. Weaver, "Translation," *Proceedings of the Conference on Mechanical Translation*, 1952.

[148] C. E. Shannon, "A mathematical theory of communication," *The Bell system technical journal*, vol. 27, no. 3, pp. 379-423, 1948.

[149] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling," *Computer Speech & Language*, vol. 13, no. 4, pp. 359-394, 1999.

[150] F. Jelinek, "Self-organized language modeling for speech recognition," *Readings in speech recognition*, pp. 450-506, 1990.

[151] B. Thorsten, A. C. Popat, P. Xu, F. J. Och and J. Dean, "Large language models in machine translation," 2007.

[152] H. Wang and B. Raj, "On the origin of deep learning," *arXiv preprint arXiv:1702.07800*, 2017.

[153] M. Z. Alom, T. M. Taha, C. Yakopcic, S. Westberg, P. Sidike, M. S. Nasrin, B. C. V. Esesn, A. A. S. Awwal and V. K. Asari, "The history began from alexnet: A comprehensive survey on deep learning approaches," *arXiv preprint arXiv:1803.01164*, 2018.

[154] D. Jeffrey, "A golden decade of deep learning: Computing systems & applications," *Daedalus*, vol. 151, no. 2, pp. 58-74, 2022.

[155] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems* , vol. 30, 2017.

[156] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," 2019.

[157] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of research and development,* vol. 44, no. 1.2, pp. 206-226, 2000.

[158] C. Zhang, C. Zhang, S. Zheng, Y. Qiao, C. Li, M. Zhang, S. K. Dam, C. M. Thwal, Y. L. Tun, L. L. Huy, D. Kim, S.-H. Bae, L.-H. Lee, Y. Yang, H. T. Shen, I. S. Kweon and C. S. Hong, "A Complete Survey on Generative AI (AIGC): Is ChatGPT from GPT-4 to GPT-5 All You Need?," *arXiv preprint arXiv:2303.11717,* 2023.

[159] E. Jiang, K. Olson, E. Toh, A. Molina, A. Donsbach, M. Terry and C. J. Cai, "Promptmaker: Prompt-based prototyping with large language models," *CHI Conference on Human Factors in Computing Systems Extended Abstracts,* 2022.

[160] L. Zhong and Z. Wang, "Can LLM Replace Stack Overflow? A Study on Robustness and Reliability of Large Language Model Code Generation," *Proceedings of the AAAI Conference on Artificial Intelligence,* vol. 38, no. 19, pp. 21841-21849, 2024.

[161] E. Olson, "Google Shares drop $100 Billion After its New AI Chatbot Makes a Mistake," NPR, 2023 February 2023. [Online]. Available: https://www.npr.org/2023/02/09/1155650909/google-chatbot--error-bard-shares. [Accessed 19 February 2023].

[162] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys,* vol. 55, no. 9, pp. 1-35, 2023.

[163] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1-7, 2021.

[164] V. Liu and L. B. Chilton, "Design guidelines for prompt engineering text-to-image generative models," *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022.

[165] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023.

[166] J. Wang, Z. Liu, L. Zhao, Z. Wu, C. Ma, S. Yu, H. Dai, Q. Yang, Y. Liu, S. Zhang, E. Shi, Y. Pan, T. Zhang, D. Zhu, X. Li, X. Jiang, B. Ge, Y. Yuan, D. Shen, T. Liu and Z, "Review of large vision models and visual prompt engineering," *Meta-Radiology*, 2023.

[167] L. Giray, "Prompt engineering with ChatGPT: a guide for academic writers," *Annals of biomedical engineering*, vol. 51, no. 12, pp. 2629-2633, 2023.

[168] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[169] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens and A. Askell, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, 2022.

[170] OpenAI, "GPT-4 Technical Report," *arXiv:2303.08774,* 2023.

[171] "Optimizing LLMs for accuracy," OpenAI, [Online]. Available: https://platform.openai.com/docs/guides/optimizing-llm-accuracy. [Accessed June 2024].

[172] O. Honovich, T. Scialom, O. Levy and T. Schick, "Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor," *arXiv preprint arXiv:2212.09689,* 2022.

[173] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Arunkumar, A. Ashok, A. S. Dhanasekaran, A. Naik, D. Stap, E. Pathak, G. Karamanolakis, H. G. Lai and I. Purohit, "Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks," *arXiv preprint arXiv:2204.07705,* 2022.

[174] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi and H. Hajishirzi, "Self-instruct: Aligning language models with self-generated instructions," *arXiv preprint arXiv:2212.10560,* 2022.

[175] A. Glaese, N. McAleese, M. Trębacz, J. Aslanides, V. Firoiu, T. Ewalds, M. Rauh, L. Weidinger, M. Chadwick, P. Thacker and L. Campbell-Gillingham, "Improving alignment of dialogue agents via targeted human judgements," *arXiv preprint arXiv:2209.14375,* 2022.

[176] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in Neural Information Processing Systems,* vol. 36, 2024.

[177] J. Chen, H. Lin, X. Han and L. Sun, "Benchmarking large language models in retrieval-augmented generation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, pp. 17754-17762, 2024.

[178] T. Procko and O. Ochoa, "Graph retrieval-augmented generation for large language models: A survey," *Available at SSRN (2024).*

[179] T. T. Procko, A. Davidoff, T. Elvira and O. Ochoa, "Towards Improved Scientific Knowledge Proliferation: Leveraging Large Language Models on the Traditional Scientific Writing Workflow," *Available at SSRN 4594836*, 2023.

[180] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga and Y. Zhang, "Holistic evaluation of language models," *arXiv preprint arXiv:2211.09110*, 2022.

[181] "A holistic framework for evaluating foundation models.," Center for Research on Foundation Models, 01 March 2024. [Online]. Available: https://crfm.stanford.edu/helm/lite/latest/#/. [Accessed March 2024].

[182] "Hello GPT-4o," OpenAI, 13 May 2024. [Online]. Available: https://openai.com/index/hello-gpt-4o/. [Accessed 25 September 2024].

[183] J. Menick, K. Lu, S. Zhao, E. Wallace, H. Ren, H. Hu, N. Stathas and F. Petroski Such, "GPT-4o mini: advancing cost-efficient intelligence," OpenAI, 18 July 2024. [Online]. Available: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/. [Accessed 25 September 2024].

[184] A. El-Kishky, D. Selsam, F. Song, G. Parascandolo, H. Ren, H. Lightman, H. W. Chung, I. Akkaya, I. Sutskever, J. Wei, J. Gordon, K. Cobbe, K. Yu, L. Kondraciuk and M.

Schwarzer, "Learning to Reason with LLMs," OpenAI, 12 September 2024. [Online]. Available: https://openai.com/index/learning-to-reason-with-llms/. [Accessed 25 September 2024].

[185] "Introducing canvas," OpenAI, 3 October 2024. [Online]. Available: https://openai.com/index/introducing-canvas/. [Accessed November 2024].

[186] OpenAI, "Introducing GPT-4.5," OpenAI, 27 February 2025. [Online]. Available: Introducing GPT-4.5. [Accessed 27 February 2025].

[187] J. Gallifant, A. Fiske, Y. A. L. Strekalova, J. S. Osorio-Valencia, R. Parke, R. Mwavu, N. Martinez, J. W. Gichoya, M. Ghassemi, D. Demner-Fushman, L. G. McCoy, L. A. Celi and R. Pierce, "Peer review of GPT-4 technical report and systems card," *PLOS Digital Health*, vol. 3, no. 1, 2024.

[188] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux and P. Stock, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

[189] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[190] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen and G.

Cucurull, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288,* 2023.

[191] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652,* 2021.

[192] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain and J. Gao, "Large language models: A survey," *arXiv preprint arXiv:2402.06196,* 2024.

[193] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin and X. Hu, "Harnessing the power of llms in practice: A survey on chatgpt and beyond," *ACM Transactions on Knowledge Discovery from Data,* vol. 18, no. 6, pp. 1-32, 2024.

[194] J. Zhang, H. Jiaxing, S. Jin and S. Lu, "Vision-language models for vision tasks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence,* 2024.

[195] T. Elvira, T. T. Procko, J. O. Couder and O. Ochoa, "A Survey on the Intersection of Research in the use of Language Models and Knowledge Graphs," *Available at SSRN,* 2024.

[196] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering,* 2024.

[197] R. C. Atkinson and R. M. Shiffrin, "Human memory: A proposed system and its control processes," *Psychology of learning and motivation,* vol. 2, pp. 89-195, 1968.

[198] T. T. Procko, T. Elvira and O. Ochoa, "GPT-4: A Stochastic Parrot or Ontological Craftsman? Discovering Implicit Knowledge Structures in Large Language Models," *2023 Fifth International Conference on Transdisciplinary AI (TransAI)*, pp. 147-154, 2023.

[199] G. Jawahar, B. Sagot and D. Sedda, "What does BERT learn about the structure of language?," *ACL 2019-57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[200] M. Dillinger, "Knowledge Graphs and/or/vs/for LLMs," 2024 May. [Online]. Available: https://www.linkedin.com/posts/mikedillinger_knowledge-graphs-andorvsfor-llms-activity-7189682533838512128-Y-ph?utm_source=share&utm_medium=member_desktop. [Accessed June 2024].

[201] H. Abu-Rasheed, C. Weber and M. Fathi, "Knowledge Graphs as Context Sources for LLM-Based Explanations of Learning Recommendations," *arXiv preprint arXiv:2403.03008*, 2024.

[202] A. H. Shariatmadari, S. Guo, S. Srinivasan and A. Zhang, "Harnessing the Power of Knowledge Graphs to Enhance LLM Explainability in the BioMedical Domain," 2024.

[203] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, W. Dai, A. Madotto and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1-38, 2023.

[204] U. Braun, S. Garfinkel, D. A. Holland, K.-K. Muniswamy-Reddy and M. I. Seltzer, "Issues in Automatic Provenance Collection," *International Provenance and Annotation Workshop,* pp. 171-183, 2006.

[205] H. U. Asuncion and R. N. Taylor, "Automated techniques for capturing custom traceability links across heterogeneous artifacts," *Software and systems traceability,* pp. 129-146, 2011.

[206] M. Hällgren and E. Maaninen-Olsson, "Deviations and the breakdown of project management principles," *International Journal of Managing Projects in Business,* vol. 2, no. 1, pp. 53-69, 2009.

[207] U. Braun, D. A. Holland, K.-K. Muniswamy-Reddy and M. Seltzer, "Coping with cycles in provenance," *2006.*

[208] T. T. Procko, O. Ochoa and C. Frederick, "Microelectronic Technology, AI and Academic Dishonesty: An Agile Engineering Approach," *2023 ASEE Annual Conference & Exposition,* 2023.

[209] M. White, I. Haddad, C. Osborne, A. Abdelmonsef and S. Varghese, "The Model Openness Framework: Promoting Completeness and Openness for Reproducibility, Transparency and Usability in AI," *arXiv preprint arXiv:2403.13784,* 2024.

[210] Y. Lu, J. Yu and S.-H. S. Huang, "Illuminating the black box: A psychometric investigation into the multifaceted nature of large language models," *arXiv preprint arXiv:2312.14202,* 2023.

[211] N. Islam, Z. Islam and N. Noor, "A survey on optical character recognition system," *arXiv preprint arXiv:1710.05703*, 2017.

[212] A. Chaudhuri, K. Mandaviya, P. Badelia, S. K. Ghosh, A. Chaudhuri, K. Mandaviya, P. Badelia and S. K. Ghosh, "Optical character recognition systems," in *Optical Character Recognition Systems for Different Languages with Soft Computing*, Springer International Publishing, 2016, pp. 9-41.

[213] C.-Y. Wang, A. Bochkovskiy and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 7464-7475, 2023.

[214] S. Feiz, J. Wu, X. Zhang, A. Swearngin, T. Barik and J. Nichols, "Understanding screen relationships from screenshots of smartphone applications," *27th International Conference on Intelligent User Interfaces*, pp. 447-458, 2022.

[215] J. Wu, X. Zhang, J. Nichols and J. P. Bigham, "Screen parsing: Towards reverse engineering of ui models from screenshots," *The 34th Annual ACM Symposium on User Interface Software and Technology*, pp. 470-483, 2021.

[216] J. Wu, R. Krosnick, E. Schoop, A. Swearngin, J. P. Bigham and J. Nichols, "Never-ending Learning of User Interfaces," *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1-13, 2023.

[217] A. Mottelson, "Why do people take Screenshots on their Smartphones?," *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, pp. 740-752, 2023.

[218] S. Shahriar, B. D. Lund, N. Reddy Mannuru, M. Arbab Arshad, K. Hayawi, R. Varma Kumar Bevara, A. Mannuru and L. Batool, "Putting gpt-4o to the sword: A comprehensive evaluation of language, vision, speech, and multimodal proficiency," *Applied Sciences*, vol. 14, no. 17, 2024.

[219] T. T. Procko and O. Ochoa, "Mapping the W3C Provenance Ontology (PROV-O) to the Basic Formal Ontology (BFO): Epistemological Considerations and Preliminary Implementation," *SSRN*, 2024.

[220] A. Cockburn, Writing Effective Use Cases, Boston: Addison-Wesley Longman Publishing Co., 2000.

[221] J. Schickore, "Doing science, writing science," *Philosophy of Science,* vol. 75, no. 3, pp. 323-343, 2008.

[222] F. L. Holmes, "Scientific Writing and Scientific Discovery," *Isis*, vol. 78, no. 2, pp. 220-235, 1987.

[223] M. Thompson and T. Fearn, "What exactly is fitness for purpose in analytical measurement?," *Analyst*, vol. 121, no. 3, pp. 275-278, 1996.

[224] F. Shull, J. Singer and D. I. K. Sjøberg, Guide to advanced empirical software engineering, London: Springer Science & Business Media, Springer Science & Business Media.

[225] Y. Jiang, E. Schoop, A. Swearngin and J. Nichols, "ILuvUI: Instruction-tuned LangUage-Vision modeling of UIs from Machine Conversations," vol. arXiv preprint arXiv:2310.04869, 2023.

[226] K. K. Vamsi and S. Samuel, "From human experts to machines: An LLM supported approach to ontology and knowledge graph construction," *arXiv:2403.08345v1,* 2024.

[227] L.-P. Meyer, C. Stadler, J. Frey, N. Radtke, K. Junghanns, R. Meissner, G. Dziwis, K. Bulert and M. Martin, "Llm-assisted knowledge graph engineering: Experiments with chatgpt," *Working conference on Artificial Intelligence Development for a Resilient and Sustainable Tomorrow*, pp. 103-115, 2023.

[228] V. K. Kommineni, B. König-Ries and S. Samuel, "From human experts to machines: An LLM supported approach to ontology and knowledge graph construction," *arXiv preprint arXiv:2403.08345,* 2024.

[229] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad and M. Stal, Pattern-Oriented Software Architecture - Volume 1: A System of Patterns, Wiley Publishing, 1996.

[230] L. S. Pheng, "Project Quality Management," in *Project Management for the Built Environment: Study Notes*, Singapore, Springer Nature, 2017, pp. 113-125.

[231] A. Holub, Taming Java Threads, Berkeley: Apress Media, 2000.

[232] Y. Ling, T. Mullen and X. Lin, "Analysis of optimal thread pool size," *ACM SIGOPS Operating Systems Review,* vol. 34, no. 2, pp. 42-55, 2000.

[233] D. L. Freire, R. Z. Frantz and F. Roos-Frantz, "Towards optimal thread pool configuration for run-time systems of integration platforms," *International Journal of Computer Applications in Technology,* vol. 62, no. 2, pp. 129-147, 2020.

[234] J. O. Wobbrock and J. A. Kientz, "Research Contributions in Human-Computer Interaction," *Interactions,* vol. 23, no. 3, pp. 38-44, 2016.

[235] V. R. Basili, "The role of experimentation in software engineering: past, current, and future," *Proceedings of IEEE 18th International Conference on Software Engineering*, pp. 442-449, 1996.

[236] M. Shaw, "What makes good research in software engineering?," *International Journal on Software Tools for Technology Transfer*, vol. 4, no. 1, pp. 1-7, 2002.

[237] J. Lazar, J. H. Feng and H. Hochheiser, Research methods in human-computer interaction, Cambridge: Morgan Kaufmann, 2017.

[238] B. Kaplan and J. A. Maxwell, "Qualitative research methods for evaluating computer information systems," *Evaluating the organizational impact of healthcare information systems*, pp. 30-55, 2005.

[239] K. Charmaz, Constructing grounded theory: A practical guide through qualitative analysis, Sage, 2006.

[240] J. F. Pimentel, J. Freire, L. Murta and V. Braganholo, "A survey on collecting, managing, and analyzing provenance from scripts," *ACM Computing Surveys (CSUR)*, vol. 52, no. 3, pp. 1-38, 2019.

[241] R. Meyes, M. Lu, C. W. de Puiseau and T. Meisen, "Ablation studies in artificial neural networks," *arXiv preprint arXiv:1901.08644*, 2019.

[242] T. C. Ford, J. M. Colombi, S. R. Graham and D. R. Jacques, "A Survey on Interoperability Measurement," *Gateways*, vol. 2, no. 3, 2007.

[243] M. Johns, T. Meurers, F. N. Wirth, A. C. Haber, A. Mueller, M. Halilovic, F. Balzer and F. Prasser, "Data provenance in biomedical research: scoping review," *Journal of Medical Internet Research,* vol. 25, 2023.

[244] S. Samuel, F. Löffler and B. König-Ries, "Machine learning pipelines: provenance, reproducibility and FAIR data principles," *International Provenance and Annotation Workshop,* pp. 226-230, 2020.

[245] R. Souza, L. G. Azevedo, V. Lourenço, E. Soares, R. Thiago, R. Brandão, D. Civitarese, E. V. Brazil, M. Moreno, P. Valduriez, M. Mattoso, R. Cerqueira and M. A. S. Netto, "Workflow provenance in the lifecycle of scientific machine learning," *Concurrency and Computation: Practice and Experience,* vol. 34, no. 14, 2022.

[246] B. Gambini, "DOD, Intelligence Community adopt resource developed by UB ontologists," University at Buffalo, 28 Februrary 2024. [Online]. Available: https://www.buffalo.edu/news/releases/2024/02/department-of-defense-ontology.html. [Accessed 28 Februrary 2025].

[247] B. Smith and W. Ceusters, "Ontological realism: A methodology for coordinated evolution of scientific ontologies," *Applied Ontology,* vol. 5, no. 3-4, pp. 139-188, 2010.

[248] M. J. Eppler and J. Mengis, "The Concept of Information Overload-A Review of Literature from Organization Science, Accounting, Marketing, MIS, and Related Disciplines," *The Information Society: An International Journal,* vol. 20, no. 5, pp. 1-20, 2004.

[249] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science,* vol. 12, no. 2, pp. 257-285, 1988.

[250] M. E. Jennex and L. Olfman, "A Model of Knowledge Management Success," *International Journal of Knowledge Management (IJKM),* vol. 2, no. 3, pp. 51-68, 2006.

[251] S. K. Boell and D. Cecez-Kecmanovic, "On Being 'Systematic' in Literature Reviews in IS," *Journal of Information Technology,* vol. 30, no. 2, pp. 161-173, 2015.

[252] C. Wohlin, "Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering," *EASE 2014: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering,* 2014.

[253] K. Otten and A. Debons, "Towards a Metascience of Information: Informatology," *Journal of the American Society for Information Science,* vol. 21, no. 1, pp. 89-94, 1970.

[254] T. D. Nies, S. Coppens, D. V. Deursen, E. Mannens and R. Van de Walle, "Automatic Discovery of High-Level Provenance Using Semantic Similarity," *International Provenance and Annotation Workshop,* pp. 97-110, 2012.

[255] J. Zhao, C. Wroe, C. Goble, R. Stevens, D. Quan and M. Greenwood, "Using Semantic Web Technologies for Representing E-science Provenance," *International Semantic Web Conference,* pp. 92-106, 2004.

[256] L. Drăgan, S. Handschuh and S. Decker, "The semantic desktop at work: interlinking notes," *Proceedings of the 7th International Conference on Semantic Systems,* pp. 17-24, 2011.

[257] S. Thursfield, "Tracker 3.0: How did we get here?," 27 October 2020. [Online]. Available: https://samthursfield.wordpress.com/2020/10/27/tracker-3-0-how-did-we-get-here/. [Accessed November 2024].

[258] "Obsidian - Sharpen your thinking.," Obsidian, [Online]. Available: https://obsidian.md/. [Accessed March 2024].

[259] "Retrace your steps with Recall," Microsoft, 2024. [Online]. Available: https://support.microsoft.com/en-us/windows/retrace-your-steps-with-recall-aa03f8a0-a78b-4b3e-b0a1-2eb8ac48701c. [Accessed May 2024].

[260] "Configuring issue linking," Atlassian, 28 April 2017. [Online]. Available: https://confluence.atlassian.com/adminjiraserver073/configuring-issue-linking-861253998.html. [Accessed November 2024].

[261] E. Dumbill, "XML Watch: Describe open source projects with XML, Part 3," IBM DeveloperWorks, 11 June 2004. [Online]. Available: https://web.archive.org/web/20040802225342/http://www-106.ibm.com/developerworks/xml/library/x-osproj3/. [Accessed March 2024].

[262] E. Wilder-James, "DOAP: Description Of A Project," [Online]. Available: https://github.com/ewilderj/doap. [Accessed March 2024].

[263] T. T. Procko and O. Ochoa, "An Ontology Based Approach to the Software Engineering Lifecycle: Application to Software Quality Assurance in the Aerospace Industry," *Available at SSRN 4703235*, 2022.

[264] "Núcleo de Estudos em Modelagem Conceitual e Ontologias - NEMO," [Online]. Available: https://nemo.inf.ufes.br/. [Accessed March 2022].

[265] M. H. Jarrahi, D. L. Blyth and C. Goray, "Mindful work and mindful technology: Redressing digital distraction in knowledge work," *Digital Business,* vol. 3, no. 1, 2023.

[266] M. Agarwal, B. Bishesh, S. Bansal and D. Kumari, "Role of social media on digital distraction: a study on university students," *Journal of content, community and communication,* vol. 13, no. 7, pp. 125-136, 2021.

[267] R. Patil, M. Brown, M. Ibrahim, J. Myers, K. Brown, M. Khan and R. Callaway, "Digital distraction outside the classroom: An empirical study," *Journal of Computing Sciences in Colleges,* vol. 34, no. 7, pp. 46-55, 2019.

[268] A. J. Dontre, "The influence of technology on academic distraction: A review," *Human Behavior and Emerging Technologies,* vol. 3, no. 3, pp. 379-390, 2021.

[269] M. Nakayama and C. C. Chen, "Digital Distraction, Non-Digital Distraction and Psychological Bearing of Remote Workers," *Interacting with Computers,* vol. 35, no. 6, pp. 789-800, 2023.

[270] M. A. Orhan, S. Castellano, I. Khelladi, L. Marinelli and F. Monge, "Technology distraction at work. Impacts on self-regulation and work engagement," *Journal of Business Research,* vol. 126, pp. 341-349, 2021.

[271] A. Sedgewick, M. Souppaya and K. Scarfone, "Guide to Application Whitelisting," *Special Publication (NIST SP) - 800-167,* 2015.

[272] J. P. A. Ioannidis, "Limitations are not properly acknowledged in the scientific literature," *Journal of Clinical Epidemiology,* vol. 60, no. 4, pp. 324-329, 2007.

[273] P. M. Johnson and A. M. Disney, "A critical analysis of PSP data quality: Results from a case study," *Empirical Software Engineering,* vol. 4, pp. 317-349, 1999.

[274] D. Erickson, M. Casado and N. McKeown, "The Effectiveness of Whitelisting: a User-Study," *CEAS,* 2008.

[275] S. T. I. Tonmoy, S. M. M. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha and A. Das, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313,* vol. 6, 2024.

[276] "Computer Security Technology Planning Study," *ESD-TR-73-51,* vol. 2, 1972.

[277] D. Schatz, R. Bashroush and J. Wall, "Towards a More Representative Definition of Cyber Security," *Journal of Digital Forensics,* vol. 12, no. 2, 2017.

[278] R. R. Faden and T. L. Beauchamp, "A history and theory of informed consent," *Oxford University Press,* 1986.

[279] R. J. Levine, "The Nature and Definition of Informed Consent in Various Research Settings," *Preliminary paper prepared for the National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research. Washington, DC: US Department of Health, Education, and Welfare,* 1975.

[280] F. Gesualdo, M. Daverio, L. Palazzani, D. Dimitriou, J. Diez-Domingo, J. Fons-Martinez, S. Jackson, P. Vignally, C. Rizzo and A. E. Tozzi, "Digital Tools in the Informed Consent Process: A Systematic Review," *BMC Medical Ethics,* vol. 22, no. 18, 2021.

[281] L. T. Eyler and D. V. Jeste, "Enhancing the Informed Consent Process: A Conceptual Overview," *Behavioral Sciences & The Law,* vol. 24, no. 4, pp. 553-568, 2006.

[282] M. M. Archibald, R. C. Ambagtsheer, M. G. Casey and M. Lawless, "Using zoom videoconferencing for qualitative data collection: perceptions and experiences of researchers and participants," *International Journal of Qualitative Methods*, vol. 18, 2019.

[283] G. Orwell, 1984, London: Secker and Warburg, 1949.

[284] L. Williams, R. R. Kessler, W. Cunningham and R. Jeffries, "Strengthening the case for pair programming," *IEEE software*, vol. 17, no. 4, pp. 19-25, 2000.

[285] L. Wang, "British English-speaking speed 2020," *Academic Journal of Humanities and Social Sciences*, vol. 4, pp. 93-100, 2021.

[286] B. Wang, Y. Liu, J. Qian and S. K. Parker, "Achieving effective remote working during the COVID-19 pandemic: A work design perspective," *Applied Psychology*, vol. 70, no. 1, pp. 16-59, 2021.

[287] J. L. Oliffe, M. T. Kelly, G. G. Motaner and W. F. Yu Ko, "Zoom Interviews: Benefits and Concessions," *International Journal of Qualitative Methods,* vol. 20, 2021.

[288] D. Serhan, "Transitioning from Face-to-Face to Remote Learning: Students' Attitudes and Perceptions of Using Zoom during COVID-19 Pandemic," *International Journal of Technology in Education and Science*, vol. 4, no. 4, pp. 335-342, 2020.

[289] K. Holmqvist, Eye Tracking: A Comprehensive Guide to Methods and Measures, Oxford University Press, 2011.

[290] B. T. Carter and S. G. Luke, "Best Practices in Eye Tracking Research," *International Journal of Psychophysiology,* vol. 155, pp. 49-62, 2020.

[291] B. He, H. Yuan, J. Meng and S. Gao, "Brain–Computer Interfaces," *Neural Engineering,* pp. 131-183, 2020.

[292] E. Waisberg, J. Ong and A. G. Lee, "Ethical Considerations of Neuralink and Brain-Computer Interfaces," *Annals of Biomedical Engineering,* vol. 52, 2024.

[293] S. Samuel and B. König-Ries, "End-to-End provenance representation for the understandability and reproducibility of scientific experiments using a semantic approach," *Journal of biomedical semantics,* vol. 13, no. 1, 2022.

[294] R. K. Cheong and E. Tsui, "The roles and values of personal knowledge management: an exploratory study," *VINE Journal of Information and Knowledge,* vol. 40, no. 2, pp. 204-227, 2010.

[295] M. Schlegel and K.-U. Sattler, "Management of machine learning lifecycle artifacts: A survey," *ACM SIGMOD Record,* vol. 51, no. 4, pp. 18-35, 2023.

[296] A. Kale, T. Nguyen, F. C. Harris Jr., C. Li, J. Zhang and X. Ma, "Provenance documentation to enable explainable and trustworthy AI: A literature review," *Data Intelligence,* vol. 5, no. 1, pp. 139-162, 2023.

[297] W. S. Humphrey, "The Personal Software Process (PSP)," *Technical Report CMU/SEI-2000-TR-022,* 2000.

[298] R. Sison, "Personal Software Process (PSP) Assistant," *12th Asia-Pacific Software Engineering Conference (APSEC'05),* 2005.

[299] W. S. Humphrey, "Introduction to the Team Software Process," *Addison-Wesley Professional*, 1999.

[300] B. W. Boehm, "Software Engineering Economics," in *Software Pioneers*, Berlin Heidelberg, Springer, 2002, pp. 641-686.

[301] S. Woodman, H. Hiden and P. Watson, "Applications of provenance in performance prediction and data storage optimisation," *Future Generation Computer Systems*, vol. 75, pp. 299-309, 2017.

[302] M. White, I. Haddad, C. Osborne, X.-Y. L. Yanglet, A. Abdelmonsef and S. Varghese, "The model openness framework: Promoting completeness and openness for reproducibility, transparency, and usability in artificial intelligence," *arXiv preprint arXiv:2403.13784*, 2024.

[303] Q. Jiang, Z. Gao and G. E. Karniadakis, "DeepSeek vs. ChatGPT vs. Claude: A Comparative Study for Scientific Computing and Scientific Machine Learning Tasks," *Theoretical and Applied Mechanics Letters*, 2025.

[304] Google, "Gemini: a family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[305] G. Booch and A. W. Brown, "Collaborative development environments," *Adv. Comput.*, vol. 59, no. 1, pp. 1-27, 2003.

[306] M. B. Miles and A. M. Huberman, Qualitative data analysis: An expanded sourcebook, Sage, 1994.

APPENDIX A: ACRONYMS

Acronym	Meaning
AI	Artificial Intelligence
BCI	Brain Computer Interface
CLT	Cognitive Load Theory
CMCS	Collaboratory for Multi-Scale Chemical Science
CI/CD	Continuous Integration and Continuous Delivery
CNS	Central Nervous System
CoEST	Center of Excellence for Software and Systems Traceability
COVID-19	Coronavirus Disease 2019
CPU	Central Processing Unit
DFD	Data Flow Diagram
DOAP	Description of a Project
ECL	Extract, Contextualize, Load
ESE	Empirical Software Engineering
ESSW	Earth System Science Workbench
ETL	Extract, Transform, Load
FAIR	Findable, Accessible, Interoperable, Reusable
FOSS	Free and Open-Source Software
GPT	Generative Pre-Trained Transformer
GREQL	Graph Repository Query Language
HCI	Human-Computer Interaction
IDE	Integrated Development Environment
KM	Knowledge Management
KMS	Knowledge Management System
LIP	Lineage Information Program
LLM	Large Language Model

ML	Machine Learning
MLLM	Multimodal Large Language Model
MOF	Model Openness Framework
NEPOMUK	Networked Environment for Personal, Ontology-based Management of Unified Knowledge
NPU	Neural Processing Unit
OCR	Optical Character Recognition
PASOA	Provenance Aware Service-oriented Architecture
PASS	Provenance-Aware Storage System
PC	Personal Computer
PC	Personal Computer
PID	Process Identifier
PIN	Personal Identification Number
PKM	Personal Knowledge Management
PSO	Particle Swarm Optimization
PSP	Personal Software Process
PSR	Problems Steps Recorder
ReDSeeDS	Requirements Driven Software Development System
RQ	Research Question
SDLC	Software Development Life Cycle
SIG	Softgoal Interdependence Graph
SLR	Systematic Literature Review
SQuaRE	Software Quality Requirements and Evaluation
SWEBoK	Software Engineering Body of Knowledge
SWfMS	Scientific Workflow Management System
TAI	Trustworthy AI
TAPT	Total Average Processing Time
TSP	Team Software Process

UI	User Interface
UML	Unified Modeling Language
VCS	Version Control System
VDG	Virtual Data Grid
XAI	Explainable AI
XML	Extensible Markup Language
XP	eXtreme Programming
XSD	XML Schema Definition

APPENDIX B: TERM DEFINITIONS – DIGITAL WORK NOISE SOURCE TAXONOMY

Taxonomy Definition / Intent: Digital work noise refers to data, events, or signals encountered during digital work that are not meaningfully or causally linked to the creation, modification, or decision-making around output artifacts. By providing a "worst-case" list of noise sources, the digital work noise source taxonomy is intended to be used as a general guide for the creation of domain- and task-specific relevant provenance source whitelists. Taxonomy terms are defined in Aristotelian form, i.e., each definition begins by stating the term it inherits its meaning from, thereafter made distinguished by its parent term with further details.

Categories:

- Personal Noise: A Digital Work Noise that pertains to explicitly personal sources of information, often volitionally interacted with, sometimes recurring from personal circumstances or interests.
- Background System Activity Noise: A Digital Work Noise that pertains to automated or passive system processes which occur independently of a user's intentional work.
- UI and Screen-Level Artifact Noise: A Digital Work Noise that pertains to visible but contextually irrelevant user interface elements, background windows, or idle screen artifacts that do not directly influence a user's work or artifacts.
- Ambient Input Noise: A Digital Work Noise that pertains to peripheral or environmental inputs that are captured but not intentionally related to a user's work.

- Cognitive Gap Noise: A Digital Work Noise that pertains to moments of inaction, idling, or undetectable internal cognition during which no externalized or traceable work activity occurs.
- System-Generated Noise: A Digital Work Noise that originates from background processes, automated alerts, or tool-generated suggestions that the user does not engage with meaningfully or integrate into their workflow.
- Human Noise: A Digital Work Noise that stems from non-task-related human activity within shared or ambient spaces that are perceptible to capture systems but irrelevant to artifact provenance.

Exemplars:

- Unrelated email: A Personal Noise that pertains to emails not directly related to work tasks, e.g., personal communications, shopping promotions, newsletters, etc.
- Instant messages / chat: A Personal Noise that pertains to real-time conversations through messaging platforms, e.g., Internet Relay Chat providers, not directly related to work tasks.
- Social media: A Personal Noise that pertains to algorithm-driven interactions with social media platforms, e.g., Instagram, Facebook, etc., that do not contribute meaningfully to artifact formation.
- Entertainment: A Personal Noise that pertains to engagement with media platforms for leisure, e.g., music or video streaming, video games, etc., not directly related to work tasks.

- Calendar alerts: A Personal Noise that pertains to reminders or notifications for events not directly related to work tasks.
- Wearable alerts: A Personal Noise that originates from devices worn on the body, e.g., smartwatches, which deliver personal communications to the wearer about, e.g., health, chats, etc., not directly related to work tasks.
- Digital assistant notifications: A Personal Noise that pertains to unsolicited or peripheral voice assistant prompts or reminders not directly related to work tasks.
- OS Notifications: A Background System Activity Noise that pertains to system-level alerts, e.g., update prompts, system messages, or background information windows not triggered by a user.
- Antivirus popups / scans: A Background System Activity Noise that pertains to security-related messages or scan reports generated by antivirus software.
- System performance widgets: A Background System Activity Noise that pertains to numeric or visual performance indicators like CPU or memory usage, etc.
- Unprompted app updates / installers: A Background System Activity Noise that pertains to software updates or installs not initiated by a user.
- Backup / sync messages: A Background System Activity Noise that pertains to automated backup or file synchronization processes manifesting in system-generated messages not initiated by a user.
- Idle browser tabs: A UI and Screen-Level Artifact Noise that pertains to open but unused tabs.

- Background applications: A UI and Screen-Level Artifact Noise that pertains to open applications not actively contributing to work tasks.
- Floating toolbars / screen overlays: A UI and Screen-Level Artifact Noise that pertains to visible overlays, e.g., video conferencing toolbars or system controls not directly related to work tasks.
- Repeated navigation patterns: A UI and Screen-Level Artifact Noise that pertains to habitual UI actions, e.g., toggling between views without the meaningful augmentation of work artifacts.
- Accidental keyboard / mouse / touchpad input: An Ambient Input Noise that pertains to unintentional interactions with input devices that leave digital traces not directly related to work tasks.
- Unintentional mouse hovers or gestures: An Ambient Input Noise that pertains to cursor movements or UI gestures that occur without deliberate user engagement.
- Unrelated secondary monitor content: An Ambient Input Noise that pertains to content visible on additional monitors not directly related to work tasks.
- Voice assistant triggers: An Ambient Input Noise that pertains to accidental or environmental activation of digital assistants.
- Idle thinking / blank stares: A Cognitive Gap Noise that pertains to mental processing, focus periods, or durations of inactivity, with no outwardly traceable interaction.
- Trial-and-error inputs: A Cognitive Gap Noise that pertains to speculative or discarded actions during problem solving that leave misleading traces or unfinalized traces.

- Browsing out of habit rather than need: A Cognitive Gap Noise that pertains to habitual platform or website visits not directly related to work tasks.
- Auto-generated method / test stubs: A System-Generated Noise that pertains to templated code elements generated by IDEs or other applications not directly engaged with by a user.
- AI completions that are deleted: A System-Generated Noise that pertains to model-generated code completions or suggestions that a user ignores or removes.
- Syntax error dialogs that are dismissed: A System-Generated Noise that pertains to transient error or warning messages not directly acted on by a user.
- Auto-linter suggestions that are ignored: A System-Generated Noise that pertains to static analysis or formatting tips manifested but not addressed by a user.
- Overhead conversations unrelated to work: A Human Noise that pertains to nearby spoken interactions not directly related to work tasks.
- Ambient room / office noise: A Human Noise that pertains to environmental sounds, e.g., typing, movement, or external voices in shared spaces.
- Coworker interruptions with no relevance: A Human Noise that pertains to spontaneous or social interruptions during work not directly related to work tasks.

APPENDIX C: TERM DEFINITIONS – RELEVANT SDLC PROVENANCE SOURCE TAXONOMY

Taxonomy Definition / Intent: Various sources of provenance influence SDLC work. The relevant SDLC provenance source taxonomy provides a domain-specific whitelist for provenance sources in SDLC work. Taxonomy terms are defined in Aristotelian form, i.e., each definition begins by stating the term it inherits its meaning from, thereafter made distinguished by its parent term with further details.

Categories:

- Requirements Phase Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the requirements phase of the SDLC.
- Design Phase Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the design phase of the SDLC.
- Implementation Phase Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the implementation phase of the SDLC.
- Testing Phase Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the testing phase of the SDLC.
- Deployment Phase Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the deployment phase of the SDLC.
- Maintenance Phase Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the maintenance phase of the SDLC.

- Sharing and Collaboration Provenance Source: A Relevant SDLC Provenance Source that pertains to provenance generated in the sharing and collaboration phase of the SDLC.

Exemplars:

- Software Requirements Specifications: A Requirements Phase Provenance Source that pertains to formal documents outlining the functional and non-functional requirements of some system.
- User stories: A Requirements Phase Provenance Source that pertains to informal, user-centric descriptions of desired system features, typically written in simple language, which influence requirements.
- Stakeholder-provided documents: A Requirements Phase Provenance Source that pertains to external materials or feedback submitted by stakeholders that influence requirements.
- Project kickoff communications: A Requirements Phase Provenance Source that pertains to initial communications that define project goals, scope, roles, expectations, etc.
- Planning meeting transcripts / minutes: A Requirements Phase Provenance Source that pertains to written (either manually or automatically) records of early planning sessions that capture decisions and task definitions influencing requirements.
- Threads / comments with feature definitions: A Requirements Phase Provenance Source that pertains to online or internal discussions that influence requirements.

- Architecture / UML diagrams: A Design Phase Provenance Source that pertains to visual models representing system structure, components, and their interactions.
- Wireframes: A Design Phase Provenance Source that pertains to low-fidelity visual designs of user interfaces used to plan layout and interactions before implementation.
- Code commits and commit messages: An Implementation Phase Provenance Source that pertains to discrete units of code changes along with human-written explanations.
- Pull / merge requests: An Implementation Phase Provenance Source that pertains to proposals to integrate changes into a codebase, often with linked discussion and review.
- Branch changes: An Implementation Phase Provenance Source that pertains to the creation, switching, or merging of development branches to isolate or integrate work.
- Code review comments: An Implementation Phase Provenance Source that pertains to annotations and suggestions made by peers on proposed code changes.
- Pair programming collaboration: An Implementation Phase Provenance Source that pertains to sessions where two or more developers work together on a task, often sharing insights not captured elsewhere.
- Unit / integration test source files: A Testing Phase Provenance Source that pertains to code written to verify the correctness of individual units or combined modules.
- Test plans / cases: A Testing Phase Provenance Source that pertains to structured outlines of test conditions, inputs, and expected outputs used to validate functionality.
- Automated test logs: A Testing Phase Provenance Source that pertains to output generated from running automated test suites, often indicating pass/fail status.

- Environment configuration changes: A Deployment Phase Provenance Source that pertains to modifications to deployment environments, e.g., variables, cloud settings, etc., that affect behavior.
- Rollback events: A Deployment Phase Provenance Source that pertains to actions taken to revert the system to a previous state due to failed deployments or bugs.
- Bug reports: A Maintenance Phase Provenance Source that pertains to formal or informal descriptions of system failures or incorrect behavior, often logged by users or quality assurance teams.
- Hotfix comments: A Maintenance Phase Provenance Source that pertains to emergency code changes made directly to fix critical issues, often outside the normal workflow.
- Internal change requests: A Maintenance Phase Provenance Source that pertains to structured proposals for modifying existing features or behavior, often from team members or management.
- Temporal comments: A Maintenance Phase Provenance Source that pertains to in-line code annotations indicating areas for future improvement or incomplete work.
- Wiki pages: A Sharing and Collaboration Phase Provenance Source that pertains to internal documentation authored by team members to describe systems, procedures, or decisions.
- Developer files: A Sharing and Collaboration Phase Provenance Source that pertains to entry-point documentation explaining how to install, run, or contribute to a system or codebase.

- Developer resource links: A Sharing and Collaboration Phase Provenance Source that pertains to external knowledge sources referenced during implementation or debugging.
- API references / language docs: A Sharing and Collaboration Phase Provenance Source that pertains to formal documentation for libraries, frameworks, or programming languages consulted or used during development.